

LINE: Queueing Analysis Algorithms

Example Manual – MATLAB Edition

Last revision: October 4, 2026
QORE Lab, Imperial College London

Contents

1	About this manual	1
2	Basic models	2
2.1	Open queueing networks	2
2.1.1	mam_transient_mapmap1	2
2.1.2	oqn_basic	3
2.1.3	oqn_bmmapt	4
2.1.4	oqn_cs_routing	5
2.1.5	oqn_fourqueues	6
2.1.6	oqn_mapt	8
2.1.7	oqn_mapt_service	9
2.1.8	oqn_mmapt	10
2.1.9	oqn_multichain_cs	11
2.1.10	oqn_nhpp	12
2.1.11	oqn_online	13
2.1.12	oqn_trace_driven	14
2.1.13	oqn_vsinks	15
2.2	Closed queueing networks	16
2.2.1	cqn_bas_blocking	16
2.2.2	cqn_bcmp_theorem	17
2.2.3	cqn_lcfs_multiclass	18
2.2.4	cqn_mmpp2_service	19
2.2.5	cqn_multichain_cs	20
2.2.6	cqn_multiserver	21
2.2.7	cqn_multiserver_nc	22
2.2.8	cqn_online	23
2.2.9	cqn_repairmen	24
2.2.10	cqn_repairmen_multi	25
2.2.11	cqn_scheduling_dps	26
2.2.12	cqn_threeclass_hyperl	27
2.2.13	cqn_twoclass_erl	28
2.2.14	cqn_twoclass_hyperl	29
2.2.15	cqn_twoqueues_multi	30
2.3	Mixed networks	31
2.3.1	mqn_basic	32
2.3.2	mqn_multichain_cs	33
2.3.3	mqn_multiserver_fcfs	34
2.3.4	mqn_multiserver_ps	35
2.3.5	mqn_singleserver_fcfs	36

2.3.6	<code>mqn_singleserver_ps</code>	38
2.4	Class switching	39
2.4.1	<code>cs_implicit</code>	39
2.4.2	<code>cs_multi_diamond</code>	40
2.4.3	<code>cs_single_diamond</code>	41
2.4.4	<code>cs_transient_class</code>	42
2.5	Priority models	44
2.5.1	<code>prio_hol_closed</code>	44
2.5.2	<code>prio_hol_open</code>	45
2.5.3	<code>prio_identical</code>	46
2.5.4	<code>prio_psprio</code>	48
2.6	Fork-join networks	49
2.6.1	<code>fj_asymm</code>	49
2.6.2	<code>fj_basic_closed</code>	50
2.6.3	<code>fj_basic_nesting</code>	51
2.6.4	<code>fj_basic_open</code>	52
2.6.5	<code>fj_complex_serial</code>	53
2.6.6	<code>fj_cs_multi_visits</code>	54
2.6.7	<code>fj_cs_postfork</code>	55
2.6.8	<code>fj_cs_prefork</code>	57
2.6.9	<code>fj_deep_nesting</code>	58
2.6.10	<code>fj_delays</code>	59
2.6.11	<code>fj_mixed_openclosed</code>	60
2.6.12	<code>fj_nojoin</code>	62
2.6.13	<code>fj_route_overlap</code>	63
2.6.14	<code>fj_serialfjs_closed</code>	64
2.6.15	<code>fj_serialfjs_open</code>	65
2.6.16	<code>fj_threebranches</code>	66
2.6.17	<code>fj_tiny_closed</code>	67
2.6.18	<code>fj_twoclasses_forked</code>	68
2.6.19	<code>fj_variable_fanout</code>	70
2.7	Layered queueing networks	71
2.7.1	<code>lqn_basic</code>	71
2.7.2	<code>lqn_bpmn</code>	72
2.7.3	<code>lqn_bpmn_trace</code>	74
2.7.4	<code>lqn_flatph</code>	75
2.7.5	<code>lqn_fork_open_arrival</code>	77
2.7.6	<code>lqn_init</code>	78
2.7.7	<code>lqn_jsq</code>	79
2.7.8	<code>lqn_moment3</code>	80
2.7.9	<code>lqn_multi_solvers</code>	81
2.7.10	<code>lqn_ofbiz</code>	82
2.7.11	<code>lqn_open_arrival</code>	84
2.7.12	<code>lqn_paramident</code>	85
2.7.13	<code>lqn_rrobin</code>	86
2.7.14	<code>lqn_serial</code>	87
2.7.15	<code>lqn_server_pools</code>	88
2.7.16	<code>lqn_setup</code>	89
2.7.17	<code>lqn_sockshop</code>	90
2.7.18	<code>lqn_srvnph</code>	91

2.7.19	lqn_transient	93
2.7.20	lqn_twotasks	94
2.7.21	lqn_workflows	95
2.8	Cache models	96
2.8.1	cache_compare_replc	97
2.8.2	cache_itemsize_costcap	98
2.8.3	cache_mmap_rr_env	99
2.8.4	cache_replc_climb	101
2.8.5	cache_replc_fifo	102
2.8.6	cache_replc_hlru	104
2.8.7	cache_replc_lru	105
2.8.8	cache_replc_qlru	107
2.8.9	cache_replc_routing	108
2.8.10	cache_replc_rr	110
2.8.11	cache_rmf_transient	111
2.8.12	cachenet_tandem	112
2.8.13	retrieval_chain	115
2.8.14	retrieval_default	116
2.8.15	retrieval_ps	117
2.8.16	retrieval_routing	119
2.8.17	retrieval_simple	120
2.9	Cluster models	122
2.9.1	cl_basic	122
2.9.2	cl_closed	123
2.9.3	cl_compare	123
2.9.4	cl_multiclass	124
2.9.5	cl_scheduling	125
2.9.6	cl_stations	125
2.9.7	cl_sweep	126
2.9.8	dispatch_closed	127
2.9.9	dispatch_mixed	128
2.9.10	dispatch_open	128
2.10	Stochastic Petri nets	129
2.10.1	spn_basic_closed	130
2.10.2	spn_basic_open	131
2.10.3	spn_closed_fourplaces	132
2.10.4	spn_closed_twoplaces	133
2.10.5	spn_colored_gspn	134
2.10.6	spn_fluid_dae	135
2.10.7	spn_fourmodes	136
2.10.8	spn_inhibiting	137
2.10.9	spn_lpbounds	138
2.10.10	spn_open_sevenplaces	139
2.10.11	spn_pareto_service	140
2.10.12	spn_productform_nc	141
2.10.13	spn_queueing_place	142
2.10.14	spn_twomodes	143
2.10.15	test_spn_nrm_open	144
2.11	Workflow models	145
2.11.1	wf_aph	145

2.11.2	<code>wf_branch</code>	146
2.11.3	<code>wf_complex</code>	146
2.11.4	<code>wf_erlang</code>	147
2.11.5	<code>wf_loop</code>	148
2.11.6	<code>wf_parallel</code>	149
2.11.7	<code>wf_serial</code>	149
3	Advanced models	151
3.1	Load-dependent stations	151
3.1.1	<code>fes_aggregation</code>	151
3.1.2	<code>fes_single_class</code>	152
3.1.3	<code>ld_class_dependence</code>	153
3.1.4	<code>ld_fes_multiclass</code>	154
3.1.5	<code>ld_fes_singleclass</code>	155
3.1.6	<code>ld_global_dependence</code>	156
3.1.7	<code>ld_joint_dependence</code>	157
3.1.8	<code>ld_multiserver_fcfs</code>	158
3.1.9	<code>ld_multiserver_ps</code>	159
3.1.10	<code>ld_multiserver_ps_twoclasses</code>	160
3.1.11	<code>ld_whittle_bandwidth</code>	161
3.2	Finite capacity regions	163
3.2.1	<code>fcf_constraints</code>	163
3.2.2	<code>fcf_lincon</code>	164
3.2.3	<code>fcf_lossn</code>	165
3.2.4	<code>fcf_mmlkdrop</code>	166
3.2.5	<code>fcf_mmlwaitq</code>	167
3.2.6	<code>fcf_oqndrop</code>	168
3.2.7	<code>fcf_oqnwaitq</code>	169
3.3	State probabilities	170
3.3.1	<code>statepr_aggr</code>	170
3.3.2	<code>statepr_aggr_large</code>	171
3.3.3	<code>statepr_allprobs_fcfs</code>	172
3.3.4	<code>statepr_allprobs_ps</code>	173
3.3.5	<code>statepr_sys_aggr</code>	174
3.3.6	<code>statepr_sys_aggr_large</code>	175
3.3.7	<code>statepr_sys_marg</code>	177
3.4	Initial state and warm start	178
3.4.1	<code>init_state_fcfs_exp</code>	178
3.4.2	<code>init_state_fcfs_nonexp</code>	179
3.4.3	<code>init_state_ps</code>	180
3.4.4	<code>ldes_warmstart</code>	181
3.5	State-dependent routing	182
3.5.1	<code>sdroute_closed</code>	182
3.5.2	<code>sdroute_jsq</code>	183
3.5.3	<code>sdroute_krzesinski</code>	184
3.5.4	<code>sdroute_multibranch</code>	185
3.5.5	<code>sdroute_open</code>	186
3.5.6	<code>sdroute_twoclasses_closed</code>	187
3.6	Cyclic polling	189
3.6.1	<code>polling_exhaustive_det</code>	189

3.6.2	polling_exhaustive_exp	190
3.6.3	polling_gated	190
3.6.4	polling_klimited	191
3.7	Switchover times	192
3.7.1	switchover_basic	192
3.8	Response time distribution	193
3.8.1	cdf_respt_closed	193
3.8.2	cdf_respt_closed_threeclasses	194
3.8.3	cdf_respt_distrib	195
3.8.4	cdf_respt_open_twoclasses	196
3.8.5	cdf_respt_populations	197
3.9	Busy periods	198
3.9.1	busyp_subnetwork	198
3.10	Stochastic network calculus	199
3.10.1	snc_delay_quantile	200
3.10.2	snc_tandem_multiclass	201
3.11	Random environments	202
3.11.1	example_mapqn2renv	202
3.11.2	renv_basic	203
3.11.3	renv_container_terminal	204
3.11.4	renv_fourstages_repairmen	205
3.11.5	renv_genqn	206
3.11.6	renv_lqn_twostages	206
3.11.7	renv_map_fallback	207
3.11.8	renv_node_breakdown	208
3.11.9	renv_rotterdam_blending	209
3.11.10	renv_threestages_repairmen	210
3.11.11	renv_twostages_repairmen	211
3.12	Layered cache-queueing models	212
3.12.1	lcq_async_cache	212
3.12.2	lcq_singlehost	213
3.12.3	lcq_threehosts	214
3.13	Pass and swap	215
3.13.1	pas_closed_tandem_fig6	216
3.13.2	pas_compatibility_5class	217
3.13.3	pas_cyclic_ctmc_vs_bruteforce	218
3.13.4	pas_cyclic_normconst_bruteforce	218
3.13.5	pas_mmk	219
3.13.6	pas_nc_sampling	220
3.13.7	pas_saturation	221
3.13.8	pas_selfloop	222
3.14	Reward models	223
3.14.1	rewardModel_aggregation	223
3.14.2	rewardModel_mmk	224
3.14.3	rewardModel_multiclass	225
3.14.4	rewardModel_templates	226
3.15	Agent models	227
3.15.1	ag_closed_network	227
3.15.2	ag_gnetwork	228
3.15.3	ag_jackson_network	229

3.15.4	ag_multiclass_closed	230
3.15.5	ag_tandem_open	231
3.15.6	ag_tandem_phasetype	232
3.16	Large-scale models	233
3.16.1	largescale_tbi	233
3.17	Custom solvers	234
3.17.1	solver_custom	234
3.17.2	solver_custom_analyzer	234
4	Model gallery	236
4.1	Reading the names	236
4.2	The catalogue	237
4.2.1	gallery_aphm1	237
4.2.2	gallery_cache_lru	238
4.2.3	gallery_cache_routing	239
4.2.4	gallery_coxm1	240
4.2.5	gallery_cqn	240
4.2.6	gallery_cqn_multiclass	241
4.2.7	gallery_detm1	242
4.2.8	gallery_dm1	243
4.2.9	gallery_erldk	244
4.2.10	gallery_erlerl1	245
4.2.11	gallery_erlerl1_reentrant	245
4.2.12	gallery_erlm1	246
4.2.13	gallery_erlm1_ps	247
4.2.14	gallery_erlm1_reentrant	248
4.2.15	gallery_erlm1ps	248
4.2.16	gallery_fcr	249
4.2.17	gallery_fj_closed	250
4.2.18	gallery_fj_open	251
4.2.19	gallery_fj_quorum	252
4.2.20	gallery_gamm1	253
4.2.21	gallery_hyperl1_feedback	254
4.2.22	gallery_hyperl1_reentrant	255
4.2.23	gallery_hyperlk	255
4.2.24	gallery_hyphyp1_linear	256
4.2.25	gallery_hyphyp1_reentrant	257
4.2.26	gallery_hyphyp1_tandem	258
4.2.27	gallery_hypm1	258
4.2.28	gallery_hypm1_reentrant	259
4.2.29	gallery_lqn_basic	260
4.2.30	gallery_lqn_random	261
4.2.31	gallery_lqn_workflows	262
4.2.32	gallery_lukumar_reentrant	263
4.2.33	gallery_mapm1	264
4.2.34	gallery_mapmk	265
4.2.35	gallery_mdk	265
4.2.36	gallery_merl1	266
4.2.37	gallery_merl1_linear	267
4.2.38	gallery_merl1_reentrant	268

4.2.39	gallery_merl1_tandem	268
4.2.40	gallery_merlk	269
4.2.41	gallery_mhyp1	270
4.2.42	gallery_mhyp1_linear	270
4.2.43	gallery_mhyp1_reentrant	271
4.2.44	gallery_mhyp1_tandem	272
4.2.45	gallery_mhypk	273
4.2.46	gallery_mm1	273
4.2.47	gallery_mm1_feedback	274
4.2.48	gallery_mm1_linear	275
4.2.49	gallery_mm1_multiclass	276
4.2.50	gallery_mm1_prio	276
4.2.51	gallery_mm1_ps	277
4.2.52	gallery_mm1_ps_feedback	278
4.2.53	gallery_mm1_ps_multiclass	279
4.2.54	gallery_mm1_ps_reentrant	279
4.2.55	gallery_mm1_reentrant	280
4.2.56	gallery_mm1_tandem	281
4.2.57	gallery_mm1_tandem_multiclass	281
4.2.58	gallery_mm1k	282
4.2.59	gallery_mmap1	283
4.2.60	gallery_mmap1_multiclass	284
4.2.61	gallery_mmapk	284
4.2.62	gallery_mmk	285
4.2.63	gallery_mpar1	286
4.2.64	gallery_multitier	287
4.2.65	gallery_multitier_storage	288
4.2.66	gallery_parm1	289
4.2.67	gallery_qn_random	290
4.2.68	gallery_renv_breakdown	291
4.2.69	gallery_repairmen	291
4.2.70	gallery_replayerm1	292
4.2.71	gallery_um1	293
5	Discrete-time models	294
5.1	The slotted models	294
5.1.1	dt_bernoulli_loaddep	294
5.1.2	dt_cycle	295
5.1.3	dt_cycle_loaddep	296
5.1.4	dt_cycle_multiclass	297
5.1.5	dt_geogeol	298
5.1.6	dt_geogeol_loss	299
6	Solver-specific examples	301
6.0.1	ctmc_spn_mm1	301
6.0.2	ctmc_spn_vs_nc_closed_qn	302
6.0.3	ctmc_tandem_mm1	303

7	Model interchange	305
7.1	The fixtures	305
7.1.1	json_balking_retrial	305
7.1.2	json_classcap_droprule	306
7.1.3	json_fcr_details	307
7.1.4	json_hetero_servers	308
7.1.5	json_join_deadline	309
7.1.6	json_signal_classes	310
8	Inference and estimation	312
8.1	The estimators	312
8.1.1	est_ekf_changepoint	312
8.1.2	est_ekf_closed	313
8.1.3	est_ekf_open	314
8.1.4	est_erps_closed	315
8.1.5	est_erps_open	316
8.1.6	est_mcmc_closed	317
8.1.7	est_mle_open	318
8.1.8	est_qmle_closed	319
8.1.9	est_trace_closed	320
8.1.10	est_ubo_changepoint	321
8.1.11	est_ubo_closed	322
8.1.12	est_ubo_open	323
8.1.13	est_ubo_variants_closed	324
8.1.14	est_ubr_changepoint	325
8.1.15	est_ubr_closed	326
8.1.16	est_ubr_open	327
8.1.17	est_vi_closed	328
9	Optimization	330
9.1	Sizing an M/M/c queue	330
9.2	Exact sizing by bisection	331
9.3	Continuous service rate tuning	331
9.4	Load balancing across heterogeneous servers	332
9.5	Population sizing in a closed network	332
9.6	Robust sizing under workload scenarios	333
9.7	Cost-performance tradeoff frontier	333
9.8	Decomposing a multi-variable problem	334
9.9	The models behind the tutorials	334
9.9.1	opt_bisection_sizing	334
9.9.2	opt_decomposition	335
9.9.3	opt_load_balancing	336
9.9.4	opt_lqn_host_demand	337
9.9.5	opt_pareto_frontier	338
9.9.6	opt_population_sizing	339
9.9.7	opt_robust_sizing	340
9.9.8	opt_sensitivity_report	341
9.9.9	opt_server_sizing	342
9.9.10	opt_service_rate	343

Chapter 1

About this manual

This manual documents the worked-example inventory of LINE, one edition per codebase. An entry is runnable only when this edition identifies a source implementation and does not mark the required operation as unsupported: `matlab/examples/`, and each chapter follows the organization of that tree. Script names agree across the four codebases, so an example can be read side by side between editions; where a codebase does not carry a family at all, the chapter says so rather than leaving the reader to infer it.

The reference material on model classes, solvers and options is in the LINE user manual, and the guided tour and the introductory tutorials are in the LINE primer. Neither is repeated here: this manual starts where the primer stops, at the point where one already knows how to build a model and wants to see the feature one needs.

Every model in the suite is written in the same four blocks, and it is worth learning to read them:

1. the nodes: stations, sources, sinks and the special nodes;
2. the classes and their arrival and service distributions;
3. the topology, as a routing matrix or a serial routing shorthand;
4. the solution: one or more solvers, and the metrics read from them.

The chapters are:

- *Basic models* (Chapter 2): the elementary model families, one per node or class feature.
- *Advanced models* (Chapter 3): features that leave the product-form setting.
- *Model gallery* (Chapter 4): a catalogue of small named models, in Kendall notation where one applies.
- *Discrete-time models* (Chapter 5): models on a slotted time scale.
- *Solver-specific examples* (Chapter 6): scripts written around one solver rather than one model.
- *Model interchange* (Chapter 7): the JSON model format and its round-trip fixtures.
- *Inference and estimation* (Chapter 8): fitting model parameters to measurements.
- *Optimization* (Chapter 9): design problems on queueing networks.

Each chapter opens with a worked example and then gives one section per script in the reference inventory. A complete entry contains a topology figure, a feature table, source code, and captured output. These components are generated from the harvested model, the implementation in the named language tree, and an actual run.

Chapter 2

Basic models

The basic examples cover one model family each: a node type, a class type or a scheduling feature, in the smallest model that shows it. They live under `matlab/examples/basic/`, one subdirectory per family, and the script names agree across the four codebases, so a model can be compared line by line between editions.

2.1 Open queueing networks

An open network has an unbounded population: jobs enter at a `Source`, visit stations according to the routing matrix and leave at a `Sink`. Stability requires the utilization of every station to stay below one, which is a property of the arrival rates and the demands, not something the model declares.

The archetype is `oqn_basic`: a Poisson stream feeds a Delay station with hyperexponential think time and an FCFS queue with exponential service. Four blocks build it – nodes, classes and their distributions, the routing matrix, and the solve – and every model in this manual follows the same four blocks.

```
model = Network('model');
%% Block 1: nodes
node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');
%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class1', 0);
node{1}.setService(jobclass{1}, HyperExp(0.5, 3.0, 10.0));
node{2}.setService(jobclass{1}, Exp(1));
node{3}.setArrival(jobclass{1}, Exp(0.1));
%% Block 3: topology
P = cell(1,1);
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);
%% Block 4: solution
AvgTable = MVA(model).getAvgTable()
```

The scripts of this family are:

2.1.1 `mam_transient_mapmap1`

Transient analysis of a MAP/MAP/1 queue started empty, through the Laplace-domain transient QBD method that `getTranAvg` auto-selects when both the arrival and the service process are correlated. The example is built and solved as follows.

```
function mam_transient_mapmap1()
% MAM_TRANSIENT_MAPMAP1 Transient analysis of a MAP/MAP/1 queue via the
```

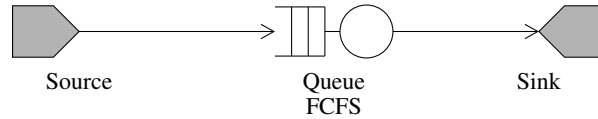


Figure 2.1: Topology of mam_transient_mapmap1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: MAP
Service	Queue – Class1: MAP

Table 2.1: Features of mam_transient_mapmap1.

```
% Laplace-domain transient QBD solver in MAM. Computes the time-dependent
% mean queue length, utilization, and throughput of a single-server queue with
% correlated (MAP) arrivals and correlated (MAP) service, starting empty.
%
% The Laplace transient QBD method is auto-selected by getTranAvg because the
% arrival and service processes are correlated MAPs (the libQBD/expm fast path
% only handles Poisson arrivals).

% Correlated arrival MAP (3 phases) and service MAP (2 phases), service scaled
% to a stable utilization rho = 0.6.
D0 = [-8, 1, 3; 0, -6, 4; 2, 0, -3];
D1 = [3, 1, 0; 0, 2, 0; 0, 0, 1];
S0 = [-3, 1; 6, -7];
S1 = [0, 2; 1, 0];
svc = map_scale({S0, S1}, 0.6 / map_lambda({D0, D1}));

model = Network('MAP/MAP/1 transient');
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
oclass = OpenClass(model, 'Class1');
source.setArrival(oclass, MAP({D0, D1}));
queue.setService(oclass, MAP(svc{1}, svc{2}));
model.link(Network.serialRouting(source, queue, sink));

... (13 source lines omitted; full implementation: matlab/examples/basic/)

% codebases print the same three labels so the cells line up.
fprintf('TranEndQLen[MAM/Queue]: %.6f\n', QNT{2,1}.metric(end));
fprintf('TranEndUtil[MAM/Queue]: %.6f\n', UNT{2,1}.metric(end));
fprintf('TranEndTput[MAM/Queue]: %.6f\n', TNT{2,1}.metric(end));

end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.2 oqn_basic

Source, Delay and an FCFS queue with one open class and a hyperexponential think time, solved with every solver that admits the model.

The example is built and solved as follows.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');
```

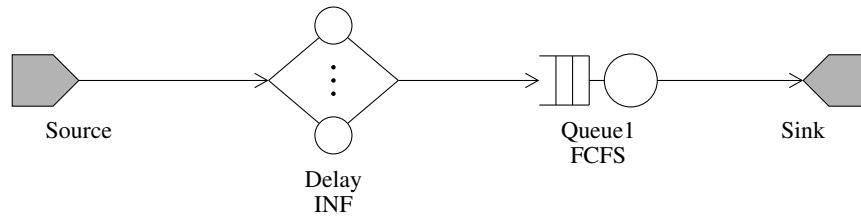


Figure 2.2: Topology of oqn_basic: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(0.1)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10) Queue1 – Class1: Exp(1)
Solvers	CTMC, FLD, MVA, MAM, NC, JMT, SSA, LDES

Table 2.2: Features of oqn_basic.

```

jobclass{1} = OpenClass(model, 'Class1', 0);

node{1}.setService(jobclass{1}, HyperExp(0.5,3.0,10.0));
node{2}.setService(jobclass{1}, Exp(1));
node{3}.setArrival(jobclass{1}, Exp(0.1));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K,K);
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];

model.link(P);
%%
options = Solver.defaultOptions;
options.keep=true;
options.verbose=1;
options.cutoff = 10;
options.seed = 23000;

... (31 source lines omitted; full implementation: matlab/examples/basic/)

solver{end+1} = LDES(model,ldeOptions);
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.3 oqn_bmmapt

A BATCH, MARKED, time-inhomogeneous MAP arrival stream: a block is indexed by segment, mark and batch size, so the schedule moves the class mix and the batch size together while the epoch rate stays put. The job rate therefore differs from the epoch rate, and per class within a segment.

The example is built and solved as follows.

```

% Open queueing network whose arrival stream is a BATCH, MARKED,
% time-inhomogeneous MAP.
%

```

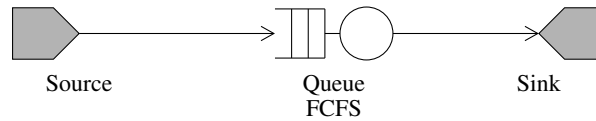


Figure 2.3: Topology of oqn_bmmapt: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Premium (open), Economy (open)
Scheduling	Queue: FCFS
Arrivals	Source – Premium: BMMAPt(True); Economy: BMMAPt(True)
Service	Queue – Premium: Exp(16); Economy: Exp(16)
Solvers	LDES

Table 2.3: Features of oqn_bmmapt.

```
% BMMAPt crosses the three axes LINE models for an arrival stream. A block is
% indexed by segment, mark and batch size: D1kb{c}{b}{j} holds the rates that,
% in segment j, release a batch of b jobs ALL carrying mark c, and
% D0{j} + sum_c sum_b D1kb{c}{b}{j} is a generator in every segment. At a Source
% the mark selects the CLASS of the arriving jobs and the batch size says HOW
% MANY of them arrive at that instant.
%
% Two derived levels come for free and are what keep the family legible:
% summing over b gives the MMAPt of the marks alone, and summing over c as well
% gives the MAPt of the epochs alone. So a consumer that ignores batches sees
% exactly the marked schedule, and one that ignores marks too sees exactly the
% unmarked one.
%
% The schedule below moves BOTH the class mix and the batch size, which is the
% point: it is the combination no existing family expresses. Every segment fires
% epochs at rate 4, so the epoch stream is statistically identical throughout;
% what changes is which class the batch carries and how big it is. In the
% morning segment the Premium class arrives in PAIRS and Economy singly; in the
% evening segment that reverses. The JOB rate therefore differs from the EPOCH
% rate, and differs per class within a segment even though the epoch rate does
% not -- a model that read either label off the time-averaged matrices would
% miss all of it.
%
... (61 source lines omitted; full implementation: matlab/examples/basic/)

% extra random draw and the reduction to MMAPt at batch size 1 is exact.
options = SolverLDES.defaultOptions;
options.samples = 4e5;
options.seed = 23000;
AvgTable = SolverLDES(model, options).getAvgTable();
AvgTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.4 oqn_cs_routing

Three open classes over two PS queues, two of which switch class at a ClassSwitch node; the topology is imported from a JSIMgraph file.

The example is built and solved as follows.

```
%model = JMT2LINE('example_openModel_3.jsimg');
model = Network('myModel');

%% Block 1: nodes
node{1} = Source(model, 'Source 1');
node{2} = Queue(model, 'Queue 1', SchedStrategy.PS);
```

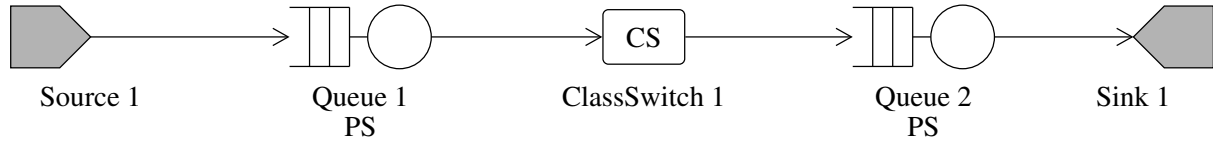


Figure 2.4: Topology of `oqn_cs_routing`: an open network over 2 queueing stations with a class-switch node.

Nodes	5: Source, Queue $\times$ 2, ClassSwitch, Sink
Classes	3: Class A (open), Class B (open), Class C (open)
Scheduling	Queue 1: PS; Queue 2: PS
Arrivals	Source 1 – Class A: Exp(2); Class B: Exp(1); Class C: Disabled
Service	Queue 1 – Class A: Exp(5); Class B: Exp(3.333); Class C: Exp(3) Queue 2 – Class A: Exp(1); Class B: Exp(1); Class C: Exp(6.667)
Solvers	CTMC, FLD, MVA, MAM, NC, JMT, SSA, LDES

Table 2.4: Features of `oqn_cs_routing`.

```
node{3} = ClassSwitch(model, 'ClassSwitch 1'); % Class switching is embedded in the routing
matrix P
node{4} = Sink(model, 'Sink 1');
node{5} = Queue(model, 'Queue 2', SchedStrategy.PS);

%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class A', 0);
jobclass{2} = OpenClass(model, 'Class B', 0);
jobclass{3} = OpenClass(model, 'Class C', 0);

node{1}.setArrival(jobclass{1}, Exp.fitMean(0.500000)); % (Source 1, Class A)
node{1}.setArrival(jobclass{2}, Exp.fitMean(1.000000)); % (Source 1, Class B)
node{1}.setArrival(jobclass{3}, Disabled.getInstance()); % (Source 1, Class C)
node{2}.setService(jobclass{1}, Exp.fitMean(0.200000)); % (Queue 1, Class A)
node{2}.setService(jobclass{2}, Exp.fitMean(0.300000)); % (Queue 1, Class B)
node{2}.setService(jobclass{3}, Exp.fitMean(0.333333)); % (Queue 1, Class C)
node{5}.setService(jobclass{1}, Exp.fitMean(1.000000)); % (Queue 2, Class A)
node{5}.setService(jobclass{2}, Exp.fitMean(1.000000)); % (Queue 2, Class B)
node{5}.setService(jobclass{3}, Exp.fitMean(0.150000)); % (Queue 2, Class C)

%% Block 3: topology
... (46 source lines omitted; full implementation: matlab/examples/basic/)

solver{end+1} = LDES(model, options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable()
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.5 `oqn_fourqueues`

A larger multiclass network over four stations mixing PS and FCFS scheduling. The example is built and solved as follows.

```
%% a large multiclass example with PS and FCFS
model = Network('MyNetwork');

%% Block 1: nodes
node{1} = Source(model, 'Source');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
```

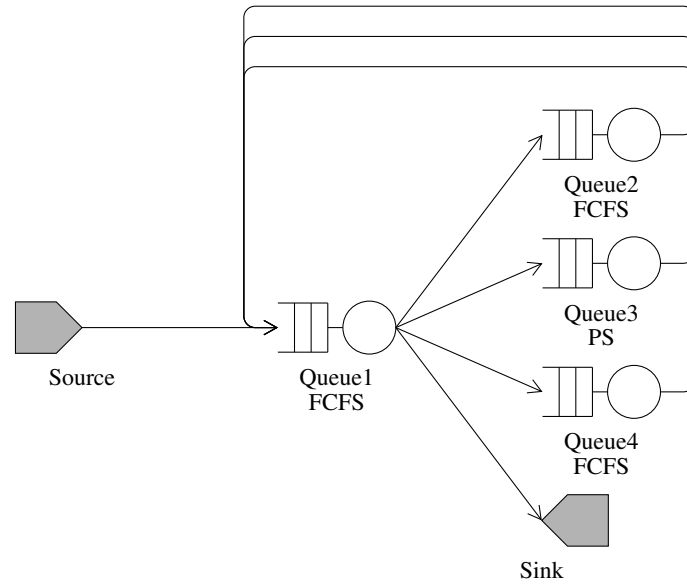


Figure 2.5: Topology of `oqn_fourqueues`: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: PS; Queue4: FCFS
Arrivals	Source – Class1: Exp(0.2); Class2: Exp(0.125); Class3: Exp(0.1429)
Service	Queue1 – Class1: Exp(3.333); Class2: Exp(2); Class3: Exp(1.667) Queue2 – Class1: Exp(0.9091); Class2: Exp(0.7692); Class3: Exp(0.6667) Queue3 – Class1: Exp(0.5); Class2: Exp(0.4762); Class3: Exp(0.5263) Queue4 – Class1: Exp(0.6667); Class2: Exp(1.111); Class3: Exp(0.4348)
Solvers	CTMC, MVA, MAM, JMT, LDES

Table 2.5: Features of `oqn_fourqueues`.


```

node{3} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{4} = Queue(model, 'Queue3', SchedStrategy.PS);
node{5} = Queue(model, 'Queue4', SchedStrategy.FCFS);
node{6} = Sink(model, 'Sink');

%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);
jobclass{3} = OpenClass(model, 'Class3', 0);

node{1}.setArrival(jobclass{1}, Exp.fitMean(5.000000)); % (Source of Customers,Class0)
node{1}.setArrival(jobclass{2}, Exp.fitMean(8.000000)); % (Source of Customers,Class1)
node{1}.setArrival(jobclass{3}, Exp.fitMean(7.000000)); % (Source of Customers,Class2)
node{2}.setService(jobclass{1}, Exp.fitMean(0.300000)); % (WebServer,Class0)
node{2}.setService(jobclass{2}, Exp.fitMean(0.500000)); % (WebServer,Class1)
node{2}.setService(jobclass{3}, Exp.fitMean(0.600000)); % (WebServer,Class2)
node{3}.setService(jobclass{1}, Exp.fitMean(1.100000)); % (Storage1,Class0)
node{3}.setService(jobclass{2}, Exp.fitMean(1.300000)); % (Storage1,Class1)
node{3}.setService(jobclass{3}, Exp.fitMean(1.500000)); % (Storage1,Class2)
node{4}.setService(jobclass{1}, Exp.fitMean(2.000000)); % (Storage2,Class0)

... (47 source lines omitted; full implementation: matlab/examples/basic/)

%solver(end+1) = NC(model,options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable()
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.6 oqn_mapt

A time-inhomogeneous MAP arrival stream and the Ko-Pender fluid and diffusion limits returned by the kp fluid method.

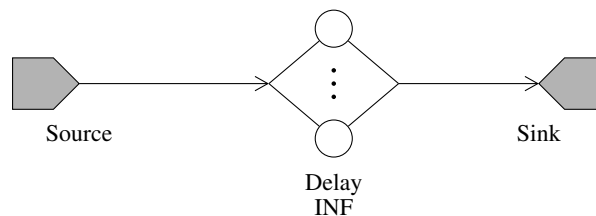


Figure 2.6: Topology of oqn_mapt: an open network over 1 queueing station.

Nodes	3: Source, Delay, Sink
Classes	1: OpenClass (open)
Scheduling	Delay: INF
Arrivals	Source – OpenClass: MAPt
Service	Delay – OpenClass: Exp(2)
Solvers	FLD

Table 2.6: Features of oqn_mapt.

The example is built and solved as follows.

```

% Open queueing network with a MAP_t arrival process and the Ko-Pender limits.
%
% MAPt is a time-inhomogeneous Markovian arrival process: segment k covers

```

```

% [breakpoints(k), breakpoints(k+1)) and carries the pair (D0{k}, D1{k}), so the
% stream is both non-renewal, through the modulating phase, and non-stationary,
% through the schedule. Setting one phase recovers an NHPP; setting one segment
% recovers an ordinary MAP.
%
% SolverFLD's 'kp' method integrates the fluid and diffusion limits of Ko and
% Pender, "Diffusion limits for the (MAP_t/Ph_t/inf)^N queueing network", Oper.
% Res. Lett. 45 (2017) 248-253: the mean and the covariance of the queue length
% are integrated jointly, so it is the only fluid method that returns a second
% moment. For infinite-server stations the rate functions are affine in the
% state, so both are exact rather than asymptotic -- the variance below is the
% exact variance of the queue length, not an approximation.

model = Network('model');

source = Source(model, 'Source');
delay = Delay(model, 'Delay');
sink = Sink(model, 'Sink');

jobclass = OpenClass(model, 'OpenClass', 0);

% Two segments of a 2-phase MAP, held for 1 and 1.5 time units and repeating.
% The second segment runs the same phase graph at roughly twice the rate.

... (22 source lines omitted; full implementation: matlab/examples/basic/)

[~,~,~,~,QNT,~,~,~,t,~,~,QVart] = solver_fluid_kp(model.getStruct(), options);
fprintf('\n   t       lambda(t)   QLen mean   QLen var\n');
for probe = [0.5 1.0 1.5 2.5 3.0 4.0 5.0]
    fprintf('%6.2f %10.4f %11.6f %11.6f\n', probe, arrival.getRateAt(probe), ...
        interp1(t, QNT{2,1}, probe), interp1(t, QVart{2,1}, probe));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.7 oqn_mapt_service

A time-inhomogeneous MAP as the SERVICE process of a finite-buffer queue: the phase walk is taken from the service start instant and the modulating phase carries across successive services, so the stream is correlated as well as non-stationary.

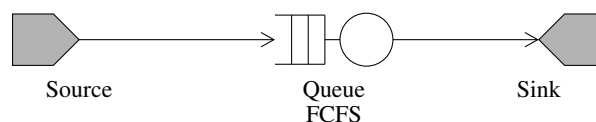


Figure 2.7: Topology of oqn_mapt_service: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: OpenClass (open)
Scheduling	Queue: FCFS
Arrivals	Source – OpenClass: Exp(10)
Service	Queue – OpenClass: MAPt
Features	finite buffer 5 at Queue
Solvers	LDES

Table 2.7: Features of oqn_mapt_service.

The example is built and solved as follows.

```

% Open queue with a MAP_t SERVICE process, simulated by LDES.
%

```

```
% A MAP_t service is the only process LDES simulates that is both non-renewal
% and non-stationary. Segment k covers [breakpoints(k), breakpoints(k+1)) and
% carries the pair (D0{k}, D1{k}); D1 fires a service COMPLETION and D0 only
% moves the modulating phase. The engine walks that phase process forward from
% the instant the job ENTERS SERVICE -- not from the instant it arrived, which
% may fall under a different segment -- and resumes from the phase the previous
% completion left behind, so successive service times are correlated exactly as
% for an ordinary MAP.
%
% The schedule below is an MMPP whose two environment states hold the same
% sojourn rates throughout (Q_env) while their arrival intensities change from
% segment to segment, so the server alternates between a fast and a slow mode
% AND the pair of modes itself changes with the clock.
%
% Because the walk is a wall-clock one, it is exact only where service runs
% continuously at unit rate once started. LDES therefore refuses a MAP_t (or
% PH_t) service under processor sharing, under a preemptive discipline, with
% load dependence and with heterogeneous server pools, rather than returning a
% number from a sample path that does not model any of those. Use INF or a
% non-preemptive FCFS/LCFS-family discipline, as here.

model = Network('model');

source = Source(model, 'Source');

... (59 source lines omitted; full implementation: matlab/examples/basic/)

tHomog = LDES(homog, 'seed', 1234, 'samples', 4e5).getAvgTable;
fprintf('\nconstant schedule vs ordinary MAP (must agree):\n');
fprintf(' MAPt   QLen=%8.5f Util=%8.5f RespT=%8.5f\n', ...
    tFlat.QLen(2), tFlat.Util(2), tFlat.RespT(2));
fprintf(' MAP    QLen=%8.5f Util=%8.5f RespT=%8.5f\n', ...
    tHomog.QLen(2), tHomog.Util(2), tHomog.RespT(2));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.8 oqn_mmapt

A MARKED time-inhomogeneous MAP arrival stream: one modulating chain drives every marked class, the mark selects the class of the arriving job, and the schedule moves the class MIX while leaving the aggregate rate flat.

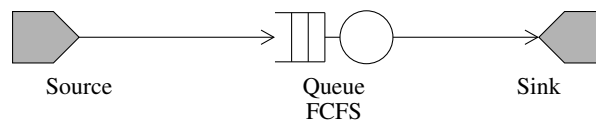


Figure 2.8: Topology of oqn_mmapt: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Morning (open), Evening (open)
Scheduling	Queue: FCFS
Arrivals	Source – Morning: MMAPt(True); Evening: MMAPt(True)
Service	Queue – Morning: Exp(8); Evening: Exp(8)
Solvers	LDES

Table 2.8: Features of oqn_mmapt.

The example is built and solved as follows.

```
% Open queueing network whose arrival stream is a MARKED, time-inhomogeneous MAP.
%
```

```

% MMAPt crosses the two axes of MAPt and MarkedMAP: arrivals are labelled with
% one of K marks, AND the matrices that generate them are functions of the wall
% clock. Segment j carries D0{j} together with the K blocks D1k{c}{j}, and
% D0{j} + sum_c D1k{c}{j} is a generator in every segment. The aggregate
% sum_c D1k{c}{j} is the D1 of the underlying MAPt, so hiding the marks recovers
% exactly that process.
%
% At a Source the mark SELECTS THE CLASS of the arriving job: one modulating
% chain drives every marked class, and only the first one carries the stream.
% That is what makes this different from declaring K independent arrival
% processes -- the classes are correlated through the shared phase, and the
% class MIX shifts with the schedule even when the total rate does not.
%
% The schedule below is built so that only the mix moves. Both segments carry an
% aggregate rate of 4, so the total arrival stream is statistically identical
% throughout; what changes is the split, 9:1 towards the first class in the
% morning segment and 1:9 towards the second in the evening one. A model that
% read the marks off the time-averaged matrices would report a flat 2:2 split.
%
% Reductions worth knowing: with K = 1 an MMAPt IS the MAPt with the same
% matrices, and with identical segments it IS the stationary MMAP. MPHT is the
% phase-type twin, stored lowered to this same form.
%
% Reference for the marked structure: Q.-M. He, "The versatility of MMAP[K] and
... (36 source lines omitted; full implementation: matlab/examples/basic/)

% the interval, so no extra randomness is introduced by the labelling.
options = SolverLDES.defaultOptions;
options.samples = 4e5;
options.seed = 23000;
AvgTable = SolverLDES(model, options).getAvgTable();
AvgTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.9 oqn_multichain_cs

Five classes grouped into three chains by class switching, the smallest model in which chains and classes differ.

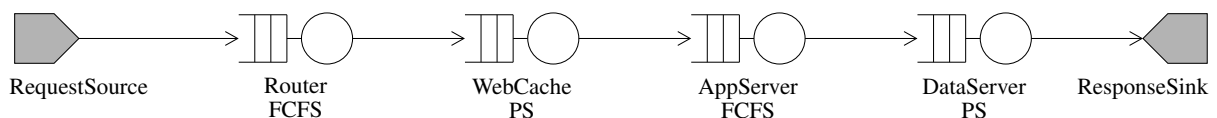


Figure 2.9: Topology of oqn_multichain_cs: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	5: InteractiveA (open), InteractiveB (open), BatchA (open), BatchB (open), RealTime (open)
Scheduling	Router: FCFS; WebCache: PS; AppServer: FCFS; DataServer: PS
Arrivals	RequestSource – InteractiveA: Exp(0.3); InteractiveB: Exp(0.1); BatchA: Exp(0.15); BatchB: Exp(0.12); RealTime: Exp(0.2)
Service	Router – InteractiveA: Exp(6.667); InteractiveB: Exp(5.556); BatchA: Exp(5); BatchB: Exp(4.545); RealTime: Exp(10) WebCache – InteractiveA: Exp(3.333); InteractiveB: Exp(2.857); BatchA: Exp(1); BatchB: Exp(0.8333); RealTime: Exp(5) AppServer – InteractiveA: Exp(2); InteractiveB: Exp(1.667); BatchA: Exp(0.6667); BatchB: Exp(0.5556); RealTime: Exp(3.333) DataServer – InteractiveA: Exp(2.5); InteractiveB: Exp(2.222); BatchA: Exp(1.25); BatchB: Exp(1); RealTime: Exp(4)
Solvers	MVA, FLD, SSA

Table 2.9: Features of oqn_multichain_cs.

The example is built and solved as follows.

```
%% Example of an open queueing network with 3 chains, 5 classes, and class switching
% Demonstrates job class switching within multiple open chains.
% Chain 1: Interactive requests (2 classes that switch)
% Chain 2: Batch jobs (2 classes that switch)
% Chain 3: Real-time tasks (1 class)

model = Network('oqn_multichain');

%% Block 1: nodes
node{1} = Source(model, 'RequestSource');
node{2} = Queue(model, 'Router', SchedStrategy.FCFS);
node{3} = Queue(model, 'WebCache', SchedStrategy.PS);
node{4} = Queue(model, 'AppServer', SchedStrategy.FCFS);
node{5} = Queue(model, 'DataServer', SchedStrategy.PS);
node{6} = Sink(model, 'ResponseSink');

%% Block 2: job classes
% Chain 1: Interactive requests (can switch between InteractiveA and InteractiveB)
jobclass{1} = OpenClass(model, 'InteractiveA', 0);
jobclass{2} = OpenClass(model, 'InteractiveB', 0);
% Chain 2: Batch jobs (can switch between BatchA and BatchB)
jobclass{3} = OpenClass(model, 'BatchA', 0);
jobclass{4} = OpenClass(model, 'BatchB', 0);
% Chain 3: Real-time tasks
jobclass{5} = OpenClass(model, 'RealTime', 0);

... (113 source lines omitted; full implementation: matlab/examples/basic/)

for s = 1:length(solver)
    fprintf('SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
    fprintf('\n');
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.10 oqn_nhpp

A cyclic non-homogeneous Poisson arrival stream, whose rate is a periodic function of time.

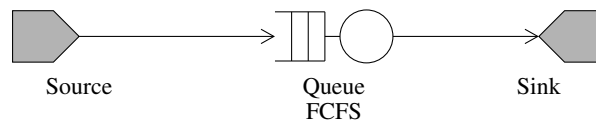


Figure 2.10: Topology of oqn_nhpp: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: OpenClass (open)
Scheduling	Queue: FCFS
Arrivals	Source – OpenClass: NHPP
Service	Queue – OpenClass: Exp(10)
Solvers	LDES

Table 2.10: Features of oqn_nhpp.

The example is built and solved as follows.

```
% Open queueing network with an NHPP (cyclic) arrival process.
%
% NHPP is a non-homogeneous Poisson process with a piecewise-constant
% intensity: segment i covers [breakpoints(i), breakpoints(i+1)) and carries
% rate rates(i). With cyclic=true the schedule repeats with period
% breakpoints(end)-breakpoints(1), giving a cyclic Poisson process. The LDES
% simulation engine honours the exact schedule; SolverFLD honours it in
% getTranAvg, where the intensity enters the closing fluid ODE as a
% time-varying rate multiplier. Steady state is the time-average rate.

model = Network('model');

source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

jobclass = OpenClass(model, 'OpenClass', 0);

% Rates 2,8,4 held for 3,1,2 time units, repeating cyclically.
source.setArrival(jobclass, NHPP([0,3,4,6],[2,8,4],true));
queue.setService(jobclass, Exp(10));

model.link(Network.serialRouting(source,queue,sink));

AvgTable{1} = LDES(model, 'seed', 1234, 'samples', 100000).getAvgTable;
AvgTable{1}

... (3 source lines omitted; full implementation: matlab/examples/basic/)

[~,~,Tnt] = solver.getTranAvg();
for t = [1.5 3.5 5.0 7.5 9.5 11.0]
    fprintf('t=%5.2f lambda(t)=%6.3f queue Tput=%6.3f\n', t, ...
        source.getArrivalProcess(jobclass).getRateAt(t), ...
        interp1(Tnt{2,1}.t, Tnt{2,1}.metric, t));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.11 oqn_online

The same tandem written in one line from the arrival, demand and think time matrices, through `Network.tandemPsInf`.

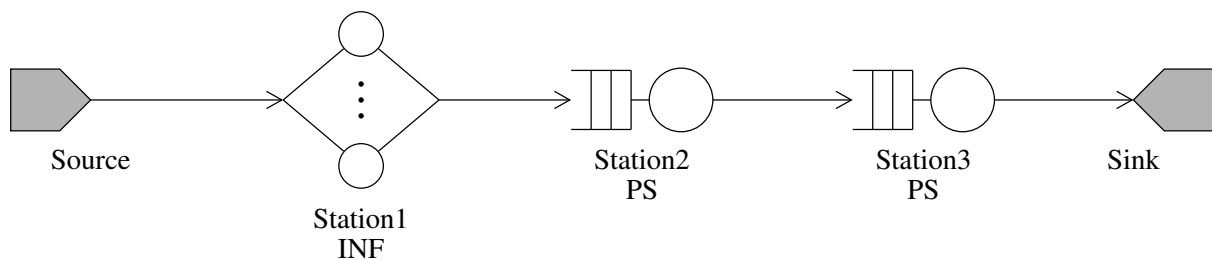


Figure 2.11: Topology of `oqn_online`: an open network over 3 queueing stations.

The example is built and solved as follows.

```
fprintf(1, 'This example shows a compact solution of a tandem open queueing network.\n');
lambda = [1,2]/50; % lambda(r) - arrival rate of class r
D = [10,5;5,9]; % D(i,r) - mean demand of class r at station i
Z = [91,92]; % Z(r) mean service time of class r at delay station i
model = Network.tandemPsInf(lambda,D,Z);
solver{1} = MVA(model);
AvgTable{1} = solver{1}.getAvgTable;
AvgTable{1}
```

Nodes	5: Source, Queue $\times$ 2, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Station1: INF; Station2: PS; Station3: PS
Arrivals	Source – Class1: Exp(0.02); Class2: Exp(0.04)
Service	Station1 – Class1: Exp(0.01099); Class2: Exp(0.01087) Station2 – Class1: Exp(0.1); Class2: Exp(0.2) Station3 – Class1: Exp(0.2); Class2: Exp(0.1111)
Solvers	MVA

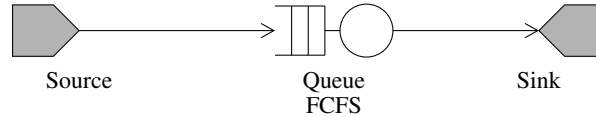
Table 2.11: Features of `oqn_online`.

```
% 1-line version: AvgTable=MVA(Network.tandemPsInf(A,D,Z)).getAvgTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.12 `oqn_trace_driven`

Arrivals and service times replayed from a measurement trace with `Replayer`.

Figure 2.12: Topology of `oqn_trace_driven`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: OpenClass (open)
Scheduling	Queue: FCFS
Arrivals	Source – OpenClass: Exp(1)
Service	Queue – OpenClass: <code>Replayer(trace)</code>
Solvers	JMT, LDES

Table 2.12: Features of `oqn_trace_driven`.

The example is built and solved as follows.

```
model = Network('model');

source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

jobclass = OpenClass(model, 'OpenClass', 0);

source.setArrival(jobclass, Exp(1));
cwd = fileparts(mfilename('fullpath'));
queue.setService(jobclass, Replayer([cwd, filesep, 'example_trace.txt']));

model.link(Network.serialRouting(source, queue, sink));

AvgTable{1} = JMT(model, 'seed', 23000).getAvgTable;
AvgTable{1}
AvgTable{2} = LDES(model, 'seed', 23000).getAvgTable;
AvgTable{2}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.1.13 oqn_vsinks

Several sinks in one model, the virtual-sink idiom for measuring departure streams separately.

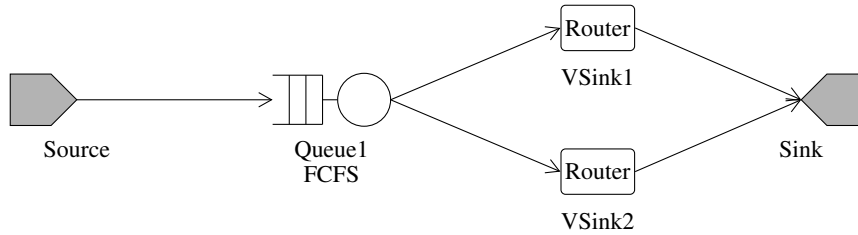


Figure 2.13: Topology of oqn_vsinks: an open network over 1 queueing station with a router.

Nodes	5: Source, Queue, Router $\times$ 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(1)
Service	Queue1 – Class1: Exp(100); Class2: Exp(100)
Solvers	MVA, MAM, NC

Table 2.13: Features of oqn_vsinks.

The example is built and solved as follows.

```
% This model exemplifies how to specify models with multiple sinks (virtual
% sink)
model = Network('model');

source = Source(model, 'Source');
queue = QueueingStation(model, 'Queue1', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
vsink1 = Router(model, 'VSink1');
vsink2 = Router(model, 'VSink2');

ocl1 = OpenClass(model, 'Class1');
ocl2 = OpenClass(model, 'Class2');

source.setArrival(ocl1, Exp(1.0));
queue.setService(ocl1, Exp(100.0));

source.setArrival(ocl2, Exp(1.0));
queue.setService(ocl2, Exp(100.0));

P = model.initRoutingMatrix;

P{ocl1}(source, queue) = 1.0;
P{ocl1}(queue, vsink1) = 0.6;
P{ocl1}(queue, vsink2) = 0.4;
P{ocl1}(vsink1, sink) = 1.0;
P{ocl1}(vsink2, sink) = 1.0;

... (12 source lines omitted; full implementation: matlab/examples/basic/)

AvgTableMAM = MAM(model).getAvgTable
AvgNodeTableMAM = MAM(model).getAvgNodeTable

AvgTableNC = NC(model).getAvgTable
AvgNodeTableNC = NC(model).getAvgNodeTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2 Closed queueing networks

A closed network holds a fixed population per class. Each closed class names a reference station, at which its visit ratio is one and through which its throughput is measured; the population is the third argument of the class constructor. A Delay station with think time Z and a queue with demand D is the interactive-system archetype, and the repairman model is its oldest form.

Because the population is finite, every metric is bounded and the network is stable by construction. What varies across the scripts below is the scheduling discipline, the service distribution and the number of servers, that is, exactly the assumptions under which a product-form solution does or does not hold.

2.2.1 cqn_bas_blocking

Blocking-after-service with finite buffers, where a completing job holds its server until the destination frees a slot.

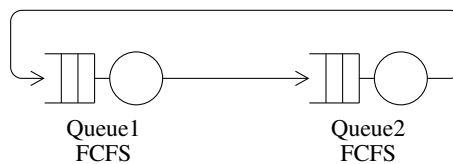


Figure 2.14: Topology of `cqn_bas_blocking`: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 2$)
Scheduling	Queue1: FCFS; Queue2: FCFS
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(0.8)
Features	drop rule {'Class1': 'blockingAfterService'} at Queue1; finite buffer 1 at Queue2
Solvers	MVA

Table 2.14: Features of `cqn_bas_blocking`.

The example is built and solved as follows.

```
% Closed network with Blocking-After-Service (BAS) finite-buffer blocking.
% Queue1 uses the BAS drop rule; Queue2 has a finite buffer (capacity 1).
% The 'sqd' (Smith Queue Decomposition) approximation handles this model;
% MVA also auto-routes BAS models to 'sqd' under its default method.
model = Network('cqn_bas_blocking');

node{1} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{2} = Queue(model, 'Queue2', SchedStrategy.FCFS);

jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1.0));
node{2}.setService(jobclass{1}, Exp(0.8));

node{2}.setCap(1);
node{1}.setDropRule(jobclass{1}, DropStrategy.BAS);

model.link(Network.serialRouting(node{1}, node{2}));

% Run solver with the SQD method explicitly
solver = MVA(model, 'sqd');
avgTable = solver.getAvgTable();
fprintf('SOLVER: MVA (method=sqd)\n');
disp(avgTable);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.2 cqn_bcmp_theorem

The product-form scheduling policies of the BCMP theorem (FCFS with class-independent exponential service, PS, LCFS-PR, INF) compared on one network.

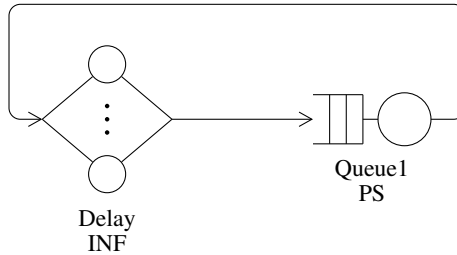


Figure 2.15: Topology of cqn_bcmp_theorem: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) Queue1 – Class1: Exp(1); Class2: Exp(1)
Solvers	CTMC

Table 2.15: Features of cqn_bcmp_theorem.

The example is built and solved as follows.

```
% comparison of product-form scheduling policies
c=1;
psmodel = Network('PS scheduling model');

node{1} = Delay(psmodel, 'Delay');
node{2} = Queue(psmodel, 'Queue1', SchedStrategy.PS);
node{2}.setNumberOfServers(c);

jobclass{1} = ClosedClass(psmodel, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(psmodel, 'Class2', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Erlang(3,2));
node{1}.setService(jobclass{2}, HyperExp(0.5,3.0,10.0));

node{2}.setService(jobclass{1}, Exp(1));
node{2}.setService(jobclass{2}, Exp(1));

P = psmodel.initRoutingMatrix;
P{1} = Network.serialRouting(node);
P{2} = Network.serialRouting(node);
psmodel.link(P);
%%
fcfsmodel = Network('FCFS scheduling model');

node{1} = Delay(fcfsmodel, 'Delay');
node{2} = Queue(fcfsmodel, 'Queue1', SchedStrategy.FCFS);

... (42 source lines omitted; full implementation: matlab/examples/basic/)

solver{end+1} = CTMC(lcfsprmodel, 'exact');
for s=1:length(solver)
    fprintf(1, 'MODEL: %s\n', solver{s}.model.getName());
end
```

```

    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.3 cqn_lcfs_multiclass

Multiclass LCFS and LCFS-PR scheduling, and the difference preemption makes.

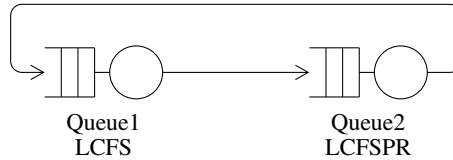


Figure 2.16: Topology of `cqn_lcfs_multiclass`: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$), Class3 (closed, $N = 1$)
Scheduling	Queue1: LCFS; Queue2: LCFS-PR
Service	Queue1 – Class1: Exp(1); Class2: Exp(0.3333); Class3: Exp(0.2) Queue2 – Class1: Exp(0.5); Class2: Exp(0.25); Class3: Exp(0.1667)
Solvers	MVA, CTMC

Table 2.16: Features of `cqn_lcfs_multiclass`.

The example is built and solved as follows.

```

% Example: Multiclass closed queueing network with LCFS and LCFS-PR scheduling
%
% This example demonstrates a 2-station closed queueing network where:
% - Station 1: LCFS (Last-Come-First-Served, non-preemptive)
% - Station 2: LCFS-PR (LCFS with Preemption-Resume)
%
% Reference:
% G. Casale, "A family of multiclass LCFS queueing networks with
% order-dependent product-form solutions", QUESTA 2026.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% Define service rates
% mu(i,r) = service rate at station i for class r
mu = 1./[1, 3, 5; 2, 4, 6]; % 2 stations x 3 classes
R = size(mu, 2); % number of classes

%% Create the network model
model = Network('LCFS Multiclass Model');

% Create nodes
node{1} = Queue(model, 'Queue1', SchedStrategy.LCFS);
node{2} = Queue(model, 'Queue2', SchedStrategy.LCFS-PR);

% Create job classes (one job per class)

... (88 source lines omitted; full implementation: matlab/examples/basic/)

fprintf(' - Utilizations do not match\n');
end
if ~tput_match
    fprintf(' - Throughputs do not match\n');

```

```
end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.4 cqn_mmpp2_service

MMPP(2) service times, a correlated service process that no product-form solver can represent exactly.

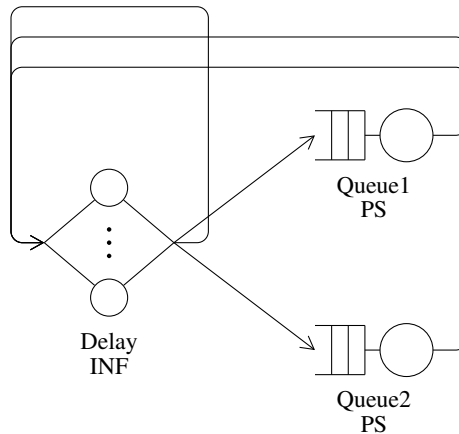


Figure 2.17: Topology of cqn_mmpp2_service: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) Queue1 – Class1: HyperExp(0.1,0.9;1,10); Class2: MMPP2 Queue2 – Class1: HyperExp(0.1,0.9;1,10); Class2: Erlang(1,2)
Features	class switching on 3 links
Solvers	JMT, LDES

Table 2.17: Features of cqn_mmpp2_service.

The example is built and solved as follows.

```
%clearvars -except exampleName;
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 1, node{1}, 0);

node{1}.setService(jobclass{1}, Erlang(3,2));
node{1}.setService(jobclass{2}, HyperExp(0.5,3.0,10.0));

node{2}.setService(jobclass{1}, HyperExp(0.1,1.0,10.0));
node{2}.setService(jobclass{2}, MMPP2(1,2,3,4));

node{3}.setService(jobclass{1}, HyperExp(0.1,1.0,10.0));
node{3}.setService(jobclass{2}, Erlang(1,2));

model.addLink(node{1}, node{1});
model.addLink(node{1}, node{2});
```

```

model.addLink(node{1}, node{3});
model.addLink(node{2}, node{1});
model.addLink(node{3}, node{1});

node{1}.setProbRouting(jobclass{1}, node{1}, 0.0)

... (28 source lines omitted; full implementation: matlab/examples/basic/)

for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.5 cqn_multichain_cs

Five classes in three chains, the closed counterpart of oqn_multichain_cs.

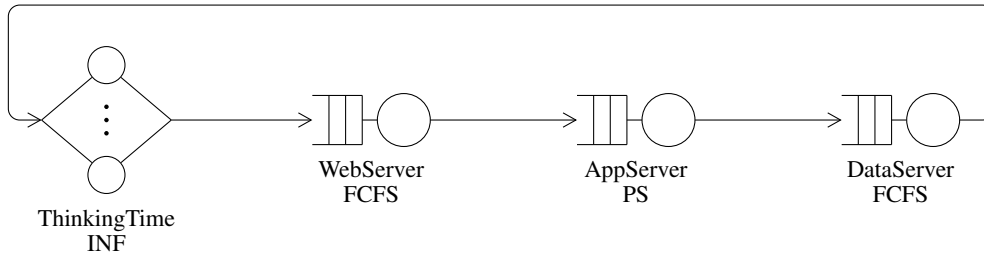


Figure 2.18: Topology of cqn_multichain_cs: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	5: HighPriorityA (closed, $N = 2$), HighPriorityB (closed, $N = 1$), RegularA (closed, $N = 3$), RegularB (closed, $N = 2$), Background (closed, $N = 2$)
Scheduling Service	ThinkingTime: INF; WebServer: FCFS; AppServer: PS; DataServer: FCFS ThinkingTime – HighPriorityA: Exp(0.6667); HighPriorityB: Exp(0.625); RegularA: Exp(0.5); RegularB: Exp(0.4762); Background: Exp(0.3333) WebServer – HighPriorityA: Exp(2.5); HighPriorityB: Exp(2.381); RegularA: Exp(2); RegularB: Exp(1.923); Background: Exp(1.25) AppServer – HighPriorityA: Exp(1.429); HighPriorityB: Exp(1.333); RegularA: Exp(1.111); RegularB: Exp(1.053); Background: Exp(0.8333) DataServer – HighPriorityA: Exp(2); HighPriorityB: Exp(1.923); RegularA: Exp(1.667); RegularB: Exp(1.613); Background: Exp(1)
Solvers	MVA, SSA

Table 2.18: Features of cqn_multichain_cs.

The example is built and solved as follows.

```

%% Example of a closed queueing network with 3 chains, 5 classes, and class switching
% Product-form (BCMP) network with class switching.
% Chain 1: High-priority users (2 classes that can switch)
% Chain 2: Regular users (2 classes that can switch)
% Chain 3: Background tasks (1 class)
%
% The network maintains product-form property:
% - All service times are exponential
% - Mix of FCFS and PS scheduling
% - All nodes are conventional queueing stations or infinite servers

model = Network('cqn_multichain');

```

```

%% Block 1: nodes
node{1} = Delay(model, 'ThinkingTime');
node{2} = Queue(model, 'WebServer', SchedStrategy.FCFS);
node{3} = Queue(model, 'AppServer', SchedStrategy.PS);
node{4} = Queue(model, 'DataServer', SchedStrategy.FCFS);

%% Block 2: job classes
% Chain 1: High-priority users (can switch between HighPriorityA and HighPriorityB)
jobclass{1} = ClosedClass(model, 'HighPriorityA', 2, node{1});
jobclass{2} = ClosedClass(model, 'HighPriorityB', 1, node{1});
% Chain 2: Regular users (can switch between RegularA and RegularB)
jobclass{3} = ClosedClass(model, 'RegularA', 3, node{1});
jobclass{4} = ClosedClass(model, 'RegularB', 2, node{1});

... (97 source lines omitted; full implementation: matlab/examples/basic/)

for s = 1:length(solver)
    fprintf('SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgChainTable();
    AvgTable{s}
    fprintf('\n');
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.6 cqn_multiserver

A closed network with multiserver FCFS and PS stations, Erlang service and a class disabled at one station.

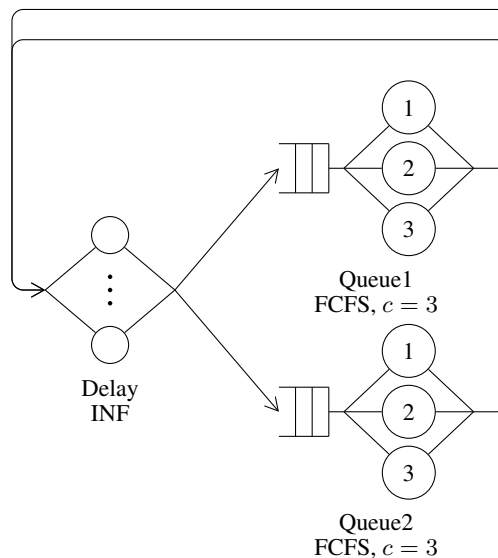


Figure 2.19: Topology of `cqn_multiserver`: a closed network over 3 queueing stations.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
%node{1} = Queue(model, 'Queue0', SchedStrategy.PS);
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{2}.setNumServers(3);
node{3}.setNumServers(3);

```

Nodes	3: Queue $\times$ 2, Delay
Classes	4: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$), Class3 (closed, $N = 2$), Class4 (closed, $N = 1$)
Scheduling Service	Delay: INF; Queue1: FCFS ($c = 3$); Queue2: FCFS ($c = 3$) Delay – Class1: Exp(1); Class2: Exp(1); Class3: Exp(10); Class4: Exp(1) Queue1 – Class1: Exp(1); Class2: Erlang(1,2); Class3: Exp(10); Class4: Exp(1) Queue2 – Class1: Disabled; Class2: Disabled; Class3: Erlang(1,2); Class4: Exp(1)
Solvers	MVA

Table 2.19: Features of `cqn_multiserver`.

```
% Default: scheduling is set as FCFS everywhere, routing as Random
jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);
jobclass{3} = ClosedClass(model, 'Class3', 2, node{1}, 0);
jobclass{4} = ClosedClass(model, 'Class4', 1, node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(1));
node{1}.setService(jobclass{3}, Exp(10));
node{1}.setService(jobclass{4}, Exp(1));

node{2}.setService(jobclass{1}, Exp(1));
node{2}.setService(jobclass{2}, Erlang(1,2));
node{2}.setService(jobclass{3}, Exp(10));
node{2}.setService(jobclass{4}, Exp(1));

node{3}.setService(jobclass{1}, Disabled());

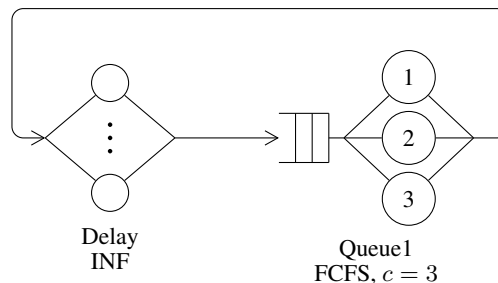
... (49 source lines omitted; full implementation: matlab/examples/basic/)

tic
solver = MVA(model);
avgTable = solver.getAvgTable();
fprintf('SOLVER: MVA\n');
disp(avgTable);
toc
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.7 `cqn_multiserver_nc`

Delay plus a multiserver FCFS queue, solved by the normalizing constant method.

Figure 2.20: Topology of `cqn_multiserver_nc`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
% Closed Delay + multiserver FCFS queue.
% Exercises the exact multiserver path of NC: with method 'exact'
```

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(1.25)
Solvers	CTMC, MVA, NC

Table 2.20: Features of `cqn_multiserver_nc`.

```
% the c-server station is converted to a load-dependent station with rate
% min(n,c) (runAnalyzer.m), solved via comomld/pfqnc_comomrm_ld. CTMC is the
% exact ground truth; MVA provides an additional cross-check.

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{2}.setNumServers(3);

jobclass{1} = ClosedClass(model, 'Class1', 5, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(0.8)); % mean = 0.8

P = model.initRoutingMatrix;
P{1} = [0,1.0;1.0,0];
model.link(P);

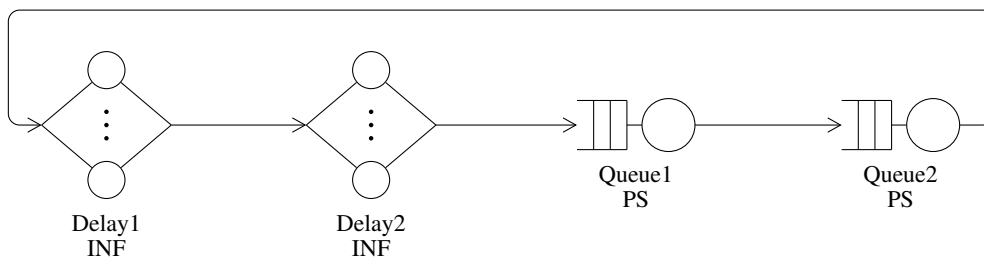
solver{1} = CTMC(model,'exact');
solver{end+1} = MVA(model);
solver{end+1} = NC(model,'exact');

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.8 `cqn_online`

The closed counterpart of the one-line construction, from the demand and think time matrices.

Figure 2.21: Topology of `cqn_online`: a closed network over 4 queueing stations.

The example is built and solved as follows.

```
fprintf(1,'This example shows a 1-line exact MVA solution of a cyclic queueing network.\n');
D = [10,5; 5,9]; % S(i,r) - mean service time of class r at station i
N = [1,2]; % N(r) - number of jobs of class r
Z = [91,92; 93,94]; % Z(r) mean service time of class r at delay station i
```


Nodes	4: Queue $\times$ 2, Delay $\times$ 2
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay1: INF; Delay2: INF; Queue1: PS; Queue2: PS
Service	Delay1 – Class1: Exp(0.01099); Class2: Exp(0.01087) Delay2 – Class1: Exp(0.01075); Class2: Exp(0.01064) Queue1 – Class1: Exp(0.1); Class2: Exp(0.2) Queue2 – Class1: Exp(0.2); Class2: Exp(0.1111)
Solvers	MVA

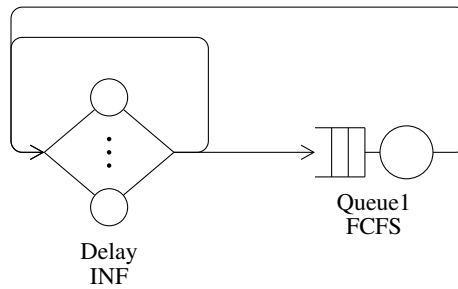
Table 2.21: Features of `cqn_online`.

```
model = Network.cyclicPsInf(N,D,Z);
avgTable = MVA(model, 'exact').getAvgTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.9 `cqn_repairmen`

The machine repairman model: a Delay station of thinking machines and one FCFS repair queue.

Figure 2.22: Topology of `cqn_repairmen`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Delay: INF; Queue1: FCFS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.6667)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES

Table 2.22: Features of `cqn_repairmen`.

The example is built and solved as follows.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(model, 'Class1', 10, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5

P = model.initRoutingMatrix;
P{1} = [0.7, 0.3; 1.0, 0];

% This may be alternatively specified as:
%P{1}(node{1}, [node{1}, node{2}]) = [0.7, 0.3];
%P{1}(node{2}, [node{1}, node{2}]) = [1.0, 0];

model.link(P);
```

```

solver{1} = CTMC(model, 'exact', 'keep', true);
solver{end+1} = JMT(model, 'seed', 23000, 'verbose', true, 'keep', true);
solver{end+1} = SSA(model, 'seed', 23000, 'verbose', true, 'samples', 5e3);
solver{end+1} = FLD(model);
solver{end+1} = MVA(model);
solver{end+1} = NC(model, 'exact');
solver{end+1} = MAM(model);
solver{end+1} = LINE(model);
solver{end+1} = LDES(model, 'seed', 23000, 'verbose', true);

AvgTable = cell(1, length(solver));
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.10 cqn_repairmen_multi

The repairman model with several repair stations.

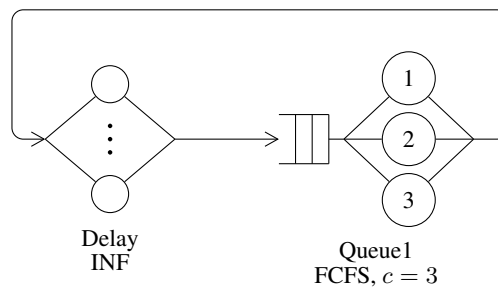


Figure 2.23: Topology of cqn_repairmen_multi: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 4$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(1); Class2: Exp(10)
Solvers	CTMC, LQNS, MVA, NC

Table 2.23: Features of cqn_repairmen_multi.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{2}.setNumServers(3);

% Default: scheduling is set as FCFS everywhere, routing as Random
jobclass{1} = ClosedClass(model, 'Class1', 4, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(1));

node{2}.setService(jobclass{1}, Exp(1));
node{2}.setService(jobclass{2}, Exp(10));

```

```

myP = model.initRoutingMatrix;
myP{1} = Network.serialRouting(node);
myP{2} = Network.serialRouting(node);
model.link(myP);
%
options = LQNS.defaultOptions;
%options.verbose = false;

solver{1} = CTMC(model, 'exact');
solver{end+1} = LQNS(model, 'qns.conway');

... (10 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = cell(1, length(solver));
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.11 cqn_scheduling_dps

Discriminatory processor sharing, with the class weights that PS ignores.

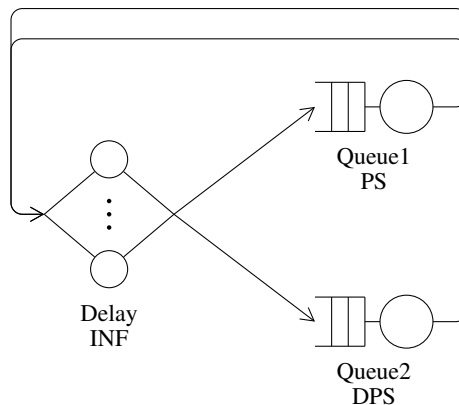


Figure 2.24: Topology of `cqn_scheduling_dps`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: DPS
Service	Delay – Class1: Exp(3); Class2: Exp(0.5) Queue1 – Class1: Exp(0.1); Class2: Exp(1) Queue2 – Class1: Exp(0.1); Class2: Exp(1)
Solvers	CTMC, JMT, FLD, MVA, LDES

Table 2.24: Features of `cqn_scheduling_dps`.

The example is built and solved as follows.

```

model = Network('model');

node1 = Delay(model, 'Delay');
node2 = Queue(model, 'Queue1', SchedStrategy.PS);
node3 = Queue(model, 'Queue2', SchedStrategy.DPS);

```

```

jobclass1 = ClosedClass(model, 'Class1', 2, node1, 0);
jobclass2 = ClosedClass(model, 'Class2', 1, node1, 0);

node1.setService(jobclass1, Exp(3));
node1.setService(jobclass2, Exp(0.5));

% this weight assignment is ignored since the node is PS
w1 = 5; node2.setService(jobclass1, Exp(0.1), w1);
w2 = 1; node2.setService(jobclass2, Exp(1), w2);

% this is not ignored since the node is DPS
w1 = 1; node3.setService(jobclass1, Exp(0.1), w1);
w2 = 5; node3.setService(jobclass2, Exp(1), w2);

P = model.initRoutingMatrix();

P.set(jobclass1, jobclass1, node1, node2, 0.300000); % (Delay,Class1) -> (Queue1,Class1)
P.set(jobclass1, jobclass1, node1, node3, 0.700000); % (Delay,Class1) -> (Queue2,Class1)
P.set(jobclass1, jobclass1, node2, node1, 1.000000); % (Queue1,Class1) -> (Delay,Class1)
P.set(jobclass1, jobclass1, node3, node1, 1.000000); % (Queue2,Class1) -> (Delay,Class1)

... (25 source lines omitted; full implementation: matlab/examples/basic/)

solver{end+1} = LDES(model,options);
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.12 cqn_threeclass_hyper1

The three-class version, with multiserver stations.

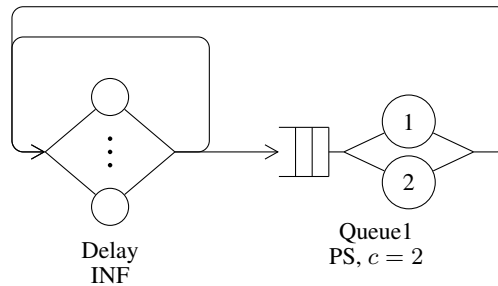


Figure 2.25: Topology of cqn_threeclass_hyper1: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	3: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$), Class3 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS ($c = 2$)
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10); Class3: Exp(1) Queue1 – Class1: HyperExp(0.1,0.9;1,10); Class2: Exp(2); Class3: Exp(3)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES

Table 2.25: Features of cqn_threeclass_hyper1.

The example is built and solved as follows.

```
model = Network('model');
```

```

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{2}.setNumberOfServers(2);

jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 0, node{1}, 0);
jobclass{3} = ClosedClass(model, 'Class3', 1, node{1}, 0);

node{1}.setService(jobclass{1}, Erlang(3,2));
node{1}.setService(jobclass{2}, HyperExp(0.5,3.0,10.0));
node{1}.setService(jobclass{3}, Exp(1));

node{2}.setService(jobclass{1}, HyperExp(0.1,1.0,10.0));
node{2}.setService(jobclass{2}, Exp(2));
node{2}.setService(jobclass{3}, Exp(3));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1,1} = [0.3,0.1; 0.2,0];
P{1,2} = [0.6,0; 0.8,0];
P{1,3} = zeros(M);

... (24 source lines omitted; full implementation: matlab/examples/basic/)

AvgChainTable{s} = solver{s}.getAvgChainTable();
AvgSysChainTable{s} = solver{s}.getAvgSysTable();
AvgTable{s}
AvgChainTable{s}
AvgSysChainTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.13 cqn_twoclass_erl

Two classes with Erlang service over PS and INF stations, exercising the RAND, round-robin and weighted round-robin routing policies.

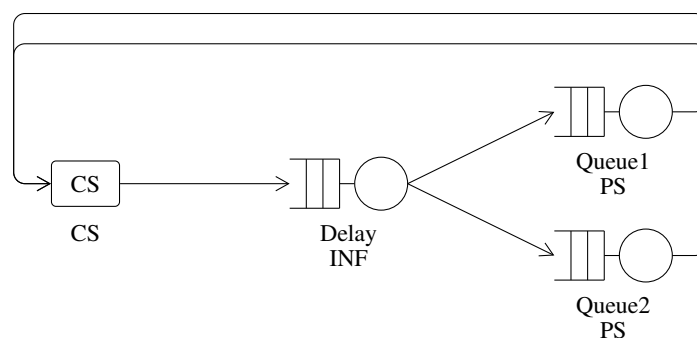


Figure 2.26: Topology of cqn_twoclass_erl: a closed network over 3 queueing stations with a class-switch node.

The example is built and solved as follows.

```

model = Network('model');

node{1} = ClassSwitch(model, 'CS', [0,1;1,0]);
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);
node{4} = Queue(model, 'Delay', SchedStrategy.INF);

```

Nodes	4: Queue $\times$ 3, ClassSwitch
Classes	2: Class1 (closed, $N = 15$), Class2 (closed, $N = 5$)
Scheduling	Queue1: PS; Queue2: PS; Delay: INF
Service	Queue1 – Class1: Exp(0.6667); Class2: Erlang(1.333,2) Queue2 – Class1: Erlang(1.333,2); Class2: Exp(0.6667) Delay – Class1: Exp(1); Class2: Exp(1)
Features	class switching on 3 links
Solvers	JMT, LDES

Table 2.26: Features of `cqn_twoclass_erl`.

```

jobclass{1} = ClosedClass(model, 'Class1', 15, node{4}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 5, node{4}, 0);

node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setService(jobclass{2}, Erlang.fitMeanAndOrder(1.5,2)); % mean = 1.5

node{3}.setService(jobclass{1}, Erlang.fitMeanAndOrder(1.5,2)); % mean = 1.5
node{3}.setService(jobclass{2}, Exp.fitMean(1.5)); % mean = 1.5

node{4}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{4}.setService(jobclass{2}, Exp.fitMean(1.0)); % mean = 1

model.addLink(node{2}, node{1});
model.addLink(node{3}, node{1});
model.addLink(node{1}, node{4});
model.addLink(node{4}, node{2});
model.addLink(node{4}, node{3});

node{1}.setRouting(jobclass{1}, RoutingStrategy.RAND);

... (15 source lines omitted; full implementation: matlab/examples/basic/)

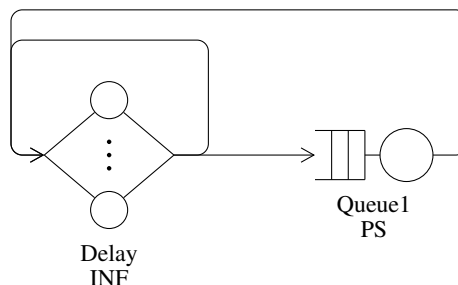
solver{end+1} = LDES(model, simoptions);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.14 `cqn_twoclass_hyperl`

Two classes with hyperexponential and Erlang service on PS stations.

Figure 2.27: Topology of `cqn_twoclass_hyperl`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```

model = Network('model');

```

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) Queue1 – Class1: HyperExp(0.1,0.9;1,10); Class2: Exp(1)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES

Table 2.27: Features of `cqn_twoclass_hyper1`.

```

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Erlang(3,2));
node{1}.setService(jobclass{2}, HyperExp(0.5,3.0,10.0));

node{2}.setService(jobclass{1}, HyperExp(0.1,1.0,10.0));
node{2}.setService(jobclass{2}, Exp(1));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1,1} = [0.3,0.1; 0.2,0];
P{1,2} = [0.6,0.0; 0.8,0];
P{2,1} = [0,0; 1,0];
P{2,2} = [0,1; 0,0];

model.link(P);
%%
% This part illustrates the execution of different solvers

... (9 source lines omitted; full implementation: matlab/examples/basic/)

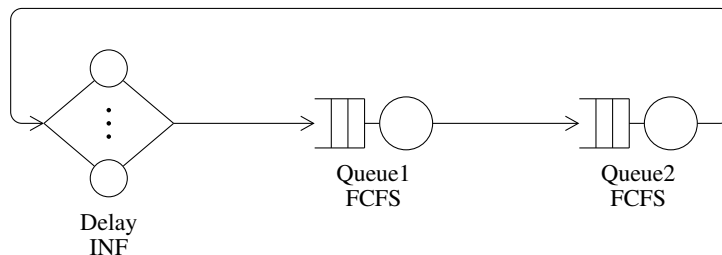
solver{end+1} = LDES(model,'seed',23000,'verbose',true,'samples',5e5); % same time of JMT,
100x samples
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.2.15 `cqn_twoqueues_multi`

Two FCFS queues and a Delay with several closed classes.

Figure 2.28: Topology of `cqn_twoqueues_multi`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 10$), Class2 (closed, $N = 10$)
Scheduling	Delay: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.6667) Queue2 – Class1: Exp(0.3333); Class2: Exp(0.3333)
Solvers	MVA

Table 2.28: Features of `cqn_twoqueues_multi`.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Queue(model, 'Queue2', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(model, 'Class1', 10, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 10, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{3}.setService(jobclass{1}, Exp.fitMean(3.0)); % mean = 2.0

node{1}.setService(jobclass{2}, Exp.fitMean(1.0)); % mean = 1
node{2}.setService(jobclass{2}, Exp.fitMean(1.5)); % mean = 1.5
node{3}.setService(jobclass{2}, Exp.fitMean(3.0)); % mean = 2.0

P = model.initRoutingMatrix;
P{1} = Network.serialRouting(node);
P{2} = Network.serialRouting(node);
%P{jobclass{1}, jobclass{1}} = [0.7, 0.3, 0; 1.0, 0, 0; 1.0, 0, 0];
%P{jobclass{2}, jobclass{2}} = [0.7, 0, 0.3; 1.0, 0, 0; 1.0, 0, 0];

% This may be alternatively specified as:
%P{1}(node{1}, [node{1}, node{2}]) = [0.7, 0.3];
%P{1}(node{2}, [node{1}, node{2}]) = [1.0, 0];

... (14 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = cell(1, length(solver));
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.3 Mixed networks

A mixed network carries open and closed classes at the same stations. The open classes contribute a fixed arrival rate, the closed ones a fixed population, and the two compete for the same servers, so neither can be solved in isolation.

The scripts pair a single-server and a multiserver station under FCFS and under PS. They are deliberately awkward instances: the load-dependent rates they induce are sparse, and they are the models on which the approximate solvers disagree first.

2.3.1 mqn_basic

Open and closed classes at the same stations, the smallest mixed model.

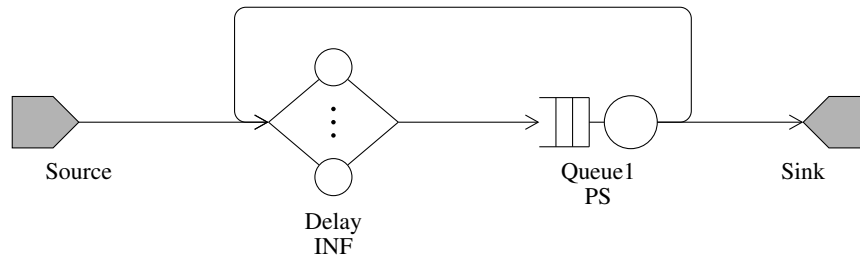


Figure 2.29: Topology of mqn_basic: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: ClosedClass (closed, $N = 2$), OpenClass (open)
Scheduling	Delay: INF; Queue1: PS
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.1)
Service	Delay – ClosedClass: Erlang(3,2); OpenClass: HyperExp(0.5,0.5;3,10) Queue1 – ClosedClass: HyperExp(0.1,0.9;1,10); OpenClass: Exp(1)
Features	class switching on 1 link
Solvers	CTMC, JMT, SSA, MVA, LDES

Table 2.29: Features of mqn_basic.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = ClosedClass(model, 'ClosedClass', 2, node{1}, 0);
jobclass{2} = OpenClass(model, 'OpenClass', 0);

node{1}.setService(jobclass{1}, Erlang(3,2));
node{1}.setService(jobclass{2}, HyperExp(0.5,3.0,10.0));

node{2}.setService(jobclass{1}, HyperExp(0.1,1.0,10.0));
node{2}.setService(jobclass{2}, Exp(1));

node{3}.setArrival(jobclass{2}, Exp(0.1));

M = model.getNumberOfNodes();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1,1} = zeros(M); P{1,1}(1:2,1:2) = circul(2);
P{1,2} = zeros(M);
P{2,2} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
P{2,1} = zeros(M);

... (31 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = MAM(model,options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.3.2 mqn_multichain_cs

A mixed network whose classes are grouped into several chains by class switching.

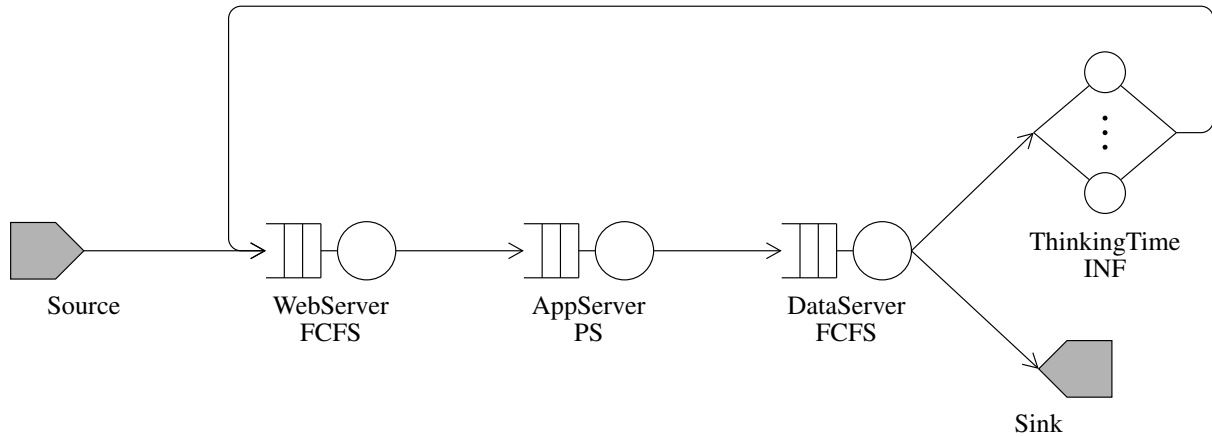


Figure 2.30: Topology of mqn_multichain_cs: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 3, Delay, Sink
Classes	5: InteractiveA (closed, $N = 3$), InteractiveB (closed, $N = 2$), BatchA (open), BatchB (open), ExternalLoad (open)
Scheduling	ThinkingTime: INF; WebServer: FCFS; AppServer: PS; DataServer: FCFS
Arrivals	Source – InteractiveA: Disabled; InteractiveB: Disabled; BatchA: Exp(0.1); BatchB: Exp(0.05); ExternalLoad: Exp(0.05)
Service	ThinkingTime – InteractiveA: Exp(0.6667); InteractiveB: Exp(0.5); BatchA: Disabled; BatchB: Disabled; ExternalLoad: Disabled WebServer – InteractiveA: Exp(2.5); InteractiveB: Exp(2); BatchA: Exp(3.333); BatchB: Exp(2.857); ExternalLoad: Exp(5) AppServer – InteractiveA: Exp(1.25); InteractiveB: Exp(1); BatchA: Exp(1.667); BatchB: Exp(1.429); ExternalLoad: Exp(2) DataServer – InteractiveA: Exp(2); InteractiveB: Exp(1.667); BatchA: Exp(0.8333); BatchB: Exp(0.6667); ExternalLoad: Exp(1.25)
Features	drop rule { 'BatchA': 'waitingQueue', 'BatchB': 'waitingQueue', 'ExternalLoad': 'waitingQueue' } at ThinkingTime; class switching on 1 link
Solvers	MVA, SSA

Table 2.30: Features of mqn_multichain_cs.

The example is built and solved as follows.

```
%% Example of a mixed queueing network with multiple chains and class switching
% Demonstrates class switching within both open and closed job classes.
% Chain 1: Closed class with 2 classes that can switch (interactive users)
% Chain 2: Open classes with 2 classes that can switch (batch requests)
% Chain 3: Open class (external load)

model = Network('mqn_multichain');

%% Block 1: nodes
node{1} = Delay(model, 'ThinkingTime');
node{2} = Queue(model, 'WebServer', SchedStrategy.FCFS);
node{3} = Queue(model, 'AppServer', SchedStrategy.PS);
node{4} = Queue(model, 'DataServer', SchedStrategy.FCFS);
node{5} = Source(model, 'Source');
node{6} = Sink(model, 'Sink');
```

```

%% Block 2: job classes
% Chain 1: Closed classes (interactive users that can switch types)
jobclass{1} = ClosedClass(model, 'InteractiveA', 3, node{1});
jobclass{2} = ClosedClass(model, 'InteractiveB', 2, node{1});
% Chain 2: Open classes (batch jobs that can switch types)
jobclass{3} = OpenClass(model, 'BatchA', 0);
jobclass{4} = OpenClass(model, 'BatchB', 0);
% Chain 3: Open class (external load)
jobclass{5} = OpenClass(model, 'ExternalLoad', 0);

... (99 source lines omitted; full implementation: matlab/examples/basic/)

for s = 1:length(solver)
    fprintf('SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgChainTable();
    AvgTable{s}
    fprintf('\n');
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.3.3 mqn_multiserver_fcfs

The multiserver FCFS version of the same model.

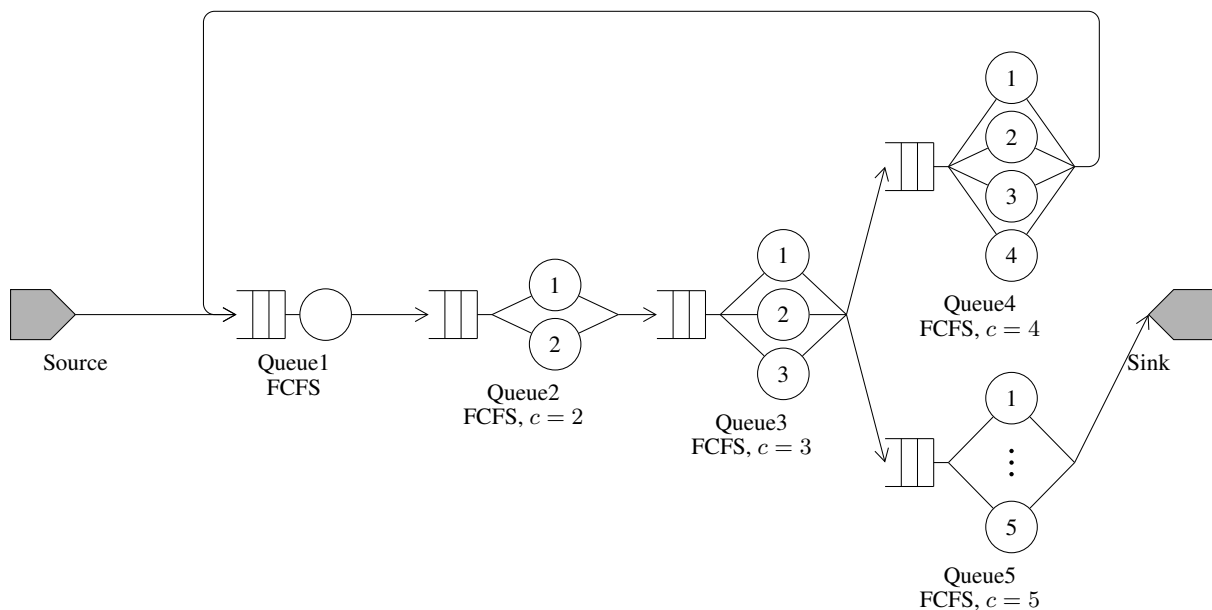


Figure 2.31: Topology of mqn_multiserver_fcfs: an open network over 5 queueing stations.

The example is built and solved as follows.

```

% this is a difficult sparse LDMX model
model = Network('model');

node{1} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{1}.setNumberOfServers(1);
node{2} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{2}.setNumberOfServers(2);
node{3} = Queue(model, 'Queue3', SchedStrategy.FCFS);
node{3}.setNumberOfServers(3);
node{4} = Queue(model, 'Queue4', SchedStrategy.FCFS); % only closed classes
node{4}.setNumberOfServers(4);

```

Nodes	7: Source, Queue $\times$ 5, Sink
Classes	2: ClosedClass (closed, $N = 3$), OpenClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS ($c = 2$); Queue3: FCFS ($c = 3$); Queue4: FCFS ($c = 4$); Queue5: FCFS ($c = 5$)
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.3)
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2) Queue5 – ClosedClass: Exp(5); OpenClass: Exp(2.236)
Features	class switching on 3 links
Solvers	CTMC, JMT, SSA, MVA, LDES

Table 2.31: Features of `mqn_multiserver_fcfs`.

```

node{5} = Queue(model, 'Queue5', SchedStrategy.FCFS); % only open classes
node{5}.setNumberOfServers(5);
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

jobclass{1} = ClosedClass(model, 'ClosedClass', 3, node{1}, 0);
jobclass{2} = OpenClass(model, 'OpenClass', 0);

for i=1:5
    node{i}.setService(jobclass{1}, Exp(i));
    node{i}.setService(jobclass{2}, Exp(sqrt(i)));
end

source.setArrival(jobclass{2}, Exp(0.3));

... (29 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = MAM(model,options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.3.4 `mqn_multiserver_ps`

The multiserver PS version.

Nodes	7: Source, Queue $\times$ 5, Sink
Classes	2: ClosedClass (closed, $N = 3$), OpenClass (open)
Scheduling	Queue1: PS; Queue2: PS ($c = 2$); Queue3: PS ($c = 3$); Queue4: PS ($c = 4$); Queue5: PS ($c = 5$)
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.3)
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2) Queue5 – ClosedClass: Exp(5); OpenClass: Exp(2.236)
Features	class switching on 3 links
Solvers	CTMC, JMT, MVA, LDES

Table 2.32: Features of `mqn_multiserver_ps`.

The example is built and solved as follows.

```

% this is a difficult sparse LDMX model
model = Network('model');

```

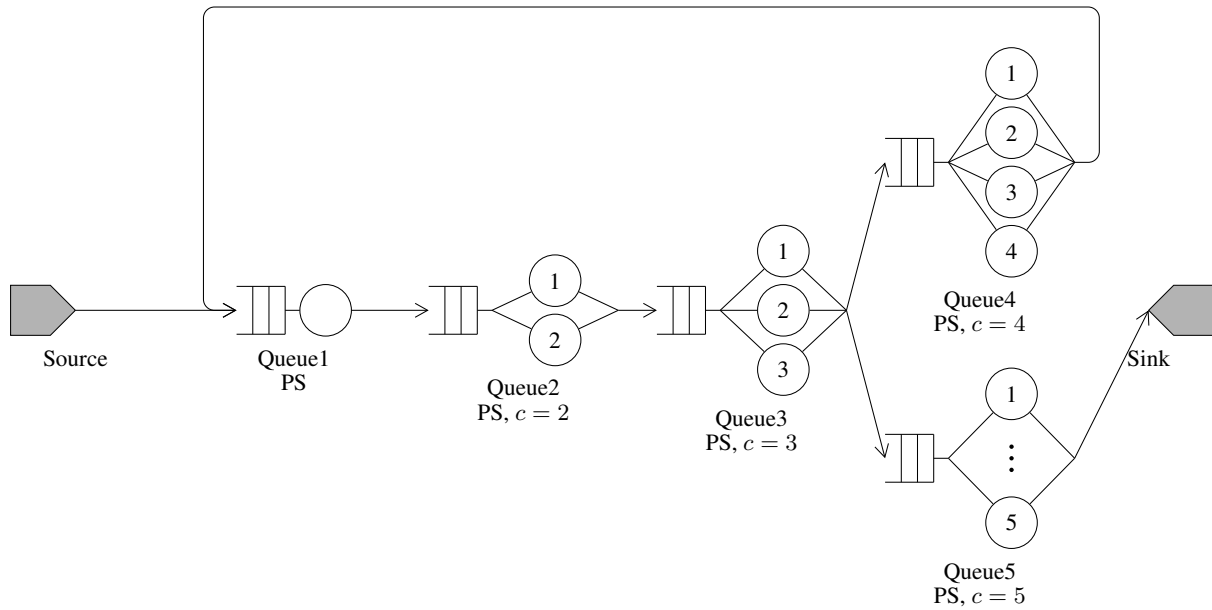


Figure 2.32: Topology of `mqn_multiserver_ps`: an open network over 5 queueing stations.

```

node{1} = Queue(model, 'Queue1', SchedStrategy.PS);
node{1}.setNumberOfServers(1);
node{2} = Queue(model, 'Queue2', SchedStrategy.PS);
node{2}.setNumberOfServers(2);
node{3} = Queue(model, 'Queue3', SchedStrategy.PS);
node{3}.setNumberOfServers(3);
node{4} = Queue(model, 'Queue4', SchedStrategy.PS); % only closed classes
node{4}.setNumberOfServers(4);
node{5} = Queue(model, 'Queue5', SchedStrategy.PS); % only open classes
node{5}.setNumberOfServers(5);
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

jobclass{1} = ClosedClass(model, 'ClosedClass', 3, node{1}, 0);
jobclass{2} = OpenClass(model, 'OpenClass', 0);

for i=1:5
    node{i}.setService(jobclass{1}, Exp(i));
    node{i}.setService(jobclass{2}, Exp(sqrt(i)));
end

source.setArrival(jobclass{2}, Exp(0.3));

... (32 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = MAM(model, mam_options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.3.5 `mqn_singleserver_fcfs`

A single-server FCFS station under a mixed workload, a sparse load-dependent multiclass instance that stresses the solvers.

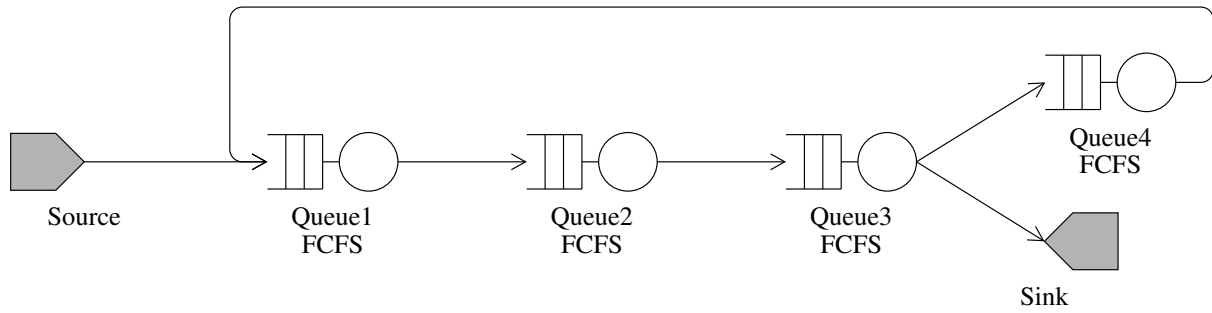


Figure 2.33: Topology of mqn_singleserver_fcfs: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	2: ClosedClass (closed, $N = 100$), OpenClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS
Arrivals	Source – ClosedClass: Disabled; OpenClass: APH
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2)
Features	class switching on 2 links
Solvers	JMT, MVA, NC, LDES

Table 2.33: Features of mqn_singleserver_fcfs.

The example is built and solved as follows.

```
% this is a difficult sparse LDMX model
model = Network('model');

M = 4;
node{1} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{2} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{3} = Queue(model, 'Queue3', SchedStrategy.FCFS);
node{4} = Queue(model, 'Queue4', SchedStrategy.FCFS); % only closed classes
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

jobclass{1} = ClosedClass(model, 'ClosedClass', 100, node{1}, 0);
jobclass{2} = OpenClass(model, 'OpenClass', 0);

for i=1:M
    node{i}.setService(jobclass{1}, Exp(i));
    node{i}.setService(jobclass{2}, Exp(sqrt(i)));
end

source.setArrival(jobclass{2}, APH.fitMeanAndSCV(3,64));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1,1} = zeros(M+1);

... (24 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = MAM(model,options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to

obtain its results.

2.3.6 mqn_singleserver_ps

The processor sharing counterpart.

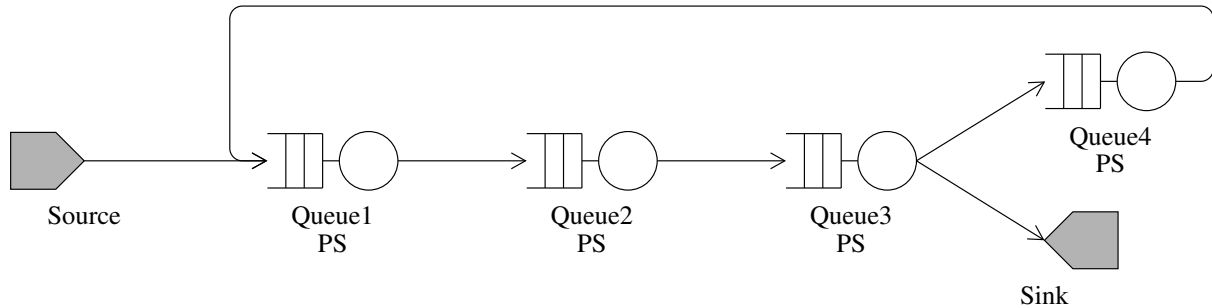


Figure 2.34: Topology of mqn_singleserver_ps: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	2: ClosedClass (closed, $N = 100$), OpenClass (open)
Scheduling	Queue1: PS; Queue2: PS; Queue3: PS; Queue4: PS
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.3)
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2)
Features	class switching on 2 links
Solvers	JMT, SSA, FLD, MVA, NC, LDES

Table 2.34: Features of mqn_singleserver_ps.

The example is built and solved as follows.

```
% this is a difficult sparse LDMX model
model = Network('model');

M = 4;
node{1} = Queue(model, 'Queue1', SchedStrategy.PS);
node{2} = Queue(model, 'Queue2', SchedStrategy.PS);
node{3} = Queue(model, 'Queue3', SchedStrategy.PS);
node{4} = Queue(model, 'Queue4', SchedStrategy.PS); % only closed classes
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

jobclass{1} = ClosedClass(model, 'ClosedClass', 100, node{1}, 0);
jobclass{2} = OpenClass(model, 'OpenClass', 0);

for i=1:M
    node{i}.setService(jobclass{1}, Exp(i));
    node{i}.setService(jobclass{2}, Exp(sqrt(i)));
end

source.setArrival(jobclass{2}, Exp(0.3));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1,1} = zeros(M+1);

... (25 source lines omitted; full implementation: matlab/examples/basic/)
```

```
%solver(end+1) = MAM(model,options);
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.4 Class switching

A job may change class as it moves. Switching is expressed either by off-diagonal blocks of the routing matrix, which give the probability of leaving node i as class r and arriving at node j as class s , or by a dedicated `ClassSwitch` node that performs the change in place.

Switching is what makes classes and chains differ: the classes that can reach one another form a chain, and it is the chain, not the class, that carries a conserved population. The diamond examples build a switching chain that is reducible on purpose, so that transient classes appear.

2.4.1 `cs_implicit`

A `ClassSwitch` node declared explicitly, against the implicit switching that a routing matrix with off-diagonal class blocks already expresses.

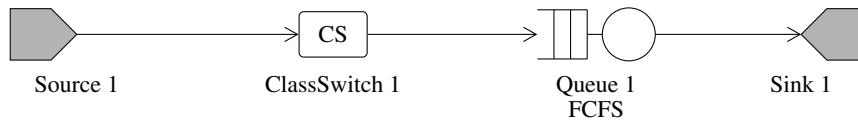


Figure 2.35: Topology of `cs_implicit`: an open network over 1 queueing station with a class-switch node.

Nodes	4: Source, Queue, ClassSwitch, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue 1: FCFS
Arrivals	Source 1 – Class1: Exp(0.1); Class2: Exp(0.5)
Service	Queue 1 – Class1: Exp(1); Class2: Exp(1)
Solvers	MVA

Table 2.35: Features of `cs_implicit`.

The example is built and solved as follows.

```
%% a basic M/M/1 with explicit definition of a ClassSwitch node
% Recommended ClassSwitch declaration style
model = Network('mm1cs');

%% Block 1: nodes
node{1} = Source(model, 'Source 1');
node{2} = Queue(model, 'Queue 1', SchedStrategy.FCFS);
node{3} = Sink(model, 'Sink 1');
node{4} = ClassSwitch(model, 'ClassSwitch 1');

%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);

node{1}.setArrival(jobclass{1}, Exp.fitMean(10.000000)); % (Source 1,Class1)
```



```

node{1}.setArrival(jobclass{2}, Exp.fitMean(2.000000)); % (Source 1,Class2)
node{2}.setService(jobclass{1}, Exp.fitMean(1.000000)); % (Queue 1,Class1)
node{2}.setService(jobclass{2}, Exp.fitMean(1.000000)); % (Queue 1,Class2)

%% Block 3: topology
% The class switching matrix can now be declared after the classes, so the
% ClassSwitch node can be declared outside Block 1.
csmatrix = node{4}.initClassSwitchMatrix; % element (i,j) = probability that class i switches
to j
csmatrix(jobclass{1},jobclass{1}) = 0.3;
csmatrix(jobclass{1},jobclass{2}) = 0.7;
csmatrix(jobclass{2},jobclass{1}) = 1.0;

... (12 source lines omitted; full implementation: matlab/examples/basic/)

model.printRoutingMatrix();

solver{1} = MVA(model);
AvgTable{1} = solver{1}.getAvgChainTable;
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.4.2 cs_multi_diamond

The same pattern with several branches.

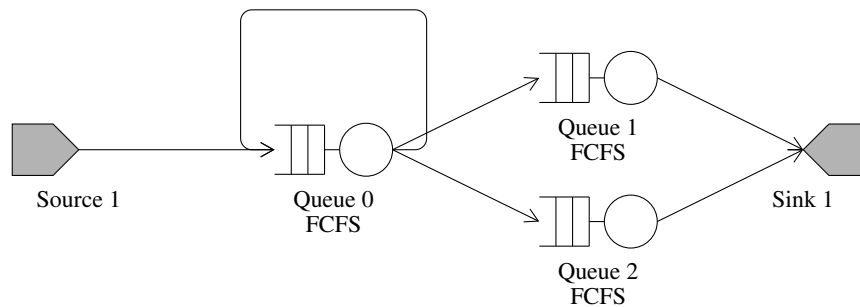


Figure 2.36: Topology of cs_multi_diamond: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	Queue 0: FCFS; Queue 1: FCFS; Queue 2: FCFS
Arrivals	Source 1 – Class1: Exp(1); Class2: Disabled; Class3: Disabled
Service	Queue 0 – Class1: Exp(10); Class2: Disabled; Class3: Disabled Queue 1 – Class1: Disabled; Class2: Exp(20); Class3: Disabled Queue 2 – Class1: Disabled; Class2: Disabled; Class3: Exp(30)
Features	drop rule {'Class2': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 0; drop rule {'Class1': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 1; drop rule {'Class1': 'waitingQueue', 'Class2': 'waitingQueue'} at Queue 2; class switching on 1 link
Solvers	MVA

Table 2.36: Features of cs_multi_diamond.

The example is built and solved as follows.

```

%% Example of class switching controlled by a reducible Markov chain
model = Network('mm1cs');

```

```

%% Block 1: nodes
node{1} = Source(model, 'Source 1');
node{2} = Queue(model, 'Queue 0', SchedStrategy.FCFS);
node{3} = Queue(model, 'Queue 1', SchedStrategy.FCFS);
node{4} = Queue(model, 'Queue 2', SchedStrategy.FCFS);
node{5} = Sink(model, 'Sink 1');

%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);
jobclass{3} = OpenClass(model, 'Class3', 0);

node{1}.setArrival(jobclass{1}, Exp(1.000000)); % (Source 1,Class1)
node{2}.setService(jobclass{1}, Exp(10.000000)); % (Queue 1,Class2)
node{3}.setService(jobclass{2}, Exp(20.000000)); % (Queue 1,Class2)
node{4}.setService(jobclass{3}, Exp(30.000000)); % (Queue 2,Class3)

P = model.initRoutingMatrix(); % initialize routing matrix
P{1,1}(1,2) = 1;
P{1,1}(2,2) = 0.2;
P{1,2}(2,3) = 0.3;
P{1,3}(2,4) = 0.5;
P{2,2}(3,5) = 1;
P{3,3}(4,5) = 1;
model.link(P);

model.printRoutingMatrix();

solver{1} = MVA(model);
AvgTable{1} = solver{1}.getAvgChainTable;
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.4.3 cs_single_diamond

Class switching driven by a reducible Markov chain over the classes, the diamond pattern with one branch.

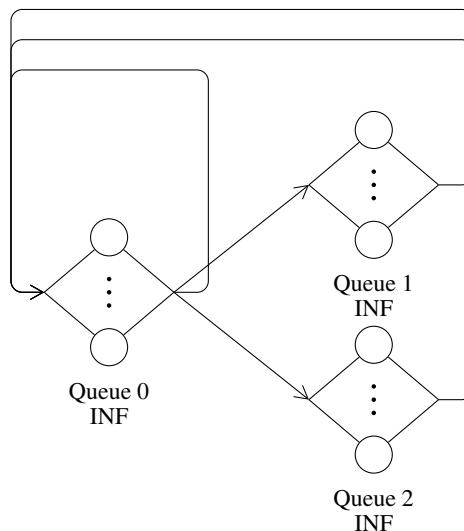


Figure 2.37: Topology of cs_single_diamond: a closed network over 3 queueing stations.

The example is built and solved as follows.

```

%% Example of class switching controlled by a reducible Markov chain

```

Nodes	3: Delay $\times$ 3
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 0$)
Scheduling	Queue 0: INF; Queue 1: INF; Queue 2: INF
Service	Queue 0 – Class1: Exp(1); Class2: Disabled; Class3: Disabled Queue 1 – Class1: Disabled; Class2: Exp(0.5); Class3: Disabled Queue 2 – Class1: Disabled; Class2: Disabled; Class3: Exp(0.3333)
Features	drop rule {'Class2': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 0; drop rule {'Class1': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 1; drop rule {'Class1': 'waitingQueue', 'Class2': 'waitingQueue'} at Queue 2
Solvers	MVA

Table 2.37: Features of `cs_single_diamond`.

```

model = Network('mm1cs');

%% Block 1: nodes
node{1} = Delay(model, 'Queue 0');
node{2} = Delay(model, 'Queue 1');
node{3} = Delay(model, 'Queue 2');

%% Block 2: classes
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1});
jobclass{2} = ClosedClass(model, 'Class2', 0, node{1});
jobclass{3} = ClosedClass(model, 'Class3', 0, node{1});

node{1}.setService(jobclass{1}, Exp.fitMean(1.000000)); % (Queue 1, Class2)
node{2}.setService(jobclass{2}, Exp.fitMean(2.000000)); % (Queue 1, Class2)
node{3}.setService(jobclass{3}, Exp.fitMean(3.000000)); % (Queue 2, Class3)

P = model.initRoutingMatrix(); % initialize routing matrix
P{1,1}(1,1) = 0.2;
P{1,2}(1,2) = 0.3;
P{1,3}(1,3) = 0.5;
P{2,1}(2,1) = 1;
P{3,1}(3,1) = 1;
model.link(P);

model.printRoutingMatrix();

solver{1} = MVA(model);
AvgTable{1} = solver{1}.getAvgChainTable;
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.4.4 `cs_transient_class`

A class that is transient in the switching chain, so it is visited but never returned to.

Nodes	3: Delay $\times$ 3
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 0$)
Scheduling	Queue 0: INF; Queue 1: INF; Queue 2: INF
Service	Queue 0 – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1) Queue 1 – Class1: Disabled; Class2: Exp(1); Class3: Disabled Queue 2 – Class1: Disabled; Class2: Disabled; Class3: Exp(1)
Features	drop rule {'Class1': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 1; drop rule {'Class1': 'waitingQueue', 'Class2': 'waitingQueue'} at Queue 2
Solvers	MVA

Table 2.38: Features of `cs_transient_class`.

The example is built and solved as follows.

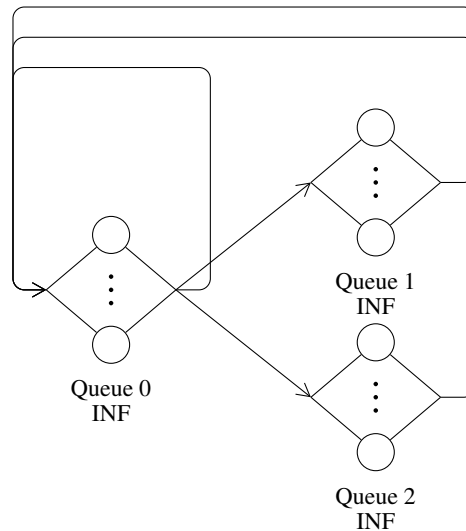


Figure 2.38: Topology of `cs_transient_class`: a closed network over 3 queueing stations.

```
%% Example of class switching controlled by a reducible Markov chain
% In this variant the job remains either in class 2 or class 3 forever
%
% This example demonstrates a transient class model where Class1 can
% switch to Class2 or Class3, but those classes never return to Class1.
% This creates a reducible routing chain with transient states (Class1)
% and recurrent states (Class2/Class3 cycles).

model = Network('mm1cs');

%% Block 1: nodes
node{1} = Delay(model, 'Queue 0');
node{2} = Delay(model, 'Queue 1');
node{3} = Delay(model, 'Queue 2');

%% Block 2: classes
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1});
jobclass{2} = ClosedClass(model, 'Class2', 0, node{1});
jobclass{3} = ClosedClass(model, 'Class3', 0, node{1});

node{1}.setService(jobclass{1}, Exp.fitMean(1.000000)); % (Queue 1,Class1)
node{1}.setService(jobclass{2}, Exp.fitMean(1.000000)); % (Queue 1,Class2)
node{1}.setService(jobclass{3}, Exp.fitMean(1.000000)); % (Queue 1,Class3)
node{2}.setService(jobclass{2}, Exp.fitMean(1.000000)); % (Queue 1,Class2)
node{3}.setService(jobclass{3}, Exp.fitMean(1.000000)); % (Queue 2,Class3)

... (9 source lines omitted; full implementation: matlab/examples/basic/)

model.printRoutingMatrix();

solver{1} = MVA(model);
AvgTable{1} = solver{1}.getAvgChainTable;
AvgTable{1}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.5 Priority models

Priorities are declared on the class and enforced by the discipline of the station. Head-of-line (HOL) service is non-preemptive: a low-priority job in service completes, and only the waiting order changes. The preemptive variants interrupt it instead, and the priority processor sharing family divides the capacity in proportion to the priority weights.

Priority breaks product form, so these models are the province of the exact CTMC solver, of simulation, and of the approximations that the MVA solver offers for HOL.

2.5.1 prio_hol_closed

The closed counterpart.

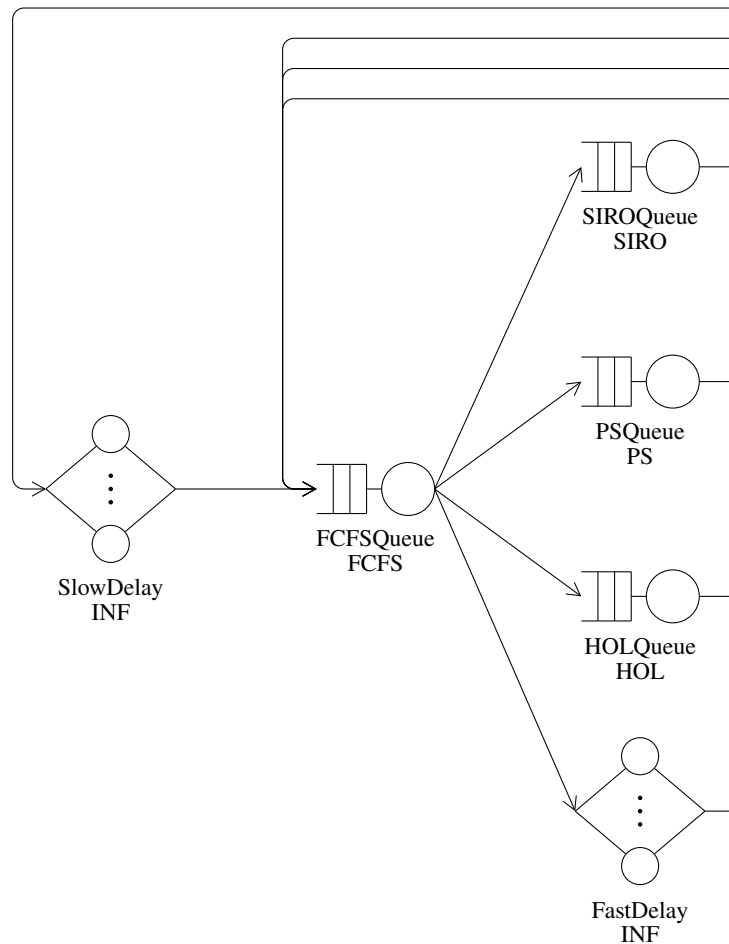


Figure 2.39: Topology of prio_hol_closed: a closed network over 6 queueing stations.

The example is built and solved as follows.

```
%% multiclass example with PS, SIRO, FCFS, HOL priority
model = Network('MyNetwork');

%% Block 1: nodes
node{1} = Delay(model, 'SlowDelay');
node{2} = Queue(model, 'FCFSQueue', SchedStrategy.FCFS);
node{3} = Queue(model, 'SIROQueue', SchedStrategy.SIRO);
node{4} = Queue(model, 'PSQueue', SchedStrategy.PS);
node{5} = Queue(model, 'HOLQueue', SchedStrategy.HOL);
node{6} = Delay(model, 'FastDelay');
```

Nodes	6: Queue $\times$ 4, Delay $\times$ 2
Classes	3: Class1 (closed, $N = 18$), Class2 (closed, $N = 18$), Class3 (closed, $N = 18$)
Scheduling	SlowDelay: INF; FCFSQueue: FCFS; SIROQueue: SIRO; PSQueue: PS; HOLQueue: HOL; FastDelay: INF
Service	SlowDelay – Class1: Exp(0.1); Class2: Exp(0.1); Class3: Exp(0.1) FCFSQueue – Class1: Exp(3.333); Class2: Exp(2); Class3: Exp(1.667) SIROQueue – Class1: Exp(0.9091); Class2: Exp(0.7692); Class3: Exp(0.6667) PSQueue – Class1: Exp(1); Class2: Exp(0.9091); Class3: Exp(0.5263) HOLQueue – Class1: Exp(0.4); Class2: Exp(0.5263); Class3: Exp(0.2326) FastDelay – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1)
Solvers	MVA, JMT, SSA, LDES

Table 2.39: Features of `prio_hol_closed`.

```

%% Block 2: classes
jobclass{1} = ClosedClass(model, 'Class1', 18, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 18, node{1}, 1);
jobclass{3} = ClosedClass(model, 'Class3', 18, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(10.000000));
node{1}.setService(jobclass{2}, Exp.fitMean(10.000000));
node{1}.setService(jobclass{3}, Exp.fitMean(10.000000));

node{2}.setService(jobclass{1}, Exp.fitMean(0.300000));
node{2}.setService(jobclass{2}, Exp.fitMean(0.500000));
node{2}.setService(jobclass{3}, Exp.fitMean(0.600000));

node{3}.setService(jobclass{1}, Exp.fitMean(1.100000));
node{3}.setService(jobclass{2}, Exp.fitMean(1.300000));

... (57 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = NC(model,options);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.5.2 prio_hol_open

Head-of-line non-preemptive priority in an open network, with PS, SIRO and FCFS stations alongside.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	WebServer: FCFS; Storage1: SIRO; Storage2: PS; Storage3: HOL
Arrivals	Source – Class1: Exp(0.1); Class2: Exp(0.1); Class3: Exp(0.1)
Service	WebServer – Class1: Exp(3.333); Class2: Exp(2); Class3: Exp(1.667) Storage1 – Class1: Exp(0.9091); Class2: Exp(0.7692); Class3: Exp(0.6667) Storage2 – Class1: Exp(0.5); Class2: Exp(0.4762); Class3: Exp(0.5263) Storage3 – Class1: Exp(0.4); Class2: Exp(0.5263); Class3: Exp(0.2326)
Solvers	CTMC, MVA, JMT, SSA

Table 2.40: Features of `prio_hol_open`.

The example is built and solved as follows.

```

%% multiclass example with PS, SIRO, FCFS, HOL priority
model = Network('MyNetwork');

```

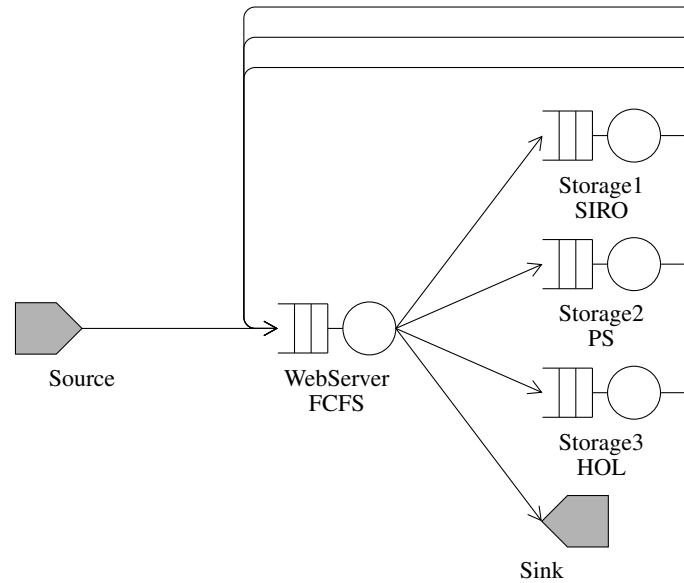


Figure 2.40: Topology of prio_hol_open: an open network over 4 queueing stations.

```
%% Block 1: nodes
node{1} = Source(model, 'Source');
node{2} = Queue(model, 'WebServer', SchedStrategy.FCFS);
node{3} = Queue(model, 'Storage1', SchedStrategy.SIRO);
node{4} = Queue(model, 'Storage2', SchedStrategy.PS);
node{5} = Queue(model, 'Storage3', SchedStrategy.HOL);
node{6} = Sink(model, 'Sink');

%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 1); % lower priority (priority 1 < priority 0)
jobclass{3} = OpenClass(model, 'Class3', 0);

node{1}.setArrival(jobclass{1}, Exp.fitMean(10.000000)); % (Source of Customers,Class0)
node{1}.setArrival(jobclass{2}, Exp.fitMean(10.000000)); % (Source of Customers,Class1)
node{1}.setArrival(jobclass{3}, Exp.fitMean(10.000000)); % (Source of Customers,Class2)
node{2}.setService(jobclass{1}, Exp.fitMean(0.300000)); % (WebServer,Class0)
node{2}.setService(jobclass{2}, Exp.fitMean(0.500000)); % (WebServer,Class1)
node{2}.setService(jobclass{3}, Exp.fitMean(0.600000)); % (WebServer,Class2)
node{3}.setService(jobclass{1}, Exp.fitMean(1.100000)); % (Storage1,Class0)
node{3}.setService(jobclass{2}, Exp.fitMean(1.300000)); % (Storage1,Class1)
node{3}.setService(jobclass{3}, Exp.fitMean(1.500000)); % (Storage1,Class2)
node{4}.setService(jobclass{1}, Exp.fitMean(2.000000)); % (Storage2,Class0)

... (53 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = NC(model,options);
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable()
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.5.3 prio_identical

Priority classes with identical parameters, the sanity check that priority alone does not move the aggregate metrics.

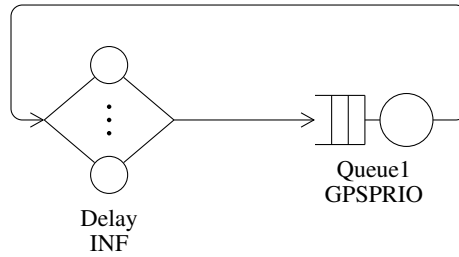


Figure 2.41: Topology of prio_identical: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	4: Class1 (closed, $N = 6$), Class2 (closed, $N = 4$), Class3 (closed, $N = 4$), Class4 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: GPSPRIO
Service	Delay – Class1: Erlang(3,2); Class2: Exp(1); Class3: Exp(1); Class4: Exp(2) Queue1 – Class1: Exp(30); Class2: Exp(2); Class3: Exp(12); Class4: Exp(1)
Solvers	CTMC, JMT

Table 2.41: Features of prio_identical.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.GPSPRIO);

jobclass{1} = ClosedClass(model, 'Class1', 6, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 4, node{1}, 1);
jobclass{3} = ClosedClass(model, 'Class3', 4, node{1}, 1);
jobclass{4} = ClosedClass(model, 'Class4', 1, node{1}, 2);

node{1}.setService(jobclass{1}, Erlang(3.0,2));
node{1}.setService(jobclass{2}, Exp(1.0));
node{1}.setService(jobclass{3}, Exp(1.0));
node{1}.setService(jobclass{4}, Exp(2.0));

w1=12; node{2}.setService(jobclass{1}, Exp(30), w1);
w2=3; node{2}.setService(jobclass{2}, Exp(2), w2);
w3=5; node{2}.setService(jobclass{3}, Exp(12), w3);
w4=1; node{2}.setService(jobclass{4}, Exp(1), w4);

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1} = Network.serialRouting(node);
P{2} = Network.serialRouting(node);

... (13 source lines omitted; full implementation: matlab/examples/basic/)

% solver{end+1} = LINE(model);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.5.4 prio_psprio

Priority processor sharing, where the share of a class is set by its priority.

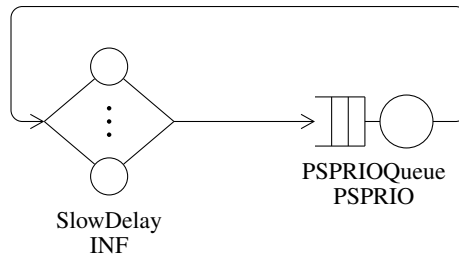


Figure 2.42: Topology of prio_psprio: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$)
Scheduling	SlowDelay: INF; PSPRIOQueue: PSPRIO
Service	SlowDelay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) PSPRIOQueue – Class1: HyperExp(0.1,0.9;1,10); Class2: Exp(1)
Solvers	CTMC, JMT, SSA

Table 2.42: Features of prio_psprio.

The example is built and solved as follows.

```
model = Network('MyNetwork');

node{1} = Delay(model, 'SlowDelay');
node{2} = Queue(model, 'PSPRIOQueue', SchedStrategy.PSPRIO);

jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 1);

node{1}.setService(jobclass{1}, Erlang(3,2));
node{1}.setService(jobclass{2}, HyperExp(0.5,3.0,10.0));

node{2}.setService(jobclass{1}, HyperExp(0.1,1.0,10.0));
node{2}.setService(jobclass{2}, Exp(1));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = model.initRoutingMatrix;
P{1} = Network.serialRouting(node);
P{2} = Network.serialRouting(node);
model.link(P);

%%
% This part illustrates the execution of different solvers
solver = {};
solver{end+1} = CTMC(model, 'exact');
solver{end+1} = JMT(model, 'seed', 23000, 'verbose', true, 'samples', 5e3);

... (5 source lines omitted; full implementation: matlab/examples/basic/)

% solver{end+1} = LINE(model);
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6 Fork-join networks

A `Fork` node splits an arriving job into sibling tasks, one per outgoing branch, and a `Join` node recombines them. The response time of the parent job is the maximum over its siblings, which is why fork-join has no product-form solution and why the join is where the modeling subtleties live: how many siblings are required (the quorum), whether the join is present at all, and what happens when branches share a station.

The family is the largest in the basic set because each of those questions has its own script. Nesting a fork inside a branch, three or more branches, asymmetric branch demands and class switching on the siblings are all covered separately, so a failure can be localized.

2.6.1 `fj_asymm`

Asymmetric branches, whose response time is set by the slowest one.

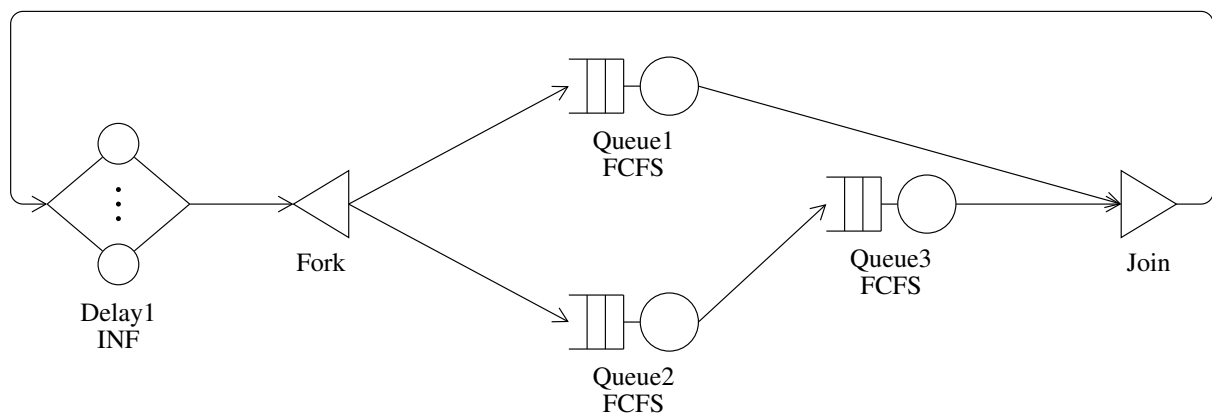


Figure 2.43: Topology of `fj_asymm`: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(2) Queue3 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.43: Features of `fj_asymm`.

The example is built and solved as follows.

```
%%clearvars -except exampleName;
model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
queue3 = Queue(model, 'Queue3', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
```

```

queue2.setService(jobclass1, Exp(2.0));
queue3.setService(jobclass1, Exp(1.0));
delay.setService(jobclass1, Exp(0.5));

P = zeros(6);
% P(source,fork) = 1;
P(delay, fork) = 1.0;
P(fork,queue1) = 1.0;
P(fork,queue2) = 1.0;
P(queue1,join) = 1.0;

... (10 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable(end+1) = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.2 fj_basic_closed

The closed counterpart, on PS branches.

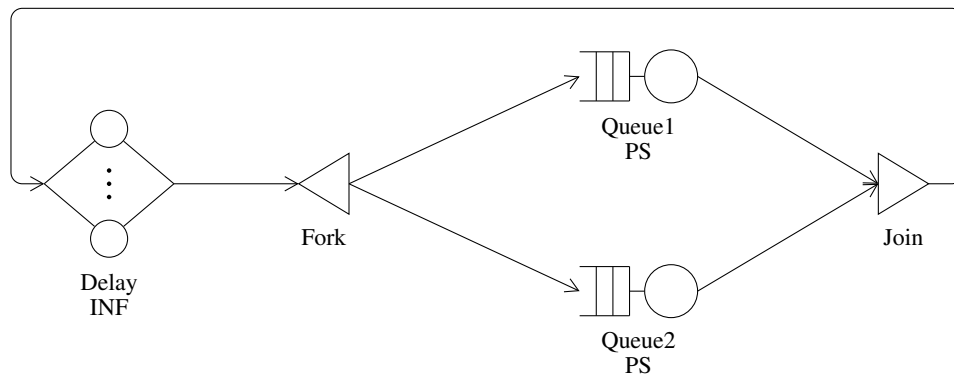


Figure 2.44: Topology of `fj_basic_closed`: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.44: Features of `fj_basic_closed`.

The example is built and solved as follows.

```

% similar to example_forkJoin_1 but closed
model = Network('model');

delay = Delay(model, 'Delay');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);

```

```

jobclass1 = ClosedClass(model, 'class1', 5, delay);

delay.setService(jobclass1, Exp(1.0));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));

P = zeros(5);
P(delay,fork) = 1;
P(fork,queue1) = 1.0;
P(fork,queue2) = 1.0;
P(queue1,join) = 1.0;
P(queue2,join) = 1.0;
P(join,delay) = 1.0;

model.link(P);

solver = {};

... (7 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.3 fj_basic_nesting

A fork inside a branch of another fork, with open and closed classes.

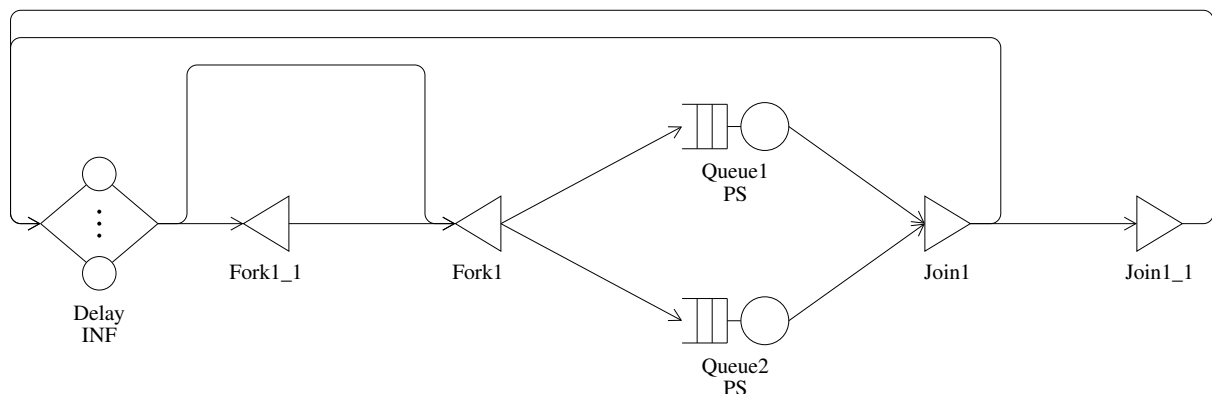


Figure 2.45: Topology of fj_basic_nesting: a closed network over 3 queueing stations with a fork-join block.

Nodes	7: Queue $\times$ 2, Delay, Fork $\times$ 2, Join $\times$ 2
Classes	2: class1 (closed, $N = 5$), class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(0.25); class2: Exp(0.25) Queue1 – class1: Exp(1); class2: Exp(2) Queue2 – class1: Exp(0.75); class2: Exp(2)
Features	class switching on 2 links
Solvers	JMT, MVA

Table 2.45: Features of fj_basic_nesting.

The example is built and solved as follows.

```
model = Network('model');

delay = Delay(model, 'Delay');
fork1 = Fork(model, 'Fork1');
fork1.setTasksPerLink(1);
fork11 = Fork(model, 'Fork1_1');
fork11.setTasksPerLink(2);
join1 = Join(model, 'Join1', fork1);
join11 = Join(model, 'Join1_1', fork11);
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);

jobclass1 = ClosedClass(model, 'class1', 5, delay);
jobclass2 = ClosedClass(model, 'class2', 2, delay);

delay.setService(jobclass1, Exp(0.25));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(0.75));

delay.setService(jobclass2, Exp(0.25));
queue1.setService(jobclass2, Exp(2.0));
queue2.setService(jobclass2, Exp(2.0));

M = model.getNumberOfNodes;
R = model.getNumberOfClasses;

... (70 source lines omitted; full implementation: matlab/examples/basic/)

% P{jobclass2, jobclass2}(queue2, join1) = 1.0;
% P{jobclass2, jobclass2}(join1, join11) = 1.0;
% P{jobclass2, jobclass2}(join11, sink) = 1.0;
%
% model.link(P);
% JMT(model, 'keep', true).getAvgNodeTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.4 fj_basic_open

One fork, two parallel FCFS branches and a join, in an open network.

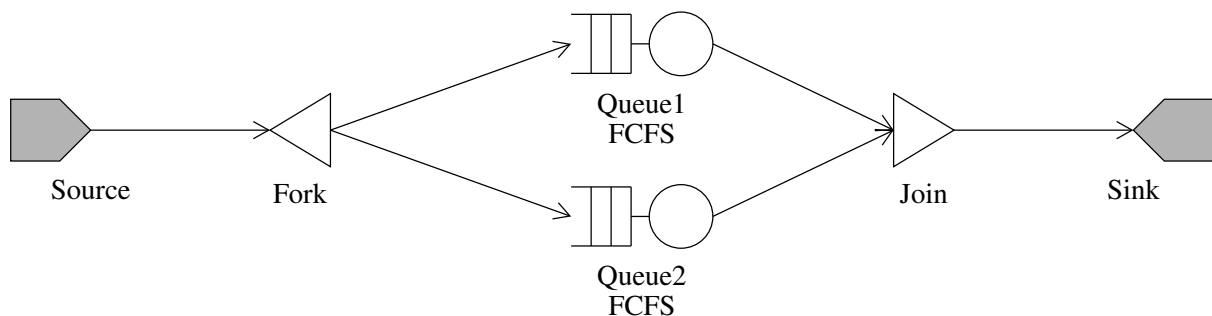


Figure 2.46: Topology of `fj_basic_open`: an open network over 2 queueing stations with a fork-join block.

The example is built and solved as follows.

```
%%clearvars -except exampleName;
model = Network('model');

source = Source(model, 'Source');
```

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	1: class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – class1: Exp(0.05)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(2)
Solvers	JMT, MVA, LDES, MAM

Table 2.46: Features of `fj_basic_open`.

```
% delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
sink = Sink(model, 'Sink');

jobclass1 = OpenClass(model, 'class1');

source.setArrival(jobclass1, Exp(0.05));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(2.0));
% delay.setService(jobclass1, Exp(0.5));

P = zeros(6);
P(source, fork) = 1;
% P(delay, fork) = 1.0;
P(fork, queue1) = 1.0;
P(fork, queue2) = 1.0;
P(queue1, join) = 1.0;
P(queue2, join) = 1.0;
P(join, sink) = 1.0;

... (8 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable(end+1) = solver{s}.getAvgTable;
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.5 `fj_complex_serial`

Several fork-join blocks in series inside a larger network.

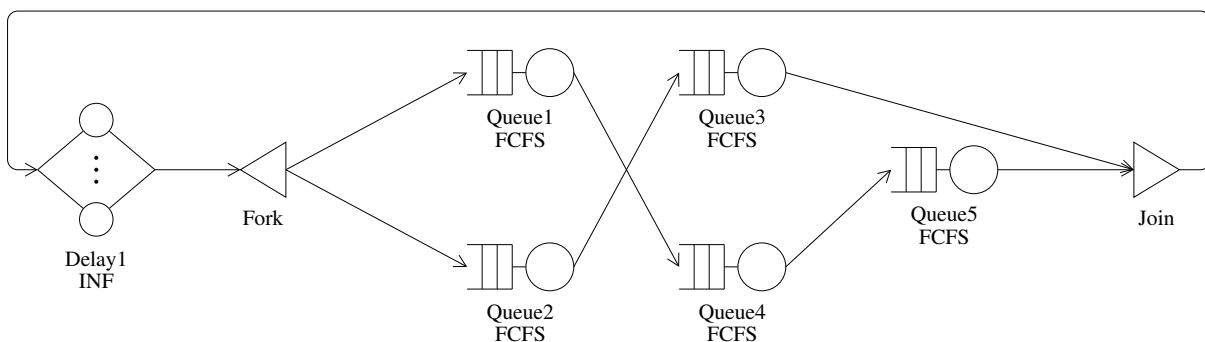


Figure 2.47: Topology of `fj_complex_serial`: a closed network over 6 queueing stations with a fork-join block.

Nodes	8: Queue $\times$ 5, Delay, Fork, Join
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS; Queue5: FCFS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(2) Queue3 – class1: Exp(1) Queue4 – class1: Exp(3) Queue5 – class1: Exp(0.8)
Solvers	JMT, MVA

Table 2.47: Features of fj_complex_serial.

The example is built and solved as follows.

```
%clearvars -except exampleName;
model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
queue3 = Queue(model, 'Queue3', SchedStrategy.FCFS);
queue4 = Queue(model, 'Queue4', SchedStrategy.FCFS);
queue5 = Queue(model, 'Queue5', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(2.0));
queue3.setService(jobclass1, Exp(1.0));
queue4.setService(jobclass1, Exp(3.0));
queue5.setService(jobclass1, Exp(0.8));
delay.setService(jobclass1, Exp(0.5));

P = zeros(5);
% P(source, fork) = 1;

... (16 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable(end+1) = solver{s}.getAvgTable;
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.6 fj_cs_multi_visits

Class switching with several visits per branch.

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	2: class1 (open), class2 (open)
Scheduling	Queue1: PS; Queue2: PS
Arrivals	Source – class1: Exp(0.1); class2: Disabled
Service	Queue1 – class1: Exp(1); class2: Exp(1) Queue2 – class1: Exp(1); class2: Exp(1)
Features	class switching on 1 link
Solvers	MVA, JMT

Table 2.48: Features of fj_cs_multi_visits.

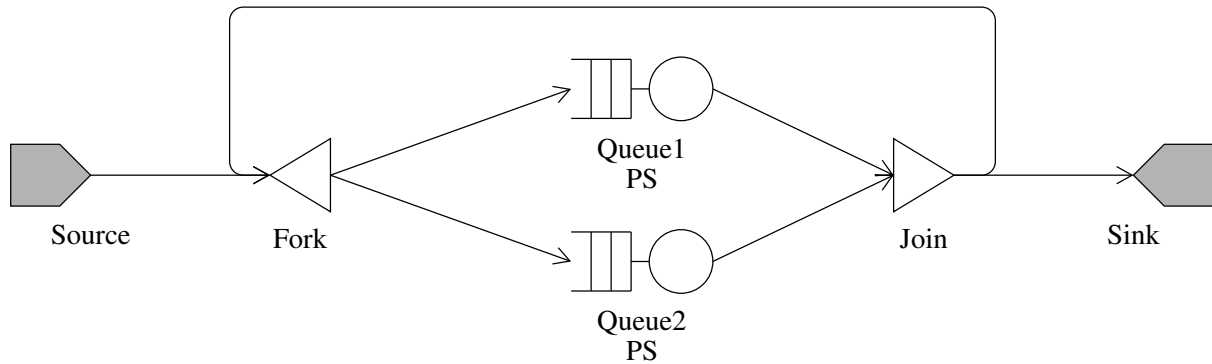


Figure 2.48: Topology of `fj_cs_multi_visits`: an open network over 2 queueing stations with a fork-join block.

The example is built and solved as follows.

```
% fork-join with multiple visits within the same chain
model = Network('model');

source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
sink = Sink(model, 'Sink');

jobclass1 = OpenClass(model, 'class1');
jobclass2 = OpenClass(model, 'class2');

source.setArrival(jobclass1, Exp(0.1));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));
queue1.setService(jobclass2, Exp(1.0));
queue2.setService(jobclass2, Exp(1.0));

P = model.initRoutingMatrix;
P{jobclass1, jobclass1}(source, fork) = 1;
P{jobclass1, jobclass1}(fork, queue1) = 1.0;
P{jobclass1, jobclass1}(fork, queue2) = 1.0;
P{jobclass1, jobclass1}(queue1, join) = 1.0;
P{jobclass1, jobclass1}(queue2, join) = 1.0;

... (12 source lines omitted; full implementation: matlab/examples/basic/)

%solver(end+1) = LDES(model, 'seed', 23000);

for s=1:length(solver)
    AvgTable{s} = solver{s}.getAvgTable;
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.7 `fj_cs_postfork`

Class switching after the fork, on the sibling tasks.

The example is built and solved as follows.

```
model = Network('model');
```

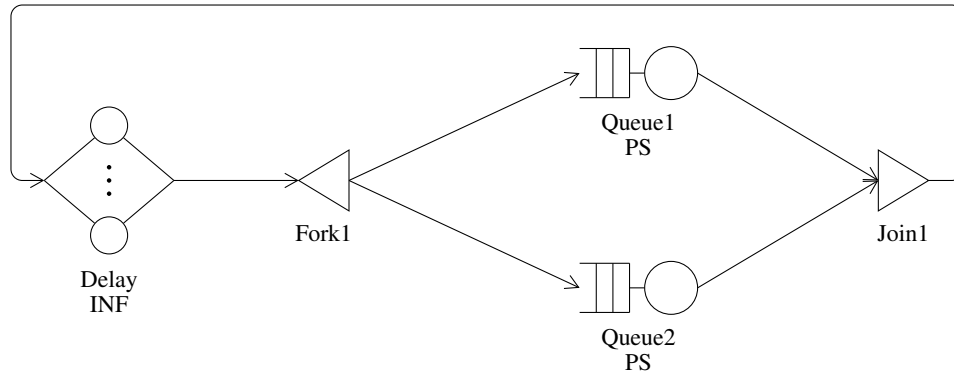



Figure 2.49: Topology of `fj_cs_postfork`: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	2: class1 (closed, $N = 1$), class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(0.25); class2: Exp(0.25) Queue1 – class1: Exp(2); class2: Exp(2) Queue2 – class1: Exp(2); class2: Exp(2)
Features	class switching on 3 links
Solvers	MVA, JMT

Table 2.49: Features of `fj_cs_postfork`.

```

delay = Delay(model, 'Delay');
fork1 = Fork(model, 'Fork1');
join1 = Join(model, 'Join1', fork1);
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);

jobclass1 = ClosedClass(model, 'class1', 1, delay);
jobclass2 = ClosedClass(model, 'class2', 1, delay);

delay.setService(jobclass1, Exp(0.25));
queue1.setService(jobclass1, Exp(2.0));
queue2.setService(jobclass1, Exp(2.0));

delay.setService(jobclass2, Exp(0.25));
queue1.setService(jobclass2, Exp(2.0));
queue2.setService(jobclass2, Exp(2.0));

M = model.getNumberOfNodes;
R = model.getNumberOfClasses;

P = model.initRoutingMatrix;
P{jobclass1, jobclass1}(delay, fork1) = 1.0;
P{jobclass1, jobclass1}(fork1, queue1) = 1.0;
P{jobclass1, jobclass1}(fork1, queue2) = 1.0;

... (12 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = LDES(model, 'seed', 23000);

for s=1:length(solver)
    AvgTable{s} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.8 fj_cs_prefork

Class switching before the fork.

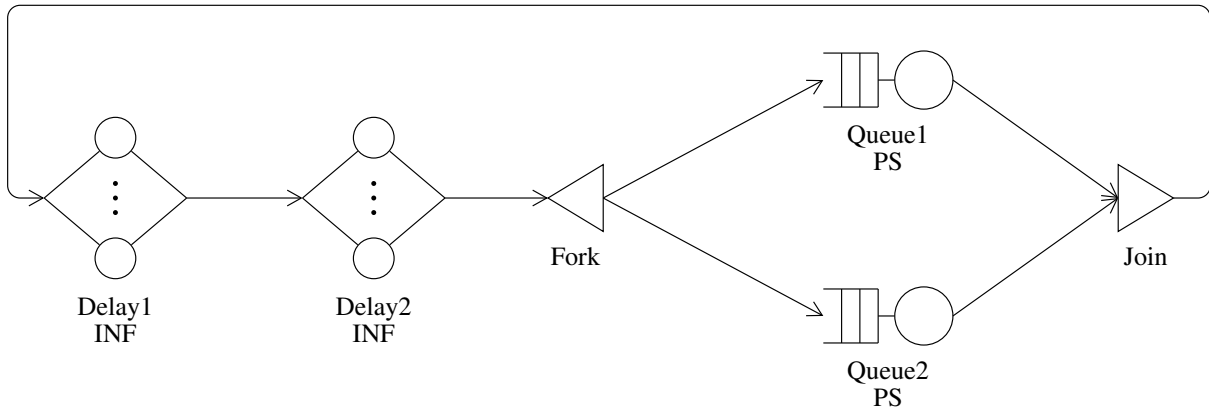


Figure 2.50: Topology of fj_cs_prefork: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 2, Delay $\times$ 2, Fork, Join
Classes	2: class1 (closed, $N = 10$), class2 (closed, $N = 10$)
Scheduling	Delay1: INF; Delay2: INF; Queue1: PS; Queue2: PS
Service	Delay1 – class1: Exp(0.5); class2: Exp(0.5) Delay2 – class1: Disabled; class2: Exp(2) Queue1 – class1: Exp(1); class2: Disabled Queue2 – class1: Exp(1); class2: Disabled
Features	drop rule {'class1': 'waitingQueue'} at Delay2; drop rule {'class2': 'waitingQueue'} at Queue1; drop rule {'class2': 'waitingQueue'} at Queue2; class switching on 5 links
Solvers	JMT, MVA

Table 2.50: Features of fj_cs_prefork.

The example is built and solved as follows.

```

%clearvars -except exampleName;
model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
delay2 = Delay(model, 'Delay2');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);
jobclass2 = ClosedClass(model, 'class2', 10, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));
delay.setService(jobclass1, Exp(0.5));
delay.setService(jobclass2, Exp(0.5));
delay2.setService(jobclass2, Exp(2));

P = model.initRoutingMatrix;
P{jobclass1, jobclass2}(delay, delay2) = 1.0;
P{jobclass2, jobclass1}(delay2, fork) = 1.0;
% P(source, fork) = 1;

... (13 source lines omitted; full implementation: matlab/examples/basic/)

```

```

AvgTable = {};
for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.9 fj_deep_nesting

Several nesting levels, the stress case for the join bookkeeping.

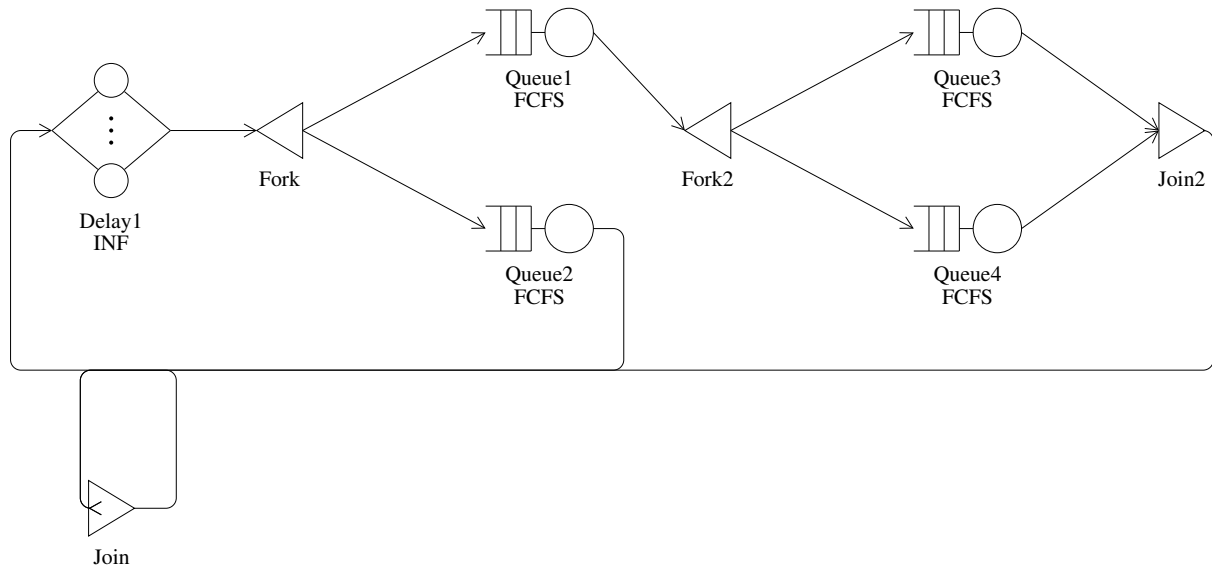


Figure 2.51: Topology of `fj_deep_nesting`: a closed network over 5 queueing stations with a fork-join block.

Nodes	9: Queue $\times$ 4, Delay, Fork $\times$ 2, Join $\times$ 2
Classes	1: class1 (closed, $N = 1$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1) Queue3 – class1: Exp(2) Queue4 – class1: Exp(2)
Solvers	JMT, MVA

Table 2.51: Features of `fj_deep_nesting`.

The example is built and solved as follows.

```

%clearvars -except exampleName;
model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
queue3 = Queue(model, 'Queue3', SchedStrategy.FCFS);
queue4 = Queue(model, 'Queue4', SchedStrategy.FCFS);

```

```

fork2 = Fork(model, 'Fork2');
join2 = Join(model, 'Join2', fork2);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 1, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));
delay.setService(jobclass1, Exp(0.5));
queue3.setService(jobclass1, Exp(2.0));
queue4.setService(jobclass1, Exp(2.0));

P = zeros(9);
% P(source, fork) = 1;

... (20 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable(end+1) = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.10 fj_delays

Delay stations inside the branches.

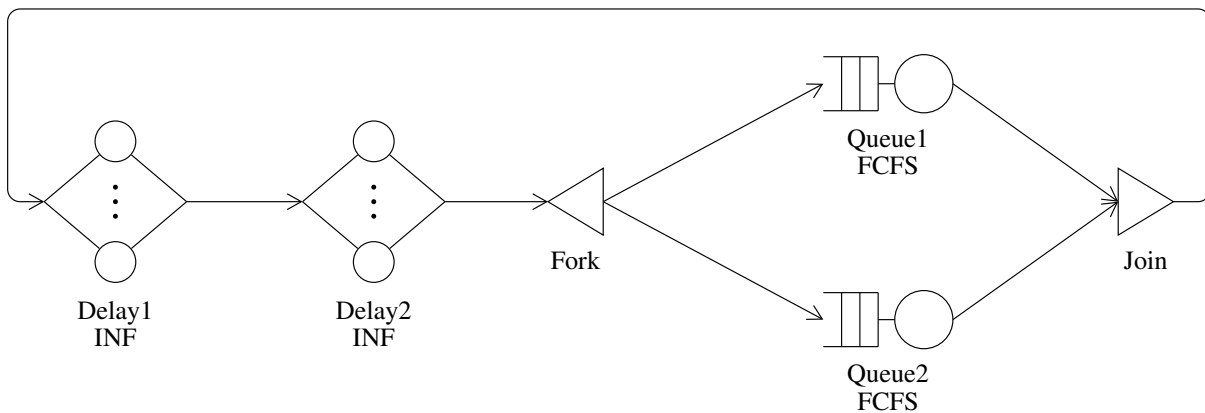


Figure 2.52: Topology of fj_delays: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 2, Delay $\times$ 2, Fork, Join
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Delay2: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay1 – class1: Exp(0.5) Delay2 – class1: Exp(2) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.52: Features of fj_delays.

The example is built and solved as follows.

```
%%clearvars -except exampleName;
```

```

model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
delay2 = Delay(model, 'Delay2');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));
delay.setService(jobclass1, Exp(0.5));
delay2.setService(jobclass1, Exp(2.0));

P = model.initRoutingMatrix;
P{jobclass1, jobclass1}(delay, delay2) = 1.0;
P{jobclass1, jobclass1}(delay2, fork) = 1.0;
% P(source, fork) = 1;
% P(delay, fork) = 1.0;
P{jobclass1, jobclass1}(fork, queue1) = 1.0;

... (11 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable(end+1) = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.11 fj_mixed_openclosed

Open and closed classes through one fork-join block, with a class disabled on a branch.

Nodes	9: Source, Queue $\times$ 4, Delay, Fork, Join, Sink
Classes	2: Open (open), Closed (closed, $N = 1$)
Scheduling	Client: INF; PreFork: FCFS; Branch1: FCFS; Branch2: FCFS; PostJoin: FCFS
Arrivals	Source – Open: Exp(0.1); Closed: Disabled
Service	Client – Open: Disabled; Closed: Exp(1) PreFork – Open: Exp(5); Closed: Exp(5) Branch1 – Open: Exp(3.333); Closed: Exp(3.333) Branch2 – Open: Exp(2.5); Closed: Exp(2.5) PostJoin – Open: Exp(10); Closed: Exp(10)
Features	class switching on 2 links
Solvers	MVA, JMT

Table 2.53: Features of fj_mixed_openclosed.

The example is built and solved as follows.

```

% Flat counterpart of lqn_fork_open_arrival.m: one fork-join traversed by an
% open class (Poisson rate 0.1) and by a closed class (1 job, think time 1), so
% the same fork carries exogenous and closed traffic at once. The MVA fork-join
% transform solves this; the layered builder refuses the analogous LQN because
% the transform's auxiliary Source collides with the layer's open-stream Source.
%
% Reference (SolverJMT, seed 23000, 2e5 samples): Branch1 open RespT 0.362,
% Branch1 closed RespT 0.310, Join open RespT 0.215, closed tput 0.540.

model = Network('ForkJoinOpenClosed');

```

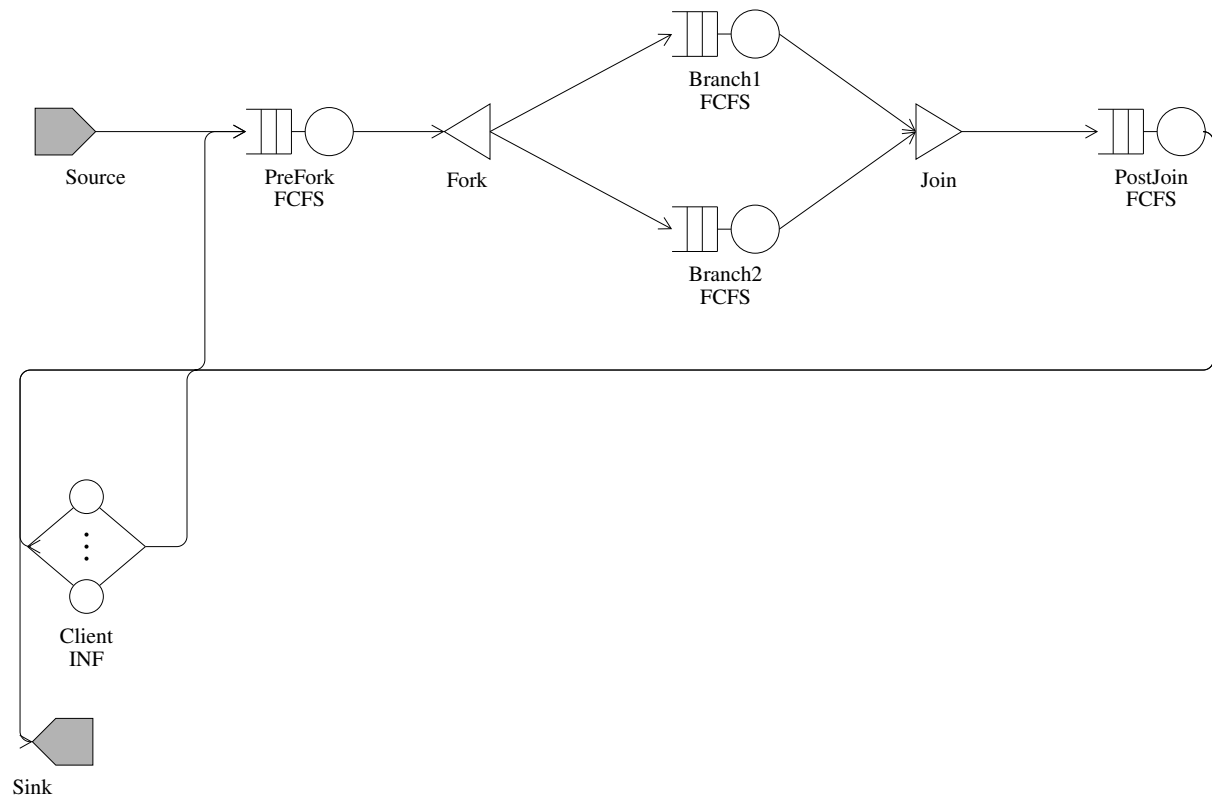


Figure 2.53: Topology of `fj_mixed_openclosed`: an open network over 5 queueing stations with a fork-join block.

```

source = Source(model, 'Source');
client = Delay(model, 'Client');
prefork = Queue(model, 'PreFork', SchedStrategy.FCFS);
branch1 = Queue(model, 'Branch1', SchedStrategy.FCFS);
branch2 = Queue(model, 'Branch2', SchedStrategy.FCFS);
postjoin = Queue(model, 'PostJoin', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
sink = Sink(model, 'Sink');

oclass = OpenClass(model, 'Open');
cclass = ClosedClass(model, 'Closed', 1, client);

source.setArrival(oclass, Exp(0.1));
client.setService(cclass, Exp.fitMean(1.0));

... (34 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.12 fj_nojoin

A fork with no join, so the siblings leave through the sink independently.

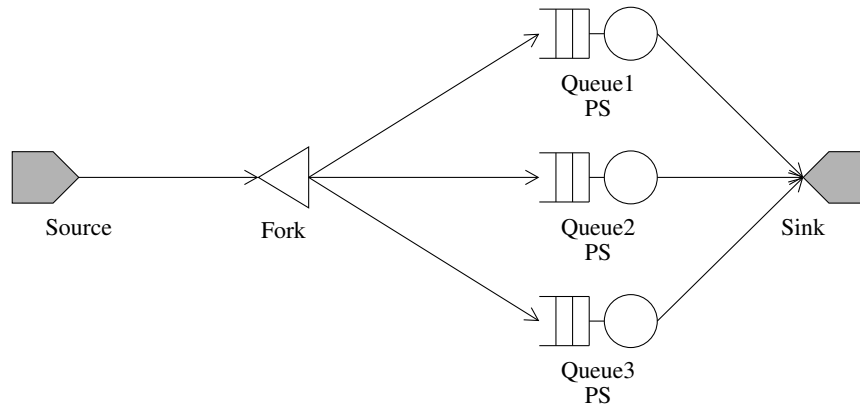


Figure 2.54: Topology of fj_nojoin: an open network over 3 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 3, Fork, Sink
Classes	1: class1 (open)
Scheduling	Queue1: PS; Queue2: PS; Queue3: PS
Arrivals	Source – class1: Exp(0.5)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(2) Queue3 – class1: Exp(3)
Solvers	MVA, JMT

Table 2.54: Features of fj_nojoin.

The example is built and solved as follows.

```
model = Network('model');

source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
queue3 = Queue(model, 'Queue3', SchedStrategy.PS);
fork = Fork(model, 'Fork');
sink = Sink(model, 'Sink');

jobclass1 = OpenClass(model, 'class1');

source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(2.0));
queue3.setService(jobclass1, Exp(3.0));

P = zeros(5);
P(source, fork) = 1;
P(fork, queue1) = 1.0;
P(fork, queue2) = 1.0;
P(fork, queue3) = 1.0;
P(queue1, sink) = 1.0;
P(queue2, sink) = 1.0;
P(queue3, sink) = 1.0;

model.link(P);

... (5 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
```

```

for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.13 fj_route_overlap

Branches that share a station, so their visits overlap.

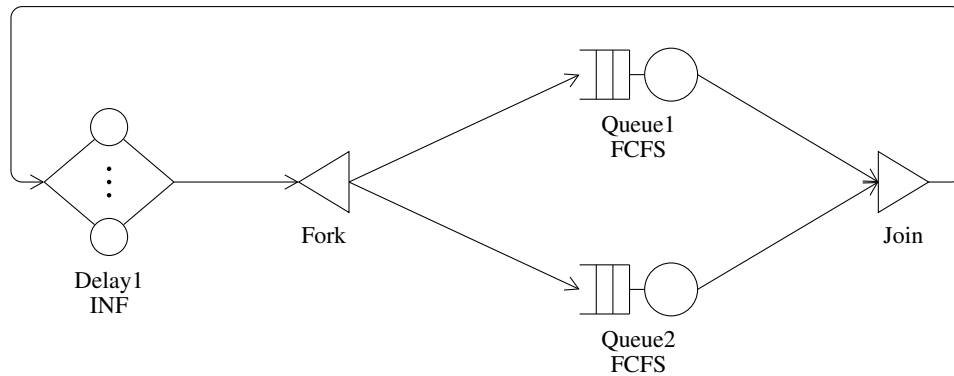


Figure 2.55: Topology of fj_route_overlap: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	2: class1 (closed, $N = 10$), class2 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay1 – class1: Exp(0.5); class2: Exp(0.2) Queue1 – class1: Exp(1); class2: Exp(1) Queue2 – class1: Exp(2); class2: Exp(2)
Solvers	JMT, MVA

Table 2.55: Features of fj_route_overlap.

The example is built and solved as follows.

```

%clearvars -except exampleName;
model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);
jobclass2 = ClosedClass(model, 'class2', 10, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(2.0));
delay.setService(jobclass1, Exp(0.5));

queue1.setService(jobclass2, Exp(1.0));
queue2.setService(jobclass2, Exp(2.0));
delay.setService(jobclass2, Exp(0.2));

```



```

P = model.initRoutingMatrix;
% P(source,fork) = 1;
P{jobclass1, jobclass1}(delay, fork) = 1.0;

... (19 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.14 fj_serialfjs_closed

The closed version.

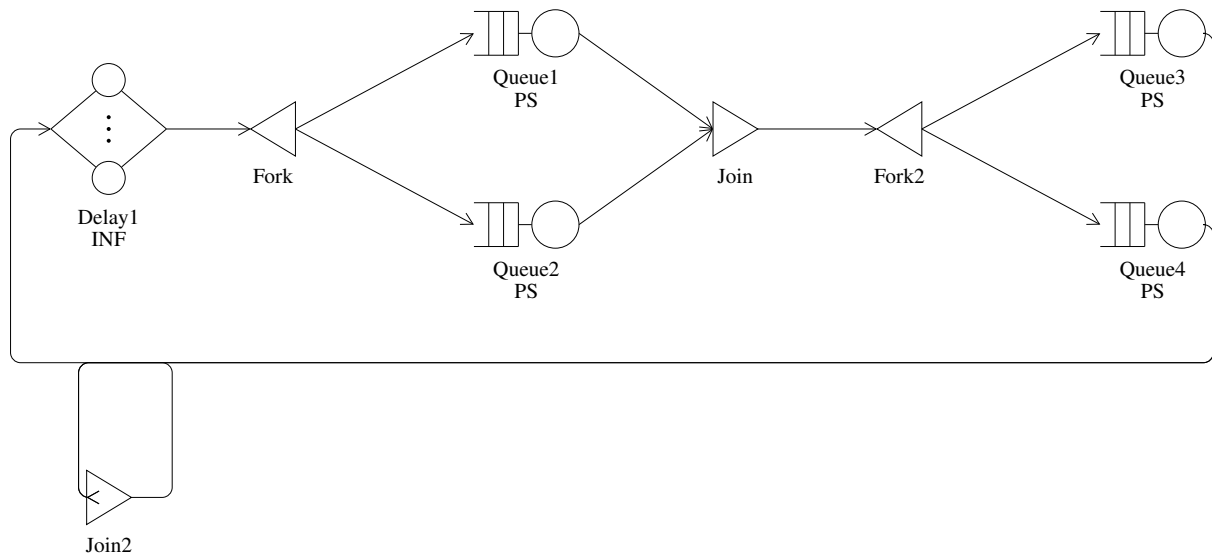


Figure 2.56: Topology of fj_serialfjs_closed: a closed network over 5 queueing stations with a fork-join block.

Nodes	9: Queue $\times$ 4, Delay, Fork $\times$ 2, Join $\times$ 2
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: PS; Queue2: PS; Queue3: PS; Queue4: PS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1) Queue3 – class1: Exp(1) Queue4 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.56: Features of fj_serialfjs_closed.

The example is built and solved as follows.

```

%clearvars -except exampleName;
model = Network('model');

% source = Source(model, 'Source');
delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);

```

```

queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
queue3 = Queue(model, 'Queue3', SchedStrategy.PS);
queue4 = Queue(model, 'Queue4', SchedStrategy.PS);
fork2 = Fork(model, 'Fork2');
join2 = Join(model, 'Join2', fork2);
% sink = Sink(model, 'Sink');

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);

% source.setArrival(jobclass1, Exp(0.5));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));
delay.setService(jobclass1, Exp(0.5));
queue3.setService(jobclass1, Exp(1.0));
queue4.setService(jobclass1, Exp(1.0));

P = zeros(5);
% P(source, fork) = 1;

... (18 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable(end+1) = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.15 fj_serialfjs_open

A chain of fork-join blocks, open.

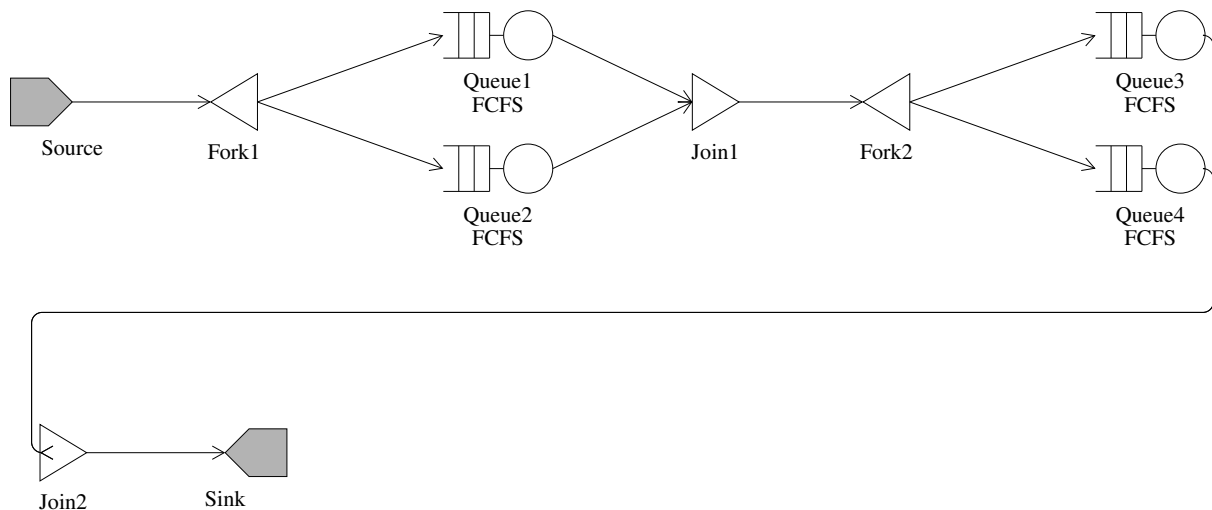


Figure 2.57: Topology of fj_serialfjs_open: an open network over 4 queueing stations with a fork-join block.

The example is built and solved as follows.

```

model = Network('model');
source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);

```

Nodes	10: Source, Queue $\times$ 4, Fork $\times$ 2, Join $\times$ 2, Sink
Classes	1: class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS
Arrivals	Source – class1: Exp(0.4)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(1) Queue3 – class1: Exp(1) Queue4 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.57: Features of `fj_serialfjs_open`.

```

queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork1 = Fork(model, 'Fork1');
join1 = Join(model, 'Join1', fork1);
queue3 = Queue(model, 'Queue3', SchedStrategy.FCFS);
queue4 = Queue(model, 'Queue4', SchedStrategy.FCFS);
fork2 = Fork(model, 'Fork2');
join2 = Join(model, 'Join2', fork2);
sink = Sink(model, 'Sink');

jobclass1 = OpenClass(model, 'class1');

source.setArrival(jobclass1, Exp(0.4));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(1.0));
queue3.setService(jobclass1, Exp(1.0));
queue4.setService(jobclass1, Exp(1.0));

P = cellzeros(2,2,6,6);
P{jobclass1,jobclass1}(source,fork1) = 1;
P{jobclass1,jobclass1}(fork1,queue1) = 1.0;
P{jobclass1,jobclass1}(fork1,queue2) = 1.0;
P{jobclass1,jobclass1}(queue1,join1) = 1.0;

... (12 source lines omitted; full implementation: matlab/examples/basic/)

%solver{end+1} = LDES(model, 'seed', 23000);

for s=1:length(solver)
    AvgTable{s} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.16 `fj_threebranches`

Three parallel branches rather than two.

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	2: class1 (closed, $N = 10$), class2 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	Delay1 – class1: Exp(0.5); class2: Exp(0.8) Queue1 – class1: Exp(1.5); class2: Exp(2.8) Queue2 – class1: Exp(1.1); class2: Exp(3) Queue3 – class1: Exp(2.5); class2: Exp(1)
Solvers	MVA

Table 2.58: Features of `fj_threebranches`.

The example is built and solved as follows.

```

model = Network('model');

delay = Delay(model, 'Delay1');

```

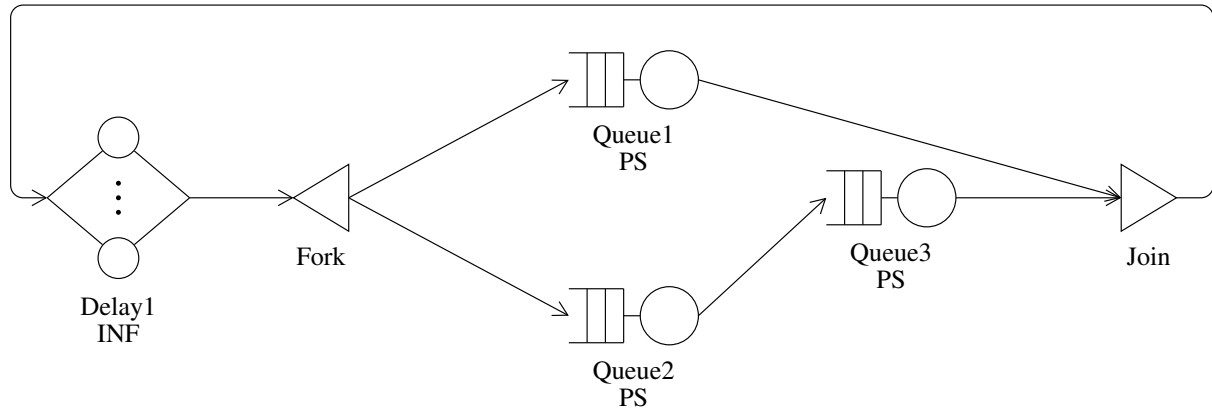


Figure 2.58: Topology of `fj_threebranches`: a closed network over 4 queueing stations with a fork-join block.

```

queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
queue3 = Queue(model, 'Queue3', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);

jobclass1 = ClosedClass(model, 'class1', 10, delay, 0);
jobclass2 = ClosedClass(model, 'class2', 10, delay, 0);

queue1.setService(jobclass1, Exp(1.5));
queue2.setService(jobclass1, Exp(1.1));
queue3.setService(jobclass1, Exp(2.5));
delay.setService(jobclass1, Exp(0.5));

queue1.setService(jobclass2, Exp(2.8));
queue2.setService(jobclass2, Exp(3));
queue3.setService(jobclass2, Exp(1.0));
delay.setService(jobclass2, Exp(0.8));

P = model.initRoutingMatrix;
P{jobclass1, jobclass1}(delay, fork) = 1.0;
P{jobclass1, jobclass1}(fork, queue1) = 1.0;
P{jobclass1, jobclass1}(fork, queue2) = 1.0;

... (10 source lines omitted; full implementation: matlab/examples/basic/)

P{jobclass2, jobclass2}(queue1, join) = 1.0;
P{jobclass2, jobclass2}(join, delay) = 1.0;

model.link(P);

MVA(model).getAvgTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.17 fj_tiny_closed

The smallest closed fork-join model, small enough for an exact CTMC solve. The example is built and solved as follows.

```

% Minimal closed fork-join model solved exactly by the native CTMC
% fork-join implementation (tag-augmented state space) and validated
% against the closed form for N=1:
% X = 1/(Z + E[max(S1,S2)]), E[max] = 1/mu1 + 1/mu2 - 1/(mu1+mu2)
model = Network('model');

```

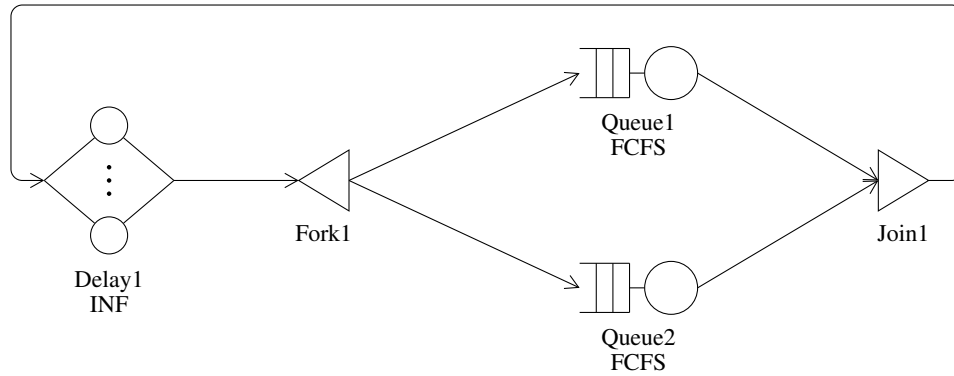


Figure 2.59: Topology of `fj_tiny_closed`: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay1 – Class1: Exp(1) Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(3)
Solvers	CTMC, SSA, JMT

Table 2.59: Features of `fj_tiny_closed`.

```

delay = Delay(model, 'Delay1');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork = Fork(model, 'Fork1');
join = Join(model, 'Join1', fork);

jobclass1 = ClosedClass(model, 'Class1', 1, delay);

delay.setService(jobclass1, Exp(1.0));
queue1.setService(jobclass1, Exp(2.0));
queue2.setService(jobclass1, Exp(3.0));

P = zeros(5);
P(delay, fork) = 1;
P(fork, queue1) = 1.0;
P(fork, queue2) = 1.0;
P(queue1, join) = 1.0;
P(queue2, join) = 1.0;
P(join, delay) = 1.0;

... (8 source lines omitted; full implementation: matlab/examples/basic/)

for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

% closed-form check (N=1): X = 1/(1 + 1/2 + 1/3 - 1/5) = 0.612244...

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.18 fj_twoclasses_forked

Two classes forked independently through the same block.
The example is built and solved as follows.

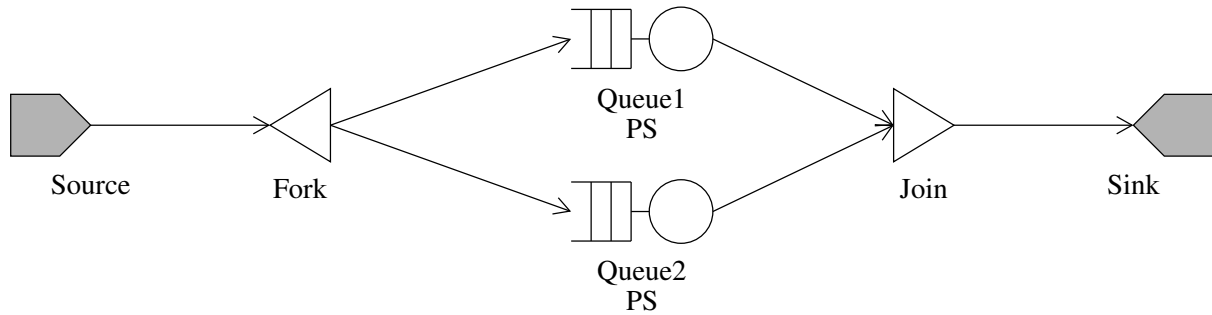


Figure 2.60: Topology of `fj_twoclasses_forked`: an open network over 2 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	2: class1 (open), class2 (open)
Scheduling	Queue1: PS; Queue2: PS
Arrivals	Source – class1: Exp(0.25); class2: Exp(0.25)
Service	Queue1 – class1: Exp(1); class2: Immediate Queue2 – class1: Exp(0.75); class2: Exp(2)
Solvers	JMT, MVA

Table 2.60: Features of `fj_twoclasses_forked`.

```

model = Network('model');

source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
fork = Fork(model, 'Fork');
fork.setTasksPerLink(2);
join = Join(model, 'Join', fork);
sink = Sink(model, 'Sink');

jobclass1 = OpenClass(model, 'class1');
jobclass2 = OpenClass(model, 'class2');

source.setArrival(jobclass1, Exp(0.25));
queue1.setService(jobclass1, Exp(1.0));
queue2.setService(jobclass1, Exp(0.75));

source.setArrival(jobclass2, Exp(0.25));
queue1.setService(jobclass2, Immediate());
queue2.setService(jobclass2, Exp(2.0));

P = cellzeros(2,2,6,6);
P{jobclass1, jobclass1}(source, fork) = 1;
P{jobclass1, jobclass1}(fork, queue1) = 1.0;
P{jobclass1, jobclass1}(fork, queue2) = 1.0;
P{jobclass1, jobclass1}(queue1, join) = 1.0;

... (16 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = {};
for s=1:length(solver)
    AvgTable{end+1} = solver{s}.getAvgTable;
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.6.19 fj_variable_fanout

A forking level that is not one fixed number: a per-link vector, a random degree, or a branch that fires only part of the time.

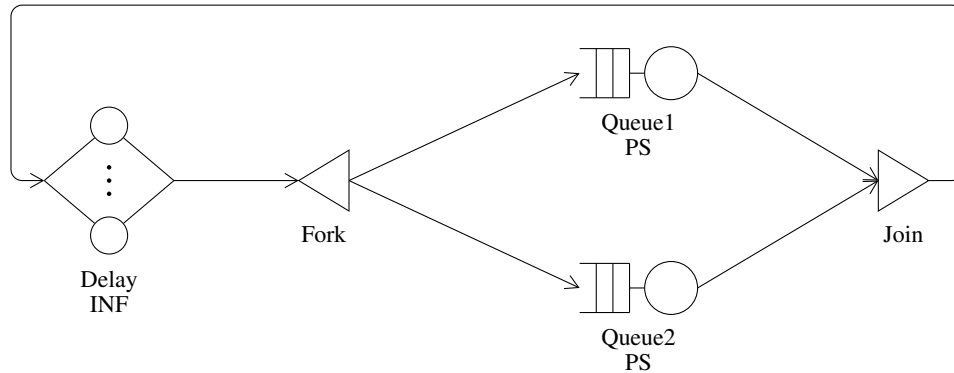


Figure 2.61: Topology of `fj_variable_fanout`: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: class1 (closed, $N = 4$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(2) Queue2 – class1: Exp(2)
Solvers	JMT

Table 2.61: Features of `fj_variable_fanout`.

The example is built and solved as follows.

```
function model = fj_variable_fanout(mode)
% FJ_VARIABLE_FANOUT Fork-join with a variable forking level
%
% MODEL = FJ_VARIABLE_FANOUT(MODE) builds a closed fork-join network whose Fork
% emits a number of tasks per link that is NOT one fixed number. MODE selects
% which way the degree varies:
%
%   'fixed'   the classic fork, one task on each of two links (the baseline)
%   'vector'  three tasks towards Queue2 and one towards Queue1
%   'random'  one or three tasks per link, each with probability 1/2
%   'prob'    the branch towards Queue2 fires only half the time
%
% Exact under SolverJMT and SolverLDES, which draw the degree at the fork epoch.
% SolverMVA's MMT method sees the EXPECTED degree. The exact CTMC/SSA path
% accepts 'fixed' and 'vector' -- on 'vector' its answer matches JMT to
% simulation noise -- and refuses 'random' and 'prob' by name, because its tag
% construction fixes the sibling count, and the sibling SET, when the state
% space is built.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

if nargin < 1
    mode = 'fixed';
end

... (42 source lines omitted; full implementation: matlab/examples/basic/)

line_printf('\n<strong>mode = %s</strong>', m{1});
AvgTable = SolverJMT(mm, 'seed', 23000).getAvgTable();
disp(AvgTable);
```

```

end
end
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7 Layered queueing networks

A layered queueing network (LQN) models software resources explicitly. A `Processor` hosts `Tasks`, each task exposes `Entry` points, and each entry runs an `Activity` graph that consumes host demand and makes synchronous or asynchronous calls to the entries of other tasks. A synchronous call blocks the caller until the reply arrives, which is what creates the layers: a task is simultaneously a server to its callers and a client of the tasks it calls.

The models are solved by `SolverLN`, which decomposes the layers into queueing submodels and iterates to a fixed point, and by the external `lqns` and `lqsim` tools. The same model is also readable from, and writable to, the `.lqn` XML interchange format.

2.7.1 `lqn_basic`

One reference task calling one server task, the smallest layered model.

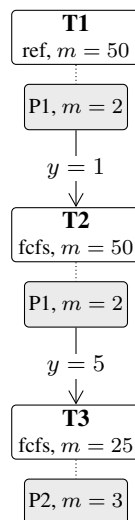


Figure 2.62: Call graph of `lqn_basic`: 3 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 2$), P2 (ps, $m = 3$)
Tasks	3: T1 (ref, $m = 50$), T2 (fcfs, $m = 50$), T3 (fcfs, $m = 25$)
Entries	3: E1, E2, E3
Activities	3, host demands AS1: 0.1, AS2: 0.05, AS3: 0.02
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 2.0

Table 2.62: Features of `lqn_basic`.

The example is built and solved as follows.


```

model = LayeredNetwork('test_LQN_4');

P1 = Processor(model, 'P1', 2, SchedStrategy.PS);
P2 = Processor(model, 'P2', 3, SchedStrategy.PS);

T1 = Task(model, 'T1', 50, SchedStrategy.REF).on(P1).setThinkTime( Exp(1 / 2));
T2 = Task(model, 'T2', 50, SchedStrategy.FCFS).on(P1).setThinkTime( Exp(1 / 3));

T3 = Task(model, 'T3', 25, SchedStrategy.FCFS).on(P2).setThinkTime( Exp(1 / 4));

E1 = Entry(model, 'E1').on(T1);
E2 = Entry(model, 'E2').on(T2);
E3 = Entry(model, 'E3').on(T3);

A1 = Activity(model, 'AS1', Exp(10)).on(T1).boundTo(E1).synchCall(E2, 1);
A2 = Activity(model, 'AS2', Exp(20)).on(T2).boundTo(E2).synchCall(E3, 5).repliesTo(E2);
A3 = Activity(model, 'AS3', Exp(50)).on(T3).boundTo(E3).repliesTo(E3);

AvgTable = LN(model, 'verbose', false).getAvgTable;
disp(AvgTable)

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.2 lqn_bpmn

A BPMN process translated into a layered model with forks, joins and immediate activities.

Processors	7: R1_Processor (fcfs, $m = 100$), R2_Processor (inf, $m = 1$), R3_Processor (fcfs, $m = 2$), R1A_Processor (fcfs, $m = 7$), R1B_Processor (fcfs, $m = 3$), R2A_Processor (fcfs, $m = 4$), R2B_Processor (fcfs, $m = 5$)
Tasks	7: R1_Task (ref, $m = 100$), R2_Task (inf, $m = 1$), R3_Task (fcfs, $m = 2$), R1A_Task (fcfs, $m = 7$), R1B_Task (fcfs, $m = 3$), R2A_Task (fcfs, $m = 4$), R2B_Task (fcfs, $m = 5$)
Entries	16: R1_Ref_Entry, R2_Synch_A2_Entry, R2_Synch_A5_Entry, R3_Synch_A9_Entry, R1A_Synch_A1_Entry, R1A_Synch_A2_Entry, R1A_Synch_A3_Entry, R1B_Synch_A4_Entry, R1B_Synch_A5_Entry, R1B_Synch_A6_Entry, R2A_Synch_A7_Entry, R2A_Synch_A8_Entry, R2A_Synch_A11_Entry, R2B_Synch_A9_Entry, R2B_Synch_A10_Entry, R2B_Synch_A12_Entry
Activities	29, host demands A1_Empty: 0.0, A2_Empty: 0.0, A5_Empty: 0.0, A6_Empty: 0.0, A3_Empty: 0.0, A4_Empty: 0.0, E4_Empty: 0.0, A7_Empty: 0.0, ...
Calls	15 (15 synchronous, 0 asynchronous)
Think times	R1_Task: 20.0

Table 2.63: Features of lqn_bpmn.

The example is built and solved as follows.

```

fprintf(1, 'This example illustrates the solution of a complex layered queueing network
         extracted from a BPMN model.\n');

%model = LayeredNetwork.parseXML([lineRootFolder, filesep, 'examples', filesep, 'lqn_bpmn.xml']);
model = LayeredNetwork('myLayeredModel');

P{1} = Processor(model, 'R1_Processor', 100, SchedStrategy.FCFS);
P{2} = Processor(model, 'R2_Processor', Inf, SchedStrategy.INF);
P{3} = Processor(model, 'R3_Processor', 2, SchedStrategy.FCFS);
P{4} = Processor(model, 'R1A_Processor', 7, SchedStrategy.FCFS);
P{5} = Processor(model, 'R1B_Processor', 3, SchedStrategy.FCFS);
P{6} = Processor(model, 'R2A_Processor', 4, SchedStrategy.FCFS);
P{7} = Processor(model, 'R2B_Processor', 5, SchedStrategy.FCFS);

T{1} = Task(model, 'R1_Task', 100, SchedStrategy.REF).on(P{1}).setThinkTime(Exp.fitMean(20));
T{2} = Task(model, 'R2_Task', Inf, SchedStrategy.INF).on(P{2}).setThinkTime(Immediate());
T{3} = Task(model, 'R3_Task', 2, SchedStrategy.FCFS).on(P{3}).setThinkTime(Immediate());

```

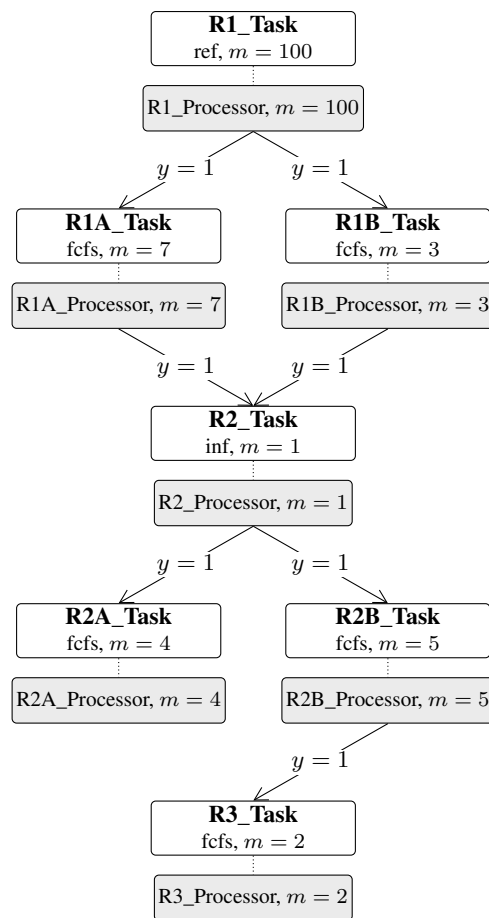


Figure 2.63: Call graph of `lqn_bpmn`: 7 tasks over 7 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```

T{4} = Task(model, 'R1A_Task', 7, SchedStrategy.FCFS).on(P{4}).setThinkTime(Immediate());
T{5} = Task(model, 'R1B_Task', 3, SchedStrategy.FCFS).on(P{5}).setThinkTime(Immediate());
T{6} = Task(model, 'R2A_Task', 4, SchedStrategy.FCFS).on(P{6}).setThinkTime(Immediate());
T{7} = Task(model, 'R2B_Task', 5, SchedStrategy.FCFS).on(P{7}).setThinkTime(Immediate());

E{1} = Entry(model, 'R1_Ref_Entry').on(T{1});
E{2} = Entry(model, 'R2_Synch_A2_Entry').on(T{2});
E{3} = Entry(model, 'R2_Synch_A5_Entry').on(T{2});
E{4} = Entry(model, 'R3_Synch_A9_Entry').on(T{3});
E{5} = Entry(model, 'R1A_Synch_A1_Entry').on(T{4});

... (89 source lines omitted; full implementation: matlab/examples/basic/)

simoptions.samples = 1e5;
simoptions.verbose = 0;
solver{4} = LQNS(model, simoptions);
AvgTable{4} = solver{4}.getAvgTable;
fprintf(1, '\nLQNS(lqsim) Results:\n');
disp(AvgTable{4});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.3 lqn_bpmn_trace

The BPMN translation with the activity trace printed.

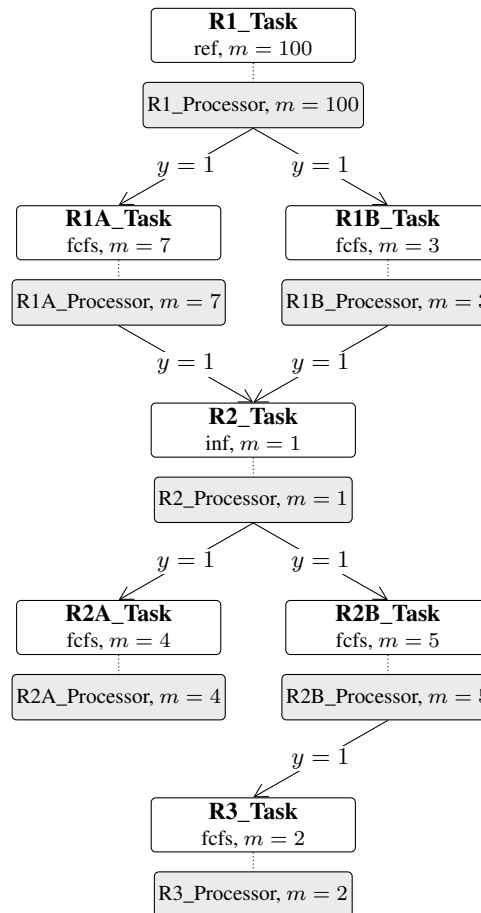


Figure 2.64: Call graph of `lqn_bpmn_trace`: 7 tasks over 7 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	7: R1_Processor (fcfs, $m = 100$), R2_Processor (inf, $m = 1$), R3_Processor (fcfs, $m = 2$), R1A_Processor (fcfs, $m = 7$), R1B_Processor (fcfs, $m = 3$), R2A_Processor (fcfs, $m = 4$), R2B_Processor (fcfs, $m = 5$)
Tasks	7: R1_Task (ref, $m = 100$), R2_Task (inf, $m = 1$), R3_Task (fcfs, $m = 2$), R1A_Task (fcfs, $m = 7$), R1B_Task (fcfs, $m = 3$), R2A_Task (fcfs, $m = 4$), R2B_Task (fcfs, $m = 5$)
Entries	16: R1_Ref_Entry, R2_Synch_A2_Entry, R2_Synch_A5_Entry, R3_Synch_A9_Entry, R1A_Synch_A1_Entry, R1A_Synch_A2_Entry, R1A_Synch_A3_Entry, R1B_Synch_A4_Entry, R1B_Synch_A5_Entry, R1B_Synch_A6_Entry, R2A_Synch_A7_Entry, R2A_Synch_A8_Entry, R2A_Synch_A11_Entry, R2B_Synch_A9_Entry, R2B_Synch_A10_Entry, R2B_Synch_A12_Entry
Activities	29, host demands A1_Empty: 0.0, A2_Empty: 0.0, A5_Empty: 0.0, A6_Empty: 0.0, A3_Empty: 0.0, A4_Empty: 0.0, E4_Empty: 0.0, A7_Empty: 0.0, ...
Calls	15 (15 synchronous, 0 asynchronous)
Think times	R1_Task: 20.0

Table 2.64: Features of `lqn_bpmn_trace`.

The example is built and solved as follows.

```
% Debug trace for LN(MVA) fix
model = LayeredNetwork('myLayeredModel');

P{1} = Processor(model, 'R1_Processor', 100, SchedStrategy.FCFS);
P{2} = Processor(model, 'R2_Processor', Inf, SchedStrategy.INF);
P{3} = Processor(model, 'R3_Processor', 2, SchedStrategy.FCFS);
P{4} = Processor(model, 'R1A_Processor', 7, SchedStrategy.FCFS);
P{5} = Processor(model, 'R1B_Processor', 3, SchedStrategy.FCFS);
P{6} = Processor(model, 'R2A_Processor', 4, SchedStrategy.FCFS);
P{7} = Processor(model, 'R2B_Processor', 5, SchedStrategy.FCFS);

T{1} = Task(model, 'R1_Task', 100, SchedStrategy.REF).on(P{1}).setThinkTime(Exp.fitMean(20));
T{2} = Task(model, 'R2_Task', Inf, SchedStrategy.INF).on(P{2}).setThinkTime(Immediate());
T{3} = Task(model, 'R3_Task', 2, SchedStrategy.FCFS).on(P{3}).setThinkTime(Immediate());
T{4} = Task(model, 'R1A_Task', 7, SchedStrategy.FCFS).on(P{4}).setThinkTime(Immediate());
T{5} = Task(model, 'R1B_Task', 3, SchedStrategy.FCFS).on(P{5}).setThinkTime(Immediate());
T{6} = Task(model, 'R2A_Task', 4, SchedStrategy.FCFS).on(P{6}).setThinkTime(Immediate());
T{7} = Task(model, 'R2B_Task', 5, SchedStrategy.FCFS).on(P{7}).setThinkTime(Immediate());

E{1} = Entry(model, 'R1_Ref_Entry').on(T{1});
E{2} = Entry(model, 'R2_Synch_A2_Entry').on(T{2});
E{3} = Entry(model, 'R2_Synch_A5_Entry').on(T{2});
E{4} = Entry(model, 'R3_Synch_A9_Entry').on(T{3});
E{5} = Entry(model, 'R1A_Synch_A1_Entry').on(T{4});
E{6} = Entry(model, 'R1A_Synch_A2_Entry').on(T{4});
E{7} = Entry(model, 'R1A_Synch_A3_Entry').on(T{4});

... (87 source lines omitted; full implementation: matlab/examples/basic/)

for i = 1:length(taskNames)
    idx = find(strcmp(AvgTable.Node, taskNames{i}));
    if ~isempty(idx)
        disp([taskNames{i} ' : Tput=' num2str(AvgTable.Tput(idx), '%.6f')]);
    end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.4 `lqn_flatph`

Method `flat.ph`: the squashed layering carrying the same composed phase-type encoding, one sub-model holding every processor and called task.

The example is built and solved as follows.

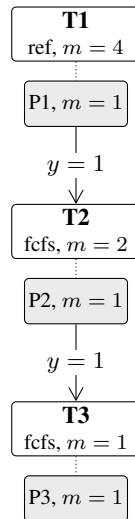


Figure 2.65: Call graph of `lqn_flatph`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (ps, $m = 1$), P2 (ps, $m = 1$), P3 (ps, $m = 1$)
Tasks	3: T1 (ref, $m = 4$), T2 (fcfs, $m = 2$), T3 (fcfs, $m = 1$)
Entries	3: E1, E2, E3
Activities	3, host demands A1: 0.2, A2: 0.2, A3: 0.2
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 1.0

Table 2.65: Features of `lqn_flatph`.

```

% Method 'flat.ph': the squashed layering with the composed phase-type encoding.
%
% A method name of SolverLN carries TWO decisions. The part before the dot is
% the LAYERING, which fixes what a submodel is; the part after it is the
% ENCODING, which fixes how an activity graph is written into that submodel. The
% four combinations are therefore 'srvn.cs', 'srvn.ph', 'flat.cs' and, as of
% this example, 'flat.ph'.
%
% Under 'flat.ph' ONE submodel holds a station for every processor and every
% called task, and a caller task is ONE closed class that visits each server it
% uses once per invocation, served there by the composed law of the demand it
% places on that server. Squashing does not conflict with the composition: a
% task station's service law is ALREADY the inflated entry law -- host demand
% plus call counts times callee service and waiting -- which the outer fixed
% point updates, exactly as lqns does under --squashed-layering.
%
% The model below is a three-deep call chain, where the middle task is at once a
% server to T1 and a caller of T3, so the two levels of the LQN accounting are
% both exercised. Against lqsim on this model 'flat.ph' reads +0.87 per cent
% where 'flat.cs' reads +2.95, 'srvn.cs' +3.59 and lqns +4.28.

model = LayeredNetwork('flatphExample');

P1 = Processor(model, 'P1', 1, SchedStrategy.PS);
P2 = Processor(model, 'P2', 1, SchedStrategy.PS);
P3 = Processor(model, 'P3', 1, SchedStrategy.PS);

... (18 source lines omitted; full implementation: matlab/examples/basic/)

solver = SolverLN(model, 'method', m{1}, 'verbose', false);
AvgTable = solver.getAvgTable();
line_printf('\n--- method=%s (built as %s, %d submodel(s))\n', ...

```

```

        m{1}, solver.lnmethod, length(solver.ensemble));
    disp(AvgTable);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.5 lqn_fork_open_arrival

An AND-fork under an open arrival stream.

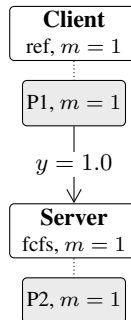


Figure 2.66: Call graph of `lqn_fork_open_arrival`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (inf, $m = 1$), P2 (inf, $m = 1$)
Tasks	2: Client (ref, $m = 1$), Server (fcfs, $m = 1$)
Entries	3: CE, SE, OE
Activities	9, host demands CA: 0.5, RA1: 0.2, RA2: 0.3, RA3: 0.4, RA4: 0.1, OA1: 0.2, OA2: 0.3, OA3: 0.4, ...
Calls	1 (1 synchronous, 0 asynchronous)
Think times	Client: 1.0

Table 2.66: Features of `lqn_fork_open_arrival`.

The example is built and solved as follows.

```

clear P T E A solver AvgTable

% An AND fork/join on a task that ALSO receives an entry-level open arrival.
% The Server task has two entries with a fork each: SE is called by the closed
% Client (rendezvous), OE takes an exogenous Poisson stream of rate 0.1. OE
% carries no reply activity because an open-arrival entry is send-no-reply,
% which lqns enforces.
%
% External references on this model: lqns 6.2.28 (valid) gives Client
% throughput 0.413391, Server task throughput 0.513391, OE throughput 0.1 with
% open-wait 1.22917 and entry service 0.841667; lqsim (T=5e5, seed 1234) gives
% 0.4154, 0.52242 and open-wait 1.10546.
%
% SolverLN does NOT solve this combination: the fork-join transform mints its
% own Source and collides with the Source the open stream is routed through.
% See fj_mixed_openclosed.m for the flat fork-join with both an open and a
% closed class, which the MVA path does solve.

model = LayeredNetwork('lqnForkOpenArrival');
%%
P{1} = Processor(model, 'P1', Inf, SchedStrategy.INF);
T{1} = Task(model, 'Client', 1, SchedStrategy.REF).on(P{1});
T{1}.setThinkTime(Exp.fitMean(1));

```

```

E{1} = Entry(model, 'CE').on(T{1});
%%
P{2} = Processor(model, 'P2', Inf, SchedStrategy.INF);

... (28 source lines omitted; full implementation: matlab/examples/basic/)

AvgTableLN = SolverLN(model, 'verbose', false).getAvgTable();
fprintf(1, '\nLN Results:\n');
disp(AvgTableLN);
catch ME
    fprintf(1, '\nLN refuses this model: %s\n', ME.message);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.6 lqn_init

A layered model whose SolverLN run is warm-started from the LQNS solution, and the elapsed time that initialization saves.

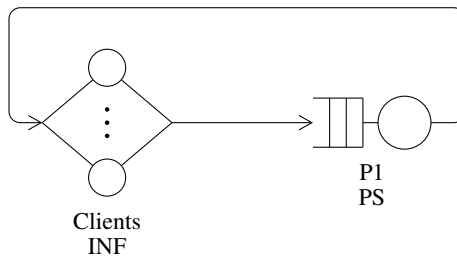


Figure 2.67: Topology of lqn_init: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: T1 (closed, $N = 10$)
Scheduling	Clients: INF; P1: PS
Service	Clients – T1: Exp(0.008971) P1 – T1: APH

Table 2.67: Features of lqn_init.

The example is built and solved as follows.

```

clear model solver options AvgTable RD

fprintf(1, 'This example illustrates the initialization of LN using the output of LQNS.\n');

cwd = fileparts(which(mfilename));
model = LayeredNetwork.parseXML([cwd, filesep, 'lqn_serial.xml']);

%% LQNS, whose solution is what an LN run can be initialized from
options = LQNS.defaultOptions;
options.keep = true; % keep the intermediate XML files of the translation

if SolverLQNS.isAvailable()
    solver{1} = LQNS(model, options);
    AvgTable{1} = solver{1}.getAvgTable();
    fprintf(1, '\nLQNS Results:\n');
    disp(AvgTable{1});
else
    fprintf(1, '\nLQNS solver not available - skipping.\n');
end

%% LN without initialization, for the elapsed time the initialization saves

```

```

fprintf(1, '\nSolve with LN without initialization:\n');
solver{2} = LN(model, @(x) MVA(x));
tic;
AvgTable{2} = solver{2}.getAvgTable();
timeElapsed = toc;

... (7 source lines omitted; full implementation: matlab/examples/basic/)

RD = FLD(ensemble{3}).getCdfRespT();
fprintf(1, 'RD (CDF of response times):\n');
disp(RD);
else
    fprintf(1, 'Model ensemble has fewer than three layers.\n');
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.7 lqn_jsq

Join-the-shortest-queue dispatching between replicated tasks.

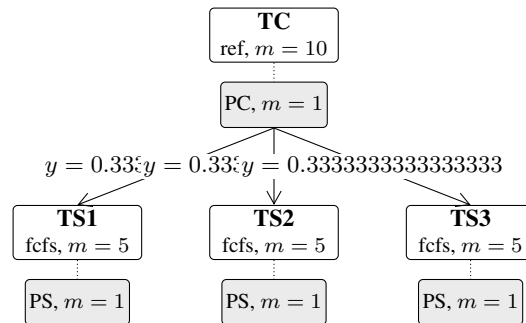


Figure 2.68: Call graph of `lqn_jsq`: 4 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: PC (inf, $m = 1$), PS (ps, $m = 1$)
Tasks	4: TC (ref, $m = 10$), TS1 (fcfs, $m = 5$), TS2 (fcfs, $m = 5$), TS3 (fcfs, $m = 5$)
Entries	4: EC, ES1, ES2, ES3
Activities	4, host demands AC: 0.5, AS1: 1.0, AS2: 1.0, AS3: 1.0
Calls	3 (3 synchronous, 0 asynchronous)
Think times	TC: 5.0

Table 2.68: Features of `lqn_jsq`.

The example is built and solved as follows.

```

function lqn_jsq()
% LQN_JSQ Join-the-shortest-queue call dispatch over a set of target tasks.
%
% The twin of LQN_RROBIN: a client task issues its synchronous calls to three
% interchangeable server tasks, but the target is the one holding the fewest
% jobs at dispatch time rather than the next one in cyclic order. What JSQ adds
% over round-robin is that the dispatch reacts to the state of the servers, so
% it also absorbs asymmetry in the service times, not only the variance of the
% branching.
%
% Only the squashed ('flat') layering can express this, and only a layer solver
% with state-dependent routing honours it (SSA here).

model = buildJSQ();

```



```

options = SolverLN.defaultOptions;
options.config.layering = 'flat';
options.verbose = false;
AvgTable = SolverLN(model, @(m) SolverSSA(m, 'verbose', false), options).getAvgTable();
disp(AvgTable);
end

function model = buildJSQ()
model = LayeredNetwork('LQN-JSQ');

PC = Processor(model, 'PC', 1, SchedStrategy.INF);

... (14 source lines omitted; full implementation: matlab/examples/basic/)

AC = Activity(model, 'AC', Exp(2)).on(TC).boundTo(EC);
AC.synchCallJSQ({ES1, ES2, ES3}, 1);
Activity(model, 'AS1', Exp(1)).on(TS1).boundTo(ES1).repliesTo(ES1);
Activity(model, 'AS2', Exp(1)).on(TS2).boundTo(ES2).repliesTo(ES2);
Activity(model, 'AS3', Exp(1)).on(TS3).boundTo(ES3).repliesTo(ES3);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.8 lqn_moment3

The third moment of the entry service time, and how it propagates up the layers.

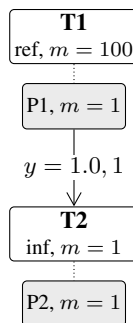


Figure 2.69: Call graph of `lqn_moment3`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 100$), T2 (inf, $m = 1$)
Entries	3: E1, E2, E3
Activities	5, host demands A1: 1.0, A20: 1.0, A21: 1.0, A22: 1.0, A3: 1.0
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 10.0
Solvers	FLD

Table 2.69: Features of `lqn_moment3`.

The example is built and solved as follows.

```

model = LayeredNetwork('myLayeredModel');

% definition of processors, tasks and entries
P1 = Processor(model, 'P1', 1, SchedStrategy.PS);
T1 = Task(model, 'T1', 100, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);

```

```

P2 = Processor(model, 'P2', 1, SchedStrategy.PS);
T2 = Task(model, 'T2', 1, SchedStrategy.INF).on(P2);
E2 = Entry(model, 'E2').on(T2);

E3 = Entry(model, 'E3').on(T2);

% definition of activities
T1.setThinkTime(Erlang.fitMeanAndOrder(10,1));

A1 = Activity(model, 'A1', Exp(1)).on(T1).boundTo(E1).synchCall(E2).synchCall(E3,1);

A20 = Activity(model, 'A20', Exp(1)).on(T2).boundTo(E2);
A21 = Activity(model, 'A21', Exp(1)).on(T2);
A22 = Activity(model, 'A22', Exp(1)).on(T2).repliesTo(E2);
T2.addPrecedence(ActivityPrecedence.Serial(A20, A21, A22));

A3 = Activity(model, 'A3', Exp(1)).on(T2).boundTo(E3).repliesTo(E3);

% LN solver with default method

... (13 source lines omitted; full implementation: matlab/examples/basic/)

solveroptions2 = Solver.defaultOptions;
solveroptions2.verbose = 0;
solver{2} = LN(model, @(1)MVA(1,solveroptions2), lnoptions2);
AvgTable{2} = solver{2}.getAvgTable;
fprintf(1, '\nLN(moment3) Results:\n');
disp(AvgTable{2});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.9 lqn_multi_solvers

One layered model solved by every layered solver, side by side.

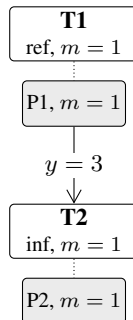


Figure 2.70: Call graph of `lqn_multi_solvers`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (inf, $m = 1$), P2 (inf, $m = 1$)
Tasks	2: T1 (ref, $m = 1$), T2 (inf, $m = 1$)
Entries	2: E1, E2
Activities	2, host demands A1: 1.0, A2: 1.0
Calls	1 (1 synchronous, 0 asynchronous)
Think times	T1: 0.0001

Table 2.70: Features of `lqn_multi_solvers`.

The example is built and solved as follows.

```

model = LayeredNetwork('LQN1');

% definition of processors, tasks and entries
P1 = Processor(model, 'P1', Inf, SchedStrategy.INF);
T1 = Task(model, 'T1', 1, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);

P2 = Processor(model, 'P2', Inf, SchedStrategy.INF);
T2 = Task(model, 'T2', Inf, SchedStrategy.INF).on(P2);
E2 = Entry(model, 'E2').on(T2);

% definition of activities
T1.setThinkTime(Erlang.fitMeanAndOrder(0.0001,2));

A1 = Activity(model, 'A1', Exp(1.0)).on(T1).boundTo(E1).synchCall(E2,3);
A2 = Activity(model, 'A2', APH.fitMeanAndSCV(1,10)).on(T2).boundTo(E2).repliesTo(E2);

%%
% instantiate solvers
options = LQNS.defaultOptions;
options.keep = true;
options.verbose = 1;
%options.method = 'lqsim';
%options.samples = 1e4;
lqnssolver = LQNS(model, options);
AvgTableLQNS = lqnssolver.getAvgTable;

... (23 source lines omitted; full implementation: matlab/examples/basic/)

disp(AvgTable{2});

% this method adapts with the features of each layer
%solver{2} = LN(model, @(model) LINE(model, LINE.defaultOptions), lnoptions);
%AvgTable{2} = solver{2}.getAvgTable
%AvgTable{2}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.10 lqn_ofbiz

The Apache OFBiz case study, read from its .lqnx file.

The example is built and solved as follows.

```

%% This example is temporarily disabled
fprintf(1,'This example illustrates the solution of a moderately large LQN.\n')

cwd = fileparts(which(mfilename));
model = LayeredNetwork.parseXML([cwd,filesep,'lqn_ofbiz.xml']);

options = LQNS.defaultOptions;
options.keep = true; % uncomment to keep the intermediate XML files generated while
    translating the model to LQNS

%% Solve with LQNS (disabled)
%solver{1} = LQNS(model);
%AvgTable{1} = solver{1}.getAvgTable;
%fprintf(1, '\nLQNS Results:\n');
%disp(AvgTable{1});

%% Solve with LN without initialization
% Pin the LN seed: SolverLN randomizes its stochastic-layer seed base when
% options.seed is unset, which makes the fixed point of this large model
% irreproducible run to run.
lnoptions = LN.defaultOptions;
lnoptions.seed = 23000;
solver{2} = LN(model, @(x) NC(x,'verbose',false), lnoptions);
Tnoinit = tic;

```

Processors	9: FrontEnd_CPU_Processor (ps, $m = 1$), USAGE_DELAY_Processor (inf, $m = 1$), UsageScenario_userType1_1_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_main_345_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_login_345_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_checkLogin_345_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_logout_345_Processor (fcfs, $m = 1$), UsageScenario_userType2_7_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_quickadd_345_Processor (fcfs, $m = 1$)
Tasks	9: FrontEnd_CPU_Task (inf, $m = 1$), USAGE_DELAY_Task (inf, $m = 1$), UsageScenario_userType1_1_Task (ref, $m = 10$), RequestHandler_HandlerIF_main_345_Task (inf, $m = 1$), RequestHandler_HandlerIF_login_345_Task (inf, $m = 1$), RequestHandler_HandlerIF_checkLogin_345_Task (inf, $m = 1$), RequestHandler_HandlerIF_logout_345_Task (inf, $m = 1$), UsageScenario_userType2_7_Task (ref, $m = 10$), RequestHandler_HandlerIF_quickadd_345_Task (inf, $m = 1$)
Entries	14: FrontEnd_CPU_Entry, InternalAction_main_Iu1-wMhoEeKON4DtRoKCMw_34_50_Entry, InternalAction_login_YgxRGw26EeSPwb7XgvxhWQ_34_50_Entry, InternalAction_CheckLogin_vHMZsMhoEeKON4DtRoKCMw_34_50_Entry, InternalAction_logout_j-2E4MhrEeKON4DtRoKCMw_34_50_Entry, InternalAction_addcart_5JEHQMhoEeKON4DtRoKCMw_34_50_Entry, USAGE_DELAY0_Entry, UsageScenario_userType1_1_Entry, RequestHandler_HandlerIF_main_345_Entry, RequestHandler_HandlerIF_login_345_Entry, RequestHandler_HandlerIF_checkLogin_345_Entry, RequestHandler_HandlerIF_logout_345_Entry, UsageScenario_userType2_7_Entry, RequestHandler_HandlerIF_quickadd_345_Entry
Activities	40, host demands FrontEnd_CPU_Activity: 0.0, InternalAction_main_Iu1-wMhoEeKON4DtRoKCMw_34_50_Activity: 0.01, InternalAction_login_YgxRGw26EeSPwb7XgvxhWQ_34_50_Activity: 0.01, InternalAction_CheckLogin_vHMZsMhoEeKON4DtRoKCMw_34_50_Activity: 0.01, InternalAction_logout_j-2E4MhrEeKON4DtRoKCMw_34_50_Activity: 0.01, InternalAction_addcart_5JEHQMhoEeKON4DtRoKCMw_34_50_Activity: 0.01, USAGE_DELAY0_Activity: 0.0, Start2: 0.0, ...
Calls	19 (19 synchronous, 0 asynchronous)
Think times	UsageScenario_userType1_1_Task: 10.0, UsageScenario_userType2_7_Task: 14.0

Table 2.71: Features of `lqn_ofbiz`.

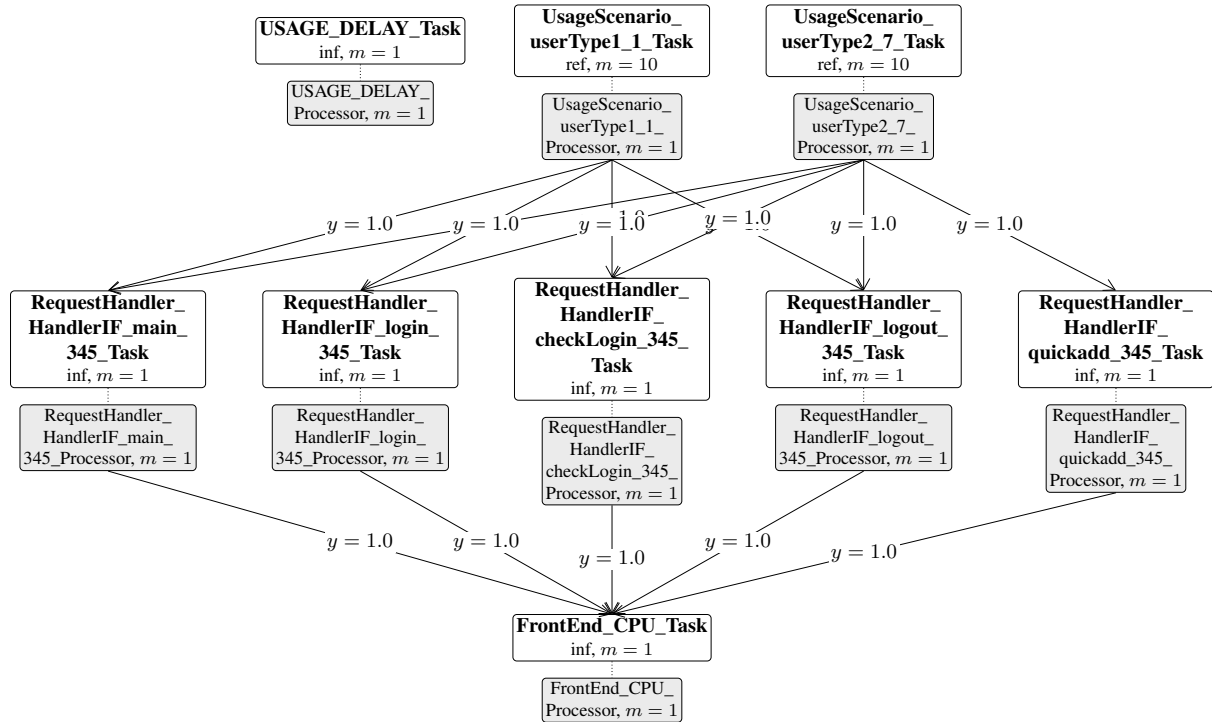


Figure 2.71: Call graph of `lqn_ofbiz`: 9 tasks over 9 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```
AvgTable{2} = solver{2}.getAvgTable;
fprintf(1, '\nLN(NC) Results:\n');
disp(AvgTable{2});
Tnoinit = toc(Tnoinit)
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.11 `lqn_open_arrival`

An open arrival stream into an entry, instead of a closed reference task.

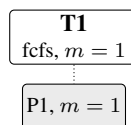


Figure 2.72: Call graph of `lqn_open_arrival`: 1 task over 1 processor, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	1: P1 (ps, $m = 1$)
Tasks	1: T1 (fcfs, $m = 1$)
Entries	1: E1
Activities	1, host demands A1: 1.6

Table 2.72: Features of `lqn_open_arrival`.

The example is built and solved as follows.

```

clear P T E A solver AvgTable

% An entry-level open arrival: an exogenous Poisson stream that is not a call.
% Rate 0.2 against a mean service of 1.6 is 0.32 of the host. Nothing else
% reaches T1, so T1 has no task layer and SolverLN represents the stream by the
% thread pool it drives -- a closed chain of mult(T1) jobs whose surrogate delay
% is closed on the known rate, the same construction a forwarding target gets
% (updateThinkTimes / updateThinkTimesPH). Reported: entry throughput 0.2, host
% utilization 0.32, entry response time 1.6, which is what lqns gives exactly and
% lqsim (0.192-0.200) and LDES (0.19986 / 0.31957 / 1.599) confirm. Both LN
% methods agree. Placing an open class on the host layer instead would double the
% load, because that chain has no other delay to cycle against: that was the
% earlier reading, 0.425 / 0.68 under 'srvn' and 0.625 / 1.000 under 'srvnph'.
% See lqn_fork_open_arrival.m for the combination with a fork.

model = LayeredNetwork('openArrivalLQN');
%%
P{1} = Processor(model, 'P1', 1, SchedStrategy.PS);
T{1} = Task(model, 'T1', 1, SchedStrategy.FCFS).on(P{1});
T{1}.setThinkTime(Immediate());
E{1} = Entry(model, 'E1').on(T{1});
E{1}.setArrival(Exp(0.2));
%%
A{1} = Activity(model, 'A1', Exp.fitMean(1.6)).on(T{1}).boundTo(E{1}).repliesTo(E{1});
%%
solver = SolverLN(model, 'verbose', false);
AvgTable = solver.getAvgTable();
fprintf(1, '\nLN Results:\n');
disp(AvgTable);

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.12 lqn_paramident

Parameter identification on a layered model, fitting host demands to measurements.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

% LQN_PARAMIDENT Identify hidden LQN parameters from measured performance.
%
% Demonstrates infer_lqn, an Extended Kalman Filter that tracks hidden LQN
% parameters (host demands, think times) from measurable performance data,
% following Zheng, Yang, Woodside, Litoiu, Iszlai, "Tracking Time-Varying
% Parameters in Software Systems with Extended Kalman Filters", CASCON 2005.
%
% Here two parameters are hidden: the reference-task think time (the paper's Z)
% and the P2 host demand of activity AS3 (the paper's service demand S_d). The
% measurable vector is [R(E1), U(P1), U(P2)] (a user response time and two
% processor utilizations). We synthesise a measurement sequence with a step
% change plus noise, then recover the parameter trajectory.

clear; lineStart;
rng(12345);

% ---- base LQN (mirrors lqn_basic.m) -----
model = LayeredNetwork('paramident_LQN');
P1 = Processor(model, 'P1', 2, SchedStrategy.PS);
P2 = Processor(model, 'P2', 3, SchedStrategy.PS);
T1 = Task(model, 'T1', 50, SchedStrategy.REF).on(P1).setThinkTime(Exp(1/2));
T2 = Task(model, 'T2', 50, SchedStrategy.FCFS).on(P1).setThinkTime(Exp(1/3));
T3 = Task(model, 'T3', 25, SchedStrategy.FCFS).on(P2).setThinkTime(Exp(1/4));
E1 = Entry(model, 'E1').on(T1);
E2 = Entry(model, 'E2').on(T2);

```

```

E3 = Entry(model, 'E3').on(T3);

... (43 source lines omitted; full implementation: matlab/examples/basic/)

    norm(info.e(:, k)));
end
fprintf('\nFinal estimate: Z = %.4f (true %.4f), Sd = %.5f (true %.5f)\n', ...
    info.ahat(1, end), aTrueSeq(1, end), info.ahat(2, end), aTrueSeq(2, end));
fprintf('RMS parameter tracking error Ea = %.4g, prediction error Er = %.4g\n', ...
    info.Ea, info.Er);

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.13 lqn_rr robin

Round-robin dispatching between replicated tasks.

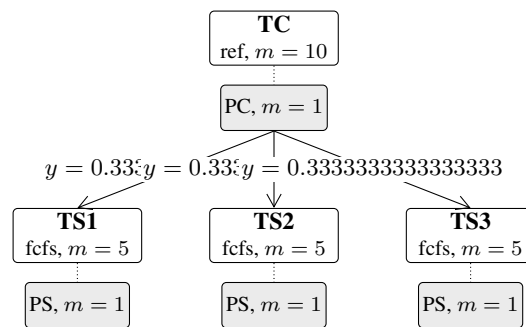


Figure 2.73: Call graph of `lqn_rr robin`: 4 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: PC (inf, $m = 1$), PS (ps, $m = 1$)
Tasks	4: TC (ref, $m = 10$), TS1 (fcfs, $m = 5$), TS2 (fcfs, $m = 5$), TS3 (fcfs, $m = 5$)
Entries	4: EC, ES1, ES2, ES3
Activities	4, host demands AC: 0.5, AS1: 1.0, AS2: 1.0, AS3: 1.0
Calls	3 (3 synchronous, 0 asynchronous)
Think times	TC: 5.0

Table 2.73: Features of `lqn_rr robin`.

The example is built and solved as follows.

```

function lqn_rr robin()
% LQN_RROBIN Round-robin call dispatch over a set of target tasks.
%
% A client task issues its synchronous calls to three interchangeable server
% tasks in cyclic order rather than by probabilistic branching. The two models
% carry the same aggregate call rate; what differs is that round-robin removes
% the variance of the branching, which smooths the server queues.
%
% Only the squashed ('flat') layering can express this: under 'srvn' each
% server task lives in its own submodel and is replaced, in the client's
% submodel, by a surrogate delay, so no node ever has arcs to more than one of
% them. The layer solver must also implement state-dependent routing (SSA
% here); MVA, NC and FLD are rejected rather than silently returning the
% probabilistic split.

model = buildRoundRobin();

options = SolverLN.defaultOptions;

```

```

options.config.layering = 'flat';
options.verbose = false;
AvgTable = SolverLN(model, @(m) SolverSSA(m, 'verbose', false), options).getAvgTable();
disp(AvgTable);
end

function model = buildRoundRobin()
model = LayeredNetwork('LQN-RRobin');

... (18 source lines omitted; full implementation: matlab/examples/basic/)

AC = Activity(model, 'AC', Exp(2)).on(TC).boundTo(EC);
AC.synchCallRoundRobin({ES1, ES2, ES3}, 1);
Activity(model, 'AS1', Exp(1)).on(TS1).boundTo(ES1).repliesTo(ES1);
Activity(model, 'AS2', Exp(1)).on(TS2).boundTo(ES2).repliesTo(ES2);
Activity(model, 'AS3', Exp(1)).on(TS3).boundTo(ES3).repliesTo(ES3);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.14 lqn_serial

A serial call chain across three layers, with an immediate activity.

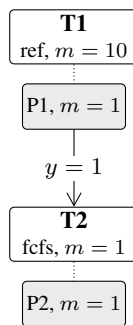


Figure 2.74: Call graph of `lqn_serial`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 10$), T2 (fcfs, $m = 1$)
Entries	2: E1, E2
Activities	4, host demands AS1: 1.6, AS2: 0.0, AS3: 5.0, AS4: 1.0
Calls	1 (1 synchronous, 0 asynchronous)
Think times	T1: 100.0

Table 2.74: Features of `lqn_serial`.

The example is built and solved as follows.

```

clear P T E A solver AvgTable

fprintf(1, 'This example illustrates the execution on a layered queueing network model.\n')
fprintf(1, 'Performance indexes now refer to processors, tasks, entries, and activities.\n')
fprintf(1, 'Indexes refer to the submodel (layer) where the processor or task acts as a server\n')
fprintf(1, 'NaN indexes indicate that the metric is not supported by the node type.\n')

%cwd = fileparts(which(mfilename));
%model = LayeredNetwork.parseXML([cwd, filesep, 'lqn_serial.xml']);
%%

```



```

model = LayeredNetwork('myLayeredModel');

P{1} = Processor(model, 'P1', 1, SchedStrategy.PS);
P{2} = Processor(model, 'P2', 1, SchedStrategy.PS);

T{1} = Task(model, 'T1', 10, SchedStrategy.REF).on(P{1}).setThinkTime(Exp.fitMean(100));
T{2} = Task(model, 'T2', 1, SchedStrategy.FCFS).on(P{2}).setThinkTime(Immediate());

E{1} = Entry(model, 'E1').on(T{1});
E{2} = Entry(model, 'E2').on(T{2});

A{1} = Activity(model, 'AS1', Exp.fitMean(1.6)).on(T{1}).boundTo(E{1});
A{2} = Activity(model, 'AS2', Immediate()).on(T{1}).synchCall(E{2},1);
A{3} = Activity(model, 'AS3', Exp.fitMean(5)).on(T{2}).boundTo(E{2});
A{4} = Activity(model, 'AS4', Exp.fitMean(1)).on(T{2}).repliesTo(E{2});

... (12 source lines omitted; full implementation: matlab/examples/basic/)

%AvgTable{2}
%CallAvgTable{2}

AvgTable{3} = LN(model,'verbose',false).getAvgTable;
fprintf(1, '\nLN Results:\n');
disp(AvgTable{3});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.15 lqn_server_pools

A layered model with heterogeneous server pools, compared with the compatibility representation that treats the servers as interchangeable.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

function lqn_server_pools()
% LQN_SERVER_POOLS A processor whose servers are not interchangeable.
%
% P1 declares three servers, but they are not a homogeneous pool: S1 is
% dedicated to task T2, S3 to task T3, and only S2 can take either. Neither task
% can therefore reach more than two of the three servers, and the model is a
% different system from a plain multiplicity-3 processor even though it holds
% the same number of servers.
%
%      P1 (3 servers)      S1 --- T2
%                          S2 --< T2, T3
%                          S3 --- T3
%
% The pools are declared with ServerType, the same class the heterogeneous
% queueing station uses, with the compatible entities being the OPERANDS of the
% layered server: the tasks of a processor, or the entries of a task.
%
% SolverLN lowers the declaration to the activated-server rate of
% SN_COMPAT_RATE, carried onto the layer station as a joint dependence, so a
% compatibility declaration is an APPROXIMATION inside a layer and is admitted
% only under the class-switching layerings ('srvn.cs', 'flat.cs').
%
% Reference:
% Dorsman, Gardner (2024). New directions in pass-and-swap queues. Queueing
% Systems 107(3):205-256, Fig. 1.
%
... (31 source lines omitted; full implementation: matlab/examples/basic/)

```

```

end

fprintf('\n--- %s pool on P1 ---\n', kind);
LN(model, @ (m) MVA(m), opt).getAvgTable
end
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.16 lqn_setup

Task setup and initialization activities before service.

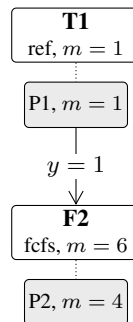


Figure 2.75: Call graph of `lqn_setup`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (inf, $m = 1$), P2 (fcfs, $m = 4$)
Tasks	2: T1 (ref, $m = 1$), F2 (fcfs, $m = 6$)
Entries	2: E1, E2
Activities	2, host demands A1: 1.0, A2: 0.3333333333333333
Calls	1 (1 synchronous, 0 asynchronous)

Table 2.75: Features of `lqn_setup`.

The example is built and solved as follows.

```

% example of layered model with a task paying setup and delay-off times
fprintf(1, 'Example of layered model with a task paying setup and delay-off times\n');

model = LayeredNetwork('setup_example');

% definition of processors, tasks and entries
P1 = Processor(model, 'P1', Inf, SchedStrategy.INF);
P2 = Processor(model, 'P2', 4, SchedStrategy.FCFS);

T1 = Task(model, 'T1', 1, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);

%T2 = Task(model, 'T2', 1, SchedStrategy.FCFS).on(P2);
T2 = SetupTask(model, 'F2', 6, SchedStrategy.FCFS).on(P2).setThinkTime(Exp.fitMean(8.0));
T2.setSetupTime(Exp(1.0));
T2.setDelayOffTime(Exp(2.0));

E2 = Entry(model, 'E2').on(T2);

% T3 = Task(model, 'T3', 1, SchedStrategy.FCFS).on(P2);
% E3 = Entry(model, 'E3').on(T3);

% definition of activities

```

```

A1 = Activity(model, 'A1', Exp(1.0)).on(T1).boundTo(E1).synchCall(E2,1);
A2 = Activity(model, 'A2', Exp(3.0)).on(T2).boundTo(E2).repliesTo(E2);
% A3 = Activity(model, 'A3', Exp(5.0)).on(T3).boundTo(E3).repliesTo(E3);

... (6 source lines omitted; full implementation: matlab/examples/basic/)

% options2 = MAM.defaultOptions;
% options2.verbose = 0;
solver{1} = LN(model, @(m) MVA(m), lnoptions);
AvgTable{1} = solver{1}.getAvgTable;
fprintf(1, '\nLN(MVA) Results:\n');
disp(AvgTable{1});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.17 lqn_sockshop

The Sock Shop microservice benchmark as a layered model.

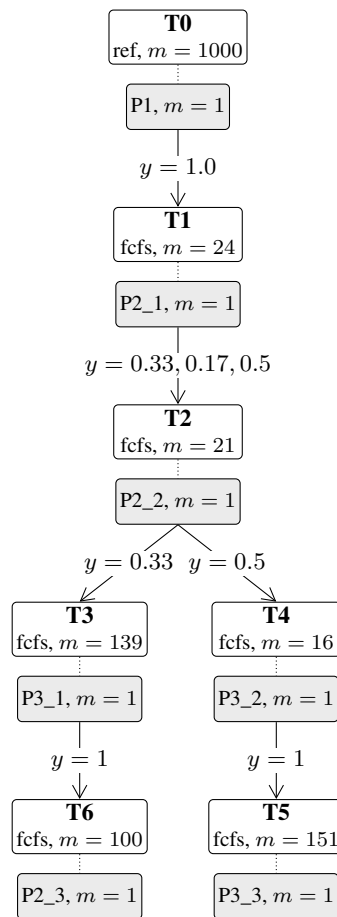


Figure 2.76: Call graph of `lqn_sockshop`: 7 tasks over 7 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

The example is built and solved as follows.

```

clear P T E A solver AvgTable

fprintf(1, 'This example illustrates a Sock Shop microservice LQN model.\n')
fprintf(1, 'It includes processor replication and fan-in/fan-out.\n')

```

Processors	7: P1 (inf, $m = 1$), P2_1 (ps, $m = 1$), P2_2 (ps, $m = 1$), P2_3 (ps, $m = 1$), P3_1 (ps, $m = 1$), P3_2 (ps, $m = 1$), P3_3 (ps, $m = 1$)
Tasks	7: T0 (ref, $m = 1000$), T1 (fcfs, $m = 24$), T2 (fcfs, $m = 21$), T6 (fcfs, $m = 100$), T3 (fcfs, $m = 139$), T4 (fcfs, $m = 16$), T5 (fcfs, $m = 151$)
Entries	12: E0, E1, E2, E3, E4, E11, E5, E6, E7, E8, E9, E10
Activities	24, host demands AS: 5e-08, AS0: 5e-08, AS1: 5e-08, AS2: 0.0022216, AH1: 5e-08, AH2: 0.0021319, AH3: 5e-08, AH4: 0.0037561, ...
Calls	14 (14 synchronous, 0 asynchronous)
Think times	T0: 7.0

Table 2.76: Features of `lqn_sockshop`.

```

model = LayeredNetwork('sockshop');

%% Processors
P{1} = Processor(model, 'P1', 1, SchedStrategy.INF);
P{2} = Processor(model, 'P2_1', 1, SchedStrategy.PS, 0.1);
P{2}.setReplication(2);
P{3} = Processor(model, 'P2_2', 1, SchedStrategy.PS, 0.1);
P{4} = Processor(model, 'P2_3', 1, SchedStrategy.PS, 0.1);
P{5} = Processor(model, 'P3_1', 1, SchedStrategy.PS, 0.1);
P{6} = Processor(model, 'P3_2', 1, SchedStrategy.PS, 0.1);
P{7} = Processor(model, 'P3_3', 1, SchedStrategy.PS, 0.1);

%% Tasks
T{1} = Task(model, 'T0', 1000, SchedStrategy.REF).on(P{1}).setThinkTime(Exp.fitMean(7.0));
% reference task
T{2} = Task(model, 'T1', 24, SchedStrategy.FCFS).on(P{2}).setThinkTime(Immediate());
% edge router
T{2}.setFanOut('T2', 1);
T{3} = Task(model, 'T2', 21, SchedStrategy.FCFS).on(P{3}).setThinkTime(Immediate());
% front end
T{3}.setFanIn('T1', 1);
T{3}.setFanOut('T3', 1);
T{3}.setFanOut('T4', 1);
T{4} = Task(model, 'T6', 100, SchedStrategy.FCFS).on(P{4}).setThinkTime(Immediate());
% cartdb

... (81 source lines omitted; full implementation: matlab/examples/basic/)

T{7}.addPrecedence(ActivityPrecedence.Serial(A{23}, A{24}));

%% Solve using LN with MVA
AvgTable{1} = LN(model, 'verbose', false).getAvgTable;
fprintf(1, '\nLN(MVA) Results:\n');
disp(AvgTable{1});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.18 `lqn_srvnph`

Method `srvnph`: each entry activity graph composed into one phase-type service law, so an AND fork-join and a loop survive as a distribution rather than as routing.

The example is built and solved as follows.

```

% Method 'srvn.ph': the activity graph of an entry as a phase-type server law.
%
% The default LN method turns every activity graph into routing: one class per
% task, entry, activity and call, plus Fork, Join, Router and ClassSwitch nodes.
% Method 'srvn.ph' composes each entry graph into a single phase-type law by the
% exact series-parallel reduction of Workflow.toPH, so a layer becomes a
% two-station cycle, Delay('Clients') + Queue(server), with one closed class per
% caller task. The sequencing survives as a distribution rather than as routing.
%

```

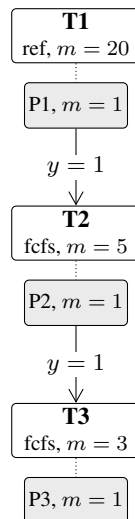


Figure 2.77: Call graph of `lqn_srvnph`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (inf, $m = 1$), P2 (ps, $m = 1$), P3 (ps, $m = 1$)
Tasks	3: T1 (ref, $m = 20$), T2 (fcfs, $m = 5$), T3 (fcfs, $m = 3$)
Entries	4: E1, E2, E3, E4
Activities	10, host demands A1: 0.1, A20: 0.2, A21: 0.3, A22: 0.2, A23: 0.1, A30: 0.1, A31: 0.2, A32: 0.1, ...
Calls	3 (3 synchronous, 0 asynchronous)
Think times	T1: 1.0

Table 2.77: Features of `lqn_srvnph`.

```
% The model below exercises the two constructs the reduction handles exactly and
% the default routing encoding only approximates: an AND fork-join, whose branch
% times are a maximum and not a sum, and a geometric loop.

model = LayeredNetwork('srvnphExample');

P1 = Processor(model, 'P1', 1, SchedStrategy.INF);
T1 = Task(model, 'T1', 20, SchedStrategy.REF).on(P1);
T1.setThinkTime(Exp.fitMean(1.0));
E1 = Entry(model, 'E1').on(T1);

P2 = Processor(model, 'P2', 1, SchedStrategy.PS);
T2 = Task(model, 'T2', 5, SchedStrategy.FCFS).on(P2);
E2 = Entry(model, 'E2').on(T2);
E3 = Entry(model, 'E3').on(T2);

P3 = Processor(model, 'P3', 1, SchedStrategy.PS);

... (43 source lines omitted; full implementation: matlab/examples/basic/)

disp(AvgTable{2});

nclasses_default = sum(cellfun(@(m) m.getNumberOfClasses(), solver{1}.ensemble));
nclasses_srvnph = sum(cellfun(@(m) m.getNumberOfClasses(), solver{2}.ensemble));
fprintf(1, '\nLayer classes: default %d, srvn.ph %d. Runtime: %.3fs vs %.3fs (%.2fx).\n', ...
    nclasses_default, nclasses_srvnph, t_default, t_srvnph, t_default/t_srvnph);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.19 lqn_transient

The transient response of a layered model, through getTranAvg.

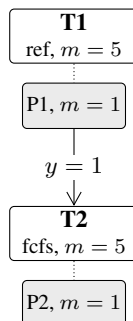


Figure 2.78: Call graph of lqn_transient: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 5$), T2 (fcfs, $m = 5$)
Entries	2: E1, E2
Activities	2, host demands A1: 0.5, A2: 0.3333333333333333
Calls	1 (1 synchronous, 0 asynchronous)
Think times	T1: 1.0

Table 2.78: Features of lqn_transient.

The example is built and solved as follows.

```
% LQN_TRANSIENT Transient (time-dependent) analysis of a layered network.
%
```

```
% Demonstrates LN.getTranAvg, which returns the transient mean queue
% length, utilization and throughput of every ensemble layer over time. The
% traces are assembled block-diagonally: layer e occupies a disjoint block of
% rows (its stations) and columns (its classes). Transient output is only
% produced by transient-capable per-layer solvers (Fluid, CTMC, SSA); here the
% layers are solved with the fluid ODE solver. Do not set a 'timespan' on the
% per-layer factory: the steady-state fixed point rejects it, and getTranAvg
% auto-selects the timespan per layer.
%
% getTranAvg first converges the LN fixed point (getAvg), pinning the
% inter-layer demands to equilibrium, then runs each layer's transient with
% those demands frozen. The initial point is the layer's default state (all
% closed jobs at the reference station, via State.initDefault), NOT the
% converged occupancy, so each curve relaxes from all-at-reference to the
% layer steady state. Use setState to seed a different start.
%
% The model is a simple 3-layer LQN: a reference task T1 on processor P1 whose
% activity synchronously calls entry E2 of task T2 on processor P2. The three
% ensemble layers are the two processor (host) layers and the T2 task layer.

model = LayeredNetwork('lqn_transient');

P1 = Processor(model, 'P1', 1, SchedStrategy.PS);
P2 = Processor(model, 'P2', 1, SchedStrategy.PS);

... (59 source lines omitted; full implementation: matlab/examples/basic/)

end
title(sprintf('Layer %d', e));
xlabel('time t');
ylabel('E[N](t)');
legend('Location', 'best');
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.20 lqn_twotasks

Two tasks over separate processors, the two-layer archetype.

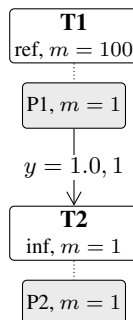


Figure 2.79: Call graph of `lqn_twotasks`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

The example is built and solved as follows.

```
model = LayeredNetwork('myLayeredModel');

% definition of processors, tasks and entries
P1 = Processor(model, 'P1', 1, SchedStrategy.PS);
T1 = Task(model, 'T1', 100, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);
```

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 100$), T2 (inf, $m = 1$)
Entries	3: E1, E2, E3
Activities	5, host demands A1: 1.0, A20: 1.0, A21: 1.0, A22: 1.0, A3: 1.0
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 10.0

Table 2.79: Features of `lqn_twotasks`.

```

P2 = Processor(model, 'P2', 1, SchedStrategy.PS);
T2 = Task(model, 'T2', 1, SchedStrategy.INF).on(P2);
E2 = Entry(model, 'E2').on(T2);

E3 = Entry(model, 'E3').on(T2);

% definition of activities
T1.setThinkTime(Erlang.fitMeanAndOrder(10,1));

A1 = Activity(model, 'A1', Exp(1)).on(T1).boundTo(E1).synchCall(E2).synchCall(E3,1);

A20 = Activity(model, 'A20', Exp(1)).on(T2).boundTo(E2);
A21 = Activity(model, 'A21', Exp(1)).on(T2);
A22 = Activity(model, 'A22', Exp(1)).on(T2).repliesTo(E2);
T2.addPrecedence(ActivityPrecedence.Serial(A20, A21, A22));

A3 = Activity(model, 'A3', Exp(1)).on(T2).boundTo(E3).repliesTo(E3);

% instantiate solvers

... (14 source lines omitted; full implementation: matlab/examples/basic/)

%options.method = 'comom';
solveroptions.verbose = 0;
solver{2} = LN(model, @(1)NC(1,solveroptions), lnoptions);
AvgTable{2} = solver{2}.getAvgTable;
fprintf(1, '\nNC Results:\n');
disp(AvgTable{2});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.7.21 `lqn_workflows`

Workflow patterns (sequence, AND-fork and join, loop) expressed as activity graphs.

Processors	3: P1 (inf, $m = 1$), P2 (inf, $m = 1$), P3 (ps, $m = 5$)
Tasks	3: T1 (ref, $m = 1$), T2 (inf, $m = 1$), T3 (inf, $m = 1$)
Entries	3: Entry, E2, E1
Activities	14, host demands A1: 1.0, A2: 2.0, A3: 3.0, B1: 0.1, B2: 0.2, B3: 0.3, B4: 0.4, B5: 0.5, ...
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 0.0

Table 2.80: Features of `lqn_workflows`.

The example is built and solved as follows.

```

clear P T E A C solver AvgTable

fprintf(1, 'This example illustrates a layered network with a loop.\n')

model = LayeredNetwork('myLayeredModel');
%%
P{1} = Processor(model, 'P1', Inf, SchedStrategy.INF);

```

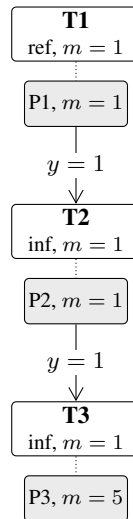



Figure 2.80: Call graph of `lqn_workflows`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```

T{1} = Task(model, 'T1', 1, SchedStrategy.REF).on(P{1});
T{1}.setThinkTime(Immediate());
E{1} = Entry(model, 'Entry').on(T{1});
%%
P{2} = Processor(model, 'P2', Inf, SchedStrategy.INF);
T{2} = Task(model, 'T2', 1, SchedStrategy.INF).on(P{2}).setThinkTime(Immediate());
E{2} = Entry(model, 'E2').on(T{2});
%%
P{3} = Processor(model, 'P3', 5, SchedStrategy.PS);
T{3} = Task(model, 'T3', 20, SchedStrategy.INF).on(P{3});
T{3}.setThinkTime(Exp.fitMean(10));
E{3} = Entry(model, 'E1').on(T{3});
%%
A{1} = Activity(model, 'A1', Exp.fitMean(1)).on(T{1}).boundTo(E{1});
A{2} = Activity(model, 'A2', Exp.fitMean(2)).on(T{1});
A{3} = Activity(model, 'A3', Exp.fitMean(3)).on(T{1}).synchCall(E{2});
%%
B{1} = Activity(model, 'B1', Exp.fitMean(0.1)).on(T{2}).boundTo(E{2});
B{2} = Activity(model, 'B2', Exp.fitMean(0.2)).on(T{2});

... (21 source lines omitted; full implementation: matlab/examples/basic/)

disp(AvgTable{1});

solver{2} = LN(model, 'verbose', false);
AvgTable{2} = solver{2}.getAvgTable();
fprintf(1, '\nLN Results:\n');
disp(AvgTable{2});

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.8 Cache models

A `Cache` node holds a finite set of items under a replacement policy. Each request names an item drawn from a popularity distribution, typically Zipf; a hit is served at once, a miss must be fetched and inserted, which evicts an item. The metric of interest is the hit ratio, and everything else in the model follows from it.

The first group of scripts fixes the workload and varies the replacement policy (LRU, FIFO, RR,

CLIMB, H-LRU, Q-LRU), so the hit ratios are directly comparable. The second group builds retrieval systems, in which the cache sits in front of a backing store and the miss path is itself a queueing network, and the delayed-hit effect appears: a request for an item already being fetched waits for that fetch rather than starting another.

2.8.1 cache_compare_replc

RR, FIFO and LRU on one item popularity. CTMC and MVA report all three hit ratios; NC does not support LRU and reports that cell as NaN rather than implying support.

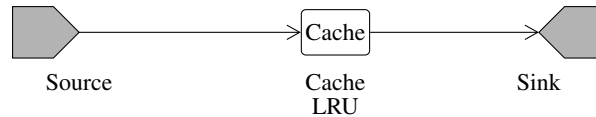


Figure 2.81: Topology of cache_compare_replc: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled
Features	cache: LRU replacement, 5 items, capacity 2,1; class switching on 2 links

Table 2.81: Features of cache_compare_replc.

The example is built and solved as follows.

```

n=5; m=[2,1]; alpha=1.0;
replStrat = {ReplacementStrategy.RR, ReplacementStrategy.FIFO, ReplacementStrategy.LRU};

AvgTable={};
solver={};
fprintf(1,'replc-strategy, ctmc, mva, nc:\n');
for s = 1:length(replStrat)
    model = Network('model');
    source = Source(model, 'Source');
    cacheNode = Cache(model, 'Cache', n, m, replStrat{s});
    sink = Sink(model, 'Sink');

    jobClass = OpenClass(model, 'InitClass', 0);
    hitClass = OpenClass(model, 'HitClass', 0);
    missClass = OpenClass(model, 'MissClass', 0);

    source.setArrival(jobClass, Exp(2));

    pAccess = Zipf(alpha, n); % Zipf-like item references
    cacheNode.setRead(jobClass, pAccess);

    cacheNode.setHitClass(jobClass, hitClass);
    cacheNode.setMissClass(jobClass, missClass);

    P = model.initRoutingMatrix;

    ... (35 source lines omitted; full implementation: matlab/examples/basic/)

else
    ncHitRatio(s) = NaN;
end

fprintf(1,'%s: %.8f, %.8f, %.8f\n',ReplacementStrategy.toString(replStrat{s}),
    ctmcHitRatio(s),mvaHitRatio(s),ncHitRatio(s));
end

```

Running this edition prints

```
rr: 0.71338230, 0.69673077, 0.70305657
fifo: 0.71338230, 0.69673077, 0.70305657
Warning [SolverFeatureSet.m]: Some features are not supported by the chosen solver (feature:
    ReplacementStrategy_LRU).
lru: 0.72430609, 0.72522055, NaN
```

2.8.2 cache_itemsize_costcap

Per-item storage costs and a cap on the cost each list may hold, so an insertion that would breach the cap serves the request without changing the cache state.

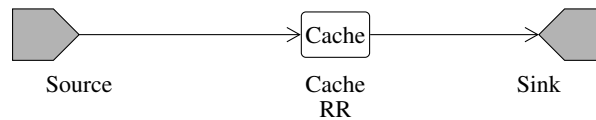


Figure 2.82: Topology of cache_itemsize_costcap: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled
Features	cache: RR replacement, 6 items, capacity 1,1; class switching on 2 links
Solvers	NC

Table 2.82: Features of cache_itemsize_costcap.

The example is built and solved as follows.

```
% Cache with per-item storage costs (sizes) and per-list cost caps.
%
% Each item i carries a storage cost sigma_i and list j may hold items of
% total cost at most k_j. A promotion that would breach a cap serves the
% request without changing the cache state. SolverNC evaluates the
% constrained normalizing constant E(m,k) of Casale-Gast (IEEE/ACM ToN
% 29(2), 2021), Sec. IX.

model = Network('model');

n = 6;           % number of items
m = [1 1];       % two lists, one item each
sizes = [1 1 1 2 2 2]; % small and large items
caps = [2 1];    % list 2 admits small items only

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.RR);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(2));
cacheNode.setRead(jobClass, DiscreteSampler((1/n)*ones(1,n)));
cacheNode.setItemSizes(sizes);

... (13 source lines omitted; full implementation: matlab/examples/basic/)

AvgTable = solver.getAvgNodeTable; AvgTable
CacheTable = solver.getAvgCacheTable; CacheTable
ItemTable = solver.getAvgItemTable; ItemTable

hitRatio = cacheNode.getHitRatio
```


Nodes	3: Source, Cache, Sink
Classes	6: Read1 (open), Read2 (open), Hit1 (open), Miss1 (open), Hit2 (open), Miss2 (open)
Arrivals	Source – Read1: MMAP; Read2: MMAP; Hit1: Disabled; Miss1: Disabled; Hit2: Disabled; Miss2: Disabled
Features	cache: RR replacement, 4 items, capacity 2; class switching on 2 links
Solvers	LDES, CTMC, FLD

Table 2.83: Features of `cache_mmap_rr_env`.

```
% (calm) emits mostly class-2 references at a low rate. The shared modulating
% chain therefore couples "which class arrives" with "how fast requests arrive".
%
% We solve the same system three ways:
%   (1) LDES - discrete-event simulation of the true MMAP-fed cache;
%   (2) CTMC - exact continuous-time Markov chain of the true system;
%   (3) ENV - three random-environment methods that view the MMPP2 phase as
%             an environment modulating phase-conditional Poisson arrivals
%             (D1 diagonal):
%             'avg' - fast-environment limit: replace the modulation by its
%                   time-average per-class Poisson rates and solve ONE
%                   rate-averaged cache;
%             'dec' - slow-environment limit: quasi-stationary
%                   decomposition, solve each phase independently and
%                   average per-phase hit ratios weighted by phase prob;
%             'statevec' - state-vector coupling: carry the cache-state
%                   distribution across phase switches and average each
%                   phase's sojourn-weighted distribution. For a
%
% ... (126 source lines omitted; full implementation: matlab/examples/basic/)

function model = local_setRates(baseModel, lambda1, lambda2)
    model = baseModel.copy();
    source = model.getNodeByName('Source');
    source.setArrival(model.classes{1}, Exp(lambda1)); % Read1 Poisson
    source.setArrival(model.classes{2}, Exp(lambda2)); % Read2 Poisson
end
```

Running this edition prints

```
LDES samples: 200000
LDES analysis [method: default; type: approximate, randomized; lang: cpp; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>      <strong>JobClass</strong>  <strong>Qlen</strong>    <strong>Util</strong>
    <strong>RespT</strong>      <strong>ResidT</strong>    <strong>ArvR</strong>      <strong>Tput</strong>
    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>
    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>
Source   Read1      0      0      0      0      0      2.5
Source   Read2      0      0      0      0      0      2.5
Cache    Read1      0      0      0      0      2.5      0
Cache    Read2      0      0      0      0      2.5      0
Cache    Hit1       0      0      0      0      0      1.3256
Cache    Miss1      0      0      0      0      0      1.1744
Cache    Hit2       0      0      0      0      0      1.323
Cache    Miss2      0      0      0      0      0      1.177
Sink     Hit1       0      0      0      0      1.3256      0
Sink     Miss1      0      0      0      0      1.1744      0
Sink     Hit2       0      0      0      0      1.323      0

... (39 captured-output lines omitted; rerun the source example for the full output)

--- Actual cache hit ratio per read class ---
                Read1      Read2
LDES (sim)    :  0.5302    0.5292
CTMC (true)   :  0.5963    0.4771
ENV (avg)     :  0.5821    0.4385
ENV (dec)     :  0.5241    0.4905
ENV (blend)   :  0.5963    0.4771
ENV (mf/FLD) :  0.5740    0.4830
```

The rows repeat for each of the 3 solvers the script runs (LDES, CTMC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.4 cache_replc_climb

The CLIMB policy, which promotes a hit by one list only.

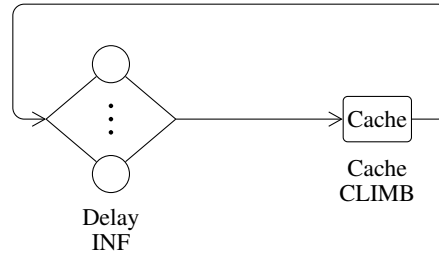


Figure 2.84: Topology of `cache_replc_climb`: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: CLIMB replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC

Table 2.84: Features of `cache_replc_climb`.

The example is built and solved as follows.

```
% CLIMB (transposition) replacement: on a hit an item moves up one position;
% on a miss it enters at the tail. Exact in CTMC; simulated in SSA/LDES.
% Not product-form, so MVA/NC/FLD reject it (use CTMC/SSA/LDES).

model = Network('model');

n = 5; % number of items
m = 2; % cache capacity

delay = Delay(model, 'Delay');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.CLIMB);

jobClass = ClosedClass(model, 'JobClass', 1, delay, 0);
hitClass = ClosedClass(model, 'HitClass', 0, delay, 0);
missClass = ClosedClass(model, 'MissClass', 0, delay, 0);

delay.setService(jobClass, Exp(1));

pAccess = Zipf(1.2, n); % Zipf-like item references
cacheNode.setRead(jobClass, pAccess);

cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);

P = model.initRoutingMatrix;
P{jobClass, jobClass}(delay, cacheNode) = 1.0;

... (4 source lines omitted; full implementation: matlab/examples/basic/)

solver{1} = CTMC(model, 'exact', 'keep', false);
AvgTable{1} = solver{1}.getAvgNodeTable; AvgTable{1}
```

```
model.reset;
solver{2} = SSA(model, 'samples', 1e4, 'method', 'serial', 'seed', 23000);
AvgTable{2} = solver{2}.getAvgNodeTable; AvgTable{2}
```

Running this edition prints

```
CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>  <strong>QLen</strong>  <strong>
Util</strong>      <strong>RespT</strong>      <strong>ResidT</strong>      <strong>ArvR</strong>      <
<strong>Tput</strong>
<strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <
_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <
<strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0
Cache      HitClass      0      0      0      0      0      0.59763
Cache      MissClass     0      0      0      0      0      0.40237
CS_Cache_to_Delay JobClass      0      0      0      0      0      1
CS_Cache_to_Delay HitClass      0      0      0      0      0.59763      0
CS_Cache_to_Delay MissClass     0      0      0      0      0.40237      0

SSA analysis [method: serial; type: approximate, randomized; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>  <strong>QLen</strong>  <strong>
Util</strong>      <strong>RespT</strong>      <strong>ResidT</strong>      <strong>ArvR</strong>      <
<strong>Tput</strong>
<strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <
_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <
<strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0
Cache      HitClass      0      0      0      0      0      0.58129
Cache      MissClass     0      0      0      0      0      0.41871
CS_Cache_to_Delay JobClass      0      0      0      0      0      1
CS_Cache_to_Delay HitClass      0      0      0      0      0.58129      0
CS_Cache_to_Delay MissClass     0      0      0      0      0.41871      0
```

The rows repeat for each of the 2 solvers the script runs (CTMC, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.5 cache_replc_fifo

First-in first-out replacement.

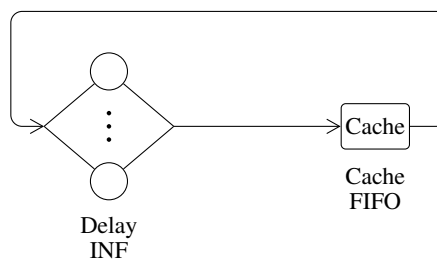


Figure 2.85: Topology of `cache_replc_fifo`: a closed network over 1 queueing station with a cache node.

The example is built and solved as follows.

```
model = Network('model');

n = 5; % number of items
m = 2; % cache capacity

delay = Delay(model, 'Delay');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.FIFO);

jobClass = ClosedClass(model, 'JobClass', 1, delay, 0);
```

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: FIFO replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC, SSA, MVA, FLD

Table 2.85: Features of `cache_replc_fifo`.

```

hitClass = ClosedClass(model, 'HitClass', 0, delay, 0);
missClass = ClosedClass(model, 'MissClass', 0, delay, 0);

delay.setService(jobClass, Exp(1));

pAccess = DiscreteSampler((1/n)*ones(1,n)); % uniform item references
cacheNode.setRead(jobClass, pAccess);

cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);

P = model.initRoutingMatrix;

P{jobClass, jobClass}(delay, cacheNode) = 1.0;
P{hitClass, jobClass}(cacheNode, delay) = 1.0;
P{missClass, jobClass}(cacheNode, delay) = 1.0;

... (9 source lines omitted; full implementation: matlab/examples/basic/)

solver{3} = MVA(model);
AvgTable{3} = solver{3}.getAvgNodeTable; AvgTable{3}

model.reset;
solver{4} = FLD(model, 'method', 'rmf');
AvgTable{4} = solver{4}.getAvgNodeTable; AvgTable{4}

```

Running this edition prints

```

CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>    <strong>QLen</strong>    <strong>
Util</strong>    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <
strong>Tput</strong>
<strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>
____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <
strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0
Cache      HitClass      0      0      0      0      0      0.4
Cache      MissClass     0      0      0      0      0      0.6
CS_Cache_to_Delay JobClass      0      0      0      0      0      1
CS_Cache_to_Delay HitClass      0      0      0      0      0.4    0
CS_Cache_to_Delay MissClass     0      0      0      0      0.6    0
SSA analysis [method: serial; type: approximate, randomized; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>    <strong>QLen</strong>    <strong>
Util</strong>    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <
strong>Tput</strong>
<strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>
____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <
strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
... (20 captured-output lines omitted; rerun the source example for the full output)

<strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>
____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <
strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0

```


Cache	HitClass	0	0	0	0	0	0.4
Cache	MissClass	0	0	0	0	0	0.6
CS_Cache_to_Delay	JobClass	0	0	0	0	0	1
CS_Cache_to_Delay	HitClass	0	0	0	0	0.4	0
CS_Cache_to_Delay	MissClass	0	0	0	0	0.6	0

The rows repeat for each of the 4 solvers the script runs (CTMC, SSA, MVA, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.6 cache_replc_hlru

H-LRU, the multi-list generalization of LRU.

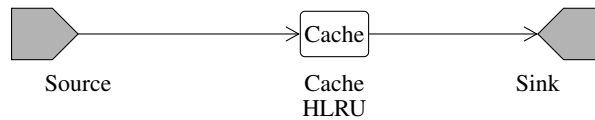


Figure 2.86: Topology of cache_replc_hlru: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	cache: HLRU replacement, 6 items, capacity 2,1; class switching on 2 links
Solvers	CTMC, MVA, SSA

Table 2.86: Features of cache_replc_hlru.

The example is built and solved as follows.

```

% h-LRU / LRU(m) replacement: h LRU lists of capacities m(1..h); a miss
% inserts the item at the head of list 1, a hit in list 1 exchanges the item
% with the tail of list 1+1. Exact in CTMC; simulated in SSA/LDES; MVA uses
% the characteristic-time (TTL) approximation of Gast and Van Houdt
% (SIGMETRICS 2015), which reduces to the Che approximation for h=1.

model = Network('model');

n = 6;          % number of items
m = [2 1];     % list capacities: list 1 holds 2 items, list 2 holds 1

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.HLRU);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(1));

pAccess = Zipf(1.2, n); % Zipf-like item references
cacheNode.setRead(jobClass, pAccess);

cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);

... (11 source lines omitted; full implementation: matlab/examples/basic/)

solver{2} = MVA(model); % TTL approximation
AvgTable{2} = solver{2}.getAvgNodeTable; AvgTable{2}

model.reset;
solver{3} = SSA(model, 'samples', 1e4, 'method', 'serial', 'seed', 23000);

```

```
AvgTable{3} = solver{3}.getAvgNodeTable; AvgTable{3}
```

Running this edition prints

```
CTMC solver using state space cutoff = 1 for open/mixed model.
Warning [runAnalyzer.m]: State space truncation may cause inaccurate results. Consider varying cutoff to
assess sensitivity.
CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>      <strong>JobClass</strong>    <strong>QLen</strong>    <strong>Util</strong>
    >    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <strong>Tput</strong>
    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>
    >    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>
    </strong>
Source    InitClass    0        0        0        0        0        1
Cache    InitClass    0        0        0        0        1        0
Cache    HitClass     0        0        0        0        0        0.70085
Cache    MissClass    0        0        0        0        0        0.29915
Sink     HitClass     0        0        0        0        0.70085    0
Sink     MissClass    0        0        0        0        0.29915    0
MVA analysis [method: default/default; type: approximate, deterministic; lang: matlab; env: 2026a]
completed in <t>s.
ans =
    <strong>Node</strong>      <strong>JobClass</strong>    <strong>QLen</strong>    <strong>Util</strong>
    >    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <strong>Tput</strong>
    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>
    >    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>
    </strong>
... (8 captured-output lines omitted; rerun the source example for the full output)

    <strong>Node</strong>      <strong>JobClass</strong>    <strong>QLen</strong>    <strong>Util</strong>
    >    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <strong>Tput</strong>
    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>
    >    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>
    </strong>
Source    InitClass    0        0        0        0        0        1
Cache    InitClass    0        0        0        0        1        0
Cache    HitClass     0        0        0        0        0        0.72712
Cache    MissClass    0        0        0        0        0        0.27288
Sink     HitClass     0        0        0        0        0.72712    0
Sink     MissClass    0        0        0        0        0.27288    0
```

The rows repeat for each of the 3 solvers the script runs (CTMC, MVA, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.7 cache_replc_lru

Least-recently-used replacement, with a closed class circulating between a Delay and the cache.

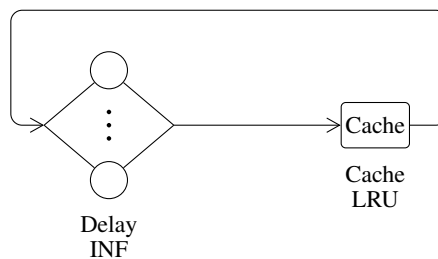


Figure 2.87: Topology of `cache_replc_lru`: a closed network over 1 queueing station with a cache node.

The example is built and solved as follows.

```
model = Network('model');
```

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: LRU replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC, SSA, MVA

Table 2.87: Features of cache_replc_lru.

```

n = 5; % number of items
m = 2; % cache capacity

delay = Delay(model, 'Delay');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.LRU);

jobClass = ClosedClass(model, 'JobClass', 1, delay, 0);
hitClass = ClosedClass(model, 'HitClass', 0, delay, 0);
missClass = ClosedClass(model, 'MissClass', 0, delay, 0);

delay.setService(jobClass, Exp(1));

pAccess = DiscreteSampler((1/n)*ones(1,n)); % uniform item references
cacheNode.setRead(jobClass, pAccess);

cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);

P = model.initRoutingMatrix;

P{jobClass, jobClass}(delay, cacheNode) = 1.0;
P{hitClass, jobClass}(cacheNode, delay) = 1.0;
P{missClass, jobClass}(cacheNode, delay) = 1.0;

... (9 source lines omitted; full implementation: matlab/examples/basic/)

solver{3} = MVA(model);
AvgTable{3} = solver{3}.getAvgNodeTable; AvgTable{3}

% No SolverFLD row: LRU has no drift-based fluid model, so the refined mean
% field carries RANDOM(m)/FIFO(m) and strict FIFO(m) only (see
% solver_fld_cacheqn_analyzer). Use CTMC/SSA/MVA/LDES for an LRU cache.

```

Running this edition prints

```

CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>    <strong>QLen</strong>    <strong>
Util</strong>    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <
    <strong>Tput</strong>
<strong>_____/>    <strong>_____/>    <strong>_____/>    <strong>_____/>    <strong>
_____/>    <strong>_____/>    <strong>_____/>    <strong>_____/>    <
    <strong>_____/>
Delay          JobClass      1      1      1      1      1      1
Cache          JobClass      0      0      0      0      1      0
Cache          HitClass      0      0      0      0      0      0.4
Cache          MissClass     0      0      0      0      0      0.6
CS_Cache_to_Delay JobClass    0      0      0      0      0      1
CS_Cache_to_Delay HitClass    0      0      0      0      0.4    0
CS_Cache_to_Delay MissClass   0      0      0      0      0.6    0
SSA analysis [method: serial; type: approximate, randomized; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>    <strong>QLen</strong>    <strong>
Util</strong>    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <
    <strong>Tput</strong>
<strong>_____/>    <strong>_____/>    <strong>_____/>    <strong>_____/>    <strong>
_____/>    <strong>_____/>    <strong>_____/>    <strong>_____/>    <
    <strong>_____/>
Delay          JobClass      1      1      1      1      1      1

```

```
... (9 captured-output lines omitted; rerun the source example for the full output)

<strong>_____</strong>    <strong>_____</strong>    <strong>____</strong>    <strong>_____</strong>
____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>    <strong>_____</strong>
strong>____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0
Cache      HitClass      0      0      0      0      0      0.4
Cache      MissClass     0      0      0      0      0      0.6
CS_Cache_to_Delay JobClass      0      0      0      0      0      1
CS_Cache_to_Delay HitClass      0      0      0      0      0.4      0
CS_Cache_to_Delay MissClass     0      0      0      0      0.6      0
```

The rows repeat for each of the 3 solvers the script runs (CTMC, SSA, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.8 cache_replc_qlru

Q-LRU, which promotes a hit with probability q .

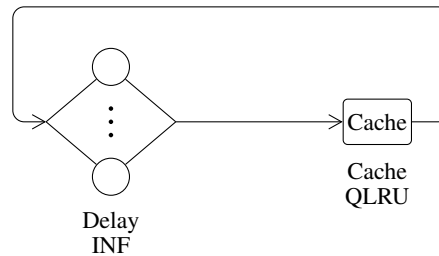


Figure 2.88: Topology of cache_replc_qlru: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: QLRU replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC

Table 2.88: Features of cache_replc_qlru.

The example is built and solved as follows.

```
% q-LRU replacement: on a miss the item is admitted (LRU head insert) with
% probability q, otherwise it passes through uncached. Admission filtering can
% raise the hit ratio over plain LRU under skewed popularity. Exact in CTMC;
% simulated in SSA/LDES. Not product-form (MVA/NC/FLD reject it).

model = Network('model');

n = 5; % number of items
m = 2; % cache capacity

delay = Delay(model, 'Delay');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.QLRU);
cacheNode.setAdmissionProb(0.5); % admit a missed item with probability q=0.5

jobClass = ClosedClass(model, 'JobClass', 1, delay, 0);
hitClass = ClosedClass(model, 'HitClass', 0, delay, 0);
missClass = ClosedClass(model, 'MissClass', 0, delay, 0);
```

```

delay.setService(jobClass, Exp(1));

pAccess = Zipf(1.2, n); % Zipf-like item references
cacheNode.setRead(jobClass, pAccess);

cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);

... (6 source lines omitted; full implementation: matlab/examples/basic/)

solver{1} = CTMC(model, 'exact', 'keep', false);
AvgTable{1} = solver{1}.getAvgNodeTable; AvgTable{1}

model.reset;
solver{2} = SSA(model, 'samples', 1e4, 'method', 'serial', 'seed', 23000);
AvgTable{2} = solver{2}.getAvgNodeTable; AvgTable{2}

```

Running this edition prints

```

CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>    <strong>QLen</strong>    <strong>
Util</strong>    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <
<strong>Tput</strong>
<strong>_____</strong>    <strong>_____</strong>    <strong>____</strong>    <strong>
____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <
<strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0
Cache      HitClass      0      0      0      0      0      0.57514
Cache      MissClass     0      0      0      0      0      0.42486
CS_Cache_to_Delay JobClass      0      0      0      0      0      1
CS_Cache_to_Delay HitClass      0      0      0      0      0.57514    0
CS_Cache_to_Delay MissClass     0      0      0      0      0.42486    0

SSA analysis [method: serial; type: approximate, randomized; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>          <strong>JobClass</strong>    <strong>QLen</strong>    <strong>
Util</strong>    <strong>RespT</strong>    <strong>ResidT</strong>    <strong>ArvR</strong>    <
<strong>Tput</strong>
<strong>_____</strong>    <strong>_____</strong>    <strong>____</strong>    <strong>
____</strong>    <strong>____</strong>    <strong>____</strong>    <strong>____</strong>    <
<strong>_____</strong>
Delay      JobClass      1      1      1      1      1      1
Cache      JobClass      0      0      0      0      1      0
Cache      HitClass      0      0      0      0      0      0.58838
Cache      MissClass     0      0      0      0      0      0.41162
CS_Cache_to_Delay JobClass      0      0      0      0      0      1
CS_Cache_to_Delay HitClass      0      0      0      0      0.58838    0
CS_Cache_to_Delay MissClass     0      0      0      0      0.41162    0

```

The rows repeat for each of the 2 solvers the script runs (CTMC, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.9 cache_replc_routing

Cache-aware routing: hits and misses leave the cache on different arcs, under RAND and round-robin routing.

Nodes	6: Source, Delay $\times$ 2, Cache, Router, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Scheduling	Delay1: INF; Delay2: INF
Arrivals	Source – InitClass: Exp(2)
Service	Delay1 – HitClass: Exp(10); MissClass: Exp(1) Delay2 – HitClass: Exp(20); MissClass: Exp(2)
Features	cache: FIFO replacement, 5 items, capacity 2; class switching on 6 links
Solvers	CTMC, SSA, MVA, NC, FLD

Table 2.89: Features of cache_replc_routing.

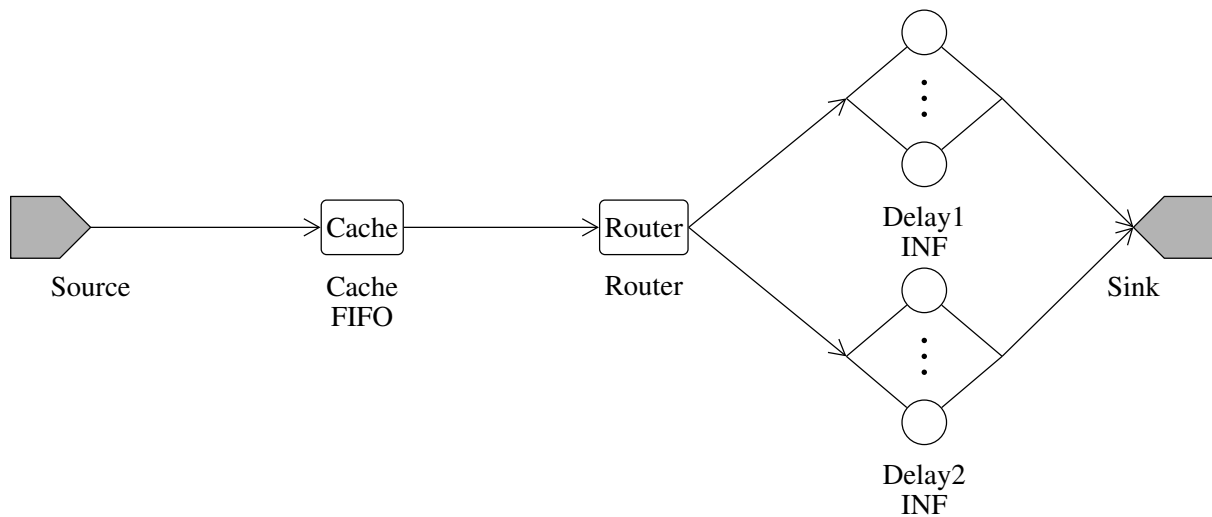


Figure 2.89: Topology of `cache_replc_routing`: an open network over 2 queueing stations with a cache node and a router.

The example is built and solved as follows.

```
% cache with state dependent routing

model = Network('model');

n = 5; % number of items
m = 2; % cache capacity

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.FIFO);
delay1 = Delay(model, 'Delay1');
delay2 = Delay(model, 'Delay2');
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(2));

delay1.setService(hitClass, Exp(10));
delay1.setService(missClass, Exp(1));
delay2.setService(hitClass, Exp(20));
delay2.setService(missClass, Exp(2));

pAccess = DiscreteSampler((1/n)*ones(1,n)); % uniform item references
cacheNode.setRead(jobClass, pAccess);

... (111 source lines omitted; full implementation: matlab/examples/basic/)

model.reset;
solver{5} = FLD(model, 'method', 'rmf');
AvgTable{5} = solver{5}.getAvgNodeTable; AvgTable{5}

hitRatio=cacheNode.getHitRatio
missRatio=cacheNode.getMissRatio
```

Running this edition prints

```
The example first illustrates an invalid routing setup for a cache that triggers the following exception:
[Node.m @ line 121] State-dependent routing not supported with caches. Add instead a Router node after the
cache.
We now illustrate the correct setup of the same model.
CTMC solver using state space cutoff = 1 for open/mixed model.
```

```
Warning [runAnalyzer.m]: State space truncation may cause inaccurate results. Consider varying cutoff to
assess sensitivity.
Warning [solver_ctmc.m]: CTMC: 40 vanishing state(s) have no immediate outgoing arc; the vanishing
predicate and the immediate-arc tagging disagree, so those rows keep their timed arcs.
CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>      <strong>JobClass</strong>      <strong>QLen</strong>      <strong>Util</strong>
    <strong>      <strong>RespT</strong>      <strong>ResidT</strong>      <strong>ArvR</strong>      <strong>      </strong>
    <strong>Tput</strong>
    <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    </strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    <strong>_____</strong>
Source      InitClass      0      0      0      0      0      1.9808
Cache      InitClass      0      0      0      0      1.9808      0
Cache      HitClass      0      0      0      0      0      0.79232
Cache      MissClass      0      0      0      0      0      1.1885
Router      HitClass      0      0      0      0      0.79232      0.79232
Router      MissClass      0      0      0      0      1.1885      1.1885

... (66 captured-output lines omitted; rerun the source example for the full output)

Delay2      HitClass      0      0      0      0      0.4      0
Delay2      MissClass      0      0      0      0      0.6      0
Sink        HitClass      0      0      0      0      0.8      0
Sink        MissClass      0      0      0      0      1.2      0
hitRatio =
    0.4000      0      0
missRatio =
    0.6000      0      0
```

The rows repeat for each of the 5 solvers the script runs (CTMC, SSA, MVA, NC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.10 cache_replc_rr

Random replacement.

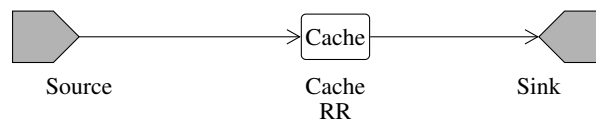


Figure 2.90: Topology of cache_replc_rr: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled
Features	cache: RR replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC, SSA, MVA, NC, FLD

Table 2.90: Features of cache_replc_rr.

The example is built and solved as follows.

```
model = Network('model');

n = 5; % number of items
m = 2; % cache capacity

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.RR);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);
```

```

source.setArrival(jobClass, Exp(2));

%pAccess = DiscreteSampler((1/n)*ones(1,n)); % uniform item references
pAccess = Zipf(1.4, n); % Zipf-like item references
cacheNode.setRead(jobClass, pAccess);

cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);

P = model.initRoutingMatrix;

P{jobClass, jobClass}(source, cacheNode) = 1.0;
P{hitClass, hitClass}(cacheNode, sink) = 1.0;

... (19 source lines omitted; full implementation: matlab/examples/basic/)

model.reset;
solver{5} = FLD(model, 'method', 'rmf');
AvgTable{5} = solver{5}.getAvgNodeTable; AvgTable{5}

hitRatio=cacheNode.getHitRatio
missRatio=cacheNode.getMissRatio

```

Running this edition prints

```

CTMC solver using state space cutoff = 1 for open/mixed model.
[runAnalyzer.m] Message casted more than once, repetitions will not be printed on screen for 60 seconds.
CTMC analysis [method: exact; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>      <strong>JobClass</strong>      <strong>QLen</strong>      <strong>Util</strong>
    >      <strong>RespT</strong>      <strong>ResidT</strong>      <strong>ArvR</strong>      <strong>Tput</strong>
    <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    >      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    </strong>
Source   InitClass    0      0      0      0      0      2
Cache    InitClass    0      0      0      0      2      0
Cache    HitClass      0      0      0      0      0      1.1808
Cache    MissClass      0      0      0      0      0      0.81921
Sink     HitClass      0      0      0      0      1.1808      0
Sink     MissClass      0      0      0      0      0.81921      0
SSA analysis [method: serial; type: approximate, randomized; lang: matlab; env: 2026a] completed in <t>s.
ans =
    <strong>Node</strong>      <strong>JobClass</strong>      <strong>QLen</strong>      <strong>Util</strong>
    >      <strong>RespT</strong>      <strong>ResidT</strong>      <strong>ArvR</strong>      <strong>Tput</strong>
    <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    >      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    </strong>
... (32 captured-output lines omitted; rerun the source example for the full output)

Cache    HitClass      0      0      0      0      0      1.1246
Cache    MissClass      0      0      0      0      0      0.87542
Sink     HitClass      0      0      0      0      1.1246      0
Sink     MissClass      0      0      0      0      0.87542      0
hitRatio =
    0.5623      0      0
missRatio =
    0.4377      0      0

```

The rows repeat for each of the 5 solvers the script runs (CTMC, SSA, MVA, NC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.11 cache_rmf_transient

The transient hit ratio from the refined mean field approximation.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.


```

%% Cache RMF Transient Analysis
% Demonstrates refined mean field (RMF) transient and steady-state
% analysis of a multi-list cache with RANDOM(m) replacement.
%
% Uses CacheRMF directly to compute:
% - Steady-state hit/miss probabilities with 1/N correction
% - Transient evolution of hit rates via coupled ODE system

clear;

% Cache parameters
n = 10;           % number of items
m = [3, 2];      % list capacities (2-list cache)
alpha = 0.8;     % Zipf exponent

% Zipf popularity distribution
p = (1:n).^(-alpha);
p = p / sum(p);

fprintf('Cache parameters: n=%d, m=[%s], Zipf(%.1f)\n', n, strjoin(arrayfun(@num2str, m, '
UniformOutput', false), ', '), alpha);

% Build DDPP model
model = CacheRMF(p, m);

% Steady-state analysis with 1/N correction
[pi, V, VW] = model.meanFieldExpansionSteadyState(1);

... (21 source lines omitted; full implementation: matlab/examples/basic/)

hr = 0;
for k = 1:length(m)
    hr = hr + model.hitRate(xt, k);
end
fprintf('  t=%7.3f: hit_rate=%.6f\n', T(idx), hr);
end

```

Running this edition prints

```

Cache parameters: n=10, m=[3, 2], Zipf(0.8)

Steady-state results (refined mean field):
Hit rate (list 1): 0.312854
Hit rate (list 2): 0.301606
Miss rate:         0.385540
Total hit prob:    0.614460
Total miss prob:   0.385540

Transient hit rates (refined, N=10):
t=  0.000: hit_rate=0.728003
t=  5.025: hit_rate=0.645750
t= 12.563: hit_rate=0.624411
t= 25.126: hit_rate=0.616758
t= 50.000: hit_rate=0.614587

```

2.8.12 cachenet_tandem

A tandem of two caches over one item set, where a miss at the first is a read of the same item at the second. Item identity is carried by a dedicated job class per item, so no class switching happens on the arc between the caches.

The example is built and solved as follows.

```

% Tandem of two caches sharing one item set.
%
% A request reads Cache1; on a miss it reads Cache2 for THE SAME item; on a
% second miss it is fetched from the origin. Item identity is carried by giving
% each item its OWN job class, so no class switching happens on an arc into a
% cache and the miss of item i at Cache1 IS the read class of item i at Cache2.

```

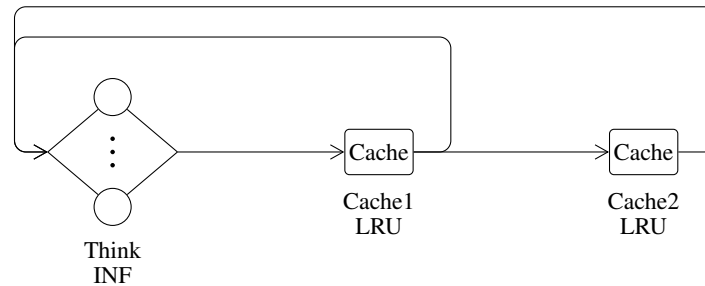


Figure 2.91: Topology of `cachenet_tandem`: a closed network over 1 queueing station with a cache node.

Nodes	3: Delay, Cache $\times$ 2
Classes	15: Read1 (closed, $N = 1$), Hit1_1 (closed, $N = 0$), Hit2_1 (closed, $N = 0$), Miss_1 (closed, $N = 0$), Read2 (closed, $N = 1$), Hit1_2 (closed, $N = 0$), Hit2_2 (closed, $N = 0$), Miss_2 (closed, $N = 0$), Read3 (closed, $N = 1$), Hit1_3 (closed, $N = 0$), Hit2_3 (closed, $N = 0$), Miss_3 (closed, $N = 0$), Cache2_item1 (closed, $N = 0$), Cache2_item2 (closed, $N = 0$), Cache2_item3 (closed, $N = 0$)
Scheduling	Think: INF
Service	Think – Read1: Exp(0.5); Hit1_1: Disabled; Hit2_1: Disabled; Miss_1: Disabled; Read2: Exp(0.3); Hit1_2: Disabled; Hit2_2: Disabled; Miss_2: Disabled; Read3: Exp(0.2); Hit1_3: Disabled; Hit2_3: Disabled; Miss_3: Disabled; Cache2_item1: Disabled; Cache2_item2: Disabled; Cache2_item3: Disabled
Features	drop rule {'Hit1_1': 'waitingQueue', 'Hit2_1': 'waitingQueue', 'Miss_1': 'waitingQueue', 'Hit1_2': 'waitingQueue', 'Hit2_2': 'waitingQueue', 'Miss_2': 'waitingQueue', 'Hit1_3': 'waitingQueue', 'Hit2_3': 'waitingQueue', 'Miss_3': 'waitingQueue', 'Cache2_item1': 'waitingQueue', 'Cache2_item2': 'waitingQueue', 'Cache2_item3': 'waitingQueue'} at Think; cache: LRU replacement, 3 items, capacity 1; cache: LRU replacement, 3 items, capacity 2; class switching on 3 links
Solvers	CTMC, SSA, LDES

Table 2.91: Features of `cachenet_tandem`.

```
%
% Item popularity is expressed by the per-class request rates rather than by a
% popularity distribution the cache draws from, which is also how the TTL
% cache-network analysis is parameterised.
%
% Note on the capacities: with LRU(1) at BOTH levels, Cache2 can never hit. A
% miss at Cache1 inserts the item there, so the next Cache1 miss is necessarily
% for a different item and consecutive arrivals at Cache2 are always distinct.
% Cache2 therefore needs room for more than one item to show any hit at all.
clc; clear solver AvgTable;

n      = 3;                                % items, shared by both caches
pAccess = [0.5 0.3 0.2];                  % item popularity
lambda = 1.0;                             % request rate scale

model = Network('CacheTandem');

think = Delay(model, 'Think');
cache1 = Cache(model, 'Cache1', n, 1, ReplacementStrategy.LRU);
cache2 = Cache(model, 'Cache2', n, 2, ReplacementStrategy.LRU);

... (22 source lines omitted; full implementation: matlab/examples/basic/)

% Cache1 -> Cache2 for each item is registered by setMissCache.
model.link(P);

CTMC(model).getAvgCacheTable
SSA(model, 'samples', 5e4, 'seed', 1).getAvgCacheTable
LDES(model, 'samples', 2e5, 'seed', 1).getAvgCacheTable
```

Running this edition prints

```
Warning [solver_ctmc.m]: CTMC: the generator has more than one closed communicating class (1944 of 3258
states cannot reach the one containing the initial state), so its stationary distribution is not
unique. Results are those of the class the initial state selects, and depend on it: an order-
preserving discipline under routing that admits no overtaking freezes the relative service order at
time zero, so declaring the classes in a different order gives a different, equally valid answer.
Product-form results (MVA, NC) normalize over the whole space instead and will not agree.
CTMC infinitesimal generator and state space saved in: <tmp>
CTMC analysis [method: default; type: exact, deterministic; lang: matlab; env: 2026a] completed in <t>s.
ans =
<strong>Node</strong>      <strong>JobClass</strong>      <strong>Item</strong>      <strong>List</strong>
<strong>ListCap</strong> <strong>Items</strong> <strong>HitProb</strong> <strong>
DelayedHitProb</strong> <strong>MissProb</strong> <strong>HitRate</strong> <strong>
DelayedHitRate</strong> <strong>MissRate</strong> <strong>ArvR</strong> <strong>ResidT</strong>
<strong>ListCost</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong>
Cache1  Read1      1      0      1      3      0.5      0      0.5
0      0      0      0      0      NaN      NaN
Cache1  Read2      2      0      1      3      0.3      0      0.7
0      0      0      0      0      NaN      NaN
Cache1  Read3      3      0      1      3      0.2      0      0.8
0      0      0      0      0      NaN      NaN
Cache2  Cache2_item1 1      0      2      3      0.67857  0      0.32143
0      0      0      0      0      NaN      NaN
Cache2  Cache2_item2 2      0      2      3      0.53571  0      0.46429
0      0      0      0      0      NaN      NaN
Cache2  Cache2_item3 3      0      2      3      0.35714  0      0.64286
0      0      0      0      0      NaN      NaN
SSA analysis [method: default/nrm; type: approximate, randomized; lang: matlab; env: 2026a] completed in <
t>s.
ans =
<strong>Node</strong>      <strong>JobClass</strong>      <strong>Item</strong>      <strong>List</strong>
<strong>ListCap</strong> <strong>Items</strong> <strong>HitProb</strong> <strong>
DelayedHitProb</strong> <strong>MissProb</strong> <strong>HitRate</strong> <strong>
DelayedHitRate</strong> <strong>MissRate</strong> <strong>ArvR</strong> <strong>ResidT</strong>
<strong>ListCost</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong>
<strong>____</strong> <strong>____</strong> <strong>____</strong> <strong>____</strong>
```

```

    strong>      <strong>_____</strong>

... (9 captured-output lines omitted; rerun the source example for the full output)

    <strong>Node</strong>      <strong>JobClass</strong>      <strong>Item</strong>      <strong>List</strong>
    strong>      <strong>ListCap</strong>      <strong>Items</strong>      <strong>HitProb</strong>      <strong>
    DelayedHitProb</strong>      <strong>MissProb</strong>      <strong>HitRate</strong>      <strong>
    DelayedHitRate</strong>      <strong>MissRate</strong>      <strong>ArvR</strong>      <strong>ResidT</strong>
    strong>      <strong>ListCost</strong>
    <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    _____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    _____</strong>      <strong>_____</strong>      <strong>_____</strong>      <strong>_____</strong>
    strong>      <strong>_____</strong>
    Cache1      Read1      1      0      1      3      0.49878      0      0.50122
    0      0      0      0      0      NaN      NaN
    Cache1      Read2      2      0      1      3      0.29924      0      0.70076
    0      0      0      0      0      NaN      NaN
    Cache1      Read3      3      0      1      3      0.20015      0      0.79985
    0      0      0      0      0      NaN      NaN
    Cache2      Cache2_item1      1      0      2      3      0.67966      0      0.32034
    0      0      0      0      0      NaN      NaN
    Cache2      Cache2_item2      2      0      2      3      0.53588      0      0.46412
    0      0      0      0      0      NaN      NaN
    Cache2      Cache2_item3      3      0      2      3      0.36105      0      0.63895
    0      0      0      0      0      NaN      NaN

```

The rows repeat for each of the 3 solvers the script runs (CTMC, SSA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.13 retrieval_chain

A chain of caches, each backing the one before it.

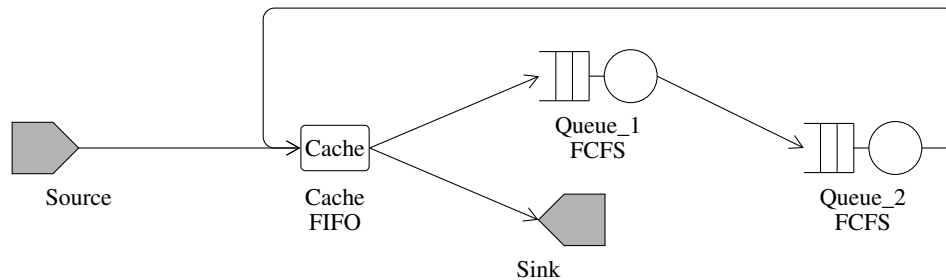


Figure 2.92: Topology of `retrieval_chain`: an open network over 2 queueing stations with a cache node.

Nodes	5: Source, Queue × 2, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	Queue_1: FCFS; Queue_2: FCFS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	Queue_1 – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(2); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2) Queue_2 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3)
Features	cache: FIFO replacement, 3 items, capacity 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_2; class switching on 4 links
Solvers	SSA, LDES, MVA, NC

Table 2.92: Features of `retrieval_chain`.

The example is built and solved as follows.

```
% Cache with a chained retrieval system (delayed hits).
clc; clear solver AvgTable;

accessProb = [0.6, 0.3, 0.1]; % per-item access probabilities (3 items)
nQueues    = 2;

model = Network('DelayedHits');

n = numel(accessProb); % number of items
capacity = [1]; % per-level cache capacity

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, capacity, ReplacementStrategy.FIFO);
queues = cell(1, nQueues);
for i = 1:nQueues
    queues{i} = Queue(model, sprintf('Queue_%d', i), SchedStrategy.FCFS);
end
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(1));

% Read class service at each retrieval queue = default per-item fetch service.
... (26 source lines omitted; full implementation: matlab/examples/basic/)

MVA(model).getAvgCacheTable
NC(model).getAvgCacheTable

% Item-level cache occupancy
MVA(model).getAvgItemTable
NC(model).getAvgItemTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.8.14 retrieval_default

The same system with a PS backing store and the default retrieval parameters.

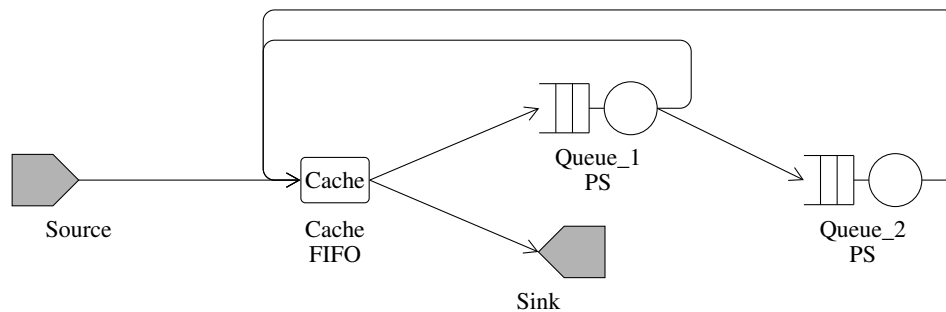


Figure 2.93: Topology of retrieval_default: an open network over 2 queueing stations with a cache node.

The example is built and solved as follows.

```
% Retrieval system whose per-item miss routing AND service are taken, by
% default, from the read class: routing from the read class's edges among the
% retrieval queues in the top-level P matrix, and service from the read class's
% service distribution at each queue. Per-item overrides
```

Nodes	5: Source, Queue $\times$ 2, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	Queue_1: PS; Queue_2: PS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	Queue_1 – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(5); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2) Queue_2 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3)
Features	cache: FIFO replacement, 3 items, capacity 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_2; class switching on 4 links
Solvers	SSA, LDES, MVA, NC

Table 2.93: Features of retrieval_default.

```
% (setItemRoutingProb with the cache as source/dest / queue.setItemServiceRate)
% then reconfigure one item at finer granularity.
clc; clear solver AvgTable;

accessProb = [0.6, 0.3, 0.1]; % per-item access probabilities (3 items)

model = Network('DelayedHits');

n = numel(accessProb); % number of items
capacity = [1]; % per-level cache capacity

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, capacity, ReplacementStrategy.FIFO);
% PS retrieval stations: per-item (class-dependent) service rates are admissible
% in the analytical retrieval algorithm (FCFS/SIRO would require identical rates).
queue1 = Queue(model, 'Queue_1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue_2', SchedStrategy.PS);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

... (35 source lines omitted; full implementation: matlab/examples/basic/)

MVA(model).getAvgCacheTable
NC(model).getAvgCacheTable

% Item-level cache occupancy
MVA(model).getAvgItemTable
NC(model).getAvgItemTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.8.15 retrieval_ps

The PS variant, with the delayed-hit behaviour made explicit.

The example is built and solved as follows.

```
% PS variant of retrieval_simple: a single processor-sharing retrieval station.
clc; clear solver AvgTable;

accessProb = [49, 49, 49, 49, 7, 1, 1] / 205; % per-item arrival probabilities (7 items)

model = Network('DelayedHits');
```

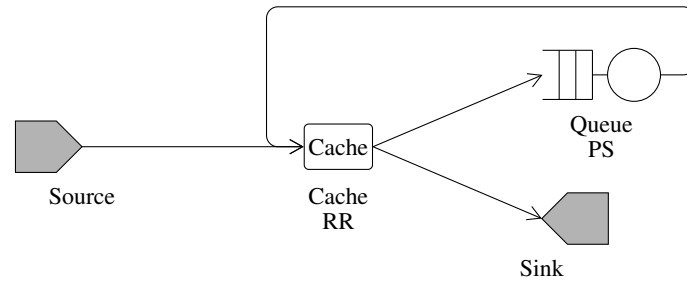


Figure 2.94: Topology of `retrieval_ps`: an open network over 1 queueing station with a cache node.

Nodes	4: Source, Queue, Cache, Sink
Classes	10: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open), InitClass_retrievalClass_4 (open), InitClass_retrievalClass_5 (open), InitClass_retrievalClass_6 (open), InitClass_retrievalClass_7 (open)
Scheduling	Queue: PS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled; InitClass_retrievalClass_4: Disabled; InitClass_retrievalClass_5: Disabled; InitClass_retrievalClass_6: Disabled; InitClass_retrievalClass_7: Disabled
Service	Queue – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(1); InitClass_retrievalClass_2: Exp(1); InitClass_retrievalClass_3: Exp(1); InitClass_retrievalClass_4: Exp(1); InitClass_retrievalClass_5: Exp(1); InitClass_retrievalClass_6: Exp(1); InitClass_retrievalClass_7: Exp(1)
Features	cache: RR replacement, 7 items, capacity 6; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue; class switching on 3 links
Solvers	SSA, LDES, MVA, NC

Table 2.94: Features of `retrieval_ps`.

```

n = numel(accessProb);           % number of items
capacity = [6];                 % per-level cache capacity (total m = 6)

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, capacity, ReplacementStrategy.RR);
queue = Queue(model, 'Queue', SchedStrategy.PS);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(1));

% Read class service at the retrieval queue = default per-item fetch service.
queue.setService(jobClass, Exp(1.0));

pAccess = DiscreteSampler(accessProb);
cacheNode.setRead(jobClass, pAccess);

... (20 source lines omitted; full implementation: matlab/examples/basic/)

MVA(model).getAvgCacheTable
NC(model).getAvgCacheTable

% Item-level cache occupancy
MVA(model).getAvgItemTable
NC(model).getAvgItemTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.8.16 retrieval_routing

Retrieval with routing that separates the hit and miss paths.

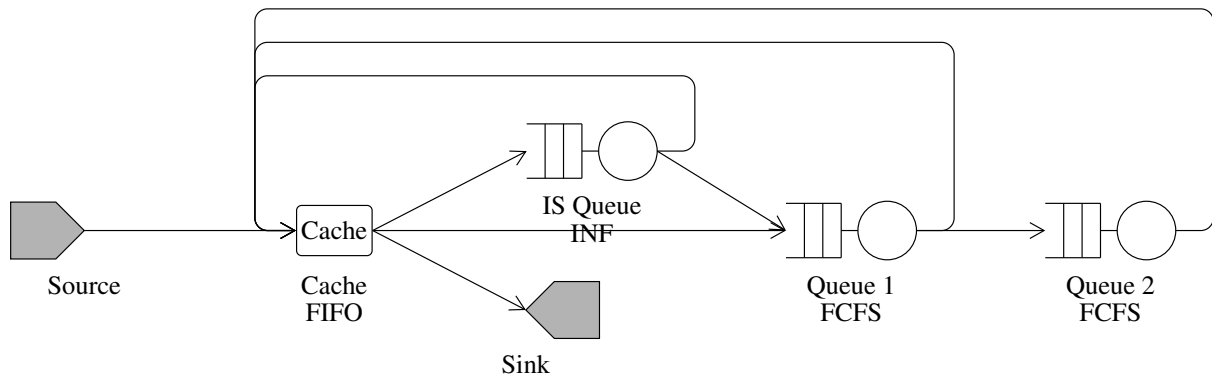


Figure 2.95: Topology of retrieval_routing: an open network over 3 queueing stations with a cache node.

The example is built and solved as follows.

```

% Cache with probabilistic per-item routing through a retrieval system.
clc; clear solver AvgTable;

accessProb = [0.6, 0.3, 0.1]; % per-item access probabilities (3 items)

model = Network('DelayedHits');

n = numel(accessProb);           % number of items
capacity = [2];                 % per-level cache capacity

```


Nodes	6: Source, Queue $\times$ 3, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	IS Queue: INF; Queue 1: FCFS; Queue 2: FCFS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	IS Queue – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(2); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2) Queue 1 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3) Queue 2 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3)
Features	cache: FIFO replacement, 3 items, capacity 2; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at IS Queue; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue 2; class switching on 5 links
Solvers	SSA, LDES, MVA, NC

Table 2.95: Features of retrieval_routing.

```

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, capacity, ReplacementStrategy.FIFO);
isQueue = Queue(model, 'IS Queue', SchedStrategy.INF);
queue1 = Queue(model, 'Queue 1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue 2', SchedStrategy.FCFS);
queues = {isQueue, queue1, queue2};
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(1));

% Read class service per queue = default per-item fetch service (uniform over items).
isQueue.setService(jobClass, Exp(2.0));

... (52 source lines omitted; full implementation: matlab/examples/basic/)

MVA(model).getAvgCacheTable
NC(model).getAvgCacheTable

% Item-level cache occupancy
MVA(model).getAvgItemTable
NC(model).getAvgItemTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.8.17 retrieval_simple

A retrieval system: a cache in front of an infinite-server backing store, hits served at once and misses fetched.

The example is built and solved as follows.

```

% Cache with a single-queue retrieval system (delayed hits).
clc; clear solver AvgTable;

accessProb = [0.6, 0.3, 0.1]; % per-item access probabilities (3 items)

model = Network('DelayedHits');

```

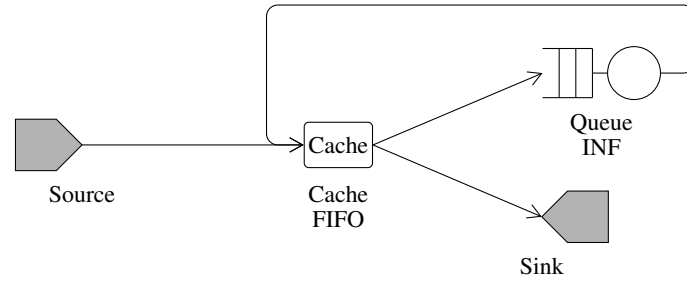


Figure 2.96: Topology of `retrieval_simple`: an open network over 1 queueing station with a cache node.

Nodes	4: Source, Queue, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	Queue: INF
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	Queue – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(2); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2)
Features	cache: FIFO replacement, 3 items, capacity 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue; class switching on 3 links
Solvers	SSA, LDES, MVA, NC

Table 2.96: Features of `retrieval_simple`.

```

n = numel(accessProb);           % number of items
capacity = [1];                 % per-level cache capacity

source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, capacity, ReplacementStrategy.FIFO);
queue = Queue(model, 'Queue', SchedStrategy.INF);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);

source.setArrival(jobClass, Exp(1));

% Read class service at the retrieval queue = default per-item fetch service.
queue.setService(jobClass, Exp(2.0));

pAccess = DiscreteSampler(accessProb);
cacheNode.setRead(jobClass, pAccess);

... (20 source lines omitted; full implementation: matlab/examples/basic/)

MVA(model).getAvgCacheTable
NC(model).getAvgCacheTable

% Item-level cache occupancy
MVA(model).getAvgItemTable
NC(model).getAvgItemTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9 Cluster models

A cluster is a dispatcher in front of a pool of servers. It is not a new node type: the shorthand builds a Router and a set of stations, and the interest is in the dispatching policy and in how the response time scales with the pool size.

2.9.1 cl_basic

A dispatcher in front of a pool of identical servers, built with the cluster shorthand.

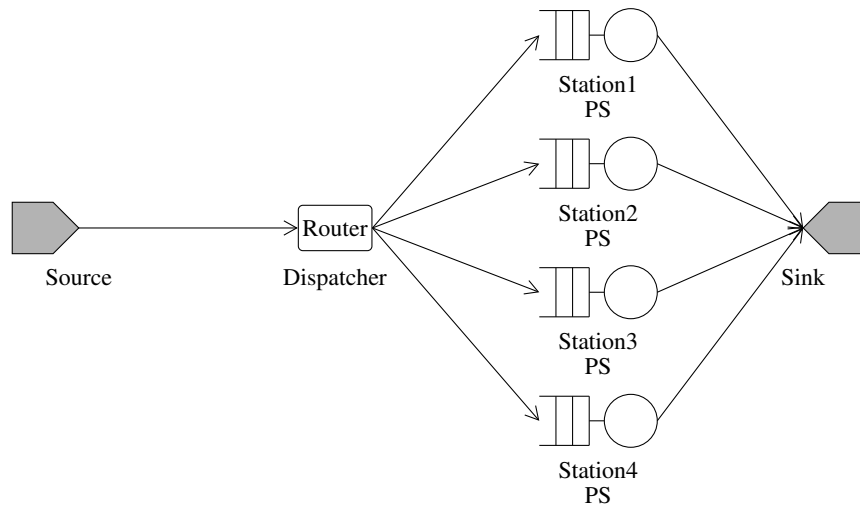


Figure 2.97: Topology of `cl_basic`: an open network over 4 queueing stations with a router.

Nodes	7: Source, Queue $\times$ 4, Router, Sink			
Classes	1: Class1 (open)			
Scheduling	Station1: PS; Station2: PS; Station3: PS; Station4: PS			
Arrivals	Source – Class1: Exp(0.4)			
Service	Station1 – Class1: Exp(1)	Station2 – Class1: Exp(1)	Station3 – Class1: Exp(1)	Station4 – Class1: Exp(1)
Features	class switching on 7 links			
Solvers	MVA			

Table 2.97: Features of `cl_basic`.

The example is built and solved as follows.

```
% SF_BASIC Open cluster with four PS servers and random dispatching.
%
% Source -> Dispatcher -> Server[1..4] -> Sink. The single call to
% Network.clusterPs collapses what would otherwise be ~20 lines of
% manual node/link wiring.

clear all

lambda = 0.4;
D = ones(4, 1); % mean service time = 1 at each server
model = Network.clusterPs(lambda, D, RoutingStrategy.RAND);

solver = MVA(model);
disp(solver.getAvgTable());
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.2 cl_closed

The closed version, with a fixed population of requests.

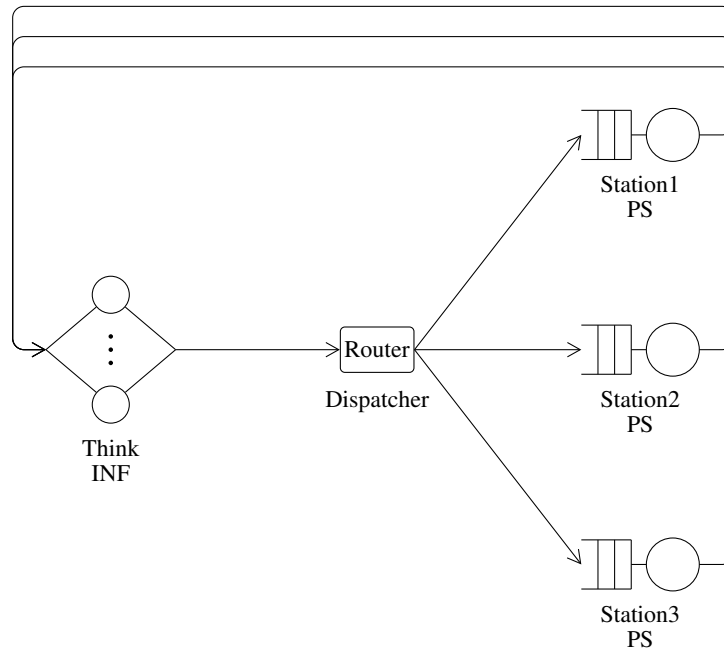


Figure 2.98: Topology of `cl_closed`: a closed network over 4 queueing stations with a router.

Nodes	5: Queue $\times$ 3, Delay, Router
Classes	1: Class1 (closed, $N = 8$)
Scheduling	Think: INF; Station1: PS; Station2: PS; Station3: PS
Service	Think – Class1: Exp(1) Station1 – Class1: Exp(1) Station2 – Class1: Exp(1) Station3 – Class1: Exp(1)
Features	class switching on 5 links
Solvers	MVA

Table 2.98: Features of `cl_closed`.

The example is built and solved as follows.

```
% SF_CLOSED Closed cluster: 8 jobs cycling between Think and 3 PS servers.

clear all

N = 8;
Z = 1.0;
D = ones(3, 1);
strategy = {SchedStrategy.PS, SchedStrategy.PS, SchedStrategy.PS};

model = Network.clusterClosed(N, Z, D, strategy, ones(3, 1), RoutingStrategy.RAND);

solver = MVA(model);
disp(solver.getAvgTable());
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.3 cl_compare

RAND against round-robin dispatching on one cluster.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	SSA
----------------	-----

Table 2.99: Features of `cl_compare`.

The example is built and solved as follows.

```
% SF_COMPARE Compare RAND vs RROBIN dispatching on the same cluster.
%
% MVA only handles RAND routing (product-form). For non-product-form
% policies we use SSA (a discrete-event simulator) with a low sample
% budget for a quick comparison.

clear all

cluster = Cluster().setNumStations(4).setArrivalRate(1.0).setServiceRate(0.4);
cluster.setScheduling(SchedStrategy.PS);

solverFcn = @(m) SSA(m, 'seed', 23000, 'samples', 2000).getAvgTable();

policies = [RoutingStrategy.RAND, RoutingStrategy.RROBIN];
results = cluster.compareDispatching(solverFcn, policies);

keys_ = keys(results);
for k = 1:numel(keys_)
    fprintf('\n=== Dispatching: %s ===\n', keys_(k));
    disp(results{keys_(k)});
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.4 cl_multiclass

Several request classes through one cluster.

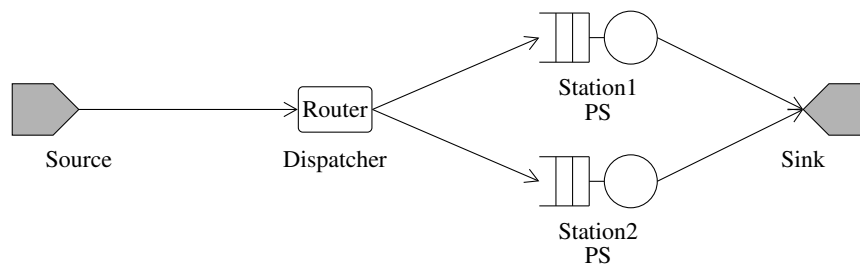


Figure 2.99: Topology of `cl_multiclass`: an open network over 2 queueing stations with a router.

Nodes	5: Source, Queue $\times$ 2, Router, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Station1: PS; Station2: PS
Arrivals	Source – Class1: Exp(0.3); Class2: Exp(0.2)
Service	Station1 – Class1: Exp(1); Class2: Exp(2) Station2 – Class1: Exp(1); Class2: Exp(2)
Features	class switching on 5 links
Solvers	MVA

Table 2.100: Features of `cl_multiclass`.

The example is built and solved as follows.

```
% SF_MULTICLASS Two-class open cluster (e.g. interactive vs batch).

clear all

lambda = [0.3, 0.2];
D = [1.0, 0.5; 1.0, 0.5]; % rows = servers, cols = classes
model = Network.clusterPs(lambda, D, RoutingStrategy.RAND);

solver = MVA(model);
disp(solver.getAvgTable());
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.5 cl_scheduling

The same cluster under FCFS and under PS.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	SSA
---------	-----

Table 2.101: Features of `cl_scheduling`.

The example is built and solved as follows.

```
% CL_SCHEDULING Compare scheduling disciplines on the same cluster.
%
% Cluster.compareScheduling rebuilds the model once per discipline and returns a
% dictionary from the discipline to its AvgTable, restoring the original setting
% on exit. With exponential service FCFS and PS agree in the mean; the contrast
% appears once the service SCV departs from 1.

clear all

cluster = Cluster().setNumStations(3).setArrivalRate(0.9).setServiceRate(0.5);
cluster.setDispatching(RoutingStrategy.RAND);
cluster.setServiceSCV(4.0);

solverFcn = @(m) SSA(m, 'seed', 23000, 'samples', 20000).getAvgTable();

disciplines = [SchedStrategy.FCFS, SchedStrategy.PS];
results = cluster.compareScheduling(solverFcn, disciplines);

% compareScheduling keys by the numeric enum value, so map it back to a name
keys_ = keys(results);
for k = 1:numel(keys_)
    fprintf('\n== Scheduling: %s ==\n', SchedStrategy.toText(str2double(keys_(k))));
    disp(results{keys_(k)});
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.6 cl_stations

Clusters whose members are declared station by station.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	MVA
---------	-----

Table 2.102: Features of `cl_stations`.

The example is built and solved as follows.

```
% CL_STATIONS Horizontal-scaling sweep: response time against the number of
% stations at a fixed total arrival rate.
%
% Cluster.sweepNumStations replicates the single-station service rate across M
% stations and returns a dictionary keyed by M. Adding stations splits the same
% arrival stream, so per-station utilization falls and response time drops
% towards the bare service time.

clear all

cluster = Cluster().setNumStations(1).setArrivalRate(1.6).setServiceRate(1.0);
cluster.setScheduling(SchedStrategy.PS).setDispatching(RoutingStrategy.RAND);

solverFcn = @(m) MVA(m).getAvgTable();
results = cluster.sweepNumStations([2, 3, 4, 6], solverFcn);

counts = sort(keys(results));
for k = 1:numel(counts)
    T = results{counts(k)};
    % getAvgTable returns an IndexedTable; getTable unwraps the MATLAB table
    tbl = T.getTable();
    isStation = ~strcmp(string(tbl.Station), "Source");
    fprintf('\n=== M = %d (mean RespT %.4f, max Util %.4f) ===\n', ...
        counts(k), mean(tbl.RespT(isStation)), max(tbl.Util(isStation)));
    disp(T);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.7 `cl_sweep`

A sweep over the cluster size, and the response time curve it traces.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	MVA
---------	-----

Table 2.103: Features of `cl_sweep`.

The example is built and solved as follows.

```
% SF_SWEEP Arrival-rate sweep on a 2-PS-cluster. Shows response time
% growing as lambda approaches saturation.

clear all

cluster = Cluster().setNumStations(2).setArrivalRate(0.1).setServiceRate(1.0);
cluster.setScheduling(SchedStrategy.PS).setDispatching(RoutingStrategy.RAND);

solverFcn = @(m) MVA(m).getAvgTable();
results = cluster.sweepArrivalRate([0.2, 0.5, 0.9, 1.5], solverFcn);

keys_ = sort(keys(results));
for k = 1:numel(keys_)
    fprintf('\n=== lambda = %g ===\n', keys_(k));
    disp(results{keys_(k)});
end
```

```
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.8 dispatch_closed

The closed version.

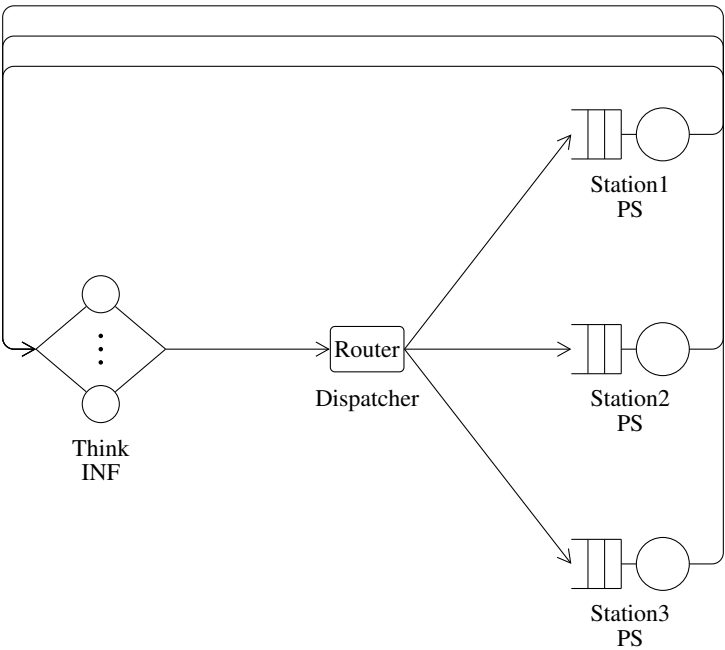


Figure 2.100: Topology of dispatch_closed: a closed network over 4 queueing stations with a router.

Nodes	5: Queue $\times$ 3, Delay, Router
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Think: INF; Station1: PS; Station2: PS; Station3: PS
Service	Think – Class1: Exp(1) Station1 – Class1: Exp(1) Station2 – Class1: Exp(1) Station3 – Class1: Exp(1)
Features	class switching on 5 links
Solvers	MVA, SSA

Table 2.104: Features of dispatch_closed.

The example is built and solved as follows.

```
% DISPATCH_CLOSED Parity test: closed cluster built via the Cluster builder.
%
% 3 PS server stations, single closed class with N = 10 jobs, think time
% Z = 1.0, mu = 1.0, RAND dispatching. Solved exactly via MVA, then
% re-solved by stochastic simulation via SSA for cross-validation. The
% same script runs unchanged under PYTHON, PYTHON-WRAPPER, MATLAB, and
% MATLAB-WRAPPER.

clear all

cluster = Cluster();
cluster.setNumStations(3);
cluster.setServiceRate(1.0);
```



```
cluster.setScheduling(SchedStrategy.PS);
cluster.setClosed(10, 1.0);

avgTable = MVA(cluster.build()).getAvgTable()
ssaTable = SSA(cluster.build(), 'seed', 23000, 'samples', 20000).getAvgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.9 dispatch_mixed

The mixed version, open and closed classes through the same dispatcher.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	MVA, LDES
----------------	-----------

Table 2.105: Features of `dispatch_mixed`.

The example is built and solved as follows.

```
% DISPATCH_MIXED Parity test: mixed cluster built via the Cluster builder.
%
% 2 PS server stations shared by one open class (lambda = 0.5) and one
% closed class (N = 3 jobs, think time Z = 1.0), mu = 2.0 for the open
% class and mu = 1.5 for the closed one, RAND dispatching. Solved by MVA
% and re-solved by simulation via LDES for cross-validation. The same
% script runs unchanged under PYTHON, MATLAB, and MATLAB-WRAPPER.

clear all

cluster = Cluster();
cluster.setNumStations(2);
cluster.setMixed(0.5, 3, 1.0);
cluster.setServiceRates([2.0 1.5; 2.0 1.5]);
cluster.setScheduling(SchedStrategy.PS);

avgTable = MVA(cluster.build()).getAvgTable()
ldesTable = LDES(cluster.build(), 'seed', 23000).getAvgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.9.10 dispatch_open

The dispatcher idiom written out in full, open workload.

Nodes	6: Source, Queue $\times$ 3, Router, Sink
Classes	1: Class1 (open)
Scheduling	Station1: PS; Station2: PS; Station3: PS
Arrivals	Source – Class1: Exp(0.4)
Service	Station1 – Class1: Exp(1) Station2 – Class1: Exp(1) Station3 – Class1: Exp(1)
Features	class switching on 6 links
Solvers	MVA, SSA

Table 2.106: Features of `dispatch_open`.

The example is built and solved as follows.

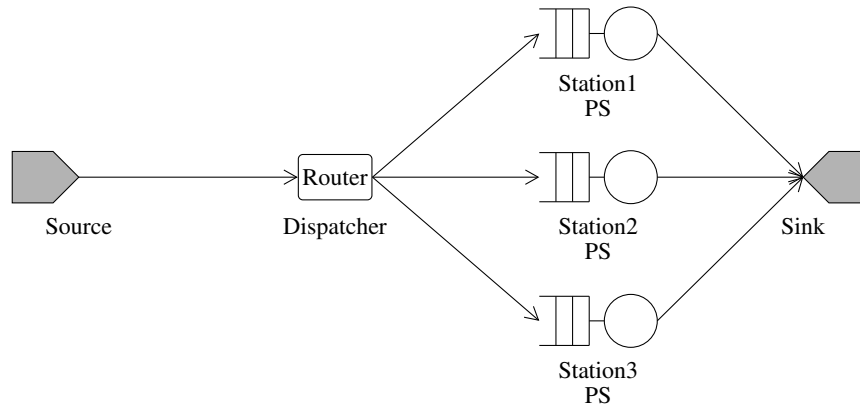


Figure 2.101: Topology of `dispatch_open`: an open network over 3 queueing stations with a router.

```
% DISPATCH_OPEN Parity test: open cluster built via the Cluster builder.
%
% 3 PS server stations, single open class, lambda = 0.4, mu = 1.0, RAND
% dispatching. Solved exactly via MVA, then re-solved by stochastic
% simulation via SSA for cross-validation. The same script runs unchanged
% under PYTHON, PYTHON-WRAPPER, MATLAB, and MATLAB-WRAPPER.

clear all

cluster = Cluster();
cluster.setNumStations(3);
cluster.setArrivalRate(0.4);
cluster.setServiceRate(1.0);
cluster.setScheduling(SchedStrategy.PS);

model = cluster.build();
avgTable = MVA(model).getAvgTable()
ssaTable = SSA(model, 'seed', 23000, 'samples', 20000).getAvgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10 Stochastic Petri nets

A stochastic Petri net is built from `Place` and `Transition` nodes. A transition fires when its input places hold enough tokens, consuming and producing tokens according to the arc weights; timed transitions fire after a random delay, immediate transitions fire at once, and an inhibitor arc disables a transition while its place is marked. A transition may have several firing modes, each with its own rate and arc weights.

Petri nets and queueing models are two notations for the same processes, and two of the scripts make that explicit by building the same system both ways and comparing the generators. Because the firing time need not be exponential, the family also carries the non-Markovian cases.

The figures of this section are drawn in the notation of the net rather than in that of a queueing network. A place is a circle, and the tokens or the number inside it are the marking the model starts from; a transition is a bar, hollow when it fires after a random delay and solid when it fires at once; an arc is plain when it moves one token and labelled with its multiplicity otherwise, and an arc ending in a small circle is an inhibitor, which holds its transition disabled for as long as its place is marked. A transition with more than one firing mode is named with the number of modes, since no single distribution describes it. A circle bisected by a bar is a queueing place, whose tokens are served by a scheduling station embedded in the place.

2.10.1 spn_basic_closed

The closed counterpart, a token circulating between two places.

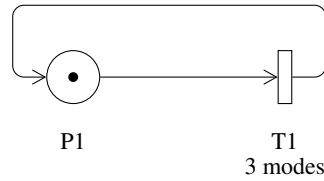


Figure 2.102: Topology of `spn_basic_closed`: a stochastic Petri net over 1 place and 1 transition; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; a transition offering several firing modes is named with their number in place of a distribution.

Nodes	2: Place, Transition
Classes	1: Class1 (closed, $N = 1$)
Features	1 transition with 3 firing modes each
Solvers	JMT

Table 2.107: Features of `spn_basic_closed`.

The example is built and solved as follows.

```
clear P T R solver AvgTable jobclass

model = Network('model');

%% Nodes
P{1} = Place(model, 'P1');
T{1} = Transition(model, 'T1');

% Source
jobclass{1} = ClosedClass(model, 'Class1', 1, P{1});

%% Parameterisation

% T1
mode1 = T{1}.addMode('Mode1');
T{1}.setDistribution(mode1, Exp.fitMean(1));
T{1}.setEnablingConditions(mode1, jobclass{1}, P{1}, 1);
T{1}.setFiringOutcome(mode1, jobclass{1}, P{1}, 1);

mode2 = T{1}.addMode('Mode2');
T{1}.setDistribution(mode2, Erlang.fitMeanAndOrder(1, 2));
T{1}.setEnablingConditions(mode2, jobclass{1}, P{1}, 1);
T{1}.setFiringOutcome(mode2, jobclass{1}, P{1}, 1);

mode3 = T{1}.addMode('Mode3');
T{1}.setDistribution(mode3, HyperExp.fitMeanAndSCV(1, 4));

... (37 source lines omitted; full implementation: matlab/examples/basic/)

solver = {};
solver{1} = JMT(model, options);
AvgTable{1} = solver{1}.getAvgTable();
if height(AvgTable{1}) > 0
    AvgTable{1}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.2 spn_basic_open

One place and one timed transition fed by an open class, the Petri net form of an M/M/1 queue.

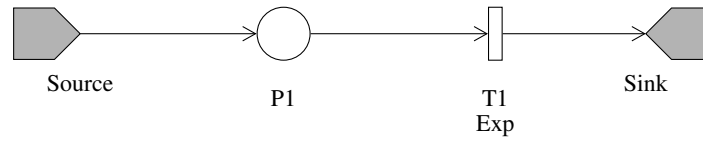


Figure 2.103: Topology of `spn_basic_open`: a stochastic Petri net over 1 place and 1 transition; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the net is open, fed by a Source and drained into a Sink.

Nodes	4: Source, Place, Transition, Sink
Classes	1: Class1 (open)
Arrivals	Source – Class1: Exp(1)
Features	1 transition with 1 firing mode each
Solvers	JMT

Table 2.108: Features of `spn_basic_open`.

The example is built and solved as follows.

```
clear P T R solver AvgTable jobclass

model = Network('model');

%% Nodes
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

P{1} = Place(model, 'P1');

T{1} = Transition(model, 'T1');

% Source
jobclass{1} = OpenClass(model, 'Class1', 0);
source.setArrival(jobclass{1}, Exp(1));

%% Parameterisation

% T1
mode = T{1}.addMode('Model');
T{1}.setNumberOfServers(mode, Inf);
T{1}.setDistribution(mode, Exp(4));
T{1}.setEnablingConditions(mode, jobclass{1}, P{1}, 1);
T{1}.setFiringOutcome(mode, jobclass{1}, sink, 1);

% Routing

... (28 source lines omitted; full implementation: matlab/examples/basic/)

% solver.getAvgTable();

solver = {};
solver{1} = JMT(model, options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.3 spn_closed_fourplaces

Four places with Erlang, Coxian and hyperexponential firing times.

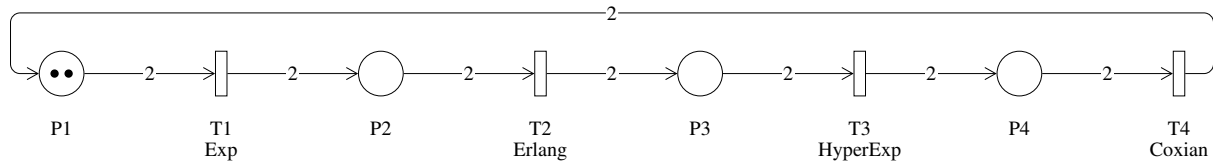


Figure 2.104: Topology of `spn_closed_fourplaces`: a stochastic Petri net over 4 places and 4 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity.

Nodes	8: Place $\times$ 4, Transition $\times$ 4
Classes	1: Class1 (closed, $N = 2$)
Features	4 transitions with 1 firing mode each
Solvers	JMT

Table 2.109: Features of `spn_closed_fourplaces`.

The example is built and solved as follows.

```
clear P T R solver AvgTable

model = Network("model");

P1 = Place(model, "P1");
P2 = Place(model, "P2");
P3 = Place(model, "P3");
P4 = Place(model, "P4");

T1 = Transition(model, "T1");
T2 = Transition(model, "T2");
T3 = Transition(model, "T3");
T4 = Transition(model, "T4");

jobclass = ClosedClass(model, "Class1", 2, P1, 0);

model = T1.addMode("Model1");
T1.setDistribution(model, Exp(2));
T1.setEnablingConditions(model, jobclass, P1, 2);
T1.setFiringOutcome(model, jobclass, P2, 2);

mode2 = T2.addMode("Mode2");
T2.setDistribution(mode2, Erlang(3, 4));
T2.setEnablingConditions(mode2, jobclass, P2, 2);
T2.setFiringOutcome(mode2, jobclass, P3, 2);

... (36 source lines omitted; full implementation: matlab/examples/basic/)

options.seed = 23000;

solver = {};
solver{1} = JMT(model, options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.4 spn_closed_twoplaces

A closed net over two places with Erlang firing times.

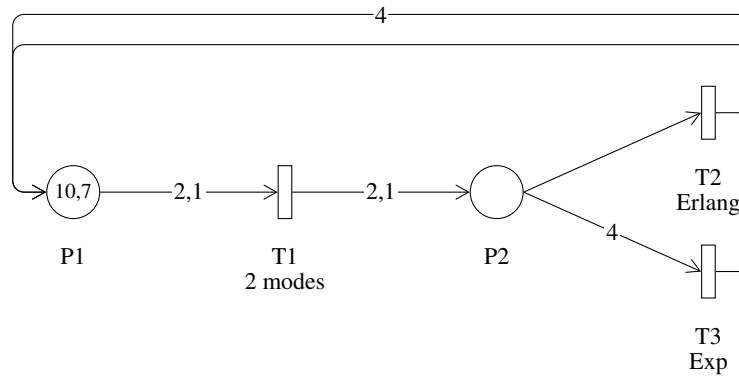


Figure 2.105: Topology of `spn_closed_twoplaces`: a stochastic Petri net over 2 places and 3 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity; a transition offering several firing modes is named with their number in place of a distribution.

Nodes	5: Place $\times$ 2, Transition $\times$ 3
Classes	2: Class1 (closed, $N = 10$), Class2 (closed, $N = 7$)
Features	3 transitions, 1 to 2 firing modes; class switching on 2 links
Solvers	JMT

Table 2.110: Features of `spn_closed_twoplaces`.

The example is built and solved as follows.

```
clear P T R solver AvgTable

model = Network("model");

P1 = Place(model, "P1");
P2 = Place(model, "P2");

T1 = Transition(model, "T1");
T2 = Transition(model, "T2");
T3 = Transition(model, "T3");

jobclass1 = ClosedClass(model, "Class1", 10, P1, 0);
jobclass2 = ClosedClass(model, "Class2", 7, P1, 0);

% T1
model = T1.addMode("Mode1");
%T1.setNumberOfServers(model, Inf);
T1.setDistribution(model, Exp(2));
T1.setEnablingConditions(model, jobclass1, P1, 2);
T1.setFiringOutcome(model, jobclass1, P2, 2);

% T1
mode2 = T1.addMode("Mode2");
%T1.setNumberOfServers(mode2, Inf);
T1.setDistribution(mode2, Exp(3));
T1.setEnablingConditions(mode2, jobclass2, P1, 1);

... (36 source lines omitted; full implementation: matlab/examples/basic/)

options.seed = 23000;
```

```

solver = {};
solver{1} = JMT(model,options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.5 spn_colored_gspn

A colored generalized stochastic Petri net: one firing mode per token color, an immediate transition whose inhibitor arcs make the server exclusive, and firing weights that arbitrate the two colors competing for it.

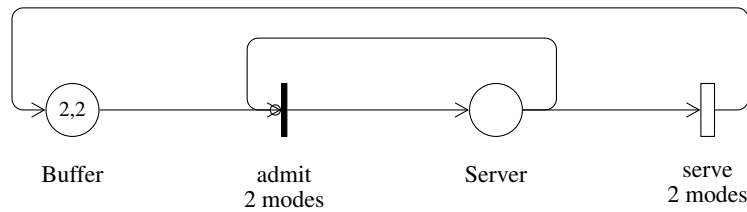


Figure 2.106: Topology of `spn_colored_gspn`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the arc ending in a circle is an inhibitor and holds admit disabled while Server is marked; a transition offering several firing modes is named with their number in place of a distribution.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	2: Gold (closed, $N = 2$), Silver (closed, $N = 2$)
Features	2 transitions with 2 firing modes each
Solvers	CTMC, SSA

Table 2.111: Features of `spn_colored_gspn`.

The example is built and solved as follows.

```

% Colored generalized stochastic Petri net (CGSPN).
%
% A generalized stochastic Petri net mixes IMMEDIATE transitions, which fire
% as soon as they are enabled and consume no time, with TIMED ones. It becomes
% COLORED when the tokens carry a type: in LINE a token color is a job class,
% so a place holds a marking per color and a transition declares one MODE per
% color it serves.
%
% Here two colors, Gold and Silver, circulate between a Buffer place and a
% Server place. The immediate transition admit moves one token into the empty
% server; its two modes are held apart by inhibiting arcs on both colors, so
% the server is a mutual-exclusion resource and the firing weights arbitrate
% between the colors whenever both are waiting. The timed transition serve
% returns the token to the buffer at a color-dependent rate.
%
% Admission is immediate, so the server is never idle and the model is exact
% by hand: the weights split the completions 2:1 in favour of Gold, hence
% X_Gold = 2*X_Silver, U_c = X_c/mu_c and U_Gold + U_Silver = 1.

model = Network('ColoredGSPN');

% Two places and two transitions. Declare every node before parameterizing
% any of it: the enabling, inhibiting and firing matrices are indexed by
% (node, class) over the nodes that exist when they are first set.

```

```

buf = Place(model, 'Buffer');
srv = Place(model, 'Server');

... (47 source lines omitted; full implementation: matlab/examples/basic/)

buf.setState([2, 2]);
srv.setState([0, 0]);

% Exact solution of the underlying CTMC, and a simulation cross-check.
AvgTableCTMC = SolverCTMC(model, 'cutoff', 4).avgTable()
AvgTableSSA = SolverSSA(model, 'seed', 23000, 'samples', 2e5).avgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.6 spn_fluid_dae

Fluid DAE analysis of stochastic Petri nets, including invariant constraints, immediate-transition flows, bounded markings and marking covariances.

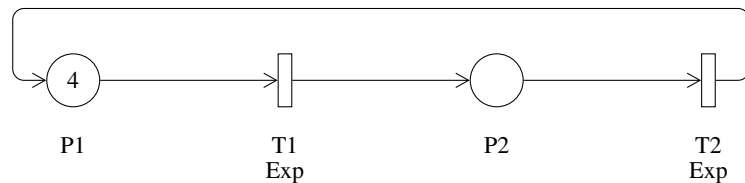


Figure 2.107: Topology of `spn_fluid_dae`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	1: Class1 (closed, $N = 4$)
Features	2 transitions with 1 firing mode each
Solvers	FLD, CTMC

Table 2.112: Features of `spn_fluid_dae`.

The example is built and solved as follows.

```

% Fluid (mean-field) analysis of a stochastic Petri net, with SolverFLD.
%
% A GSPN is a density-dependent Markov population process: the marking is the
% population, a transition mode is a reaction, and the firing rate
% lambda*min(enabling degree, servers) is the same min() non-linearity the
% min-normal closure of SolverFLD exists to smooth. The 'dae' method is the one
% that can carry it, because a Petri net needs three things stated as EQUATIONS
% rather than integrated:
%
% * the P-invariants, which hold to solver tolerance instead of integrator
%   tolerance -- and supply the rank the drift Jacobian is missing;
% * the firing FLOW of an immediate transition, an algebraic unknown pinned by
%   the constraint that its input place holds no mass;
% * a bounded place, a linear inequality on the marking.
%
% SolverFLD(model) resolves to 'dae' on any model holding a Transition node, so
% no method has to be named. Unlike every other solver of a Petri net in LINE it
% also returns a SECOND MOMENT: the marking covariance of the linear noise
% approximation, through getMoments().

clear model P T R jc solver

```



```

%% -- a closed net whose fluid answer is EXACT -----
% Every mode is infinite-server with one input arc, so min(m/w, Inf) = m and the
% drift is LINEAR: the fluid mean is then the exact mean, and the covariance the
% exact covariance (a binomial marking).

... (62 source lines omitted; full implementation: matlab/examples/basic/)

Rr.set(jr, jr, W1, V1, 1.0); Rr.set(jr, jr, V1, W3, 1.0);
Rr.set(jr, jr, W3, V3, 1.0); Rr.set(jr, jr, V3, W1, 1.0);
red.link(Rr);
W1.setState(jr.population); W3.setState(0);

AvgTableReduced = SolverFLD(red).getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.7 spn_fourmodes

Four firing modes, and the mode selection probabilities.

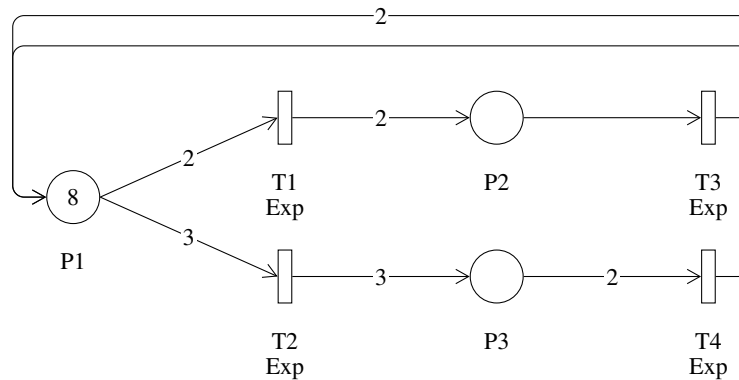


Figure 2.108: Topology of `spn_fourmodes`: a stochastic Petri net over 3 places and 4 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity.

Nodes	7: Place $\times$ 3, Transition $\times$ 4
Classes	1: Class1 (closed, $N = 8$)
Features	4 transitions with 1 firing mode each
Solvers	JMT

Table 2.113: Features of `spn_fourmodes`.

The example is built and solved as follows.

```

clear P T R solver AvgTable

model = Network("model");

P1 = Place(model, "P1");
P2 = Place(model, "P2");
P3 = Place(model, "P3");

T1 = Transition(model, "T1");
T2 = Transition(model, "T2");
T3 = Transition(model, "T3");
T4 = Transition(model, "T4");

jobclass = ClosedClass(model, "Class1", 8, P1, 0);

```

```

model1 = T1.addMode("Model1");
%T1.setNumberOfServers(model1, Inf);
T1.setDistribution(model1, Exp(2));
T1.setEnablingConditions(model1, jobclass, P1, 2);
T1.setFiringOutcome(model1, jobclass, P2, 2);

mode2 = T2.addMode("Mode2");
%T2.setNumberOfServers(mode2, Inf);
T2.setDistribution(mode2, Exp(1));
T2.setEnablingConditions(mode2, jobclass, P1, 3);
T2.setFiringOutcome(mode2, jobclass, P3, 3);

... (35 source lines omitted; full implementation: matlab/examples/basic/)

options.seed = 23000;

solver = {};
solver{1} = JMT(model, options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.8 spn_inhibiting

An inhibitor arc, which disables a transition while its place is marked.

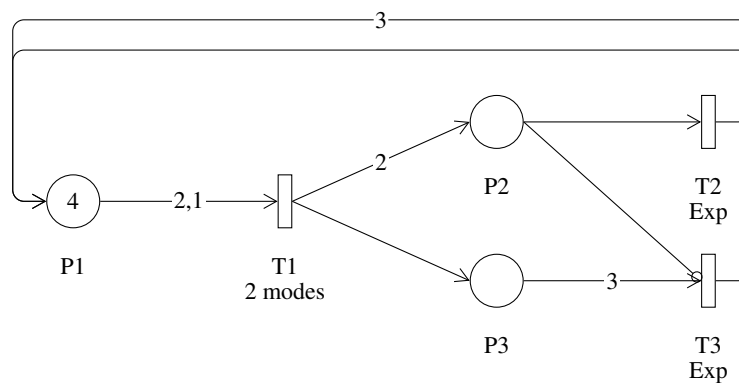


Figure 2.109: Topology of `spn_inhibiting`: a stochastic Petri net over 3 places and 3 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity; the arc ending in a circle is an inhibitor and holds T3 disabled while P2 is marked; a transition offering several firing modes is named with their number in place of a distribution.

Nodes	6: Place $\times$ 3, Transition $\times$ 3
Classes	1: Class1 (closed, $N = 4$)
Features	3 transitions, 1 to 2 firing modes
Solvers	JMT

Table 2.114: Features of `spn_inhibiting`.

The example is built and solved as follows.

```

clear P T R solver AvgTable

model = Network("model");

```

```

P1 = Place(model, "P1");
P2 = Place(model, "P2");
P3 = Place(model, "P3");

T1 = Transition(model, "T1");
T2 = Transition(model, "T2");
T3 = Transition(model, "T3");

jobclass = ClosedClass(model, "Class1", 4, P1, 0);

model = T1.addMode("Model");
T1.setDistribution(model, Exp(2));
T1.setEnablingConditions(model, jobclass, P1, 2);
T1.setFiringOutcome(model, jobclass, P2, 2);

mode2 = T1.addMode("Mode2");
T1.setDistribution(mode2, Exp(1));
T1.setEnablingConditions(mode2, jobclass, P1, 1);
T1.setFiringOutcome(mode2, jobclass, P3, 1);

mode3 = T2.addMode("Mode3");
T2.setDistribution(mode3, Exp(4));

... (32 source lines omitted; full implementation: matlab/examples/basic/)

options.seed = 23000;

solver = {};
solver{1} = JMT(model,options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.9 spn_lpbounds

Linear-programming bounds on a Petri net with SolverBA: the stationary chain is relaxed to a moment polytope and every reported measure minimised and maximised over it, reproducing Table 2 of Liu (1998) on a blocking production line and then bracketing the exact marking of an inhibitor net.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC
---------	------

Table 2.115: Features of spn_lpbounds.

The example is built and solved as follows.

```

% SPN_LPBOUNDS  Bound a stochastic Petri net by linear programming.
%
% SolverBA's 'spnlp' family is the first BOUNDING route LINE offers for a Petri
% net. SolverCTMC builds the explicit generator, SolverSSA and SolverLDES
% simulate, SolverFLD fluidises, and SolverNC 'rec' needs a product form; this
% one needs none of that. It relaxes the stationary chain to a MOMENT POLYTOPE
% -- the uniformized evolution equation written for E[X_p], E[X_p^2] and
% E[X_p1 X_p2], plus behavioural and probabilistic inequalities -- and then
% minimises and maximises each reported measure over it. Every stationary point
% of the true chain satisfies every row, so the two optima bracket the exact
% value.
%
%   SPN_LPBND    assembles the polytope and solves the LPs
%   SPN_SINVARIANTS  supplies the weighted place invariants, which are both the

```

```

% conservation equalities and the a priori per-place caps
%
% Reference: Z. Liu, "Performance Analysis of Stochastic Timed Petri Nets Using
% Linear Programming Approach", IEEE Trans. Software Engineering 24(11), 1998,
% 1014-1030.
%% The reference's own Fig. 2b, and its Table 2
%
% Four servers in a line, blocking before service: server i cannot start until
% the downstream buffer has a free slot. The buffers hold 3, 2 and 4, and each
% is a conserved pair of places -- (p5,p2), (p4,p1), (p3,p0) -- so the net is a
% strongly connected marked graph and all four transitions carry the same
%
... (109 source lines omitted; full implementation: matlab/examples/basic/)
R.set(jc,jc,P2,T3,1.0); R.set(jc,jc,P3,T3,1.0);
R.set(jc,jc,T1,P2,1.0); R.set(jc,jc,T1,P3,1.0);
R.set(jc,jc,T2,P1,1.0); R.set(jc,jc,T3,P1,1.0);
model.link(R);
P1.setState(n); P2.setState(0); P3.setState(0);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.10 spn_open_sevenplaces

A seven-place open net, large enough that the state space matters.

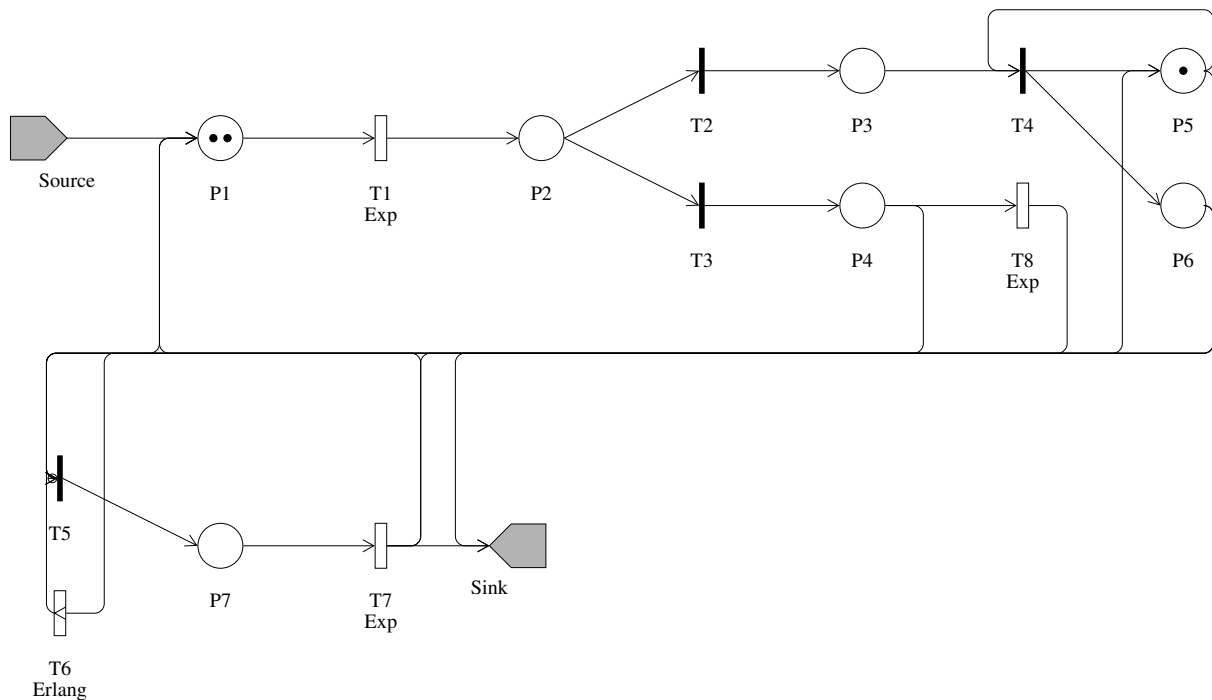


Figure 2.110: Topology of `spn_open_sevenplaces`: a stochastic Petri net over 7 places and 8 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the arc ending in a circle is an inhibitor and holds T5 disabled while P6 is marked; the net is open, fed by a Source and drained into a Sink.

The example is built and solved as follows.

```
clear P T R solver AvgTable
```

Nodes	17: Source, Place $\times$ 7, Transition $\times$ 8, Sink
Classes	1: Class1 (open)
Arrivals	Source – Class1: Exp(1)
Features	8 transitions with 1 firing mode each
Solvers	JMT

Table 2.116: Features of `spn_open_sevenplaces`.

```

model = Network('model');

%% Nodes
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

P{1} = Place(model, 'P1');
P{2} = Place(model, 'P2');
P{3} = Place(model, 'P3');
P{4} = Place(model, 'P4');
P{5} = Place(model, 'P5');
P{6} = Place(model, 'P6');
P{7} = Place(model, 'P7');

T{1} = Transition(model, 'T1');
T{2} = Transition(model, 'T2');
T{3} = Transition(model, 'T3');
T{4} = Transition(model, 'T4');
T{5} = Transition(model, 'T5');
T{6} = Transition(model, 'T6');
T{7} = Transition(model, 'T7');
T{8} = Transition(model, 'T8');

% Source

... (137 source lines omitted; full implementation: matlab/examples/basic/)

% solver.getAvgTable();

solver = {};
solver{1} = JMT(model, options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.11 `spn_pareto_service`

A Pareto firing time, a heavy-tailed non-Markovian net.

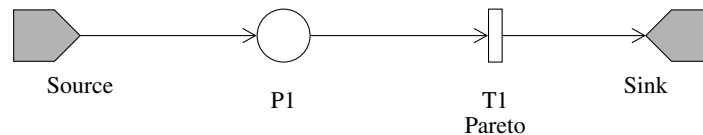


Figure 2.111: Topology of `spn_pareto_service`: a stochastic Petri net over 1 place and 1 transition; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the net is open, fed by a Source and drained into a Sink.

The example is built and solved as follows.

```
clear P T R solver AvgTable jobclass
```

Nodes	4: Source, Place, Transition, Sink
Classes	1: Class1 (open)
Arrivals	Source – Class1: Exp(0.5)
Features	1 transition with 1 firing mode each
Solvers	JMT

Table 2.117: Features of `spn_pareto_service`.

```

model = Network('model');

%% Nodes
source = Source(model, 'Source');
sink = Sink(model, 'Sink');

P{1} = Place(model, 'P1');

T{1} = Transition(model, 'T1');

% Source
jobclass{1} = OpenClass(model, 'Class1', 0);
source.setArrival(jobclass{1}, Exp(0.5)); % arrival rate 0.5

%% Parameterisation

% T1 with Pareto service time
% Pareto(shape, scale) - shape must be >= 2
% With shape=3 and scale=1, mean service time = 3*1/(3-1) = 1.5
mode = T{1}.addMode('Model1');
T{1}.setNumberOfServers(mode, 1);
T{1}.setDistribution(mode, Pareto(3, 1)); % Pareto with shape=3, scale=1
T{1}.setEnablingConditions(mode, jobclass{1}, P{1}, 1);
T{1}.setFiringOutcome(mode, jobclass{1}, sink, 1);

... (14 source lines omitted; full implementation: matlab/examples/basic/)

options.samples = 1e4; % limit samples for faster execution

solver = {};
solver{1} = JMT(model, options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.12 `spn_productform_nc`

Product-form analysis of cyclic and fork–join stochastic Petri nets with SolverNC, compared against an explicit CTMC and accompanied by the structural certificate and marking invariants.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	NC, CTMC
----------------	----------

Table 2.118: Features of `spn_productform_nc`.

The example is built and solved as follows.

```

% SPN_PRODUCTFORM_NC Solve a stochastic Petri net analytically with SolverNC.
%
% SolverNC's 'rec' method is the first ANALYTICAL route LINE offers for a Petri
% net: SolverCTMC builds the explicit generator, SolverSSA and SolverLDES

```

```
% simulate, SolverFLD fluidises. It works in three steps, each with its own
% reference:
%
%   SPN_PF      decides whether the net has a product form and derives the
%               per-place factors  $g_l$ , by complex balance
%               (Coleman-Henderson-Taylor, Perform. Eval. 26(3), 1996)
%   MDD_REC     evaluates  $G = \text{sum over the reachable set of prod}_l g_l(m_l)$  by
%               ONE memoised walk of the decision diagram holding that set
%               (Balsamo-Marin-Stojic, FGCS 111 (2020) 475-490)
%   SPN_METRICS reads the mean tokens, the utilisations and the throughputs off
%               masked walks of the same diagram
%
% Two nets are solved here. The first is a closed cycle, which a queueing
% network could also express. The second FORKS: its transition Tf consumes one
% token and produces two, and Tj consumes two and produces one, so the marking
% is not a conserved job population and there is no queueing-network
% counterpart -- which is exactly the limitation the MDD-rec paper opens with.

%% A closed cycle, where the exact CTMC gives the reference
model = i_cyclic(4);
nc = SolverNC(model);
line_printf('\n--- 3-place cyclic net, N = 4 ---\n');

... (66 source lines omitted; full implementation: matlab/examples/basic/)

R{1,1}(P{1}, Tf) = 1; R{1,1}(Tf, P{2}) = 1; R{1,1}(Tf, P{3}) = 1;
R{1,1}(P{2}, Tj) = 1; R{1,1}(P{3}, Tj) = 1; R{1,1}(Tj, P{4}) = 1;
R{1,1}(P{4}, Tb) = 1; R{1,1}(Tb, P{1}) = 1;
model.link(R);
for i = 1:4, P{i}.setState((i == 1) * njobs); end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.13 spn_queueing_place

A queueing station and a Petri net in one model, the two formalisms side by side.

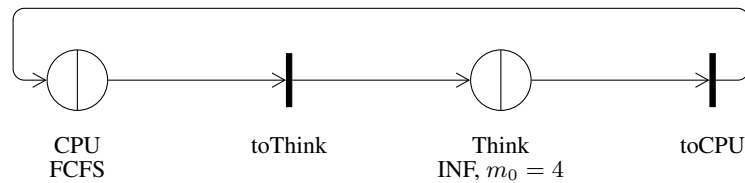


Figure 2.112: Topology of `spn_queueing_place`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the 2 bisectioned circles are queueing places, whose tokens are served by an embedded station.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	1: Jobs (closed, $N = 4$)
Service	CPU – Jobs: Exp(1.5) Think – Jobs: Exp(0.5)
Features	2 transitions with 1 firing mode each
Solvers	MVA, LDES

Table 2.119: Features of `spn_queueing_place`.

The example is built and solved as follows.

```
% Closed queueing Petri net (QPN) with two queueing places.
```

```
%
% A queueing place embeds a scheduling station inside a Petri-net place: an
% arriving token is served by the place's embedded queue and, on completion,
% moves to a depository from which the output transitions consume it.
%
% Here a CPU (single-server FCFS queueing place) and a think stage (infinite-
% server queueing place) exchange a fixed population of N tokens through two
% immediate transitions. This is the queueing-Petri-net rendering of a machine-
% repairman / finite-population model, so its exact solution is available from
% SolverMVA on the equivalent Delay+Queue network for cross-validation.

N = 4;
model = Network('QueueingPetriNet');

% Two queueing places: setService turns each Place into a queueing place whose
% embedded queue is served under the scheduling strategy of its constructor.
cpu = Place(model, 'CPU', SchedStrategy.FCFS); % single-server embedded queue
think = Place(model, 'Think', SchedStrategy.INF); % infinite-server embedded queue

jobs = ClosedClass(model, 'Jobs', N, think, 0);
cpu.setService(jobs, Exp(1.5)); % embedded FCFS service, rate 1.5
think.setService(jobs, Exp(0.5)); % embedded think time, mean 2

% Two immediate transitions cycle one token per firing between the places.
toCPU = Transition(model, 'toCPU');

... (28 source lines omitted; full implementation: matlab/examples/basic/)

queue = Queue(ref, 'CPU', SchedStrategy.FCFS);
cc = ClosedClass(ref, 'Jobs', N, delay, 0);
delay.setService(cc, Exp(0.5));
queue.setService(cc, Exp(1.5));
ref.link(Network.serialRouting(delay, queue));
AvgTableMVA = SolverMVA(ref).avgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.14 spn_twomodes

A transition with two firing modes.

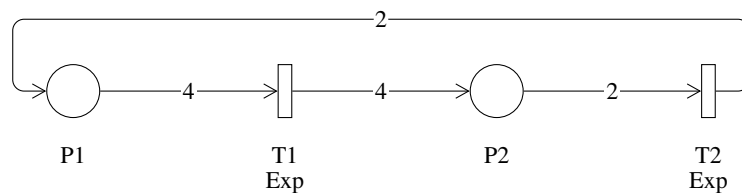


Figure 2.113: Topology of `spn_twomodes`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	1: Class1 (closed, $N = 10$)
Features	2 transitions with 1 firing mode each
Solvers	JMT

Table 2.120: Features of `spn_twomodes`.

The example is built and solved as follows.


```

clear P T R solver AvgTable

model = Network('model');

P1 = Place(model, 'P1');
P2 = Place(model, 'P2');
T1 = Transition(model, 'T1');
T2 = Transition(model, 'T2');

jobclass = ClosedClass(model, 'Class1', 10, P1, 0); % automatically added to P1 at
initialization

% T1
mode = T1.addMode('Model1');
%T1.setNumberOfServers(mode, Inf)
T1.setDistribution(mode, Exp(2));
T1.setEnablingConditions(mode, jobclass, P1, 4);
T1.setFiringOutcome(mode, jobclass, P2, 4);

% T2
mode = T2.addMode('Mode2');
%T2.setNumberOfServers(mode, Inf)
T2.setDistribution(mode, Exp(3));
T2.setEnablingConditions(mode, jobclass, P2, 2);
T2.setFiringOutcome(mode, jobclass, P1, 2);

routingMatrix = model.initRoutingMatrix();

... (12 source lines omitted; full implementation: matlab/examples/basic/)

options.seed = 23000;

solver = {};
solver{1} = JMT(model,options);
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.15 test_spn_nrm_open

The open net solved by the next-reaction method, against the exact generator.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	JMT
---------	-----

Table 2.121: Features of test_spn_nrm_open.

The example is built and solved as follows.

```

% test_spn_nrm_open
% Validates the SSA Next-Reaction-Method (NRM) stochastic-Petri-net path on OPEN
% nets: a Source feeds a Place whose tokens drain through a Transition to a Sink.
% Before the Source-arrival reaction was added the fed Place stayed empty and the
% run threw "Deadlock: no transition is enabled".
%
% Each net asserts (a) the solver actually ran method 'nrm' (never a silent
% serial fallback) and (b) the simulated marking mean and throughput match the
% analytic M/M/1 result. A Source Exp(lambda) feeding a single-server Transition
% Exp(mu) is an M/M/1 queue at the Place: mean tokens = rho/(1-rho), throughput =
% lambda (rho = lambda/mu < 1). The canonical net is cross-checked against JMT.
clear;

```

```

RTOL = 0.04;
SAMPLES = 3e5;
SEED = 23000;

%% --- Net 1: M/M/1 SPN, Source Exp(0.5) -> P1 -> T1 Exp(1.0) -> Sink -----
lambda = 0.5; mu = 1.0; rho = lambda/mu;
qExact = rho/(1-rho); % = 1.0
model = mmlspn(lambda, mu);
solver = SolverSSA(model, 'method', 'nrm', 'samples', SAMPLES, 'seed', SEED);
[Qn,~,~,Tn] = solver.getAvg();
assert(~isempty(strfind(solver.result.Avg.method, 'nrm')), 'net1 did not run NRM');
% stations: Source(1), P1(2)
assert(abs(Qn(2)-qExact)/qExact < RTOL, sprintf('net1 P1 tokens %g vs exact %g', Qn(2),
    qExact));

... (51 source lines omitted; full implementation: matlab/examples/basic/)

T2.setFiringOutcome(m2, jobclass, sink, 1);
R = model.initRoutingMatrix();
R{1,1}(source,P1) = 1; R{1,1}(P1,T1) = 1;
R{1,1}(T1,P2) = 1; R{1,1}(P2,T2) = 1; R{1,1}(T2,sink) = 1;
model.link(R);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11 Workflow models

The workflow models express the standard control-flow patterns – sequence, probabilistic branch, parallel split and join, loop – as activity graphs, and solve them for the completion time of the whole workflow rather than for station metrics.

2.11.1 wf_aph

Acyclic phase-type activity durations.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

%% Workflow with APH Distributions
% A workflow using APH (Acyclic Phase-Type) distributions
% for flexible variability modeling.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;

%% Define the workflow
wf = Workflow('APHWorkflow');

% Add activities with different SCVs using APH
A = wf.addActivity('A', APH.fitMeanAndSCV(1.0, 0.5)); % Low variability
B = wf.addActivity('B', APH.fitMeanAndSCV(2.0, 2.0)); % High variability
C = wf.addActivity('C', APH.fitMeanAndSCV(1.5, 1.0)); % Standard variability

% Define serial precedence
wf.addPrecedence(Workflow.Serial(A, B, C));

%% Convert to phase-type distribution

```

```

ph = wf.toPH();

%% Display results
fprintf('APH Workflow: A -> B -> C\n');
fprintf('Activity A: APH(mean=1.0, SCV=0.5)\n');
fprintf('Activity B: APH(mean=2.0, SCV=2.0)\n');
fprintf('Activity C: APH(mean=1.5, SCV=1.0)\n');
fprintf('Expected total mean: %.2f\n', 1.0 + 2.0 + 1.5);
fprintf('Computed PH mean: %.4f\n', ph.getMean());
fprintf('Computed PH SCV: %.4f\n', ph.getSCV());
fprintf('Number of phases: %d\n', size(ph.getSubgenerator(), 1));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11.2 wf_branch

A probabilistic branch and its merge.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

%% Probabilistic Branching Workflow Example (OR-fork/join)
% A workflow with probabilistic choice between branches.
%
% Workflow:
%   +--> B (60%) --+
%   A --|           |--> D
%   +--> C (40%) --+
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;

%% Define the workflow
wf = Workflow('BranchingWorkflow');

% Add activities
A = wf.addActivity('A', Exp.fitMean(1.0));
B = wf.addActivity('B', Exp.fitMean(2.0));
C = wf.addActivity('C', Exp.fitMean(5.0));
D = wf.addActivity('D', Exp.fitMean(0.5));

% Define precedences with probabilities
wf.addPrecedence(Workflow.OrFork(A, {B, C}, [0.6, 0.4]));
wf.addPrecedence(Workflow.OrJoin({B, C}, D));

... (6 source lines omitted; full implementation: matlab/examples/basic/)

fprintf('Activity means: A=1.0, B=2.0, C=5.0, D=0.5\n');
expected_branch = 0.6 * 2.0 + 0.4 * 5.0;
fprintf('Expected branch mean: 0.6*2.0 + 0.4*5.0 = %.2f\n', expected_branch);
fprintf('Expected total mean: %.2f\n', 1.0 + expected_branch + 0.5);
fprintf('Computed PH mean: %.4f\n', ph.getMean());
fprintf('Number of phases: %d\n', size(ph.getSubgenerator(), 1));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11.3 wf_complex

Nested branches, forks and loops in one workflow.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
%% Complex Workflow Example
% A workflow combining serial, parallel, and branching patterns.
%
% Workflow:
%
%      +--> C --+
%      |           |
%  A -> B -> [AND] |           | [AND] -> F -> G
%      |           |
%      +--> D --+
%
%      |
%      +--> [OR] -> E (30%)
%
%      +--> skip (70%)
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;

%% Define the workflow
wf = Workflow('ComplexWorkflow');

% Add activities
A = wf.addActivity('A', Exp.fitMean(0.5));
B = wf.addActivity('B', Exp.fitMean(1.0));
C = wf.addActivity('C', Exp.fitMean(2.0));
D = wf.addActivity('D', Exp.fitMean(1.5));
F = wf.addActivity('F', Exp.fitMean(1.0));

... (26 source lines omitted; full implementation: matlab/examples/basic/)

fprintf('\nSampling 1000000 workflow execution times...\n');
samples = ph.sample(1000000);
fprintf('Sample mean: %.4f\n', mean(samples));
fprintf('Sample std: %.4f\n', std(samples));
fprintf('Sample min: %.4f\n', min(samples));
fprintf('Sample max: %.4f\n', max(samples));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11.4 wf_erlang

Erlang activity durations.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
%% Workflow with Erlang Distribution
% A workflow using Erlang distributions for lower variability.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;

%% Define the workflow
wf = Workflow('ErlangWorkflow');

% Add activities with Erlang distributions (SCV < 1)
A = wf.addActivity('A', Erlang.fitMeanAndOrder(2.0, 4)); % 4 phases, SCV=0.25
```

```

B = wf.addActivity('B', Erlang.fitMeanAndOrder(3.0, 2)); % 2 phases, SCV=0.5
C = wf.addActivity('C', Exp.fitMean(1.0)); % 1 phase, SCV=1.0

% Define serial precedence
wf.addPrecedence(Workflow.Serial(A, B, C));

%% Convert to phase-type distribution
ph = wf.toPH();

%% Display results
fprintf('Erlang Workflow: A -> B -> C\n');
fprintf('Activity A: Erlang(mean=2.0, phases=4), SCV=0.25\n');
fprintf('Activity B: Erlang(mean=3.0, phases=2), SCV=0.50\n');
fprintf('Activity C: Exp(mean=1.0), SCV=1.00\n');
fprintf('Expected total mean: %.2f\n', 2.0 + 3.0 + 1.0);
fprintf('Computed PH mean: %.4f\n', ph.getMean());
fprintf('Computed PH SCV: %.4f\n', ph.getSCV());
fprintf('Number of phases: %d\n', size(ph.getSubgenerator(), 1));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11.5 wf_loop

A loop with a repetition count.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

%% Loop Workflow Example
% A workflow with a repeated activity.
%
% Workflow: A -> [B x 3] -> C
% Activity B is executed 3 times.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;

%% Define the workflow
wf = Workflow('LoopWorkflow');

% Add activities
A = wf.addActivity('A', Exp.fitMean(1.0));
B = wf.addActivity('B', Exp.fitMean(2.0));
C = wf.addActivity('C', Exp.fitMean(0.5));

% Define loop: A runs once, B runs 3 times, C runs once
wf.addPrecedence(Workflow.Loop(A, {B, C}, 3));

%% Convert to phase-type distribution
ph = wf.toPH();

%% Display results
fprintf('Loop Workflow: A -> [B x 3] -> C\n');
fprintf('Activity means: A=1.0, B=2.0, C=0.5\n');
fprintf('Expected total mean: 1.0 + 3*2.0 + 0.5 = %.2f\n', 1.0 + 3*2.0 + 0.5);
fprintf('Computed PH mean: %.4f\n', ph.getMean());
fprintf('Number of phases: %d\n', size(ph.getSubgenerator(), 1));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11.6 wf_parallel

An AND-fork, parallel activities and a join.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
%% Parallel Workflow Example (AND-fork/join)
% A workflow with parallel activities that synchronize.
%
% Workflow:
%   +--> B --+
%   A --|       |--> D
%   +--> C --+
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;

%% Define the workflow
wf = Workflow('ParallelWorkflow');

% Add activities
A = wf.addActivity('A', Exp.fitMean(1.0));
B = wf.addActivity('B', Exp.fitMean(2.0));
C = wf.addActivity('C', Exp.fitMean(3.0));
D = wf.addActivity('D', Exp.fitMean(0.5));

% Define precedences
wf.addPrecedence(Workflow.AndFork(A, {B, C}));
wf.addPrecedence(Workflow.AndJoin({B, C}, D));

... (10 source lines omitted; full implementation: matlab/examples/basic/)

lambda_C = 1/3; % rate for mean 3.0
expected_max = 2.0 + 3.0 - 1/(lambda_B + lambda_C);
fprintf('Expected max(B,C) mean: %.4f\n', expected_max);
fprintf('Expected total mean: %.4f\n', 1.0 + expected_max + 0.5);
fprintf('Computed PH mean: %.4f\n', ph.getMean());
fprintf('Number of phases: %d\n', size(ph.getSubgenerator(), 1));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.11.7 wf_serial

A sequence of activities.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
%% Serial Workflow Example
% A simple workflow with three sequential activities.
%
% Workflow: A -> B -> C
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear;
lineStart;
```

```
%% Define the workflow
wf = Workflow('SerialWorkflow');

% Add activities with exponential service times
A = wf.addActivity('A', Exp.fitMean(1.0));
B = wf.addActivity('B', Exp.fitMean(2.0));
C = wf.addActivity('C', Exp.fitMean(1.5));

% Define serial precedence
wf.addPrecedence(Workflow.Serial(A, B, C));

%% Convert to phase-type distribution
ph = wf.toPH();

%% Display results
fprintf('Serial Workflow: A -> B -> C\n');
fprintf('Activity means: A=1.0, B=2.0, C=1.5\n');
fprintf('Expected total mean: %.2f\n', 1.0 + 2.0 + 1.5);
fprintf('Computed PH mean: %.4f\n', ph.getMean());
fprintf('Computed PH SCV: %.4f\n', ph.getSCV());
fprintf('Number of phases: %d\n', size(ph.getSubgenerator(), 1));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Chapter 3

Advanced models

The advanced examples cover the features that leave the product-form setting: load dependence, blocking, state-dependent routing, rewards, random environments and the disciplines that no closed form covers. They live under `matlab/examples/advanced/`.

Each of these features narrows the set of solvers that can be used, and the scripts say which. A model that a solver cannot represent is refused rather than approximated silently, so the feature set of a solver is discoverable from the model itself.

3.1 Load-dependent stations

A load-dependent station has a service rate that is a function of the number of jobs it holds, declared through `setLoadDependence` as a rate vector $\mu(n)$. A multiserver station is the special case $\mu(n) = \min(n, c)\mu$, and writing it out as a load-dependent station is how the scripts check the two against each other. Class dependence, declared through `setClassDependence`, lets the rate depend on the class mix rather than on the total only.

The same mechanism supports the flow-equivalent server construction: solve a subnetwork in isolation for every population, tabulate its throughput, and install that table as the rate vector of a single station that replaces it.

3.1.1 fes_aggregation

Aggregating a subnetwork into one station and checking the aggregate against the full model.

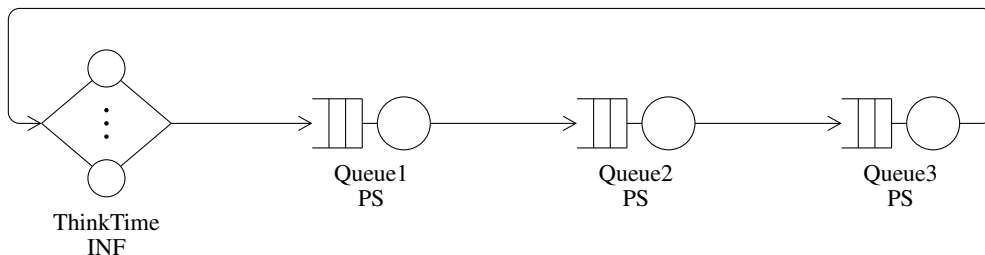


Figure 3.1: Topology of `fes_aggregation`: a closed network over 4 queueing stations.

The example is built and solved as follows.

```
% Flow-Equivalent Server (FES) Aggregation Example
%
% This example demonstrates how to use ModelAdapter.aggregateFES to replace
% a subset of stations in a closed product-form queueing network with a
% single Flow-Equivalent Server (FES).
```


Nodes	4: Queue $\times$ 3, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	ThinkTime: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	ThinkTime – Class1: Exp(0.2); Class2: Exp(0.25) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.5) Queue2 – Class1: Exp(1); Class2: Exp(0.8333) Queue3 – Class1: Exp(1.25); Class2: Exp(1)
Solvers	MVA

Table 3.1: Features of fes_aggregation.

```
%
% The FES has Limited Joint Dependence (LJD) service rates where the rate
% for class-c in state (n1,...,nK) equals the throughput of class-c in an
% isolated subnetwork consisting only of the subset stations.

fprintf('=== Flow-Equivalent Server (FES) Aggregation Example ===\n\n');

%% Create original 4-station tandem network with 2 classes
fprintf('Creating original 4-station network...\n');

N1 = 3; % number of class-1 jobs
N2 = 2; % number of class-2 jobs

model = Network('OriginalModel');

% Create stations
node{1} = Delay(model, 'ThinkTime');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);
node{4} = Queue(model, 'Queue3', SchedStrategy.PS);

... (95 source lines omitted; full implementation: matlab/examples/advanced/)

for i = 1:length(ME.stack)
    fprintf(' %s (line %d)\n', ME.stack(i).name, ME.stack(i).line);
end
end

fprintf('\n=== Example Complete ===\n');
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.2 fes_single_class

The flow-equivalent construction step by step: solve the subnetwork, tabulate its throughput, install it as a load-dependent station.

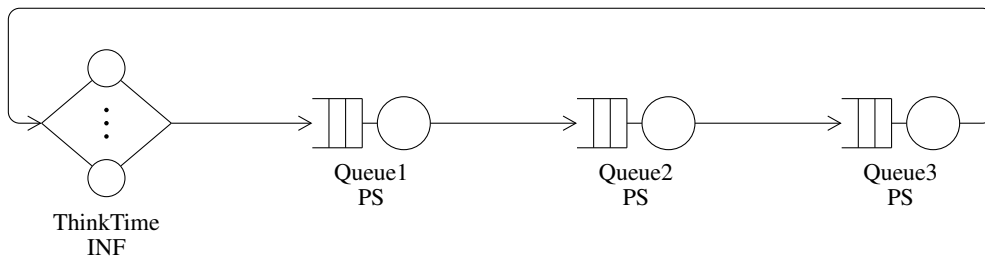


Figure 3.2: Topology of fes_single_class: a closed network over 4 queueing stations.

The example is built and solved as follows.

Nodes	4: Queue $\times$ 3, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	ThinkTime: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	ThinkTime – Class1: Exp(0.2) Queue1 – Class1: Exp(0.6667) Queue2 – Class1: Exp(1) Queue3 – Class1: Exp(1.25)
Solvers	MVA

Table 3.2: Features of `fes_single_class`.

```
% Single-class FES Aggregation Example
%
% This tests FES aggregation with a single job class, where it should be exact.

fprintf('=== Single-Class FES Aggregation Example ===\n\n');

%% Create original 4-station tandem network with 1 class
fprintf('Creating original 4-station network...\n');

N1 = 5; % number of jobs

model = Network('OriginalModel');

% Create stations
node{1} = Delay(model, 'ThinkTime');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);
node{4} = Queue(model, 'Queue3', SchedStrategy.PS);

% Create job class
jobclass{1} = ClosedClass(model, 'Class1', N1, node{1}, 0);

% Set service times
node{1}.setService(jobclass{1}, Exp.fitMean(5.0));
node{2}.setService(jobclass{1}, Exp.fitMean(1.5));
node{3}.setService(jobclass{1}, Exp.fitMean(1.0));

... (72 source lines omitted; full implementation: matlab/examples/advanced/)

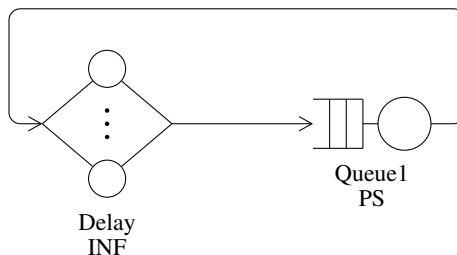
for i = 1:length(ME.stack)
    fprintf(' %s (line %d)\n', ME.stack(i).name, ME.stack(i).line);
end
end

fprintf('\n=== Example Complete ===\n');
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.3 1d_class_dependence

Service rates that depend on the class mix at the station, not only on the total population.

Figure 3.3: Topology of `1d_class_dependence`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 16$), Class2 (closed, $N = 8$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4)
Solvers	CTMC, MVA

Table 3.3: Features of `ld_class_dependence`.

The example is built and solved as follows.

```
% class-dependent (product-form) model.
% Each class sees a service-rate scaling beta_{i,r} that depends ONLY on its
% own per-class population n_{i,r} at the station. This is the BCMP
% product-form case of QD-AMVA (Casale, Perez, Wang, IFIP PERFORMANCE 2015):
% D_{i,r}(n) = theta_{i,r} * beta_{i,r}(n_{i,r}). The handle returns a length-R
% vector [beta_{i,1}(n), beta_{i,2}(n)], of which the solver picks class r.
% Contrast ld_joint_dependence.m, where the scalar min(ni(1),c) reads a foreign
% class marginal and is therefore non-product-form (setJointDependence).
N = 16; % number of jobs
c = 2;
%%
cdmodel = Network('model');
node{1} = Delay(cdmodel, 'Delay');
node{2} = Queue(cdmodel, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(cdmodel, 'Class1', N, node{1}, 0);
jobclass{2} = ClosedClass(cdmodel, 'Class2', N/2, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{1}.setService(jobclass{2}, Exp.fitMean(2.0)); % mean = 2
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setService(jobclass{2}, Exp.fitMean(2.5)); % mean = 2.5
% beta_{i,r}(n_{i,r}): class 1 scales up to c servers with its OWN count, class
% 2 is always single-server. Both entries read only their own marginal, so the
% demands satisfy the product-form recurrence. Peak rate scaling [c 1] per
% class normalizes Util = T*S/peak.
node{2}.setClassDependence(@(ni) [min(ni(1),c), 1], [c 1]);

... (6 source lines omitted; full implementation: matlab/examples/advanced/)

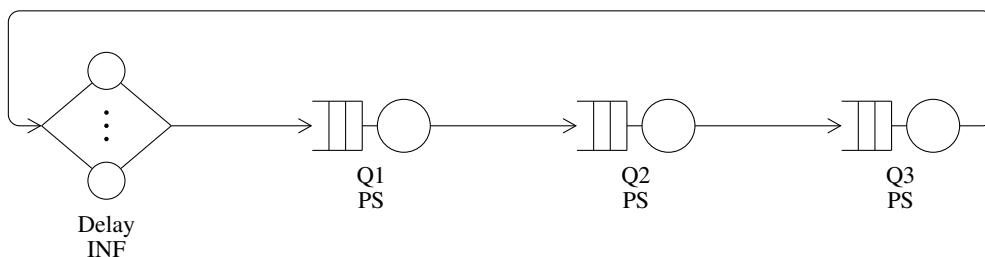
cdAvgTableCD=MVA(cdmodel,'method','qd').getAvgTable
% JMT is not solved here: the JSIM writer has no representation for the
% class-dependence handle, so SolverJMT rejects the model rather than
% silently solving it unscaled (see SolverJMT.getFeatureSet).

model = cdmodel; % for test compatibility
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.4 ld_fes_multiclass

The multiclass flow-equivalent server.

Figure 3.4: Topology of `ld_fes_multiclass`: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Q1: PS; Q2: PS; Q3: PS
Service	Delay – Class1: Exp(1); Class2: Exp(0.6667) Q1 – Class1: Exp(2); Class2: Exp(1.25) Q2 – Class1: Exp(3.333); Class2: Exp(1.667) Q3 – Class1: Exp(2.5); Class2: Exp(1.429)
Solvers	MVA, NC

Table 3.4: Features of `ld_fes_multiclass`.

The example is built and solved as follows.

```
% Multiclass FES Aggregation with Norton's Theorem Verification
%
% This example demonstrates Norton's theorem for closed multiclass
% queueing networks: a subset of stations is replaced by a single
% Flow-Equivalent Server (FES) whose class-dependent service rates equal the
% throughputs of the isolated subnetwork.

N1 = 3; % class-1 jobs
N2 = 2; % class-2 jobs

%% Original 4-station tandem network with 2 classes
model = Network('OriginalModel');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Q1', SchedStrategy.PS);
node{3} = Queue(model, 'Q2', SchedStrategy.PS);
node{4} = Queue(model, 'Q3', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', N1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N2, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.5));
node{2}.setService(jobclass{1}, Exp.fitMean(0.5));
node{2}.setService(jobclass{2}, Exp.fitMean(0.8));
node{3}.setService(jobclass{1}, Exp.fitMean(0.3));

... (15 source lines omitted; full implementation: matlab/examples/advanced/)

[fesModel, ~, ~] = ModelAdapter.aggregateFES(model, ...
    {node{2}, node{3}, node{4}}, struct('solver','mva','verbose',false));

fprintf('NC (FES model):\n');
AvgNC = NC(fesModel, 'method', 'exact').getAvgTable;
disp(AvgNC);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.5 `ld_fes_singleclass`

A flow-equivalent server replacing a subnetwork, single class.

Nodes	4: Queue $\times$ 3, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Q1: PS; Q2: PS; Q3: PS
Service	Delay – Class1: Exp(0.2) Q1 – Class1: Exp(0.6667) Q2 – Class1: Exp(1) Q3 – Class1: Exp(1.25)
Solvers	MVA, NC

Table 3.5: Features of `ld_fes_singleclass`.

The example is built and solved as follows.

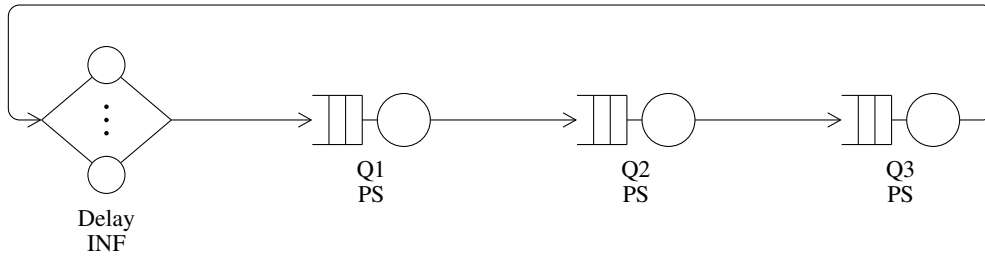


Figure 3.5: Topology of `ld_fes_singleclass`: a closed network over 4 queueing stations.

```
% Single-class FES Aggregation with Norton's Theorem Verification
%
% This example demonstrates Norton's theorem for a closed single-class
% queueing network: a subset of stations is replaced by a single
% Flow-Equivalent Server (FES) whose load-dependent service rates
% equal the throughputs of the isolated subnetwork.

N1 = 5; % number of jobs

%% Original 4-station tandem network with 1 class
model = Network('OriginalModel');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Q1', SchedStrategy.PS);
node{3} = Queue(model, 'Q2', SchedStrategy.PS);
node{4} = Queue(model, 'Q3', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', N1, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(5.0));
node{2}.setService(jobclass{1}, Exp.fitMean(1.5));
node{3}.setService(jobclass{1}, Exp.fitMean(1.0));
node{4}.setService(jobclass{1}, Exp.fitMean(0.8));

P = model.initRoutingMatrix();
P{1,1} = [0 1 0 0; 0 0 1 0; 0 0 0 1; 1 0 0 0];

... (8 source lines omitted; full implementation: matlab/examples/advanced/)

[fesModel, ~, ~] = ModelAdapter.aggregateFES(model, ...
{node{2}, node{3}, node{4}}, struct('solver','mva','verbose',false));

fprintf('NC (FES model):\n');
AvgNC = NC(fesModel, 'method', 'exact').getAvgTable;
disp(AvgNC);
```

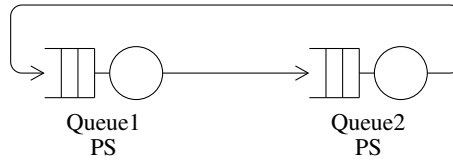
No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.6 ld_global_dependence

A globally state-dependent (Whittle) model: `setGlobalDependence` scales the service rate by a function of the FULL station-class population matrix rather than of the population local to one station. The scaling used here satisfies the Whittle balance property, so the chain is reversible, keeps a product form and is insensitive to the service distribution beyond its mean.

The example is built and solved as follows.

```
% Globally state-dependent (Whittle) model: setGlobalDependence declares a rate
% scaling phi(n) over the FULL (nstations x nclasses) population matrix, not
% just the population local to one station. Here two PS stations share one unit
% of capacity, phi_s(n) = n_s/|n|, which is the single-link allocation every
% alpha-fair rule collapses to.
```

Figure 3.6: Topology of `ld_global_dependence`: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 3$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(2)
Solvers	CTMC, SSA

Table 3.6: Features of `ld_global_dependence`.

```
%
% phi satisfies the Whittle balance property
%   phi_s(n) phi_t(n-e_s) = phi_t(n) phi_s(n-e_t),
% so the chain is reversible, has the product form pi(n) ~ Phi(n) prod rho^n and
% is INSENSITIVE: the means below do not change when the exponential service is
% replaced by an Erlang or a hyperexponential of the same mean.
N = 3; % number of jobs
%%
gdmodel = Network('model');
node{1} = Queue(gdmodel, 'Queue1', SchedStrategy.PS);
node{2} = Queue(gdmodel, 'Queue2', SchedStrategy.PS);
jobclass{1} = ClosedClass(gdmodel, 'Class1', N, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp.fitMean(0.5));
gdmodel.link(gdmodel.serialRouting(node));

sn = gdmodel.getStruct();
i1 = sn.nodeToStation(gdmodel.getNodeIndex('Queue1'));
i2 = sn.nodeToStation(gdmodel.getNodeIndex('Queue2'));
M = sn.nstations; K = sn.nclasses;
% peak scaling is 1: no station ever receives more than the whole link

... (32 source lines omitted; full implementation: matlab/examples/advanced/)

if tot == 0
    x = zeros(numel(n),1);
else
    x = n(:) / tot;
end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.7 `ld_joint_dependence`

Joint dependence on the populations of several classes.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 16$), Class2 (closed, $N = 8$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4)
Solvers	CTMC, MVA

Table 3.7: Features of `ld_joint_dependence`.

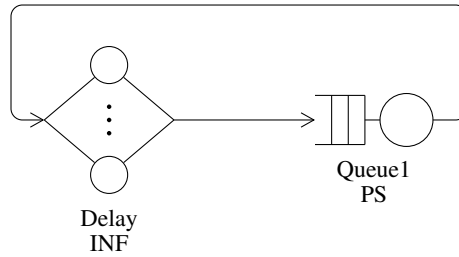


Figure 3.7: Topology of `ld_joint_dependence`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
% joint-dependent (non-product-form) model
N = 16; % number of jobs
c = 2;
%%
jdmodel = Network('model');
node{1} = Delay(jdmodel, 'Delay');
node{2} = Queue(jdmodel, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(jdmodel, 'Class1', N, node{1}, 0);
jobclass{2} = ClosedClass(jdmodel, 'Class2', N/2, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{1}.setService(jobclass{2}, Exp.fitMean(2.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setService(jobclass{2}, Exp.fitMean(2.5)); % mean = 1.5
node{2}.setJointDependence(@(ni) min(ni(1),c), c); % NON-product-form: rate reads the class-1
    marginal only (eta_i), shared across classes; peak rate scaling = c (Util = T*S/c)

P = jdmodel.initRoutingMatrix();
P{1,1} = jdmodel.serialRouting(node);
P{2,2} = jdmodel.serialRouting(node);
jdmodel.link(P);

jdAvgTableCTMC=CTMC(jdmodel,'exact').getAvgTable
jdAvgTableJD=MVA(jdmodel,'method','qd').getAvgTable
% JMT is not solved here: the JSIM writer has no representation for the
% joint-dependence handle, so SolverJMT rejects the model rather than
% silently solving it unscaled (see SolverJMT.getFeatureSet).

model = jdmodel; % for test compatibility
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.8 `ld_multiserver_fcfs`

The FCFS counterpart, with class-dependent scaling on top.

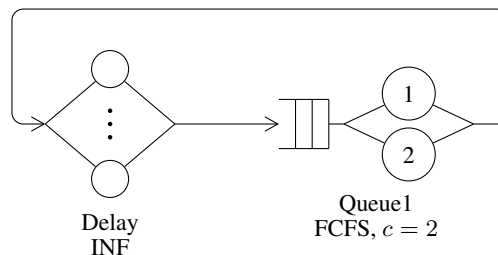


Figure 3.8: Topology of `ld_multiserver_fcfs`: a closed network over 2 queueing stations.

The example is built and solved as follows.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 16$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 2$)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.6667)
Solvers	NC, CTMC, MVA, JMT

Table 3.8: Features of `ld_multiserver_fcfs`.

```

N = 16; % number of jobs
c = 2; % number of servers

model = Network('model');
node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setNumberOfServers(c);

model.link(model.serialRouting(node));

msT=NC(model).getAvgTable

%% casted with scaling function that depends on the total queue
ldmodel = Network('model');
node{1} = Delay(ldmodel, 'Delay');
node{2} = Queue(ldmodel, 'Queue1', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(ldmodel, 'Class1', N, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setLoadDependence(min(1:N,c)); % multi-server with c servers

ldmodel.link(ldmodel.serialRouting(node));

... (23 source lines omitted; full implementation: matlab/examples/advanced/)

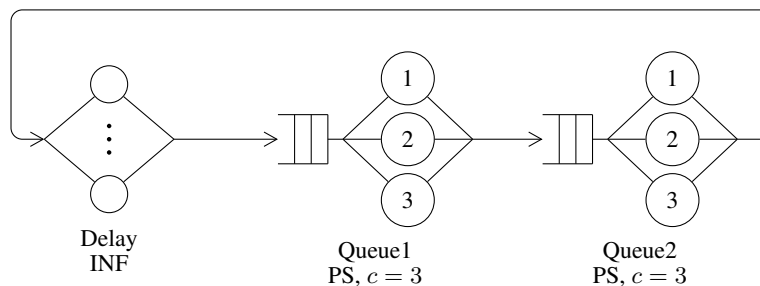
cdAvgTableCTMC=CTMC(cdmodel,'exact').getAvgTable
cdAvgTableCD=MVA(cdmodel,'method','qd').getAvgTable
% JMT is not solved here: the JSIM writer has no representation for the
% class-dependence handle, so SolverJMT rejects the model rather than
% silently solving it unscaled (see SolverJMT.getFeatureSet). The lld
% model above is JMT-solvable because lldscaling maps onto the server count.

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.9 `ld_multiserver_ps`

A multiserver PS station written as a load-dependent station, and the rate vector that reproduces it.

Figure 3.9: Topology of `ld_multiserver_ps`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 4$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS ($c = 3$); Queue2: PS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4) Queue2 – Class1: Exp(0.2857); Class2: Exp(0.2222)
Solvers	MVA, CTMC, NC, JMT

Table 3.9: Features of `ld_multiserver_ps`.

The example is built and solved as follows.

```

N = 4; % number of jobs
c = 3; % number of servers
%%
model = Network('model');
node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N/2, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{1}.setService(jobclass{2}, Exp.fitMean(2.0)); % mean = 1

node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setService(jobclass{2}, Exp.fitMean(2.5)); % mean = 1.5
node{2}.setNumberOfServers(c);

node{3}.setService(jobclass{1}, Exp.fitMean(3.5)); % mean = 1.5
node{3}.setService(jobclass{2}, Exp.fitMean(4.5)); % mean = 1.5
node{3}.setNumberOfServers(c);

P = model.initRoutingMatrix();
P{1,1} = model.serialRouting(node);
P{2,2} = model.serialRouting(node);
model.link(P);

msT=MVA(model,'exact').getAvgTable

... (28 source lines omitted; full implementation: matlab/examples/advanced/)

l1dAvgTableNRL=NC(ldmodel,'method','nrl').getAvgTable

l1dAvgTableMVALD=MVA(ldmodel,'method','exact').getAvgTable
l1dAvgTableQD=MVA(ldmodel,'method','qd').getAvgTable

l1dAvgTableJMT=JMT(ldmodel,'seed',23000).getAvgTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.10 `ld_multiserver_ps_twoclasses`

The two-class version.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 4$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4)
Solvers	MVA, CTMC, NC, JMT

Table 3.10: Features of `ld_multiserver_ps_twoclasses`.

The example is built and solved as follows.

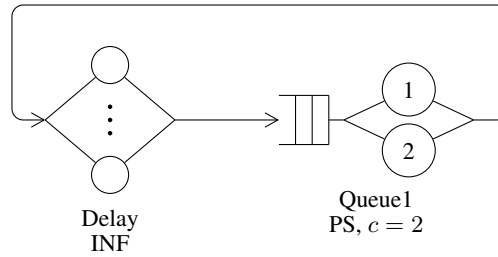


Figure 3.10: Topology of `ld_multiserver_ps_twoclasses`: a closed network over 2 queueing stations.

```

N = 4; % number of jobs
c = 2; % number of servers
%%
model = Network('model');
node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N/2, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0)); % mean = 1
node{1}.setService(jobclass{2}, Exp.fitMean(2.0)); % mean = 1
node{2}.setService(jobclass{1}, Exp.fitMean(1.5)); % mean = 1.5
node{2}.setService(jobclass{2}, Exp.fitMean(2.5)); % mean = 1.5
node{2}.setNumberOfServers(c);

P = model.initRoutingMatrix();
P{1,1} = model.serialRouting(node);
P{2,2} = model.serialRouting(node);
model.link(P);

mst=MVA(model,'exact').getAvgTable
%%
ldmodel = Network('ldmodel');
node{1} = Delay(ldmodel, 'Delay');
node{2} = Queue(ldmodel, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(ldmodel, 'Class1', N, node{1}, 0);
jobclass{2} = ClosedClass(ldmodel, 'Class2', N/2, node{1}, 0);

... (16 source lines omitted; full implementation: matlab/examples/advanced/)

lldAvgTableNRL=NC(ldmodel,'method','nrl').getAvgTable

lldAvgTableMVALD=MVA(ldmodel,'method','exact').getAvgTable
lldAvgTableQD=MVA(ldmodel,'method','qd').getAvgTable

lldAvgTableJMT=JMT(ldmodel,'seed',23000,'samples',5000).getAvgTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.1.11 `ld_whittle_bandwidth`

An open bandwidth-sharing network in which one route holds several links at once, which no per-station rate scaling can express. Capacity is shared by balanced fairness, whose rate function satisfies the Whittle property by construction, so the stationary law is again of product form and insensitive.

The example is built and solved as follows.

```

% Open Whittle network: a bandwidth-sharing model in which one route holds
% SEVERAL links at once, which no per-station rate scaling can express. This is
% the 2-link linear network: route 1 crosses both links, routes 2 and 3 use one
% link each.
%
%   A = [1 1 0;          link 1 carries routes 1 and 2

```

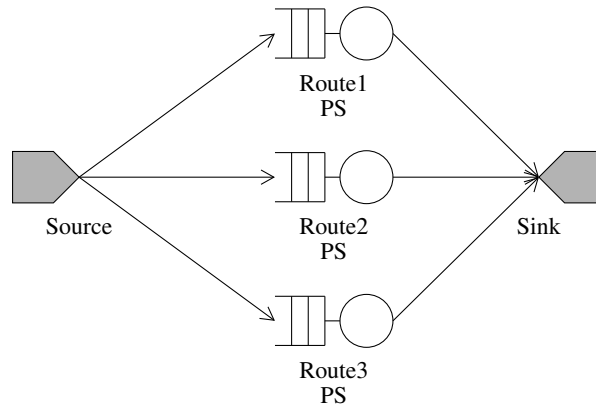


Figure 3.11: Topology of ld_whittle_bandwidth: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	3: Route1Flows (open), Route2Flows (open), Route3Flows (open)
Scheduling	Route1: PS; Route2: PS; Route3: PS
Arrivals	Source – Route1Flows: Exp(0.2); Route2Flows: Exp(0.3); Route3Flows: Exp(0.3)
Service	Route1 – Route1Flows: Exp(1); Route2Flows: Disabled; Route3Flows: Disabled Route2 – Route1Flows: Disabled; Route2Flows: Exp(1); Route3Flows: Disabled Route3 – Route1Flows: Disabled; Route2Flows: Disabled; Route3Flows: Exp(1)
Solvers	CTMC

Table 3.11: Features of ld_whittle_bandwidth.

```
%      1 0 1]      link 2 carries routes 1 and 3
%
% Capacity is shared by BALANCED FAIRNESS, whose rates come from the recursion
%   Phi(n) = max_l (1/C_l) sum_{s in l} Phi(n-e_s),   x_s(n) = Phi(n-e_s)/Phi(n).
% That construction satisfies the Whittle balance property by design, so the
% stationary law is pi(n) ~ Phi(n) prod rho_s^{n_s} and is insensitive.
%
% Each route is modelled as its own PS queue fed by its own open class, so the
% queue populations ARE the coordinates of the Whittle state n.
A = [1 1 0; 1 0 1];
C = [1; 1];
nu = [0.20 0.30 0.30]; % flow arrival rates
mu = [1.00 1.00 1.00]; % 1/mean file size
cutoff = 6; % per-route truncation of the open state space
%%
[Phi, base] = bfBalanceFunction(A, C, cutoff*ones(3,1));

bwmodel = Network('model');
node{1} = Source(bwmodel, 'Source');
node{2} = Queue(bwmodel, 'Route1', SchedStrategy.PS);

... (120 source lines omitted; full implementation: matlab/examples/advanced/)

i = 1; mult = 1;
for s = 1:numel(n)
    i = i + n(s)*mult;
    mult = mult*base(s);
end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2 Finite capacity regions

A finite capacity region caps the number of jobs inside a set of stations. A job that would breach the cap is either dropped or made to wait outside the region, per class, and per-class caps may be declared alongside the global one. The region is a property of the model, not of a station, which is what lets it span several stations.

The flagship comparison is `fcr_mm1kdrop`: a region of capacity K around a single queue, with dropping, is an M/M/1/K queue, and the script builds both models and solves them side by side.

```
% Model 1: one queue inside a region of capacity K, with dropping
modell = Network('FCR Dropping');
source1 = Source(modell, 'Source');
queue1 = Queue(modell, 'Queue', SchedStrategy.FCFS);
sink1 = Sink(modell, 'Sink');
jobclass1 = OpenClass(modell, 'Class1', 0);
source1.setArrival(jobclass1, Exp(0.8));
queue1.setService(jobclass1, Exp(1.0));
modell.link(Network.serialRouting(source1, queue1, sink1));

fcr = modell.addRegion({queue1});
fcr.setGlobalMaxJobs(K);
fcr.setDropRule(jobclass1, true); % true = drop, false = wait outside

% Model 2: the same system as an M/M/1/K queue
queue2.setCapacity(K);
```

3.2.1 fcr_constraints

Per-class capacities inside one region, and the interaction with the global limit.

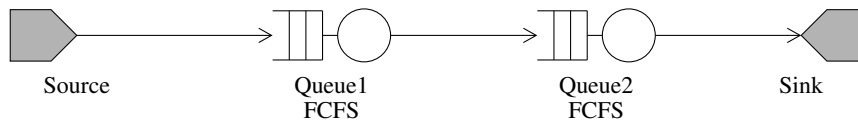


Figure 3.12: Topology of `fcr_constraints`: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: HighPriority (open), LowPriority (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – HighPriority: Exp(0.3); LowPriority: Exp(0.5)
Service	Queue1 – HighPriority: Exp(1); LowPriority: Exp(0.8) Queue2 – HighPriority: Exp(1.2); LowPriority: Exp(1)
Solvers	JMT

Table 3.12: Features of `fcr_constraints`.

The example is built and solved as follows.

```
%% Finite Capacity Region - Constraint Types
% This example demonstrates different FCR constraint types:
% - Global max jobs: limits total jobs across all classes
% - Per-class max jobs: limits jobs of a specific class
% - Drop rule: determines blocking vs dropping behavior

clear; clc;

%% Create a multiclass open network
model = Network('FCR Constraints Demo');
```

```

% Nodes
source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

% Job classes
class1 = OpenClass(model, 'HighPriority', 0);
class2 = OpenClass(model, 'LowPriority', 1);

% Arrival and service rates
source.setArrival(class1, Exp(0.3)); % High priority: lower arrival rate
source.setArrival(class2, Exp(0.5)); % Low priority: higher arrival rate
queue1.setService(class1, Exp(1.0));
queue1.setService(class2, Exp(0.8));

... (26 source lines omitted; full implementation: matlab/examples/advanced/)

fcr.setDropRule(class1, true); % true = drop
fcr.setDropRule(class2, true); % true = drop

%% Solve with JMT
solver = JMT(model, 'seed', 23000, 'samples', 100000);
avgTable = solver.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2.2 fcr_lincon

A linear constraint over the class populations, the general form of a region limit.

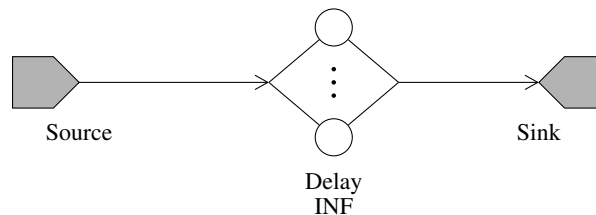


Figure 3.13: Topology of fcr_lincon: an open network over 1 queueing station.

Nodes	3: Source, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF
Arrivals	Source – Class1: Exp(0.3); Class2: Exp(0.2)
Service	Delay – Class1: Exp(1); Class2: Exp(0.8)
Solvers	LDES

Table 3.13: Features of fcr_lincon.

The example is built and solved as follows.

```

%% Linear Admission Constraints for Finite Capacity Regions
% Open network with a Delay node inside an FCR with DROP policy and
% general linear admission constraints: A*n <= b.
%
% Constraint: 2*n_1 + 1*n_2 <= 5, 1*n_1 + 3*n_2 <= 7
% (general weighted cross-class coupling)

clear; clc;

model = Network('FCR LinCon');

```

```

source = Source(model, 'Source');
delay = Delay(model, 'Delay');
sink = Sink(model, 'Sink');

class1 = OpenClass(model, 'Class1', 0);
class2 = OpenClass(model, 'Class2', 0);

source.setArrival(class1, Exp(0.3));
source.setArrival(class2, Exp(0.2));
delay.setService(class1, Exp(1.0));
delay.setService(class2, Exp(0.8));

P = model.initRoutingMatrix();
P.set(class1, class1, source, delay, 1.0);
P.set(class1, class1, delay, sink, 1.0);

... (9 source lines omitted; full implementation: matlab/examples/advanced/)

fcr.setDropRule(class2, true);
fcr.setGlobalMaxJobs(1000);
fcr.setLinearConstraints([2 1; 1 3], [5; 7]);

solver = LDES(model, 'seed', 23000, 'samples', 500000);
avgTable = solver.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2.3 fcr_lossn

A loss network: every request that cannot be admitted is lost, and the blocking probabilities are the quantity of interest.

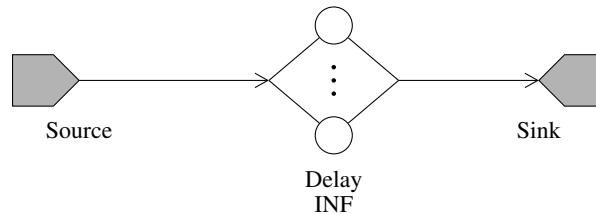


Figure 3.14: Topology of fcr_lossn: an open network over 1 queueing station.

Nodes	3: Source, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF
Arrivals	Source – Class1: Exp(0.3); Class2: Exp(0.2)
Service	Delay – Class1: Exp(1); Class2: Exp(0.8)
Solvers	JMT

Table 3.14: Features of fcr_lossn.

The example is built and solved as follows.

```

%% Loss Network with Finite Capacity Region (NC Solver)
% This example demonstrates the NC solver's ability to analyze open
% loss networks using the Erlang fixed-point approximation.
% The model has a multiclass Delay node inside an FCR with DROP policy.

clear; clc;

```

```

%% Create network
model = Network('FCR Loss Network');

%% Add nodes
source = Source(model, 'Source');
delay = Delay(model, 'Delay');
sink = Sink(model, 'Sink');

%% Add job classes
class1 = OpenClass(model, 'Class1', 0);
class2 = OpenClass(model, 'Class2', 1);

%% Set arrival and service rates
lambda1 = 0.3; lambda2 = 0.2; % arrival rates
mu1 = 1.0; mu2 = 0.8; % service rates

source.setArrival(class1, Exp(lambda1));
source.setArrival(class2, Exp(lambda2));
delay.setService(class1, Exp(mu1));

... (50 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf('Tput %-10s %12.4f %12.4f %10.2f%%\n', ...
    string(classNames(i)), tputNC(i), tputJMT(i), relErr);
end

fprintf('\nNote: NC uses analytical Erlang fixed-point approximation.\n');
fprintf('JMT uses discrete-event simulation.\n');

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2.4 fcr_mm1kdrop

A region of capacity K around one queue with dropping, which reproduces an M/M/1/K queue.

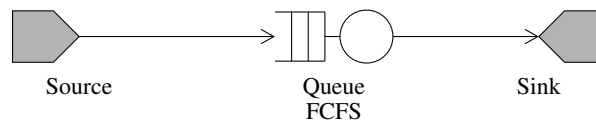


Figure 3.15: Topology of fcr_mm1kdrop: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.8)
Service	Queue – Class1: Exp(1)
Solvers	JMT

Table 3.15: Features of fcr_mm1kdrop.

The example is built and solved as follows.

```

%% Finite Capacity Region with Dropping vs M/M/1/K
% This example shows that an FCR with dropping around a single queue
% behaves like an M/M/1/K queue, where K is the FCR capacity.
% Jobs arriving when the region is full are dropped (lost).

clear; clc;

%% Parameters
arrivalRate = 0.8;
serviceRate = 1.0;

```

```

K = 3;

%% Model 1: Queue with FCR (dropping)
modell1 = Network('FCR Dropping');

source1 = Source(modell1, 'Source');
queue1 = Queue(modell1, 'Queue', SchedStrategy.FCFS);
sink1 = Sink(modell1, 'Sink');

jobclass1 = OpenClass(modell1, 'Class1', 0);

source1.setArrival(jobclass1, Exp(arrivalRate));
queue1.setService(jobclass1, Exp(serviceRate));

P1 = modell1.initRoutingMatrix();
P1.set(jobclass1, jobclass1, source1, queue1, 1.0);

... (27 source lines omitted; full implementation: matlab/examples/advanced/)

%% Solve both models and compare results
solver1 = JMT(modell1, 'seed', 23000, 'samples', 100000);
avgTable1 = solver1.getAvgTable();

solver2 = JMT(modell2, 'seed', 23000, 'samples', 100000);
avgTable2 = solver2.getAvgTable();

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2.5 fcr_mm1waitq

The same region with waiting instead of dropping, so blocked jobs queue outside the region.

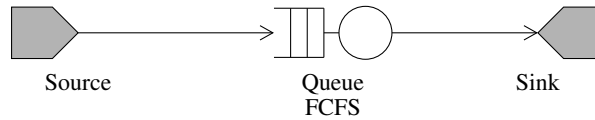


Figure 3.16: Topology of fcr_mm1waitq: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.5)
Service	Queue – Class1: Exp(1)
Solvers	JMT

Table 3.16: Features of fcr_mm1waitq.

The example is built and solved as follows.

```

%% Finite Capacity Region with Blocking vs M/M/1
% This example shows that an FCR with blocking (waitq) around a single
% queue behaves identically to a standard M/M/1 queue, since jobs
% simply wait when the region is "full" (no actual capacity limit effect).

clear; clc;

%% Parameters
arrivalRate = 0.5;
serviceRate = 1.0;

%% Model 1: Queue with FCR (blocking)
modell1 = Network('FCR Blocking');

```



```

source1 = Source(model1, 'Source');
queue1 = Queue(model1, 'Queue', SchedStrategy.FCFS);
sink1 = Sink(model1, 'Sink');

jobclass1 = OpenClass(model1, 'Class1', 0);

source1.setArrival(jobclass1, Exp(arrivalRate));
queue1.setService(jobclass1, Exp(serviceRate));

P1 = model1.initRoutingMatrix();
P1.set(jobclass1, jobclass1, source1, queue1, 1.0);
P1.set(jobclass1, jobclass1, queue1, sink1, 1.0);

... (24 source lines omitted; full implementation: matlab/examples/advanced/)

%% Solve both models and compare results
solver1 = JMT(model1, 'seed', 23000, 'samples', 100000);
avgTable1 = solver1.getAvgNodeTable()

solver2 = JMT(model2, 'seed', 23000, 'samples', 100000);
avgTable2 = solver2.getAvgNodeTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2.6 fcr_oqndrop

A region spanning several stations of an open network, with dropping.

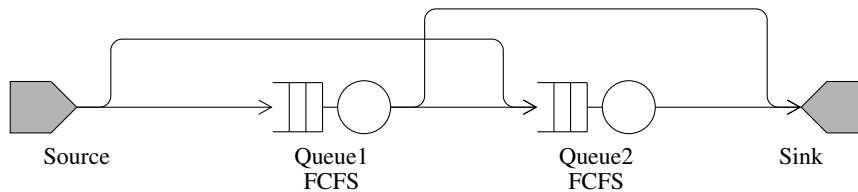


Figure 3.17: Topology of fcr_oqndrop: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.4); Class2: Exp(0.3)
Service	Queue1 – Class1: Exp(1); Class2: Exp(0.9) Queue2 – Class1: Exp(1.1); Class2: Exp(1)
Solvers	JMT

Table 3.17: Features of fcr_oqndrop.

The example is built and solved as follows.

```

%% Finite Capacity Region with Dropping (Multiclass Open Network)
% This example creates a multiclass open network with 2 queues and a
% finite capacity region with global and per-class constraints.
% When the region is full, arriving jobs are dropped (lost).

clear; clc;

%% Create network
model = Network('FCR Dropping Example');

%% Add nodes
source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);

```

```

queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

%% Add job classes
class1 = OpenClass(model, 'Class1', 0);
class2 = OpenClass(model, 'Class2', 1);

%% Set arrival and service rates
source.setArrival(class1, Exp(0.4));
source.setArrival(class2, Exp(0.3));
queue1.setService(class1, Exp(1.0));
queue1.setService(class2, Exp(0.9));
queue2.setService(class1, Exp(1.1));

... (26 source lines omitted; full implementation: matlab/examples/advanced/)

fcr.setDropRule(class1, true);      % true = drop jobs
fcr.setDropRule(class2, true);      % true = drop jobs

%% Run JMT
solver = JMT(model, 'seed', 23000, 'samples', 50000);
avgTable = solver.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.2.7 fcr_oqnwaitq

The same region with waiting.

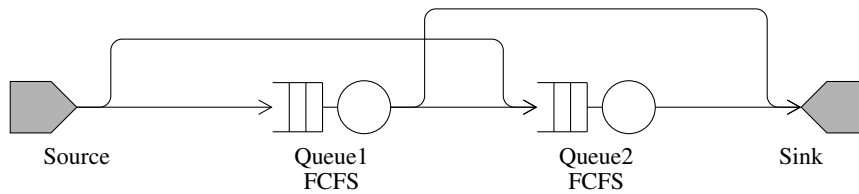


Figure 3.18: Topology of fcr_oqnwaitq: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.4); Class2: Exp(0.3)
Service	Queue1 – Class1: Exp(1); Class2: Exp(0.9) Queue2 – Class1: Exp(1.1); Class2: Exp(1)
Solvers	JMT

Table 3.18: Features of fcr_oqnwaitq.

The example is built and solved as follows.

```

%% Finite Capacity Region with Blocking (Multiclass Open Network)
% This example creates a multiclass open network with 2 queues and a
% finite capacity region with global and per-class constraints.
% When the region is full, arriving jobs are blocked (wait in queue).

clear; clc;

%% Create network
model = Network('FCR Blocking Example');

%% Add nodes
source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);

```

```

queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

%% Add job classes
class1 = OpenClass(model, 'Class1', 0);
class2 = OpenClass(model, 'Class2', 1);

%% Set arrival and service rates
source.setArrival(class1, Exp(0.4));
source.setArrival(class2, Exp(0.3));
queue1.setService(class1, Exp(1.0));
queue1.setService(class2, Exp(0.9));
queue2.setService(class1, Exp(1.1));

... (26 source lines omitted; full implementation: matlab/examples/advanced/)

fcr.setDropRule(class1, false); % false = block (wait)
fcr.setDropRule(class2, false); % false = block (wait)

%% Run JMT
solver = JMT(model, 'seed', 23000, 'samples', 10000);
avgTable = solver.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3 State probabilities

Beyond the averages, the solvers that build a state space can return probabilities. `getProb` gives the probability of a detailed station state, `getProbAggr` of an aggregate one (the queue length only, with the phases and the in-service order summed out), and the `Sys` variants give the joint state of the whole network.

The detailed state is discipline-dependent: under FCFS it records the order of the jobs in service, under PS it does not, and with non-exponential service it carries the phase of each job. That is why the same script exists in an FCFS and a PS version.

3.3.1 statepr_aggr

The marginal probability of an aggregate station state, through `getProbAggr`.

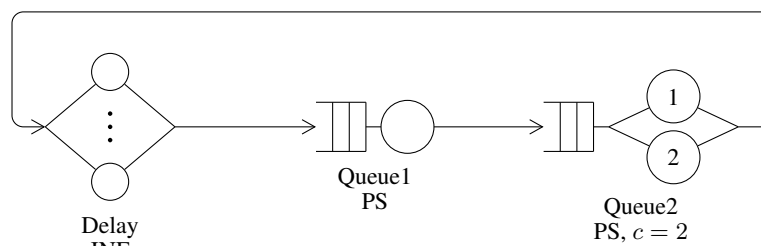


Figure 3.19: Topology of `statepr_aggr`: a closed network over 3 queueing stations.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);
node{3}.setNumServers(2);

```

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4) Queue2 – Class1: Exp(1); Class2: Exp(3)
Solvers	CTMC, NC

Table 3.19: Features of `statepr_aggr`.

```

N=[2,0];
jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(1));

node{2}.setService(jobclass{1}, Exp(3));
node{2}.setService(jobclass{2}, Exp(4));

node{3}.setService(jobclass{1}, Exp(1));
node{3}.setService(jobclass{2}, Exp(3));

K = length(jobclass);
P = cell(K,K);

P{1} = [0,1,0; 0,0,1; 1,0,0];
P{2} = [0,1,0; 0,0,1; 1,0,0];

... (27 source lines omitted; full implementation: matlab/examples/advanced/)

Pmarg_ctmc = Pr

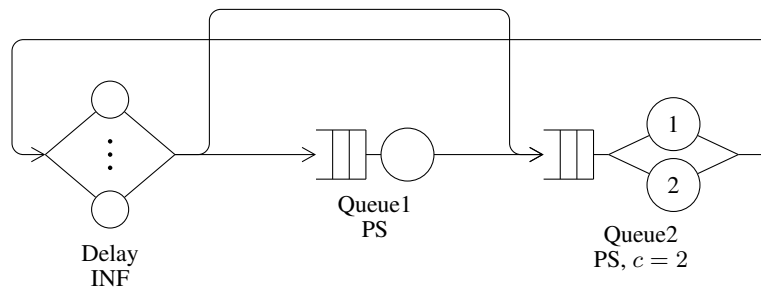
solver = NC(model,options);
Pr = solver.getProbAggr(node{i});
fprintf(1, 'Station %d is in state %s with probability %.15g\n', i, mat2str(state{i}), Pr);
Pmarg_nc = Pr

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3.2 `statepr_aggr_large`

The same on a model whose state space is large enough to need the aggregate form.

Figure 3.20: Topology of `statepr_aggr_large`: a closed network over 3 queueing stations.

The example is built and solved as follows.

```

model = Network('model');
node{1} = Delay(model, 'Delay');

```

Nodes	3: Queue $\times$ 2, Delay
Classes	4: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 4$), Class4 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(2); Class3: Exp(1); Class4: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4); Class3: Exp(5); Class4: Exp(1) Queue2 – Class1: Exp(1); Class2: Exp(3); Class3: Exp(5); Class4: Exp(2)
Features	class switching on 1 link
Solvers	CTMC, NC

Table 3.20: Features of `statepr_aggr_large`.

```

node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);

node{3}.setNumServers(2);

N=[1,0,4,0];
jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);
jobclass{3} = ClosedClass(model, 'Class3', N(3), node{1}, 0);
jobclass{4} = ClosedClass(model, 'Class4', N(4), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(2));
node{1}.setService(jobclass{3}, Exp(1));
node{1}.setService(jobclass{4}, Exp(1));

node{2}.setService(jobclass{1}, Exp(3));
node{2}.setService(jobclass{2}, Exp(4));
node{2}.setService(jobclass{3}, Exp(5));
node{2}.setService(jobclass{4}, Exp(1));

node{3}.setService(jobclass{1}, Exp(1));
node{3}.setService(jobclass{2}, Exp(3));

... (58 source lines omitted; full implementation: matlab/examples/advanced/)

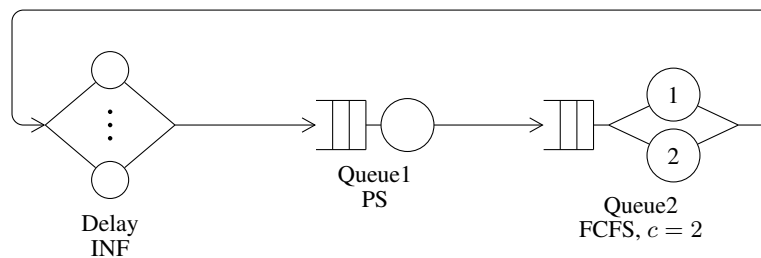
solver = NC(model,options);
for i=M
    Pr = solver.getProbAggr(node{i});
    fprintf(1,'Station %d is in state %s with probability %.15g\n',i,mat2str(state{i}),Pr);
end
Pr_nc = Pr;

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3.3 `statepr_allprobs_fcfs`

The FCFS version, where the detailed state carries the order of the jobs in service.

Figure 3.21: Topology of `statepr_allprobs_fcfs`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: FCFS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4) Queue2 – Class1: Exp(3); Class2: Exp(3)
Solvers	CTMC, NC, SSA, JMT

Table 3.21: Features of `statepr_allprobs_fcfs`.

The example is built and solved as follows.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{3}.setNumServers(2);

N=[2,0];
jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(1));

node{2}.setService(jobclass{1}, Exp(3));
node{2}.setService(jobclass{2}, Exp(4));

node{3}.setService(jobclass{1}, Exp(3));
node{3}.setService(jobclass{2}, Exp(3));

K = length(jobclass);
P = model.initRoutingMatrix;

P{1,1} = [0,1,0; 0,0,0; 1,0,0];
P{1,2} = [0,0,0; 0,0,1; 0,0,0];
P{2,1} = [0,1,0; 0,0,0; 1,0,0];

... (92 source lines omitted; full implementation: matlab/examples/advanced/)

Pjoint_ssa = Pr

% options.samples = 3e5;
% solver = JMT(model,options);
% Pr = solver.getProbSys();
% Pjoint_jmt = Pr
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3.4 `statepr_allprobs_ps`

Every probability API on one PS model: detailed and aggregate, per station and system-wide.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4) Queue2 – Class1: Exp(1); Class2: Exp(3)
Solvers	CTMC, NC, SSA, JMT

Table 3.22: Features of `statepr_allprobs_ps`.

The example is built and solved as follows.

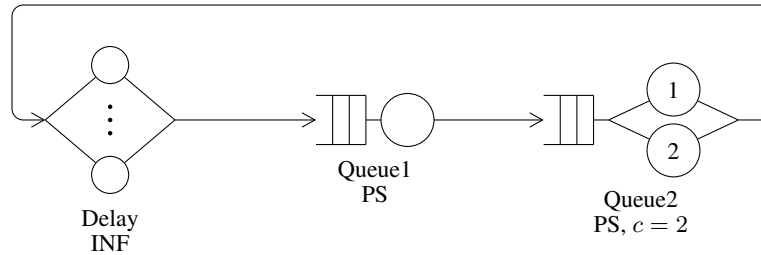


Figure 3.22: Topology of statepr_allprobs_ps: a closed network over 3 queueing stations.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);
node{3}.setNumServers(2);

N=[2,0];
jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(1));

node{2}.setService(jobclass{1}, Exp(3));
node{2}.setService(jobclass{2}, Exp(4));

node{3}.setService(jobclass{1}, Exp(1));
node{3}.setService(jobclass{2}, Exp(3));

K = length(jobclass);
P = model.initRoutingMatrix;

P{1,1} = [0,1,0; 0,0,0; 1,0,0];
P{1,2} = [0,0,0; 0,0,1; 0,0,0];
P{2,1} = [0,1,0; 0,0,0; 1,0,0];

... (99 source lines omitted; full implementation: matlab/examples/advanced/)

Pjoint_ssa = Pr

% options.samples = 3e5;
% solver = JMT(model,options);
% Pr = solver.getProbSys();
% Pjoint_jmt = Pr
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3.5 statepr_sys_aggr

The joint aggregate state of the whole network, through getProbSysAggr.
The example is built and solved as follows.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);

node{3}.setNumServers(2);

N=[1,0,3,0];
```

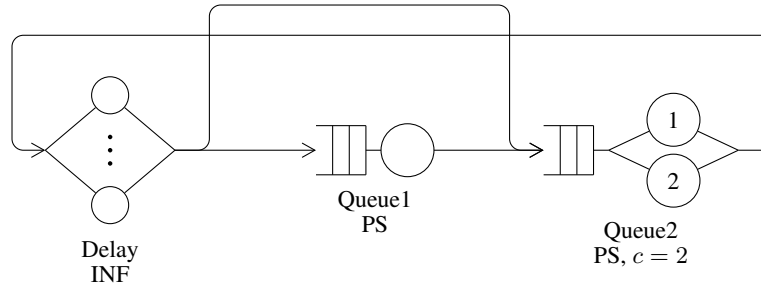


Figure 3.23: Topology of statepr_sys_aggr: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	4: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 3$), Class4 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(2); Class3: Exp(1); Class4: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4); Class3: Exp(5); Class4: Exp(1) Queue2 – Class1: Exp(1); Class2: Exp(3); Class3: Exp(5); Class4: Exp(2)
Features	class switching on 1 link
Solvers	CTMC, NC, JMT

Table 3.23: Features of statepr_sys_aggr.

```

jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);
jobclass{3} = ClosedClass(model, 'Class3', N(3), node{1}, 0);
jobclass{4} = ClosedClass(model, 'Class4', N(4), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(2));
node{1}.setService(jobclass{3}, Exp(1));
node{1}.setService(jobclass{4}, Exp(1));

node{2}.setService(jobclass{1}, Exp(3));
node{2}.setService(jobclass{2}, Exp(4));
node{2}.setService(jobclass{3}, Exp(5));
node{2}.setService(jobclass{4}, Exp(1));

node{3}.setService(jobclass{1}, Exp(1));
node{3}.setService(jobclass{2}, Exp(3));

... (57 source lines omitted; full implementation: matlab/examples/advanced/)

Pr_nc = solver.getProbSysAggr();
fprintf(1, 'NC: Pr_nc = %.15g\n', Pr_nc);

solver = JMT(model, 'samples', 1e4, 'seed', 532733);
Pr_jmt = solver.getProbSysAggr();
fprintf(1, 'JMT: Pr_jmt = %.15g\n', Pr_jmt);

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3.6 statepr_sys_aggr_large

The large-model version.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Queue(model, 'Queue1', SchedStrategy.PS);

```

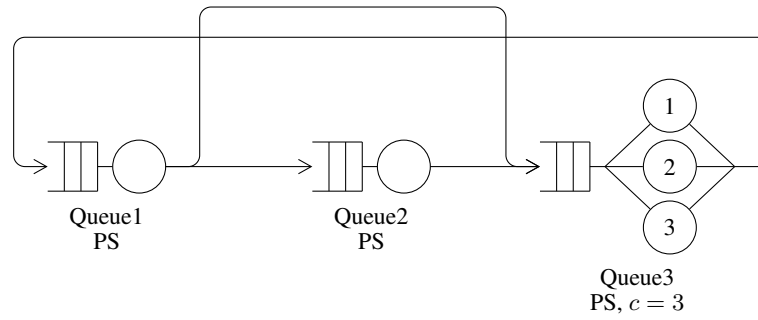



Figure 3.24: Topology of statepr_sys_aggr_large: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	4: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$), Class3 (closed, $N = 1$), Class4 (closed, $N = 1$)
Scheduling	Queue1: PS; Queue2: PS; Queue3: PS ($c = 3$)
Service	Queue1 – Class1: Exp(1); Class2: Exp(2); Class3: Exp(1); Class4: Exp(1) Queue2 – Class1: Exp(3); Class2: Exp(4); Class3: Exp(5); Class4: Exp(1) Queue3 – Class1: Exp(1); Class2: Exp(3); Class3: Exp(5); Class4: Exp(2)
Features	class switching on 1 link
Solvers	CTMC

Table 3.24: Features of statepr_sys_aggr_large.

```

node{2} = Queue(model, 'Queue2', SchedStrategy.PS);
node{3} = Queue(model, 'Queue3', SchedStrategy.PS);

node{3}.setNumServers(3);

N=[1,1,1,1];
jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);
jobclass{3} = ClosedClass(model, 'Class3', N(3), node{1}, 0);
jobclass{4} = ClosedClass(model, 'Class4', N(4), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(2));
node{1}.setService(jobclass{3}, Exp(1));
node{1}.setService(jobclass{4}, Exp(1));

node{2}.setService(jobclass{1}, Exp(3));
node{2}.setService(jobclass{2}, Exp(4));
node{2}.setService(jobclass{3}, Exp(5));
node{2}.setService(jobclass{4}, Exp(1));

node{3}.setService(jobclass{1}, Exp(1));
node{3}.setService(jobclass{2}, Exp(3));

... (60 source lines omitted; full implementation: matlab/examples/advanced/)

tic;
solver = NC(model, 'seed', 23000, 'method', 'ls', 'samples', 10000);
Pr_nc = solver.getProbSysAggr();
fprintf(1, 'NC: Pr_nc = %.15g\n', Pr_nc);

toc

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.3.7 statepr_sys_marg

The joint law of the per-station TOTAL queue lengths, all classes summed out, through getProbSysMarg.

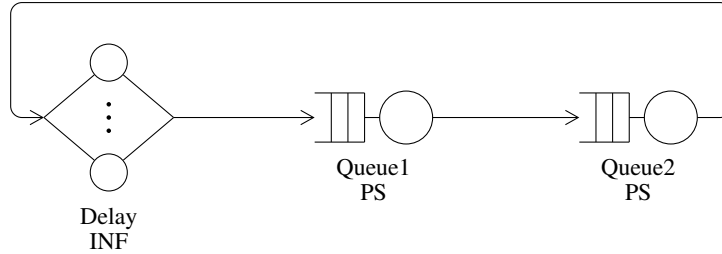


Figure 3.25: Topology of statepr_sys_marg: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: Exp(0.6667); Class2: Exp(0.5) Queue1 – Class1: Exp(1.429); Class2: Exp(2.5) Queue2 – Class1: Exp(3.333); Class2: Exp(1.111)
Solvers	CTMC

Table 3.25: Features of statepr_sys_marg.

The example is built and solved as follows.

```
% Joint probability of the per-station TOTAL queue lengths, all classes summed
% out, from SolverNC.getProbSysMarg.
%
% Compare with statepr_sys_aggr.m, which fixes the PER-CLASS population of
% every station: each probability here is the sum of that one over every
% per-class table with these row sums. The fibre grows combinatorially, so the
% quantity is evaluated as a permanent of the demand matrix replicated once per
% job rather than by enumerating it.

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);

N = [2,1];
jobclass{1} = ClosedClass(model, 'Class1', N(1), node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', N(2), node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1/1.5));
node{1}.setService(jobclass{2}, Exp(1/2.0));
node{2}.setService(jobclass{1}, Exp(1/0.7));
node{2}.setService(jobclass{2}, Exp(1/0.4));
node{3}.setService(jobclass{1}, Exp(1/0.3));
node{3}.setService(jobclass{2}, Exp(1/0.9));

... (27 source lines omitted; full implementation: matlab/examples/advanced/)

pb(j) = solver.getProbSysMarg(states(j,:), 'bethe');
end
pb = pb / sum(pb);
line_printf(' Bethe engine, mean relative error : %.2f%%\n', 100*mean(abs(pb-p)./p));

solver.citations()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.4 Initial state and warm start

Transient analysis needs an initial state. `initFromMarginal` builds one from marginal queue lengths, filling in the phases and the in-service order in a way consistent with the discipline; a state may also be set explicitly. For closed classes the initial state must conserve the population, which is what makes the ordered form necessary rather than convenient.

A simulation can also be started from the steady state computed by another solver, which removes the warm-up transient from the sample path.

3.4.1 `init_state_fcfs_exp`

An initial state set from marginal queue lengths, and the transient it produces on an FCFS network.

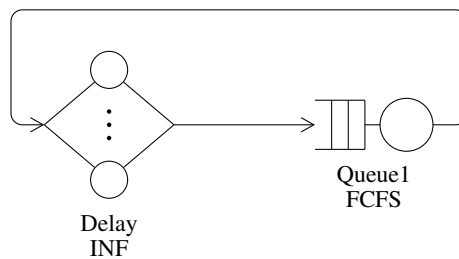


Figure 3.26: Topology of `init_state_fcfs_exp`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: FCFS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.7)

Table 3.26: Features of `init_state_fcfs_exp`.

The example is built and solved as follows.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(model, 'Class1', 5, node{2}, 0);

node{1}.setService(jobclass{1}, Exp(1));
node{2}.setService(jobclass{1}, Exp(0.7));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K,K);
P{1} = circul(2);

model.link(P);
[Qt,Ut,Tt] = model.getTranHandles();
options = Solver.defaultOptions;
options.verbose=0;
options.samples=1e4;
options.stiff=true;
options.timespan = [0,40];
```

```

%% This part illustrates the execution of different solvers
solver={};
solver{end+1} = CTMC(model, 'exact', options);

... (45 source lines omitted; full implementation: matlab/examples/advanced/)

title('Prior on state with 3 jobs in station 2');
ylabel('Queue length - station 2, class 1');
ylim([3,5])
xlabel('Time t');
xlim(options.timespan)
%legend('ctmc', 'fluid')

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.4.2 init_state_fcfs_nonexp

The same with non-exponential service, where the phase of the job in service is part of the state.

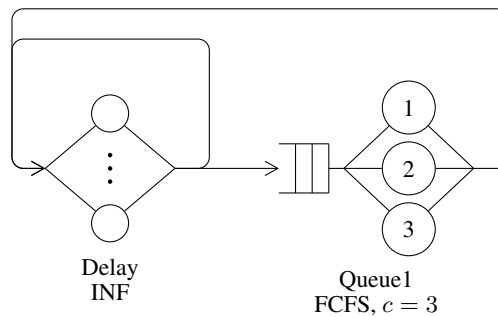


Figure 3.27: Topology of init_state_fcfs_nonexp: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(1.2); Class2: Erlang(2,2)
Solvers	CTMC, FLD

Table 3.27: Features of init_state_fcfs_nonexp.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(model, 'Class1', 3, node{2}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{2}, 0);
node{2}.setNumServers(3);

node{1}.setService(jobclass{1}, Exp(1));
node{1}.setService(jobclass{2}, Exp(1));
node{2}.setService(jobclass{1}, Exp(1.2));
node{2}.setService(jobclass{2}, Erlang.fitMeanAndSCV(1.0,0.5));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K,K);
P{1,1} = [0.3,0.1; 0.2,0];

```

```

P{1,2} = [0.6,0; 0.8,0];
P{2,2} = [0,1; 0,0];
P{2,1} = [0,0; 1,0];

model.link(P);
[Qt,Ut,Tt] = model.getTranHandles();
options = Solver.defaultOptions;
options.verbose=1;

... (73 source lines omitted; full implementation: matlab/examples/advanced/)

ylabel('QLen- station 2, class 1');
ylim([0,5])
xlabel('Time t');
xlim(options.timespan)
legend('ctmc','fluid')
hold off

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.4.3 init_state_ps

The processor sharing version.

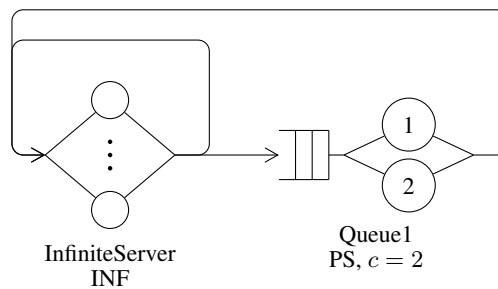


Figure 3.28: Topology of init_state_ps: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 1$)
Scheduling	InfiniteServer: INF; Queue1: PS ($c = 2$)
Service	InfiniteServer – Class1: Exp(3); Class2: Exp(0.5) Queue1 – Class1: Exp(0.1); Class2: Exp(1)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC

Table 3.28: Features of init_state_ps.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'InfiniteServer');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{2}.setNumServers(2);

jobclass{1} = ClosedClass(model, 'Class1', 3, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 1, node{1}, 0);

node{1}.setService(jobclass{1}, Exp(3));
node{1}.setService(jobclass{2}, Exp(0.5));

node{2}.setService(jobclass{1}, Exp(0.1));
node{2}.setService(jobclass{2}, Exp(1));

```

```

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K,K);
P{1,1} = [0.3,0.1; 0.2,0];
P{1,2} = [0.6,0; 0.8,0];
P{2,2} = [0,1; 0,0];
P{2,1} = [0,0; 1,0];

model.link(P);
model.initFromMarginalAndStarted([2,1;1,0],[0,0;1,0]);

... (19 source lines omitted; full implementation: matlab/examples/advanced/)

solver{end+1} = NC(model,options);
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.4.4 ldes_warmstart

Seeding a simulation from the steady state of another solver, so the warm-up phase can be skipped.

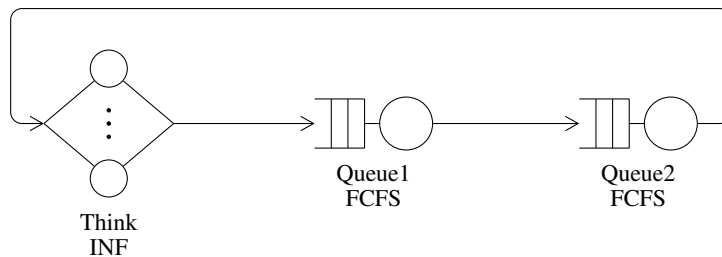


Figure 3.29: Topology of ldes_warmstart: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	1: Jobs (closed, $N = 100$)
Scheduling	Think: INF; Queue1: FCFS; Queue2: FCFS
Service	Think – Jobs: Exp(1) Queue1 – Jobs: Exp(1) Queue2 – Jobs: Exp(0.98)
Solvers	MVA, CTMC, LDES

Table 3.29: Features of ldes_warmstart.

The example is built and solved as follows.

```

%% LDES warm start from an auxiliary solver
% An auxiliary solver is passed to SolverLDES as an argument, its steady-state
% distribution is computed, and that distribution decides the initial state of
% the simulation. Since the simulation then starts (approximately) in steady
% state, the initialization bias vanishes and no warmup samples are discarded,
% so a target accuracy is reached with fewer simulated events (and less time)
% than a cold-started run.
%
% Auxiliary-solver dispatch inside SolverLDES.initFromSolver:
% - SolverCTMC: the exact stationary distribution over the aggregate state
%   space is computed and the initial state is its mode;
% - any other solver (e.g. SolverMVA): the steady-state mean queue lengths
%   are rounded to a placement conserving the closed populations.

```

```
%
% The benchmark model is a closed NEAR-BALANCED tandem, whose job split mixes
% slowly: the bias of the default cold start (all jobs at the reference
% station) persists beyond what the MSER-5 transient filter can remove.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

N = 100;
seeds = [23000 23001 23002];
budgets = [10000 50000 200000];

% Exact reference solution (exact MVA; the model is product-form)

... (66 source lines omitted; full implementation: matlab/examples/advanced/)

P = model.initRoutingMatrix();
P{1}(think, queue1) = 1.0;
P{1}(queue1, queue2) = 1.0;
P{1}(queue2, think) = 1.0;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.5 State-dependent routing

Under RAND or probabilistic routing the destination is drawn from a fixed distribution. State-dependent policies read the network state instead: JSQ sends the job to the shortest queue, k -choices samples k queues and picks the shortest of those, and round-robin and its weighted form cycle deterministically. Because the routing depends on the state, the network is no longer product-form, and JSQ in particular is only available to the simulators.

3.5.1 `sdroute_closed`

The closed counterpart.

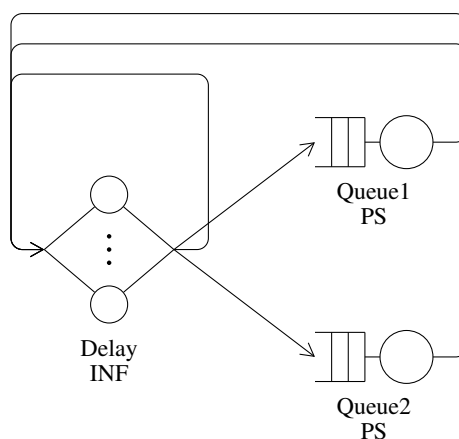


Figure 3.30: Topology of `sdroute_closed`: a closed network over 3 queueing stations.

The example is built and solved as follows.

```
model = Network('model');
node{1} = Delay(model, 'Delay');
```

Nodes	3: Queue $\times$ 2, Delay
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: HyperExp(0.99,0.01;1.534,0.02819) Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(2)
Features	class switching on 1 link
Solvers	CTMC, JMT, SSA

Table 3.30: Features of `sdroute_closed`.

```

node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);

node{1}.setService(jobclass{1}, HyperExp.fitMeanAndSCV(1,25));
node{2}.setService(jobclass{1}, Exp(1));
node{3}.setService(jobclass{1}, Exp(2));

model.addLink(node{1}, node{1});
model.addLink(node{1}, node{2});
model.addLink(node{1}, node{3});
model.addLink(node{2}, node{1});
model.addLink(node{3}, node{1});

node{1}.setRouting(jobclass{1}, RoutingStrategy.RROBIN);
node{2}.setProbRouting(jobclass{1}, node{1}, 1.0);
node{3}.setProbRouting(jobclass{1}, node{1}, 1.0);

solver={};
solver{end+1} = CTMC(model, 'exact', 'keep', true);
solver{end+1} = JMT(model, 'samples', 1e5, 'seed', 23000);
solver{end+1} = SSA(model, 'verbose', true, 'samples', 1e4, 'seed', 23000);

AvgTable = {};
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgTable();
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.5.2 `sdroute_jsq`

Join-the-shortest-queue dispatching over a pool of FCFS queues.

Nodes	6: Source, Queue $\times$ 3, Router, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Arrivals	Source – Class1: Exp(2.1)
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(1) Queue3 – Class1: Exp(1)
Features	class switching on 6 links
Solvers	JMT, LDES

Table 3.31: Features of `sdroute_jsq`.

The example is built and solved as follows.

```

model = Network('myModel');

N = 3;      % number of queues

```

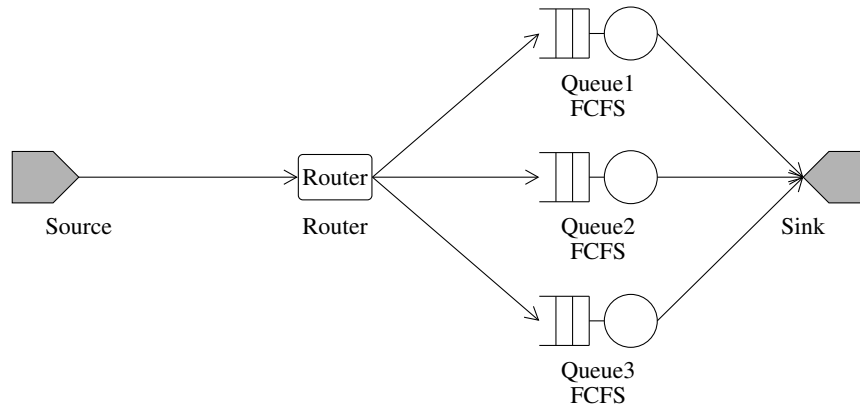



Figure 3.31: Topology of `sdroute_jsq`: an open network over 3 queueing stations with a router.

```

rho = 0.7; % per-queue load
mu = 1;    % service rate per queue
lambda = N * rho * mu; % total arrival rate

% Block 1: nodes
source = Source(model, 'Source');
router = Router(model, 'Router');
queue = cell(1, N);
for i = 1:N
    queue{i} = Queue(model, sprintf('Queue%d', i), SchedStrategy.FCFS);
end
sink = Sink(model, 'Sink');

% Block 2: classes
oclass = OpenClass(model, 'Class1');
source.setArrival(oclass, Exp(lambda));
for i = 1:N
    queue{i}.setService(oclass, Exp(mu));
end

% Block 3: topology
model.addLink(source, router);
for i = 1:N
    ... (10 source lines omitted; full implementation: matlab/examples/advanced/)

AvgTable = {};
for s = 1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgNodeTable();
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.5.3 `sdroute_krzesinski`

Adaptive routing with balanced loading (Krzesinski 1987): a central server and four single-center branches whose routing probabilities depend on the branch populations, and which keep a product form. The example is built and solved as follows.

```

% Product-form state-dependent routing, Krzesinski (1987), Perform. Eval. 7:125-143.
%
% Section 5.1.1, adaptive routing with balanced loading: a central server and
% four peripheral centers forming four single-center branches under one level
% of subnetwork nesting. With C_1 = -1 and d_{1,i} = d the entry center routes

```

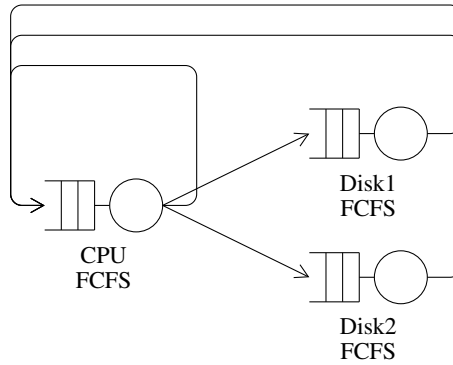


Figure 3.32: Topology of sdroute_krzesinski: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	1: Class1 (closed, $N = 3$)
Scheduling	CPU: FCFS; Disk1: FCFS; Disk2: FCFS
Service	CPU – Class1: Exp(1) Disk1 – Class1: Exp(0.8) Disk2 – Class1: Exp(0.5)
Solvers	NC, CTMC

Table 3.32: Features of sdroute_krzesinski.

```
%
%   P_{1,i}(N) = (d - n_i) / (4d - V),      V = n_2 + n_3 + n_4 + n_5,
%
% which prefers the least congested peripheral center and caps each of them at
% d customers. The state-independent counterpart of eq. (15) is P_{1,i} = 1/4.
% The throughput gain of SDR over SIR grows with the CPU/IO speed ratio, which
% is the paper's Fig. 4.

d = 1;           % routing coefficient d_{1,i}
N = 4;           % network population
mul = 100;       % central server rate
mup = 20;        % peripheral rate

model = Network('sdroute_krzesinski');
node{1} = Queue(model, 'CPU', SchedStrategy.FCFS);
for i = 2:5
    node{i} = Queue(model, sprintf('IO%d',i-1), SchedStrategy.FCFS);
end
jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);
node{1}.setService(jobclass{1}, Exp(mul));
for i = 2:5

    ... (33 source lines omitted; full implementation: matlab/examples/advanced/)

SIRTable = SolverNC(sirmodel).getAvgTable

Tsdr = sum(AvgTable.Tput(2:5));
Tsir = sum(SIRTable.Tput(2:5));
line_printf('\nPeripheral throughput: SDR %.4f, SIR %.4f, gain %.1f%%\n', ...
    Tsdr, Tsir, 100*(Tsdr-Tsir)/Tsir);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.5.4 sdroute_multibranch

The same product form on branches holding several centers, where the coefficients are the branch traffic equations.

No topology is drawn for this example: the captured run yielded no Network or LayeredNetwork to

draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC
----------------	------

Table 3.33: Features of `sdroute_multibranch`.

The example is built and solved as follows.

```
% Multi-center branches under Krzesinski (1987) state-dependent routing.
%
% Checks that the product form of eq. (16) extends to a branch holding several
% centers, and that the coefficients xi are the branch traffic equations rather
% than the paper Section 3.2 shorthand xi = xi_e, which is exact only when the
% branch departure center is visited once. See _kb/16-state-dependent-routing.md
clear; lineStart;

fprintf('\n==== SDR with multi-center branches ==== \n');
run_case('A: branch 2 = 2a -> 2b (series)', 0.0);
run_case('B: branch 2 = 2a -> 2b, 2b -> 2a w.p. 0.5 (feedback onto the branch departure)',
        0.5);

function run_case(label, pback)
mu = [1 0.9 0.7 0.5]; % centers 1, 2a, 2b, 3
N = 3;
model = Network('sdr_multi');
nd{1} = Queue(model, 'CPU', SchedStrategy.FCFS);
nd{2} = Queue(model, 'B2a', SchedStrategy.FCFS);
nd{3} = Queue(model, 'B2b', SchedStrategy.FCFS);
nd{4} = Queue(model, 'B3', SchedStrategy.FCFS);
cls = ClosedClass(model, 'Class1', N, nd{1}, 0);
for i = 1:4, nd{i}.setService(cls, Exp(mu(i))); end
model.addLink(nd{1}, nd{1});
model.addLink(nd{1}, nd{2});
model.addLink(nd{1}, nd{4});
model.addLink(nd{2}, nd{3});

... (34 source lines omitted; full implementation: matlab/examples/advanced/)

% the reading asserted verbatim by the paper for E+D: xi = xi_e everywhere
xiflat = ones(4,1);
[Q2, X2] = pfqn_sdr(S, xiflat, N, sn.sdr);
fprintf('    xi==1 everywhere: max|dQ| = %.3e    max|dX| = %.3e\n', max(abs(Qc-Q2)), max(abs(
    Xc-X2)));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.5.5 `sdroute_open`

Round-robin dispatching from a Router node in an open network.

Nodes	5: Source, Queue $\times$ 2, Router, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(2)
Features	class switching on 5 links
Solvers	JMT, CTMC

Table 3.34: Features of `sdroute_open`.

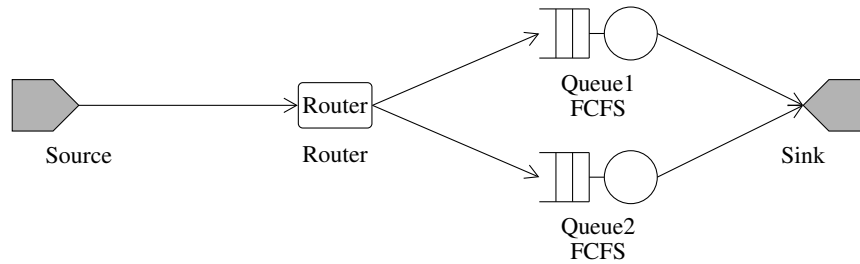


Figure 3.33: Topology of `sdroute_open`: an open network over 2 queueing stations with a router.

The example is built and solved as follows.

```
model = Network('myModel');

% Block 1: nodes
source = Source(model, 'Source');
router = Router(model, 'Router');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

% Block 2: classes
oclass = OpenClass(model, 'Class1');
source.setArrival(oclass, Exp(1));
queue1.setService(oclass, Exp(2));
queue2.setService(oclass, Exp(2));

% Block 3: topology
model.addLink(source, router);
model.addLink(router, queue1);
model.addLink(router, queue2);
model.addLink(queue1, sink);
model.addLink(queue2, sink);

router.setRouting(oclass, RoutingStrategy.RROBIN);

solver = {};
solver{1} = JMT(model, 'seed', 23000);
solver{2} = CTMC(model, 'exact', 'cutoff', 5);

AvgTable = {};
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', strrep(solver{s}.getName(), 'Solver', ''));
    AvgTable{s} = solver{s}.getAvgNodeTable();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.5.6 `sdroute_twoclasses_closed`

Two classes with MAP, MMPP(2) and phase-type service under round-robin dispatching.

The example is built and solved as follows.

```
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Queue(model, 'Queue2', SchedStrategy.FCFS);

jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);
```

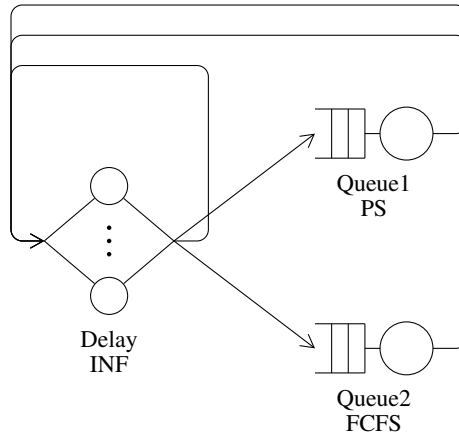


Figure 3.34: Topology of `sdroute_twoclasses_closed`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS; Queue2: FCFS
Service	Delay – Class1: HyperExp(0.99,0.01;1.534,0.02819); Class2: HyperExp(0.99,0.01;1.534,0.02819) Queue1 – Class1: APH; Class2: PH Queue2 – Class1: MAP; Class2: MAP
Features	class switching on 1 link
Solvers	CTMC, JMT, SSA

Table 3.35: Features of `sdroute_twoclasses_closed`.

```
% renewal
map21 = APH([1,0],[-2,2; 0,-0.5]);
map22 = PH(1,-1);
% non-renewal
map31 = MAP([-20,0; 0,-1],[0,20;0.8,0.2]);
map32 = MAP([-4, 3; 4, -6],[1,0;0,2]);
%map32 = MMPP2(1,2,3,4).toMAP();

node{1}.setService(jobclass{1}, HyperExp.fitMeanAndSCV(1,25));
node{2}.setService(jobclass{1}, map21);
node{3}.setService(jobclass{1}, map31);

node{1}.setService(jobclass{2}, HyperExp.fitMeanAndSCV(1,25));
node{2}.setService(jobclass{2}, map22);
node{3}.setService(jobclass{2}, map32);

model.addLink(node{1}, node{1});

... (18 source lines omitted; full implementation: matlab/examples/advanced/)

AvgTable = {};
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',strrep(solver{s}.getName(),'Solver',''));
    AvgTable{s} = solver{s}.getAvgTable();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.6 Cyclic polling

A polling server visits its queues in a cycle. Under gated service it serves the jobs that were waiting when it arrived, under exhaustive service it stays until the queue empties, and under k -limited service it serves at most k jobs per visit. The discipline is declared as `SchedStrategy.POLLING` and the variant is a parameter of the station.

The polling server parks at a queue rather than roving: the visit order and the ratio between the queue lengths are what a polling model must be validated on, not the aggregate throughput, which is insensitive to the discipline.

3.6.1 polling_exhaustive_det

Exhaustive service with deterministic (immediate) switchover.

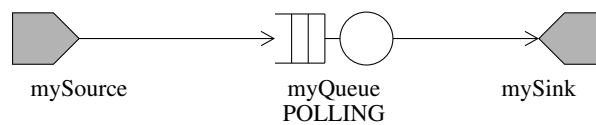


Figure 3.35: Topology of `polling_exhaustive_det`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Det(1); myClass2: Det(1)
Service	myQueue – myClass1: Det(0.001); myClass2: Det(0.001)
Solvers	JMT

Table 3.36: Features of `polling_exhaustive_det`.

The example is built and solved as follows.

```
model = Network('M[2]/M[2]/1-Gated');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.POLLING);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Det.fitMean(1.0));
queue.setService(oclass1, Det.fitMean(0.001));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Det.fitMean(1.0));
queue.setService(oclass2, Det.fitMean(0.001));

queue.setPollingType(PollingType.EXHAUSTIVE)
queue.setSwitchover(oclass2, Immediate())
queue.setSwitchover(oclass1, Immediate())
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue, sink);
P{2} = Network.serialRouting(source, queue, sink);
model.link(P);

MVA(model).getAvgTable() % solution is approximate in general
JMT(model, 'seed', 23000, 'samples', 1e5).getAvgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.6.2 polling_exhaustive_exp

Exhaustive service with exponential switchover times.

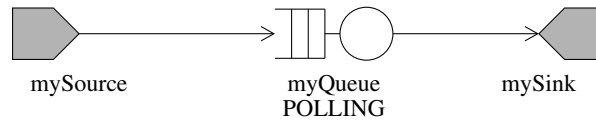


Figure 3.36: Topology of `polling_exhaustive_exp`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Exp(0.1); myClass2: Exp(0.1)
Service	myQueue – myClass1: Exp(1); myClass2: Exp(1.5)
Solvers	JMT

Table 3.37: Features of `polling_exhaustive_exp`.

The example is built and solved as follows.

```
model = Network('M[2]/M[2]/1-Gated');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.POLLING);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(0.1));
queue.setService(oclass1, Exp(1.0));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.1));
queue.setService(oclass2, Exp(1.5));

queue.setPollingType(PollingType.EXHAUSTIVE)
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue, sink);
P{2} = Network.serialRouting(source, queue, sink);
model.link(P);

MVA(model).getAvgTable() % solution is approximate in general
JMT(model, 'seed', 23000).getAvgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.6.3 polling_gated

A gated polling server: at each visit it serves the jobs that were waiting when it arrived.

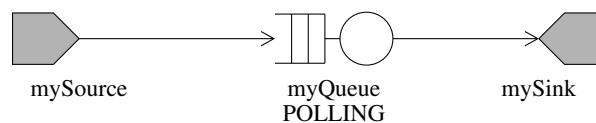


Figure 3.37: Topology of `polling_gated`: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.8)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(1.5)
Solvers	JMT

Table 3.38: Features of `polling_gated`.

```

model = Network('M[2]/M[2]/1-Exhaustive');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.POLLING);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(1.0));
queue.setService(oclass1, Exp(4.0));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.8));
queue.setService(oclass2, Exp(1.5));

queue.setPollingType(PollingType.GATED)
queue.setSwitchover(oclass1, Exp(1.0))
queue.setSwitchover(oclass2, Exp(0.5))
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue, sink);
P{2} = Network.serialRouting(source, queue, sink);
model.link(P);

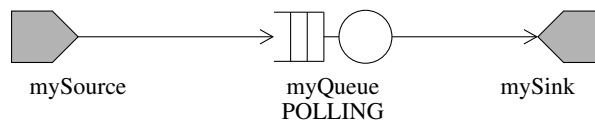
MVA(model).getAvgTable() % solution is exact
JMT(model, 'seed', 23000, 'samples', 1e6).getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.6.4 `polling_klimited`

k -limited service, at most k jobs per visit.

Figure 3.38: Topology of `polling_klimited`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Exp(0.2); myClass2: Exp(0.3)
Service	myQueue – myClass1: Exp(1); myClass2: Exp(1.5)
Solvers	JMT

Table 3.39: Features of `polling_klimited`.

The example is built and solved as follows.


```

model = Network('M[2]/M[2]/1-Gated');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.POLLING);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(0.2));
queue.setService(oclass1, Exp(1.0));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.3));
queue.setService(oclass2, Exp(1.5));

queue.setPollingType(PollingType.KLIMITED, 1)
queue.setSwitchover(oclass1, Exp(1))
queue.setSwitchover(oclass2, Immediate())
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue, sink);
P{2} = Network.serialRouting(source, queue, sink);
model.link(P);

MVA(model).getAvgTable() % solution is approximate
JMT(model, 'samples', 1e5, 'seed', 23000).getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.7 Switchover times

Moving the server between queues costs time. A switchover distribution is declared per arc of the polling cycle, and its mean adds directly to the cycle time, so the waiting time grows even though no extra work is done.

3.7.1 switchover_basic

Pairwise switchover metadata on an ordinary FCFS queue. The JMT run warns that non-polling queues do not consume these times and solves the underlying saturated two-class queue without them; use the cyclic-polling examples for honoured switchover delays.

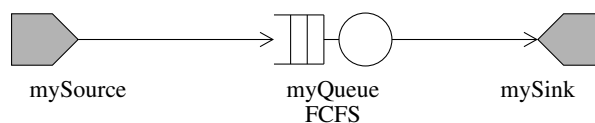


Figure 3.39: Topology of switchover_basic: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass1: Exp(0.2); myClass2: Exp(0.8)
Service	myQueue – myClass1: Exp(0.1); myClass2: Exp(1.5)
Solvers	JMT

Table 3.40: Features of switchover_basic.

The example is built and solved as follows.

```

model = Network('M[2]/M[2]/1-Gated');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(0.2));
queue.setService(oclass1, Exp(0.1));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.8));
queue.setService(oclass2, Exp(1.5));

queue.setSwitchover(oclass1, oclass2, Exp(1))
queue.setSwitchover(oclass2, oclass1, Erlang(1,2))
% Note: JMT does not support switchover times for non-polling (FCFS) queues.
% For JMT simulation with switchover, use SchedStrategy.POLLING.
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source,queue,sink);
P{2} = Network.serialRouting(source,queue,sink);
model.link(P);

%MVA(model).getAvgTable()
JMT(model, 'seed', 23000, 'keep', true).getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.8 Response time distribution

`getCdfRespT` returns the cumulative distribution of the response time, per station and per class, rather than its mean. The distribution is what a percentile service level objective is written against, and it is far more sensitive to the service distribution than the mean is: two models with the same mean response time can have very different tails.

3.8.1 cdf_respt_closed

The response time CDF at a station of a closed PS network.

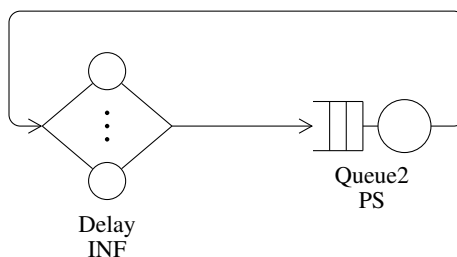


Figure 3.40: Topology of `cdf_respt_closed`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue2', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);

```

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Delay: INF; Queue2: PS
Service	Delay – Class1: Exp(10) Queue2 – Class1: Erlang(3,3)
Solvers	JMT, FLD

Table 3.41: Features of `cdf_respt_closed`.

```

servProc1 = Exp(1/0.1);
node{1}.setService(jobclass{1}, servProc1);
servProc2 = Erlang.fitMeanAndSCV(1,1/3);
node{2}.setService(jobclass{1}, servProc2);

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K);
P{1,1} = circular(2);

model.link(P);
%%
solver = JMT(model, 'seed', 23000, 'samples', 1e4);
RDSim = solver.getCdfRespT();
fprintf(1, '\n')
for i=1:model.getNumberOfStations
    for c=1:model.getNumberOfClasses
        % plot(FC{i,c}(:,2), FC{i,c}(:,1)); hold all;

        ... (36 source lines omitted; full implementation: matlab/examples/advanced/)

AvgRespTfromTheory = [servProc1.getMean; servProc2.getMean]
AvgRespTfromCDFSIm
AvgRespTfromCDFFluid
SqCoeffOfVariationRespTfromTheory = [servProc1.getSCV; servProc2.getSCV]
SqCoeffOfVariationRespTfromCDFSIm
SqCoeffOfVariationRespTfromCDFFluid

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.8.2 `cdf_respt_closed_threeclasses`

The three-class version, with Erlang service.

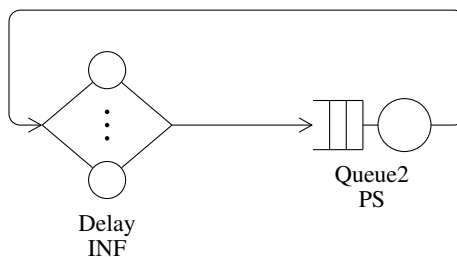


Figure 3.41: Topology of `cdf_respt_closed_threeclasses`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```

model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue2', SchedStrategy.PS);

```

Nodes	2: Queue, Delay
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 0$)
Scheduling	Delay: INF; Queue2: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1) Queue2 – Class1: Exp(1); Class2: Erlang(0.5,2); Class3: Exp(100)
Solvers	FLD

Table 3.42: Features of `cdf_respt_closed_threeclasses`.

```

jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 0, node{1}, 0);
jobclass{3} = ClosedClass(model, 'Class3', 0, node{1}, 0);

jobclass{1}.completes = false;

node{1}.setService(jobclass{1}, Exp(1/1));
node{1}.setService(jobclass{2}, Exp(1/1));
node{1}.setService(jobclass{3}, Exp(1/1));

node{2}.setService(jobclass{1}, Exp(1/1));
node{2}.setService(jobclass{2}, Erlang(1/2,2));
node{2}.setService(jobclass{3}, Exp(1/0.01));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K,K);

P{1,1} = [0,1; 0,0];
P{1,2} = [0,0; 1,0];

... (26 source lines omitted; full implementation: matlab/examples/advanced/)

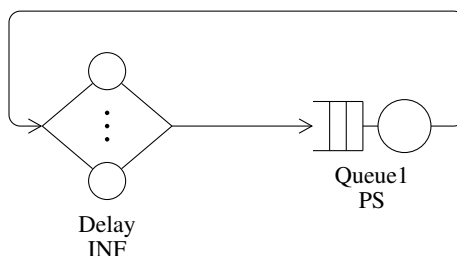
Variance_R(i,c) = PowerMoment2_R(i,c)-AvgRespTfromCDF(i,c)^2; %variance
SqCoeffOfVariationRespTfromCDF(i,c) = (Variance_R(i,c))/AvgRespTfromCDF(i,c)^2; %scv
end
end
AvgRespTfromCDF
SqCoeffOfVariationRespTfromCDF

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.8.3 `cdf_respt_distrib`

The distribution object returned by `getCdfRespT` and the percentiles read from it.

Figure 3.42: Topology of `cdf_respt_distrib`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
model = Network('model');
```

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Erlang(0.5,2) Queue1 – Class1: Exp(0.5); Class2: Hyper-Exp(0.99,0.01;0.324,0.005143)
Solvers	FLD, JMT

Table 3.43: Features of `cdf_respt_distrib`.

```

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);

jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp.fitMean(2.0));

jobclass{2} = ClosedClass(model, 'Class2', 3, node{1}, 0);
node{1}.setService(jobclass{2}, Erlang.fitMeanAndOrder(4.0,2));
node{2}.setService(jobclass{2}, HyperExp.fitMeanAndSCV(5.0,30.0));

P = cell(2);
P{1,1} = circul(2);
P{1,2} = zeros(2);
P{2,1} = zeros(2);
P{2,2} = circul(2);

% model
model.link(P);
RDfluid = FLD(model).getCdfRespt();
jmtoptions = JMT.defaultOptions;
jmtoptions.samples = 1e5;
jmtoptions.seed = 23000;
RDsim = JMT(model, jmtoptions).getTranCdfRespt();

... (22 source lines omitted; full implementation: matlab/examples/advanced/)

AvgResptfromCDFfluid(i,c) = diff(RDfluid{i,c}(:,1))'*RDfluid{i,c}(2:end,2);
AvgResptfromCDFsim(i,c) = diff(RDsim{i,c}(:,1))'*RDsim{i,c}(2:end,2);
end
end
AvgResptfromCDFfluid
AvgResptfromCDFsim

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.8.4 `cdf_respt_open_twoclasses`

The open two-class version, on FCFS stations.

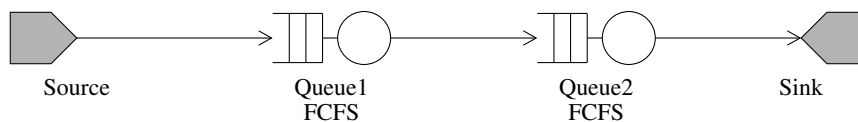


Figure 3.43: Topology of `cdf_respt_open_twoclasses`: an open network over 2 queueing stations.

The example is built and solved as follows.

```

model = Network('myModel');

%% Block 1: nodes

```

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.25); Class2: Exp(0.25)
Service	Queue1 – Class1: Exp(1); Class2: Exp(1) Queue2 – Class1: Exp(1); Class2: Exp(1)
Solvers	FLD, JMT

Table 3.44: Features of `cdf_respt_open_twoclasses`.

```

node{1} = Source(model, 'Source');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{4} = Sink(model, 'Sink');

%% Block 2: classes
jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);

node{1}.setArrival(jobclass{1}, Exp.fitMean(4.000000)); % (Source,Class1)
node{1}.setArrival(jobclass{2}, Exp.fitMean(4.000000)); % (Source,Class2)

node{2}.setService(jobclass{1}, Exp.fitMean(1.000000)); % (Queue1,Class1)
node{2}.setService(jobclass{2}, Exp.fitMean(1.000000)); % (Queue1,Class2)

node{3}.setService(jobclass{1}, Exp.fitMean(1.000000)); % (Queue2,Class1)
node{3}.setService(jobclass{2}, Exp.fitMean(1.000000)); % (Queue2,Class2)

%% Block 3: topology
P = model.initRoutingMatrix(); % initialize routing matrix
P{1,1}(1,2) = 1;
P{1,2}(2,3) = 1;
P{2,1}(3,4) = 1;

... (38 source lines omitted; full implementation: matlab/examples/advanced/)

AvgRespTfromCDFsim(i,c) = diff(RDsim{i,c}(:,1))'*RDsim{i,c}(2:end,2);
end
end

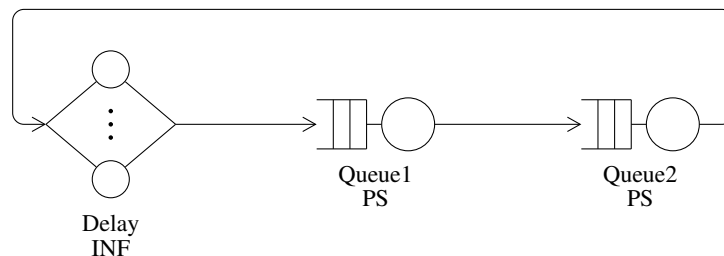
AvgRespTfromCDFfluid
AvgRespTfromCDFsim

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.8.5 `cdf_respt_populations`

How the response time distribution moves as the closed population grows.

Figure 3.44: Topology of `cdf_respt_populations`: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 2, Delay
Classes	1: Class1 (closed, $N = 8$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.5) Queue2 – Class1: Exp(0.5)
Solvers	FLD

Table 3.45: Features of `cdf_respt_populations`.

```

figure;
label = {};
nJobs = [1,4,8];
plotColors = {'k','b--','r-.'};
for ni = 1:length(nJobs)
    N = nJobs(ni);
    model = Network('model');

    node{1} = Delay(model, 'Delay');
    node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
    node{3} = Queue(model, 'Queue2', SchedStrategy.PS);

    jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);

    jobclass{1}.completes = false;

    node{1}.setService(jobclass{1}, Exp(1/1));
    node{2}.setService(jobclass{1}, Exp(1/2));
    node{3}.setService(jobclass{1}, Exp(1/2));

    M = model.getNumberOfStations();
    K = model.getNumberOfClasses();

    P = circul(M);
    model.link(P);
    %%

    ... (26 source lines omitted; full implementation: matlab/examples/advanced/)

end
legend(label, 'Location', 'SouthEast');
xlim([0,200])
title('Response time CDF at station 3 under increasing populations')
ylabel('Pr[RespT < t]')
xlabel('Response time t')

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.9 Busy periods

A busy period of order n runs from the instant an arrival raises the number of jobs held by the target to n until it falls back below n . The target may be a station, a station-class pair, or a declared set of stations, and the order n ranges over $1..K$. The analytical reference is the busy period formula of Daduna, exposed by the normalizing constant solver; the simulator measures the same quantity from the sample path.

3.9.1 busyp_subnetwork

Mean busy periods of order n for a station, for a station-class pair and for a declared subnetwork of stations.

The example is built and solved as follows.

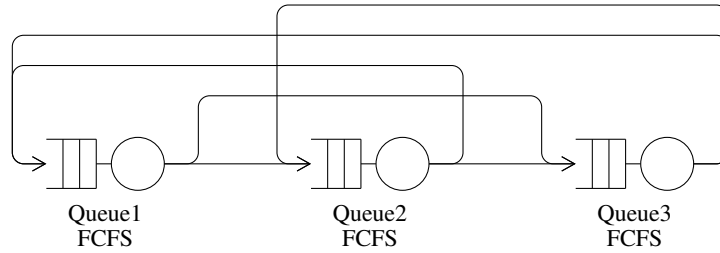


Figure 3.45: Topology of busyp_subnetwork: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Class1: Exp(1.5) Queue2 – Class1: Exp(0.9) Queue3 – Class1: Exp(2)
Solvers	LDES

Table 3.46: Features of busyp_subnetwork.

```
% Mean busy period of order n for a subnetwork of a closed queueing network.
%
% The busy period of order n for a set of stations is the time from the
% instant a job entering the set finds n-1 jobs in it up to the next instant
% when fewer than n remain. SolverNC evaluates it exactly from the
% normalizing constants of the subnetwork and of its complement (H. Daduna,
% "Busy Periods for Subnetworks in Stochastic Networks: Mean Value Analysis",
% J. ACM 35(3), 1988); SolverLDES measures the same quantity along a
% simulated sample path.

N = 5;
model = Network('busyPeriodModel');

node{1} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{2} = Queue(model, 'Queue2', SchedStrategy.FCFS);
node{3} = Queue(model, 'Queue3', SchedStrategy.FCFS);

jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);

node{1}.setService(jobclass{1}, Exp(1.5));
node{2}.setService(jobclass{1}, Exp(0.9));
node{3}.setService(jobclass{1}, Exp(2.0));

P = model.initRoutingMatrix();
P{jobclass{1}, jobclass{1}} = [0 0.6 0.4; 0.7 0 0.3; 0.5 0.5 0];
model.link(P);

... (15 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf(1, '%-18s %10s %10s %10s\n', 'subnetwork', 'n=1', 'n=3', 'n=5');
for s = 1:numel(subnets)
    I = subnets{s};
    b = ldesSolver.getAvgBusyPeriod(I, -1, [1 3 5]);
    fprintf(1, '%-18s %10.4f %10.4f %10.4f\n', mat2str(I), b(1), b(2), b(3));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.10 Stochastic network calculus

Every other family in these examples reports a mean. The stochastic network calculus reports a *tail*: given a violation probability ε it returns a delay or a queue length that is exceeded with probability at

most ε , under any work-conserving scheduling policy at the station. Each flow is carried by a moment-generating-function envelope and each server by the counting process of its service; the min-plus operations compose them across a feed-forward network, at the cost of the burstiness a server adds to the flow it forwards. The bound on the mean that follows by integration is loose, and the decay rate of the tail is asymptotically exact, so the quantile is what these examples read.

3.10.1 snc_delay_quantile

A delay quantile with a certified violation probability, and how it tightens deep in the tail.

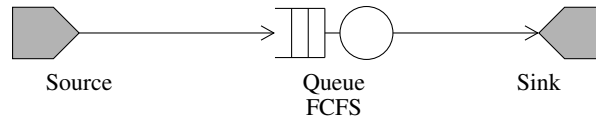


Figure 3.46: Topology of snc_delay_quantile: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.6)
Service	Queue – Class1: Exp(1)

Table 3.47: Features of snc_delay_quantile.

The example is built and solved as follows.

```
% Stochastic network calculus: a delay quantile with a certified violation
% probability.
%
% Every other solver in LINE answers with a MEAN. The 'snc' family of SolverBA
% answers with a TAIL: given a violation probability eps, it returns a delay d
% for which P{D > d} <= eps holds, and the guarantee is valid for any
% work-conserving scheduling policy at the station. That is the quantity a
% service-level objective is written against.
%
% The model is a single M/M/1 station, whose exact tail is known in closed
% form, so every number below can be checked.

%% Block 1: model
lambda = 0.6;
mu = 1;
model = Network('SncQuantile');
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
jobclass = OpenClass(model, 'Class1');
source.setArrival(jobclass, Exp(lambda));
queue.setService(jobclass, Exp(mu));
model.link(Network.serialRouting(source, queue, sink));

solver = SolverBA(model, 'method', 'snc.upper');

... (48 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf('\napi: d(1e-3) = %.4f at theta = %.4f\n', d, theta);
fprintf('    the optimal theta approaches log(mu/lambda) = %.4f, which is\n', log(mu/lambda)
);
fprintf('    what makes the backlog decay rate exact\n');
[epsAt, thetaAt] = snc_bound_delay(arv, srv, d);
fprintf('api: P{D > %.4f} <= %.3e (theta = %.4f), exact tail %.3e\n', ...
d, epsAt, thetaAt, exp(-(mu-lambda)*d));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.10.2 snc_tandem_multiclass

Envelope propagation across a feed-forward tandem, pay-bursts-only-once, blind multiplexing, and the model classes the family refuses.

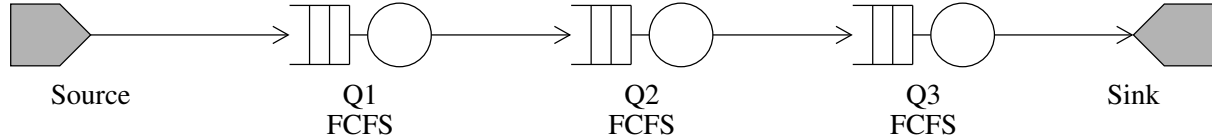


Figure 3.47: Topology of snc_tandem_multiclass: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	1: Class1 (open)
Scheduling	Q1: FCFS; Q2: FCFS; Q3: FCFS
Arrivals	Source – Class1: Exp(0.6)
Service	Q1 – Class1: Exp(1.5) Q2 – Class1: Exp(1.2) Q3 – Class1: Exp(1)

Table 3.48: Features of snc_tandem_multiclass.

The example is built and solved as follows.

```
% Stochastic network calculus on a feed-forward network: envelope propagation,
% blind multiplexing, and the model classes the family refuses.
%
% The bound of snc_delay_quantile.m is a single station. This example is what
% the network calculus adds on top of it: a departure envelope carries a flow
% to the next hop, cross traffic is subtracted from a shared server, and both
% operations cost burstiness, which is visible as the bound loosening downstream
% and under sharing.
%
% The reference throughput is the exact Jackson-network solution, since every
% station here is an M/M/1 with Poisson input by Burke's theorem.

%% Block 1: a three-station tandem
lambda = 0.6;
rates = [1.5 1.2 1.0]; % service rates, the last is the bottleneck
model = Network('SncTandem');
source = Source(model, 'Source');
q1 = Queue(model, 'Q1', SchedStrategy.FCFS);
q2 = Queue(model, 'Q2', SchedStrategy.FCFS);
q3 = Queue(model, 'Q3', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
jobclass = OpenClass(model, 'Class1');
source.setArrival(jobclass, Exp(lambda));
q1.setService(jobclass, Exp(rates(1)));
q2.setService(jobclass, Exp(rates(2)));
q3.setService(jobclass, Exp(rates(3)));

... (138 source lines omitted; full implementation: matlab/examples/advanced/)

P = model.initRoutingMatrix();
P{classA, classA}(source, queue) = 1; P{classA, classA}(queue, sink) = 1;
P{classB, classB}(source, queue) = 1; P{classB, classB}(queue, sink) = 1;
model.link(P);
SolverBA(model, 'method', 'snc.upper').getAvgTable();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11 Random environments

A random environment is a Markov process over stages, with one model per stage. The environment moves between stages according to a transition process, and the network parameters change with it; the metrics reported are the environment average, weighted by the stage probabilities, but the transient handoff between stages matters when the stage sojourn is comparable with the relaxation time of the network.

Server breakdown and repair is the smallest instance: two stages, one with the server up and one with it down. The case studies at the end of the family are larger, and are the models on which the environment solver is validated.

3.11.1 example_mapqn2renv

Converting a queueing network with MMPP(2) service into an equivalent random environment model.

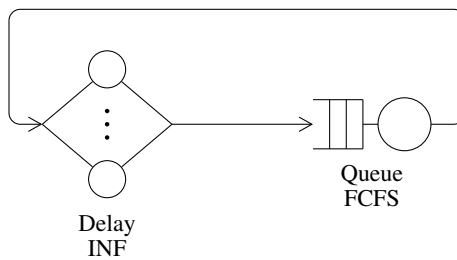


Figure 3.48: Topology of example_mapqn2renv: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue: FCFS
Service	Delay – Class1: Exp(1) Queue – Class1: MMPP2
Solvers	JMT

Table 3.49: Features of example_mapqn2renv.

The example is built and solved as follows.

```
% Example: Transform MMPP service queue to random environment model
%
% This example shows how to transform a closed queueing network with MMPP2
% service into a random environment model with exponential services
% modulated by the MMPP phases.
%
% MMPP2 (2-phase Markov Modulated Poisson Process) is a point process
% where service completions occur at rates modulated by a 2-state Markov chain.
% This transformation converts such a network into an equivalent random
% environment model where:
%   - Environment has 2 stages (one per MMPP phase)
%   - Service rates are exponential with rates from MMPP D1 diagonal
%   - Environment transitions follow MMPP D0 matrix structure

clear; close all;

% Create closed queueing network with MMPP2 service
model = Network('MMPP_ClosedQN');

% Create nodes
delay = Delay(model, 'Delay');           % Think time station
queue = Queue(model, 'Queue', SchedStrategy.FCFS); % Service station with MMPP2
```

```
% Create closed job class with N=5 jobs
N = 5; % Number of jobs in the system
jobclass = ClosedClass(model, 'Class1', N, delay);

... (146 source lines omitted; full implementation: matlab/examples/advanced/)

end
catch ME2
    fprintf('Stage solver error: %s\n', ME2.message);
end

fprintf('\n=== Transformation Complete ===\n');
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.2 renv_basic

The smallest random environment: two stages, one model per stage, and the stage transition process. No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	FLD, MVA
---------	----------

Table 3.50: Features of `renv_basic`.

The example is built and solved as follows.

```
% Example 11: Random environments and ENV
% This tutorial illustrates how to model a queueing system operating in
% a random environment, where system parameters (e.g., service rates)
% change according to an underlying environmental process.
%
% Scenario: A server that alternates between "Fast" and "Slow" modes.
% In Fast mode, service rate is 4.0. In Slow mode, service rate is 1.0.
% The environment switches from Fast->Slow at rate 0.5 and Slow->Fast at rate 1.0.

warning off;

%% Block 1: Create base network model
% First, we define a closed queueing network with a delay and a queue
baseModel = Network('BaseModel');
delay = Delay(baseModel, 'ThinkTime');
queue = Queue(baseModel, 'Fast/Slow Server', SchedStrategy.FCFS);

% Closed class with 5 jobs
N = 5;
jobclass = ClosedClass(baseModel, 'Jobs', N, delay);
delay.setService(jobclass, Exp(1.0)); % Think time = 1.0
queue.setService(jobclass, Exp(2.0)); % Placeholder service rate

% Connect nodes in a cycle
baseModel.link(Network.serialRouting(delay, queue));

... (55 source lines omitted; full implementation: matlab/examples/advanced/)

stageModel = env.envGraph.Nodes.Model{e};
fprintf('\nStage: %s (prob = %.4f)\n', stageName, env.probEnv(e));
mvaSolver = MVA(stageModel);
stageTable = mvaSolver.getAvgTable();
disp(stageTable);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.3 `renv_container_terminal`

A container terminal case study: MAP arrivals, multiserver and load-dependent stations, several environment stages.

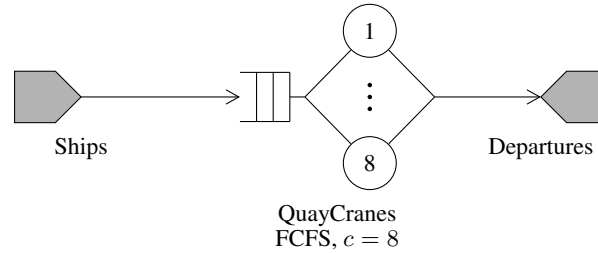


Figure 3.49: Topology of `renv_container_terminal`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Containers (open)
Scheduling	QuayCranes: FCFS ($c = 8$)
Arrivals	Ships – Containers: MAP
Service	QuayCranes – Containers: Exp(16)

Table 3.51: Features of `renv_container_terminal`.

The example is built and solved as follows.

```
% renv_container_terminal - Daily-cycle container terminal in a random environment
%
% Advanced example for ENV's STATE-VECTOR analyzer (options.method =
% 'statevec'), which carries the full per-stage state distribution across
% environment switches instead of collapsing it to marginal mean queue lengths.
%
% Case study (after the fyp26 SOQN blending port): the Rotterdam container
% terminal of Dhingra et al. Container handling demand varies over the 24 hours
% of a day; each hour is one environment "stage" with its own demand intensity
% and (random, exponential) duration. The environment visits the 24 hourly
% stages in a fixed daily cycle 1->2->...->24->1.
%
% Here the semi-open SOQN of the original study is rendered as the equivalent
% finite-token CLOSED network required by ENV: N straddle carriers cycle
% between a yard staging Delay and a multi-server quay-crane Queue. The hourly
% demand modulates the yard staging rate (the closed-network analog of the
% time-varying external arrival intensity): busier hours stage containers to the
% cranes faster, so the cranes congest during the daily peaks.
%
% The script compares, for the day-averaged metrics:
% (a) statevec - full state-vector blending (this feature),
% (b) meanfield - the default mean-field (marginal mean-queue-length) coupling,
% (c) exact - the stationary solution of the full joint
% (hour x network-state) random-environment CTMC.
% The state-vector blend tracks the exact joint solution far more closely than
% the mean-field collapse.

... (226 source lines omitted; full implementation: matlab/examples/advanced/)

[QNe, UNe, ~, TNe] = solver_ctmc_avg_from_pi(sne, blk, SSe, SSae, arve, depe);
QN = QN + probEnv(e)*QNe;
UN = UN + probEnv(e)*UNe;
TN = TN + probEnv(e)*TNe;
```

```
end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.4 `renv_fourstages_repairmen`

The four-stage version.

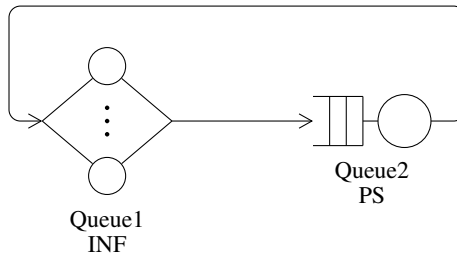


Figure 3.50: Topology of `renv_fourstages_repairmen`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 30$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(4) Queue2 – Class1: Exp(1)
Solvers	ENV

Table 3.52: Features of `renv_fourstages_repairmen`.

The example is built and solved as follows.

```
N = 30;
M = 3;
E = 4;
envModel = Environment('MyEnv');
envName = {'Stage1', 'Stage2', 'Stage3', 'Stage4'};
envType = {'UP', 'DOWN', 'FAST', 'SLOW'};

rate = ones(M,E); rate(M,1:E)=(1:E); rate(1,1:E)=(E:-1:1);

qn1 = renv_genqn(rate(:,1),N);
qn2 = renv_genqn(rate(:,2),N);
qn3 = renv_genqn(rate(:,3),N);
qn4 = renv_genqn(rate(:,4),N);
envSubModel = {qn1,qn2,qn3,qn4};
for e=1:E
    envModel.addStage(envName{e}, envType{e}, envSubModel{e});
end

envRates = [0,1,0,0; 0,0,1,1; 1,0,0,1; 1,1,0,0]/2;
for e=1:E
    for h=1:E
        if envRates(e,h)>0
            envModel.addTransition(envName{e}, envName{h}, APH.fitMeanAndSCV(1/envRates(e,h),0.5));
        end
    end
end
end

... (14 source lines omitted; full implementation: matlab/examples/advanced/)
```

```

options.timespan = [0,Inf];
options.verbose = 0;
envSolver = ENV(envModel, @(model) FLD(model, options), options);
[QN,UN,TN] = envSolver.getAvg();
AvgTable = envSolver.getAvgTable();
AvgTables = envSolver.getEnsembleAvgTables();

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.5 `renv_genqn`

A randomly generated network placed in an environment.

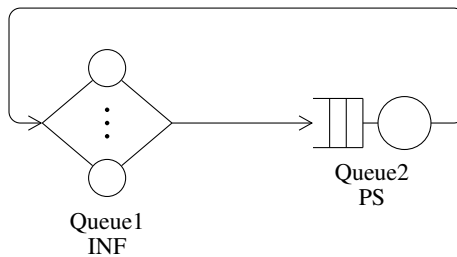


Figure 3.51: Topology of `renv_genqn`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 4$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(0.5)
Solvers	MVA

Table 3.53: Features of `renv_genqn`.

The example is built and solved as follows.

```

function qnet = renv_genqn(rate, N)
%% sn1
qnet = Network('qn1');

node{1} = Delay(qnet, 'Queue1');
node{2} = Queue(qnet, 'Queue2', SchedStrategy.PS);

jobclass{1} = ClosedClass(qnet, 'Class1', N, node{1}, 0);

node{1}.setService(jobclass{1}, Exp(rate(1)));
node{2}.setService(jobclass{1}, Exp(rate(2)));

K = 1;
P = cell(K,K);
P{1} = circul(length(node));

qnet.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.6 `renv_lqn_twostages`

A layered model in a two-stage environment, including its transient.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
function renv_lqn_twostages()
% RENV_LQN_TWOSTAGES LayeredNetwork operating in a two-stage random environment.
%
% Demonstrates SolverENV running over a LayeredNetwork (LQN) base model, using
% the uniform model/solver interface (no branching in ENV). The environment
% alternates between an UP stage (fast database) and a DOWN stage (slow
% database); the environment-averaged total throughput must lie between the two
% single-stage LQN solutions.

warning off;

%% Two structurally identical LQN stages, differing only in DB host demand.
upModel = buildLQN('LQN_UP', 0.8); % fast DB activity
downModel = buildLQN('LQN_DOWN', 3.0); % slow DB activity (degraded)

%% Random environment: UP <-> DOWN
env = Environment('DBReliability');
env.addStage('UP', 'operational', upModel);
env.addStage('DOWN', 'degraded', downModel);
env.addTransition('UP', 'DOWN', Exp(0.2)); % mean UP time = 5
env.addTransition('DOWN', 'UP', Exp(1.0)); % mean DOWN time = 1
env.init();

%% Solve with SolverENV over SolverLN(.,@SolverFLD)
% The transient window is set on SolverLN (not the layer factory): the layered
% fixed-point iteration solves each layer in steady state, and SolverLN applies

... (130 source lines omitted; full implementation: matlab/examples/advanced/)

function v = tailValue(cell_ik)
v = 0;
if isstruct(cell_ik) && isfield(cell_ik, 'metric') && ~isempty(cell_ik.metric)
    v = cell_ik.metric(end);
end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.7 renv_map_fallback

An MMPP(2) stage transition process, and the fallback taken when it cannot be represented exactly.

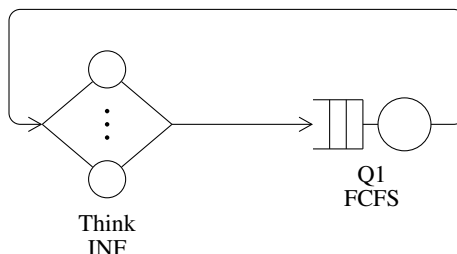


Figure 3.52: Topology of `renv_map_fallback`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
% Random-environment fallback for MAP/MMPP models
%
% A solver that cannot consume a non-renewal (MAP/MMPP/MMAP) process does not
```


Nodes	2: Queue, Delay
Classes	1: C1 (closed, $N = 5$)
Scheduling	Think: INF; Q1: FCFS
Service	Think – C1: Exp(1) Q1 – C1: MMPP2
Solvers	MVA, ENV, CTMC

Table 3.54: Features of `renv_map_fallback`.

```
% reject the model any longer: NetworkSolver intercepts it, replaces every
% modulating chain by a set of random-environment stages in which the process
% is exponential with its phase-conditional intensity (map2renv), and solves
% the stages with the same solver through SolverENV.
%
% The interception is method-aware: a method that handles the process natively
% (MVA 'rqna', MAM, CTMC, FLD, SSA, JMT, LDES) runs unchanged.

clear;

%% Closed network with an MMPP2 server: MVA and NC have no native MAP support
model = Network('mmppClosed');
delay = Delay(model, 'Think');
queue = Queue(model, 'Q1', SchedStrategy.FCFS);
jobclass = ClosedClass(model, 'C1', 5, delay);
delay.setService(jobclass, Exp(1.0));
queue.setService(jobclass, MMPP2(1.0, 10.0, 0.2, 0.3)); % slow phase 1, fast phase 10
model.link(Network.serialRouting(delay, queue));

%% The environment image built for the solver
[renv, info] = map2renv(model);
fprintf('Random-environment image: %d stages, phase orders %s, MMPP image: %d\n', ...
    info.nstages, mat2str(info.orders), info.isMMPP);

... (15 source lines omitted; full implementation: matlab/examples/advanced/)

options.config.map_env = 'off';
try
    MVA(model, options).getAvgTable();
catch ME
    fprintf('map_env='off': %s\n', ME.message);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.8 `renv_node_breakdown`

Server breakdown and repair as an environment over a queueing model.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	FLD, ENV
----------------	----------

Table 3.55: Features of `renv_node_breakdown`.

The example is built and solved as follows.

```
% Example demonstrating the node breakdown/repair API for random environments
%
% This example shows how to easily model a server that can break down and be
% repaired using the new Env convenience methods.
%
% Copyright (c) 2012-2026, Imperial College London
```

```
% All rights reserved.

%% Create base queueing network model
model = Network('ServerWithFailures');

% Define nodes
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');

% Define job class
jobclass = OpenClass(model, 'Jobs');

% Set service and arrival rates (UP state)
source.setArrival(jobclass, Exp(0.8)); % Arrival rate
queue.setService(jobclass, Exp(2.0)); % Service rate when UP
queue.setNumberOfServers(1);

% Set routing
P = model.initRoutingMatrix();

... (114 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf('- Availability: %.4f (%.2f%%)\n', ReliabilityTable.Value(4), ReliabilityTable.Value
(4)*100);
fprintf(' Fraction of time the system is operational\n');

% Expose final environment model for round-trip harness
envModel = env1;
clear model;
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.9 renv_rotterdam_blending

The Rotterdam blending case study.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
% renv_rotterdam_blending - Exact reproduction of the fyp26 SOQN blending result
%
% Reproduces the 24-environment exponential blending accuracy result of the
% fyp26 Rotterdam container-terminal study (Dhingra 2018), using the same
% level-dependent QBD blocks and the cyclic resolvent blending that the
% ENV state-vector analyzer uses internally.
%
% Per-environment model (PoissonSOQNSpec): a semi-open SOQN with N tokens, a
% Poisson arrival rate lambda_h (hour h of the day), and two flow-equivalent
% server (FES) subnetworks in tandem (upstream S1, downstream S2), each a
% load-dependent station whose throughput curve mu1(n)/mu2(n) is calibrated by
% exact MVA on the underlying closed subnetwork. The LD-QBD state is (n,k):
% level n = jobs upstream of S2 (S1 + external backlog), phase k = S2 occupancy.
%
% The 24 hourly environments are visited cyclically with exponential sojourns
% (mean = hour duration). Each visit applies the resolvent s*pi*(sI-Q)^{-1}
% (exit = time-average, memoryless), entry vectors are chained to an L1 fixed
% point, and the blend weights stages by their time fraction. Mean external
% wait W and external queue length Qex are compared to the Dhingra simulation
% (Table 13).
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear; warning off;
```

```

... (142 source lines omitted; full implementation: matlab/examples/advanced/)

    if k>=1, tput = tput + mu2(k+1)*p; end
end
end
Qex = QLex;
W = (QLex + QL1 + QL2) / tput;      % external wait + internal time (Little)
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.10 `renv_threestages_repairmen`

The three-stage version.

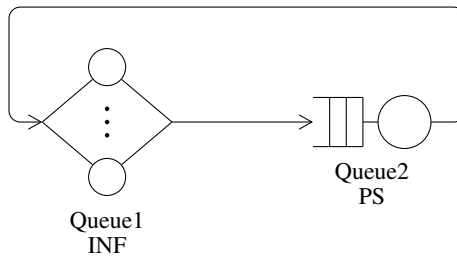


Figure 3.53: Topology of `renv_threestages_repairmen`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 2$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(3) Queue2 – Class1: Exp(1)
Solvers	ENV

Table 3.56: Features of `renv_threestages_repairmen`.

The example is built and solved as follows.

```

% This example shows the computation of the generator
N = 2;
M = 2;
E = 3;
envModel = Environment('MyEnv');
envName = {'Stage1', 'Stage2', 'Stage3'};
envType = {'UP', 'DOWN', 'FAST'};

rate = ones(M,E); rate(M,1:E)=(1:E); rate(1,1:E)=(E:-1:1);

qn1 = renv_genqn(rate(:,1),N);
qn2 = renv_genqn(rate(:,2),N);
qn3 = renv_genqn(rate(:,3),N);

envSubModel = {qn1,qn2,qn3};
for e=1:E
    envModel.addStage(envName{e}, envType{e}, envSubModel{e});
end

envRates = circul(3);
for e=1:E
    for h=1:E

```

```

        if envRates(e,h)>0
            envModel.addTransition(envName{e}, envName{h}, Erlang.fitMeanAndOrder(1/envRates(e,h),e+h));
        end
    end

    ... (16 source lines omitted; full implementation: matlab/examples/advanced/)

options.verbose = 0;
options.stiff = false;
envSolver = ENV(envModel, @(model) CTMC(model, 'exact', options), options);
[infGen,stageInfGen] = envSolver.getGenerator()

envSolver.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.11 renv_twestages_repairmen

A repairman model in a two-stage random environment.

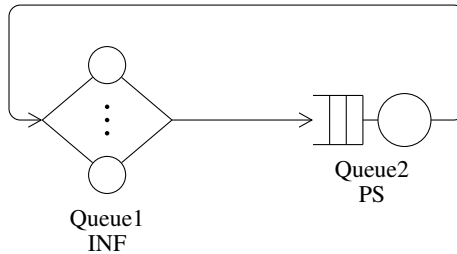


Figure 3.54: Topology of renv_twestages_repairmen: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(1)
Solvers	ENV

Table 3.57: Features of renv_twestages_repairmen.

The example is built and solved as follows.

```

N = 1;
M = 2;

E = 2;
envModel = Environment('MyEnv');
envName = {'Stage1', 'Stage2'};
envType = {'UP', 'DOWN'};

rate = zeros(M,E); rate(M,1:E)=(1:E); rate(1,1:E)=(E:-1:1);
envSubModel = {renv_genqn(rate(:,1),N), renv_genqn(rate(:,2),N)};
for e=1:E
    envModel.addStage(envName{e}, envType{e}, envSubModel{e});
end

envRates = [0,1; 0.5,0.5];
for e=1:E
    for h=1:E
        if envRates(e,h)>0

```

```

        envModel.addTransition(envName{e}, envName{h}, Exp(envRates(e,h)));
    end
end
end

%
fprintf(1,'The metasolver considers an environment with 2 stages and a queueing network with
2 stations.\n')
fprintf(1,'Every time the stage changes, the queueing network will modify the service rates
of the stations.\n')

... (18 source lines omitted; full implementation: matlab/examples/advanced/)

%scoptions = CTMC.defaultOptions;
%scoptions.timespan = [0,1e3];
%scoptions.verbose = false;
%envSolver = ENV(envModel,@(model) CTMC(model, 'exact', scoptions),options);
%[QnC,UNc,TNc] = envSolver.getAvg();
%AvgTableC = envSolver.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.12 Layered cache-queueing models

A CacheTask inside a layered model exposes ItemEntry points, and a call to one is a cache request whose hit or miss outcome selects the downstream branch. The replacement policy is simulated directly, so hit and miss are exact sample-path events rather than a hit-ratio parameter.

3.12.1 lcq_async_cache

An ASYNCHRONOUS (non-blocking) call into the cache task, as a prefetch or a warm-up makes: the client continues without waiting for the reply while the request still resolves through POST_CACHE hit/miss branching.

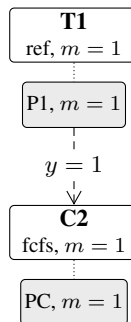


Figure 3.55: Call graph of `lcq_async_cache`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), PC (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 1$), C2 (fcfs, $m = 1$)
Entries	2: E1, I2
Activities	4, host demands A1: 0.0, AC2: 0.0, AC2h: 1.0, AC2m: 2.0
Calls	1 (0 synchronous, 1 asynchronous)

Table 3.58: Features of `lcq_async_cache`.

The example is built and solved as follows.

```

clear model solver AvgTable;

% Layered cache queueing model with asynchronous (non-blocking) cache access.
% The client makes an asynchronous call to the cache and continues without
% waiting for the reply, which is what a prefetch or a cache warm-up does; the
% cache still resolves the request through POST_CACHE hit/miss branching.
% Compare with lcq_singlehost, whose call is synchronous.

model = LayeredNetwork('AsyncCacheLQN');

%% client
P1 = Processor(model, 'P1', 1, SchedStrategy.PS);
T1 = Task(model, 'T1', 1, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);

%% cachetask
totalitems = 4;
cachecapacity = 2;
pAccess = DiscreteSampler((1/totalitems)*ones(1,totalitems));
PC = Processor(model, 'PC', 1, SchedStrategy.PS);
C2 = CacheTask(model, 'C2', totalitems, cachecapacity, ReplacementStrategy.LRU, 1).on(PC);
I2 = ItemEntry(model, 'I2', totalitems, pAccess).on(C2);

%% definition of activities
A1 = Activity(model, 'A1', Immediate()).on(T1).boundTo(E1).asynchCall(I2,1);
AC2 = Activity(model, 'AC2', Immediate()).on(C2).boundTo(I2);

... (7 source lines omitted; full implementation: matlab/examples/advanced/)

options = MVA.defaultOptions;
options.verbose = VerboseLevel.SILENT;
solver{1} = LN(model, @(model) MVA(model, options), lnoptions);
AvgTable = {};
AvgTable{1} = solver{1}.getAvgTable;
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.12.2 lcq_singlehost

A cache task and its item entries inside a layered model on one processor.

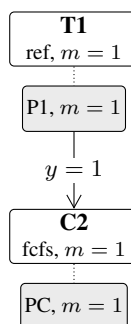


Figure 3.56: Call graph of lcq_singlehost: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

The example is built and solved as follows.

```

model = LayeredNetwork('cacheInLayeredNetwork');

%% client
P1 = Processor(model, 'P1', 1, SchedStrategy.PS);

```

Processors	2: P1 (ps, $m = 1$), PC (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 1$), C2 (fcfs, $m = 1$)
Entries	2: E1, I2
Activities	4, host demands A1: 0.0, AC2: 0.0, AC2h: 1.0, AC2m: 2.0
Calls	1 (1 synchronous, 0 asynchronous)

Table 3.59: Features of `lcq_singlehost`.

```

T1 = Task(model, 'T1', 1, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);

%% cachetask
totalitems = 4;
cachecapacity = 2;
pAccess = DiscreteSampler((1/totalitems)*ones(1,totalitems));
PC = Processor(model, 'PC', 1, SchedStrategy.PS);
C2 = CacheTask(model, 'C2', totalitems, cachecapacity, ReplacementStrategy.RR, 1).on(PC);
I2 = ItemEntry(model, 'I2', totalitems, pAccess).on(C2);

%% definition of activities
A1 = Activity(model, 'A1', Immediate()).on(T1).boundTo(E1).synchCall(I2,1);
AC2 = Activity(model, 'AC2', Immediate()).on(C2).boundTo(I2);
AC2h = Activity(model, 'AC2h', Exp(1.0)).on(C2).repliesTo(I2);
AC2m = Activity(model, 'AC2m', Exp(0.5)).on(C2).repliesTo(I2);

C2.addPrecedence(ActivityPrecedence.CacheAccess(AC2, {AC2h, AC2m}));

lnoptions = LN.defaultOptions;
%lnoptions.iter_max = 1;
lnoptions.verbose = VerboseLevel.STD;
options = MVA.defaultOptions;
options.verbose = VerboseLevel.SILENT;
solver{1} = LN(model, @(model) MVA(model, options), lnoptions);
AvgTable = {};
AvgTable{1} = solver{1}.getAvgTable;
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.12.3 `lcq_threehosts`

The three-processor version, where the cache and its clients are on separate hosts.

Processors	3: P1 (ps, $m = 1$), Pc (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	3: T1 (ref, $m = 1$), CT (fcfs, $m = 1$), T2 (fcfs, $m = 1$)
Entries	3: E1, IE, E2
Activities	5, host demands A1: 0.0, Ac: 0.0, Ac_hit: 1.0, Ac_miss: 2.0, A2: 0.2
Calls	2 (2 synchronous, 0 asynchronous)

Table 3.60: Features of `lcq_threehosts`.

The example is built and solved as follows.

```

% Example of Layered Network with a multi-level cache

model = LayeredNetwork('LQNwithCaching');
nusers = 1;
ntokens = 1;

%% client
P1 = Processor(model, 'P1', 1, SchedStrategy.PS);
T1 = Task(model, 'T1', nusers, SchedStrategy.REF).on(P1);
E1 = Entry(model, 'E1').on(T1);

```

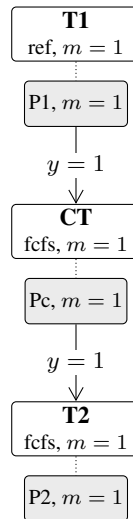


Figure 3.57: Call graph of `lcq_threehosts`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```
%% cachetask
totalitems = 4;
cachecapacity = [1 1];
pAccess = DiscreteSampler((1/totalitems)*ones(1,totalitems));
PC = Processor(model, 'Pc', 1, SchedStrategy.PS);
C2 = CacheTask(model, 'CT', totalitems, cachecapacity, ReplacementStrategy.RR, ntokens).on(PC);
I2 = ItemEntry(model, 'IE', totalitems, pAccess).on(C2);

P3 = Processor(model, 'P2', 1, SchedStrategy.PS);
T3 = Task(model, 'T2', 1, SchedStrategy.FCFS).on(P3);
E3 = Entry(model, 'E2').on(T3);
A3 = Activity(model, 'A2', Exp(5.0)).on(T3).boundTo(E3).repliesTo(E3);

%% definition of activities
A1 = Activity(model, 'A1', Immediate()).on(T1).boundTo(E1).synchCall(I2,1);
AC2 = Activity(model, 'Ac', Immediate()).on(C2).boundTo(I2);

... (12 source lines omitted; full implementation: matlab/examples/advanced/)

options2 = MVA.defaultOptions;
options2.verbose = 0;
solver{2} = LN(model, @(model) MVA(model, options2), lnoptions);
AvgTable{2} = solver{2}.getAvgTable;
AvgTable{2}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13 Pass and swap

Under the pass-and-swap discipline a job that finds an incompatible job in service swaps with it rather than waiting, subject to a compatibility relation between classes. The initial placement of a closed population is therefore part of the model: the state must be set in the order the discipline assumes, and a placement that ignores it changes the answer.

3.13.1 pas_closed_tandem_fig6

The closed tandem of the reference paper, figure 6.

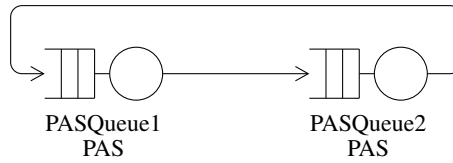


Figure 3.58: Topology of pas_closed_tandem_fig6: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	6: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$), Class3 (closed, $N = 1$), Class4 (closed, $N = 1$), Class5 (closed, $N = 1$), Class6 (closed, $N = 1$)
Scheduling	PASQueue1: PAS; PASQueue2: PAS
Service	PASQueue1 – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1); Class4: Exp(1); Class5: Exp(1); Class6: Exp(1) PASQueue2 – Class1: Exp(1.3); Class2: Exp(1.3); Class3: Exp(1.3); Class4: Exp(1.3); Class5: Exp(1.3); Class6: Exp(1.3)
Features	finite buffer 6 at PASQueue1; finite buffer 6 at PASQueue2; class switching on 2 links
Solvers	CTMC

Table 3.61: Features of pas_closed_tandem_fig6.

The example is built and solved as follows.

```
function pas_closed_tandem_fig6()
% PAS_CLOSED_TANDEM_FIG6 Closed tandem of two pass-and-swap (PAS) queues,
% reproducing Figures 5 and 6 of Comte & Dorsman, "Pass-and-Swap Queues"
% (2021, arXiv:2009.12299; Queueing Systems).
%
% Topology (Fig. 6): --> PASQueue1 --> PASQueue2 --> (closed tandem)
%
% There are I=6 customer classes with a SINGLE customer of each class. Both
% queues share the swapping graph of Figure 5 (undirected; classes as nodes):
%
%      6          edges: 1-3, 1-4, 2-4, 2-5, 3-6, 4-6, 5-6
%    / | \      (class 6 at the top, classes 1,2 at the bottom).
%   3 4 5
%  \ / \ /
%   1  2
%
% As in the paper's illustration, only the customer at the HEAD of each queue
% receives a positive service rate (single server): mu(c)=mu1 at queue 1 and
% nu(d)=mu2 at queue 2, independent of class so that the total rate depends
% only on the customer multiset (order-independence). Upon a service
% completion the departing customer is selected by the pass-and-swap scan over
% the swapping graph and is routed to the back of the OTHER queue as a
% customer of the same class.
%
% The model is solved exactly by CTMC and by the LDES simulator,
% and both are validated against an independent product-form reference that
%
% ... (129 source lines omitted; full implementation: matlab/examples/advanced/)
%
newlist = list;
for i = 1:numel(chain)-1
    newlist(chain(i+1)) = list(chain(i));
end
newlist(chain(1)) = [];
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.2 pas_compatibility_5class

Five classes and the compatibility matrix that says which may swap with which.

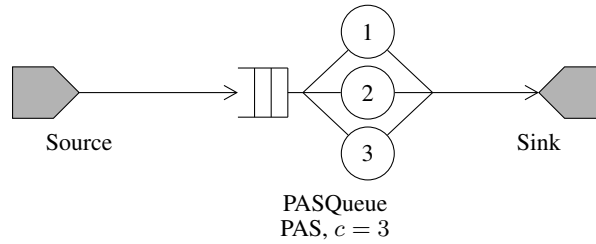


Figure 3.59: Topology of pas_compatibility_5class: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	5: Class1 (open), Class2 (open), Class3 (open), Class4 (open), Class5 (open)
Scheduling	PASQueue: PAS ($c = 3$)
Arrivals	Source – Class1: Exp(0.5); Class2: Exp(0.4); Class3: Exp(0.3); Class4: Exp(0.2); Class5: Exp(0.1)
Service	PASQueue – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1); Class4: Exp(2); Class5: Exp(1)
Features	finite buffer 3 at PASQueue
Solvers	CTMC

Table 3.62: Features of pas_compatibility_5class.

The example is built and solved as follows.

```
% Pass-and-swap (PAS) queue: an order-independent queue with a swapping graph.
% Reproduces the five-class, three-server compatibility example of
% Dorsman & Gardner (2024), "New directions in pass-and-swap queues",
% Queueing Systems 107:205-256 (Figs 1-2). A PAS station is parameterized by
% the total service-rate function  $\mu(c)$  of the ordered state vector  $c$  and by a
% swapping graph  $G$ ; both are properties of the Queue object. The stationary
% distribution is the order-independent product form and is invariant to  $G$ .

% Server compatibility (Fig 1): three unit-rate servers over five classes.
%  $\mu(c)$  = number of servers compatible with at least one class present in  $c$ .
comp = [1 0 0 1 0; % server 1: classes {1,4}
        0 1 0 1 0; % server 2: classes {2,4}
        0 0 1 0 1]; % server 3: classes {3,5}
muFun = @(c) sum( any(comp(:, c(c>0))), 2 );

model = Network('PAScompatibility');
source = Source(model, 'Source');
queue = Queue(model, 'PASQueue', SchedStrategy.PAS);
sink = Sink(model, 'Sink');

lambda = [0.5 0.4 0.3 0.2 0.1];
jobclass = cell(1,5);
for r = 1:5
    jobclass{r} = OpenClass(model, sprintf('Class%d', r));
    source.setArrival(jobclass{r}, Exp(lambda(r)));
end

... (13 source lines omitted; full implementation: matlab/examples/advanced/)

for r = 1:5
    P{jobclass{r}} = Network.serialRouting(source, queue, sink);
end
model.link(P);

AvgTable = CTMC(model, 'exact', 'cutoff', 3).getAvgTable
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.3 pas_cyclic_ctmc_vs_bruteforce

A cyclic pass-and-swap network solved by CTMC and by brute-force enumeration, as a cross-check. No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC, LDES
---------	------------

Table 3.63: Features of `pas_cyclic_ctmc_vs_bruteforce`.

The example is built and solved as follows.

```
function pas_cyclic_ctmc_vs_bruteforce()
% PAS_CYCLIC_CTMC_VS_BRUTEFORCE Validate LINE's CTMC solver on a closed
% cyclic network of two pass-and-swap (PAS) queues against the exact
% product-form (order-independent) brute-force enumeration.
%
% Topology: --> Queue1 --> Queue2 --> (cyclic, R closed classes)
%
% PAS queues are order-independent, so the network is product-form. The
% brute force enumerates the whole ordered state space to obtain the
% normalizing constant G, hence exact per-class throughputs and mean
% queue lengths, which are compared against CTMC.

if exist('Network','class') ~= 8
    lineStart;
end

% ---- shared model parameters -----
% Both stations must be valid order-independent (OI) queues: the TOTAL
% service rate mu_i(c) depends only on the customer multiset.
% station 1: M/M/k OI queue, class-independent -> mu1(c) = min(n,k1)*s1
% station 2: infinite-server, class-dependent -> mu2(c) = sum_j beta2(c_j)
K = [2 2]; % per-class closed populations
s1 = 1.0; k1 = 2; % station 1: 2-server OI (M/M/2)
beta2 = [1.5 1.0]; % station 2: per-class infinite-server rates
R = numel(K);

... (196 source lines omitted; full implementation: matlab/examples/advanced/)

else
    idx(r) = 0;
end
end
end
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.4 pas_cyclic_normconst_bruteforce

The same cross-check on the normalizing constant.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

function G = pas_cyclic_normconst_bruteforce()
% PAS_CYCLIC_NORMCONST_BRUTEFORCE Brute-force normalizing constant of a
% closed cyclic network of two pass-and-swap (PAS) queues.
%
% Topology:  --> Queue1 --> Queue2 -->    (cyclic, R closed classes)
%
% A pass-and-swap queue (Dorsman & Gardner 2024, Queueing Syst.
% 107:205-256) is an order-independent (OI) station: its total service
% rate is a function  $\mu_i(c)$  of the ordered class sequence  $c$  at the
% station, and the stationary law is product-form and INVARIANT to the
% swap graph. Hence the swap graph is irrelevant to the normalizing
% constant and is not needed here.
%
% For a closed network of OI/PAS stations the unnormalized stationary
% weight of a global ordered state  $(c_1, c_2)$  factorizes as
%
%  $w(c_1, c_2) = \Phi_1(c_1) * \Phi_2(c_2),$ 
%  $\Phi_i(c) = \prod_{j=1..n} e_{\{i, c_j\}} / \mu_i(c_1, \dots, c_j),$ 
%
% where  $e_{\{i, r\}}$  is the per-class visit ratio (relative arrival rate at
% station  $i$ ) and  $\mu_i(\text{prefix})$  is the total service sum rate of the ordered
% prefix. The normalizing constant is the exact sum over the whole state
% space,
%
%  $G = \sum_{\{c_1, c_2\}} \Phi_1(c_1) * \Phi_2(c_2)$ 
%  $= \sum_{\{m \leq K\}} S_1(m) * S_2(K-m),$ 
%
% ... (144 source lines omitted; full implementation: matlab/examples/advanced/)

Gbf = norm_const(N, e1, e2, muf1, muf2);
Gcf = sum((1/muf1).^ (0:N) .* (1/muf2).^ (N:-1:0));
assert(abs(Gbf - Gcf) <= 1e-12 * Gcf, ...
    'self-test failed: brute-force %.15g vs closed-form %.15g', Gbf, Gcf);
fprintf('Self-test (single-class Gordon-Newell): PASS (G=%.12g)\n', Gbf);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.5 pas_mmk

The pass-and-swap discipline on an M/M/k station.

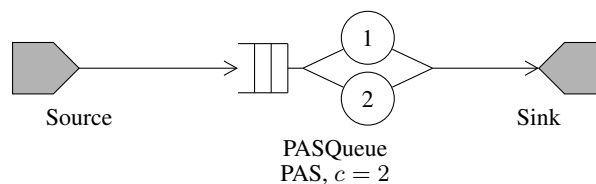


Figure 3.60: Topology of pas_mmk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	PASQueue: PAS ($c = 2$)
Arrivals	Source – Class1: Exp(0.7); Class2: Exp(0.5)
Service	PASQueue – Class1: Exp(1); Class2: Exp(1)
Features	finite buffer 4 at PASQueue
Solvers	CTMC

Table 3.64: Features of pas_mmk.

The example is built and solved as follows.

```

% Pass-and-swap (PAS) station as a multiserver order-independent (M/M/K) queue.
% The M/M/K queue satisfies the order-independence conditions (Dorsman &
% Gardner 2024, Sect. 2.2): with K unit-rate servers the total service rate in
% state c is  $\mu(c) = \min(n, K)$ , where n is the number of jobs. With an empty
% swapping graph the PAS station reduces to a plain order-independent queue.

K = 2; % number of servers
muFun = @(c) min(numel(c(c>0)), K);

model = Network('PASmmk');
source = Source(model, 'Source');
queue = Queue(model, 'PASQueue', SchedStrategy.PAS);
sink = Sink(model, 'Sink');

lambda = [0.7 0.5];
jobclass = cell(1,2);
for r = 1:2
    jobclass{r} = OpenClass(model, sprintf('Class%d', r));
    source.setArrival(jobclass{r}, Exp(lambda(r)));
end

queue.setService(muFun);
queue.setSwapGraph(zeros(2)); % empty graph: plain order-independent queue
queue.setNumberOfServers(K); % K servers (used for utilization reporting)
queue.setCap(4);

P = model.initRoutingMatrix;
for r = 1:2
    P{jobclass{r}} = Network.serialRouting(source, queue, sink);
end
model.link(P);

AvgTable = CTMC(model, 'exact', 'cutoff', 4).getAvgTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.6 pas_nc_sampling

The normalizing constant of a pass-and-swap network, by sampling.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

function pas_nc_sampling()
% PAS_NC_SAMPLING Importance-sampling normalizing-constant analysis of a closed
% pass-and-swap (P&S) tandem via SolverNC method='sampling' (Casale, Comte &
% Dorsman, 2026). The model is the Figure 6 closed tandem of Comte & Dorsman
% (2021, arXiv:2009.12299) also used in PAS_CLOSED_TANDEM_FIG6.
%
% A non-empty swap graph makes the ordered-state chain reducible; the recurrent
% communicating class carries the per-class product form  $\pi(c) = \Phi_1 \Phi_2 / G_C$ .
% SolverNC importance sampling estimates  $G_C$  and the mean queue lengths by auto-normalized
% importance sampling (API PFQN_PAS_IS), scaling to populations where the exact
% ordered-state CTMC is intractable. Here we validate it against the exact CTMC.

mu1 = 1.0; mu2 = 1.3;
edges = [1 3; 1 4; 2 4; 2 5; 3 6; 4 6; 5 6];
G = zeros(6);
for k = 1:size(edges,1)
    G(edges(k,1),edges(k,2)) = 1;
    G(edges(k,2),edges(k,1)) = 1;
end

model = Network('PASsampling');

```

```

q1 = Queue(model, 'PASQueue1', SchedStrategy.PAS);
q2 = Queue(model, 'PASQueue2', SchedStrategy.PAS);
jobclass = cell(1,6);
for r = 1:6
    jobclass{r} = ClosedClass(model, sprintf('Class%d', r), 1, q1);

    ... (22 source lines omitted; full implementation: matlab/examples/advanced/)

end
err = max(abs(qn - qc));
fprintf('\nNC-samp vs CTMC: max|dQ| = %.3e (importance-sampling noise)\n', err);
assert(err <= 5e-2, 'NC sampling does not match the exact CTMC within IS tolerance');
fprintf('PASS: SolverNC ''sampling'' matches the exact CTMC within importance-sampling noise
.\n');
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.7 pas_saturation

The saturated regime, where the swap rule sets the throughput.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC, LDES
---------	------------

Table 3.65: Features of pas_saturation.

The example is built and solved as follows.

```

% Pass-and-swap (PAS) saturation handling for large / unbounded buffers.
%
% An order-independent service rate mu(c) usually SATURATES: beyond some
% per-class count threshold tau_r, adding more class-r jobs no longer changes
% mu (e.g. a compatibility / rank function saturates once each present class
% has one job; an M/M/K rate saturates at K jobs). A large finite buffer can
% therefore approximate an unbounded queue cheaply.
%
% This uses the Comte (thesis Sect. 4.1) compatibility queue: I=2 classes,
% S=3 servers, S_1={1,2}, S_2={2,3}, rank function
%   mu(A) = sum_{s : exists present class in A compatible with s} cap_s,
% so mu({1})=mu({2})=cap_1+cap_2 = 3, mu({1,2})=cap_1+cap_2+cap_3 = 5, and mu
% is constant once both classes are present (cutoffs tau=[1,1]).
%
% It checks that (i) LDES matches CTMC on a finite buffer and
% (ii) an unset/infinite buffer raises a clean, actionable error (saturating
% rate -> suggest setCap; unbounded additive rate -> setCap required).

% compatibility comp(s,i) = 1 iff server s serves class i; capacities cap_s
comp = [1 0;      % server 1 -> class 1
        1 1;      % server 2 -> classes 1, 2 (shared)
        0 1];     % server 3 -> class 2
cap_s = [2; 1; 2];
lambda = [1.5 1.0];
I = size(comp, 2);
S = size(comp, 1);

... (66 source lines omitted; full implementation: matlab/examples/advanced/)

function q = qlen(solver, I)
QN = solver.getAvgQLen;          % nstations x nclasses mean queue length
sn = solver.model.getStruct;
sidx = sn.nodeToStation(find(strcmp(sn.nodenames, 'PASQueue'), 1));
q = QN(sidx, 1:I);

```

```
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.13.8 pas_selfloop

A self-looping class under pass-and-swap.

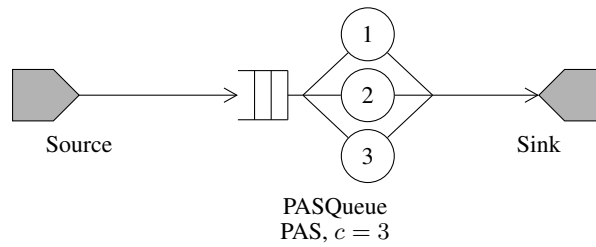


Figure 3.61: Topology of pas_selfloop: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	PASQueue: PAS ($c = 3$)
Arrivals	Source – Class1: Exp(0.6); Class2: Exp(0.4); Class3: Exp(0.3)
Service	PASQueue – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1)
Features	finite buffer 3 at PASQueue
Solvers	CTMC

Table 3.66: Features of pas_selfloop.

The example is built and solved as follows.

```
% Pass-and-swap (PAS) queue with a swapping graph that includes a self-loop.
% Per Dorsman & Gardner (2024), Sect. 2.3, self-loops in the swapping graph are
% permissible: a completing class- $i$  job may take the place of another class- $i$ 
% job further back in the queue. The order-independent product form (Theorem 2)
% still holds and remains invariant to the swapping graph.

% Three unit-rate servers, one per class (an M/M/1-per-class compatibility):
%  $\mu(c)$  = number of distinct classes present in  $c$ .
comp = eye(3);
muFun = @(c) sum( any(comp(:, c(c>0)), 2) );

model = Network('PASselfloop');
source = Source(model, 'Source');
queue = Queue(model, 'PASQueue', SchedStrategy.PAS);
sink = Sink(model, 'Sink');

lambda = [0.6 0.4 0.3];
jobclass = cell(1,3);
for r = 1:3
    jobclass{r} = OpenClass(model, sprintf('Class%d', r));
    source.setArrival(jobclass{r}, Exp(lambda(r)));
end

queue.setService(muFun);
% Swapping graph with a self-loop on class 1 and an edge (2,3)
G = zeros(3);

... (7 source lines omitted; full implementation: matlab/examples/advanced/)

for r = 1:3
```

```

    P{jobclass{r}} = Network.serialRouting(source, queue, sink);
end
model.link(P);

AvgTable = CTMC(model, 'exact', 'cutoff', 3).getAvgTable

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.14 Reward models

A reward model attaches a rate to each state, or a value to each transition, and reports the expected accumulated reward. Cost per waiting job, revenue per completion and penalty per loss are the usual three, and together they turn a performance model into an economic one. Rewards may be declared over aggregate states when the detailed state space is too large to enumerate.

3.14.1 rewardModel_aggregation

Rewards over aggregated states, when the detailed state is too large to enumerate.

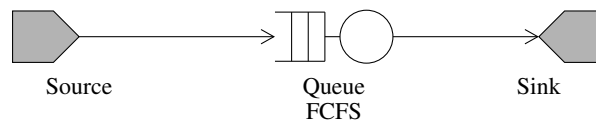


Figure 3.62: Topology of rewardModel_aggregation: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: HighPriority (open), LowPriority (open)
Scheduling	Queue: FCFS
Arrivals	Source – HighPriority: Exp(1); LowPriority: Exp(0.8)
Service	Queue – HighPriority: Exp(3); LowPriority: Exp(3)
Features	finite buffer 4 at Queue

Table 3.67: Features of rewardModel_aggregation.

The example is built and solved as follows.

```

%% Example: Reward Aggregation Operations
% This example demonstrates the aggregation capabilities of the RewardState API.
%
% The RewardStateView class supports aggregation operations:
% - total() : Sum all values
% - max()   : Maximum value
% - min()   : Minimum value
% - count() : Count non-zero entries
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear; clc;

%% Model Definition
% Create a single queue with multiple job classes

model = Network('RewardAggregationExample');

% Nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);

```



```

sink = Sink(model, 'Sink');

% Queue parameters
queue.setNumberOfServers(1);

... (62 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf(' %-20s: %10.6f\n', 'ClassCount', R(3));
fprintf(' %-20s: %10.6f\n', 'HP_Jobs', R(4));
fprintf(' %-20s: %10.6f\n', 'LP_Jobs', R(5));
fprintf(' %-20s: %10.6f\n', 'WeightedLoad', R(6));

fprintf('\nExample completed successfully.\n');

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.14.2 rewardModel_mm1k

State rewards on an M/M/1/K queue: cost per waiting job, revenue per completion, penalty per loss.

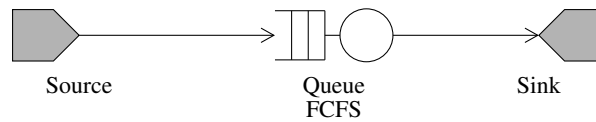


Figure 3.63: Topology of rewardModel_mm1k: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(2)
Service	Queue – Class1: Exp(3)
Features	finite buffer 3 at Queue
Solvers	CTMC

Table 3.68: Features of rewardModel_mm1k.

The example is built and solved as follows.

```

%% Example: Using setReward for reward-based CTMC analysis
% This example demonstrates the setReward feature for defining custom
% reward functions on a queueing network model and computing steady-state
% expected rewards using the CTMC solver.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear; clc;

%% Model Definition
% Create a simple M/M/1/K queue with finite buffer

model = Network('RewardExample');

% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

% Set finite buffer capacity
queue.setNumberOfServers(1);
queue.setCapacity(3); % Maximum jobs in the system

```

```

% Block 2: job classes
oclass = OpenClass(model, 'Class1');

... (111 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf('%15s: LINE = %.6f, Analytical = %.6f, Error = %.2e\n', ...
    'BlockingProb', R(3), B_analytical, abs(R(3) - B_analytical));
fprintf('%15s: LINE = %.6f, Analytical = %.6f, Error = %.2e\n', ...
    'QueueCost', R(4), Cost_analytical, abs(R(4) - Cost_analytical));

fprintf('\nExample completed successfully.\n');

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.14.3 rewardModel_multiclass

Per-class rewards on a multiclass PS station.

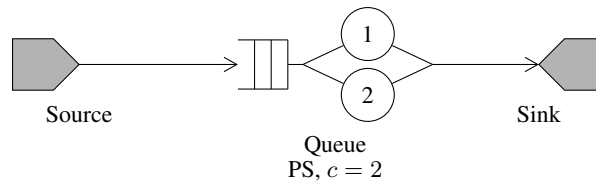


Figure 3.64: Topology of rewardModel_multiclass: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Interactive (open), Batch (open)
Scheduling	Queue: PS ($c = 2$)
Arrivals	Source – Interactive: Exp(2); Batch: Exp(1.5)
Service	Queue – Interactive: Exp(0.5); Batch: Exp(1)
Features	finite buffer 6 at Queue

Table 3.69: Features of rewardModel_multiclass.

The example is built and solved as follows.

```

%% Example: Multi-Class Reward Analysis
% This example demonstrates reward-based CTMC analysis in a multi-class queueing
% network, showing how to define per-class metrics and analyze class-specific behavior.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear; clc;

%% Model Definition
% Create an M/M/2 queue with two job classes (prioritized service)

model = Network('MultiClassRewardExample');

% Nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.PS); % Processor Sharing
sink = Sink(model, 'Sink');

% Server capacity and buffer limit
queue.setNumberOfServers(2);
queue.setCapacity(6); % Limit state space

```

```
% Job classes with different characteristics
class_interactive = OpenClass(model, 'Interactive');
class_batch = OpenClass(model, 'Batch');

... (134 source lines omitted; full implementation: matlab/examples/advanced/)

fprintf('\nEstimated Response Times (Little''s Law):\n');
fprintf('  Interactive: %.6f time units\n', est_resp_time_int);
fprintf('  Batch: %.6f time units\n', est_resp_time_batch);

fprintf('\n? Example completed successfully.\n');
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.14.4 rewardModel_templates

The reward templates that cover the common cases without writing a reward function.

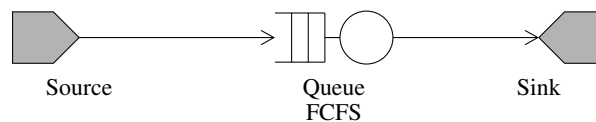


Figure 3.65: Topology of rewardModel_templates: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1.5)
Service	Queue – Class1: Exp(2)
Features	finite buffer 10 at Queue

Table 3.70: Features of rewardModel_templates.

The example is built and solved as follows.

```
%% Example: Using Reward Templates
% This example demonstrates the use of Reward template factory methods
% for quickly defining common metrics like queue length, utilization, and blocking.
%
% The new Reward templates provide a convenient way to define metrics
% without writing lambda expressions.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear; clc;

%% Model Definition
% Create an M/M/1 queue with finite buffer

model = Network('RewardTemplatesExample');

% Nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

% Queue parameters
queue.setNumberOfServers(1);
queue.setCapacity(10);
```

```

... (78 source lines omitted; full implementation: matlab/examples/advanced/)

'Utilization', R(3), U_analytical, abs(R(3) - U_analytical));
fprintf('%-25s: LINE = %10.6f, Analytical = %10.6f, Error = %.2e\n', ...
'BlockingProb', R(5), B_analytical, abs(R(5) - B_analytical));

fprintf('\n? Example completed successfully.\n');
fprintf(' All template-based rewards match analytical results!\n');

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.15 Agent models

The agent interface builds a network from the point of view of the entities that move through it rather than from the stations. The resulting models are ordinary networks – Jackson networks, closed multiclass networks, G-networks with their negative customers – and are solved by the same solvers.

3.15.1 ag_closed_network

The closed counterpart.

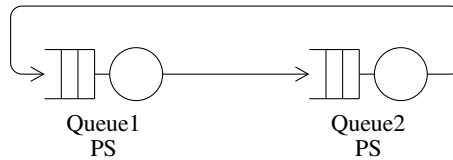


Figure 3.66: Topology of ag_closed_network: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(1)
Solvers	MVA, CTMC

Table 3.71: Features of ag_closed_network.

The example is built and solved as follows.

```

%% Closed Network with AG
%
% This example demonstrates SolverAG, the agent-based solver, on a closed queueing network
% with processor-sharing (PS) queues using the INAP algorithm.
%
% The solver decomposes the network
% into interacting stochastic processes and uses fixed-point iteration
% to compute equilibrium measures.
%
% Copyright (c) 2012-2025, Imperial College London
% All rights reserved.

clear; clc;

%% Parameters
N = 10;           % Number of jobs
mu1 = 2.0;        % Service rate at queue 1
mu2 = 1.0;        % Service rate at queue 2

```

```

%% Create model: Queue1 <-> Queue2 (closed loop)
model = Network('Closed-2Q');

queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);

cclass = ClosedClass(model, 'Class1', N, queue1);

... (12 source lines omitted; full implementation: matlab/examples/advanced/)

%% Solve with CTMC for exact results (if state space is small)
if N <= 20
    solverCTMC = CTMC(model, 'exact');
    avgTableCTMC = solverCTMC.getAvgTable()
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.15.2 ag_gnetwork

A G-network, with the negative customers that remove jobs on arrival.

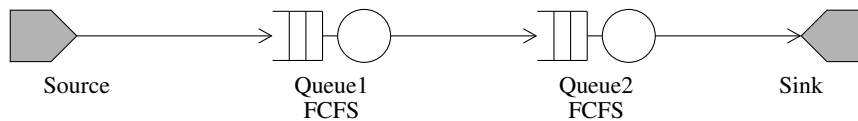


Figure 3.67: Topology of ag_gnetwork: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	2: Positive (open), Negative (signal)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Positive: Exp(1); Negative: Exp(0.3)
Service	Queue1 – Positive: Exp(2); Negative: Exp(2) Queue2 – Positive: Exp(3); Negative: Exp(3)

Table 3.72: Features of ag_gnetwork.

The example is built and solved as follows.

```

%% G-Network (Gelenbe Network) with Negative Customers
%
% This example demonstrates SolverAG, the agent-based solver, on a G-network with negative
% customers.
% Negative customers (signals) remove jobs from queues when they arrive,
% modeling job cancellations or service interrupts.
%
% Network topology:
% - Source generates positive customers (Positive) and negative signals (Negative)
% - Positive customers flow: Source -> Queue1 -> Queue2 -> Sink
% - Negative signals flow: Source -> Queue1, where they are CONSUMED,
%   each removing one job from Queue1
%
% Queue1 is where the signal acts, and the reported figures are Gelenbe's
% product form exactly:
% rho1 = lambda_pos / (mu1 + lambda_neg) = 1.0/2.3 = 0.43478, QLen1 = 0.76923
% Queue1 throughput = mu1*rho1 = 0.86957 -- the balance is killed, not served
% rho2 = 0.86957/mu2 = 0.28986, QLen2 = 0.40816
% Queue2 is affected only INDIRECTLY, through Queue1's reduced departure rate;
% no signal reaches it.
%
% Reference: Gelenbe, E. (1991). "Product-form queueing networks with
% negative and positive customers", Journal of Applied Probability

```

```
%
% Copyright (c) 2012-2025, Imperial College London
% All rights reserved.

... (60 source lines omitted; full implementation: matlab/examples/advanced/)

% "A Numerical Algorithm for the Decomposition of Cooperating Structured Markov
% Processes", MASCOTS 2012.
solverInf = AG(model, 'method', 'inapinf');
avgTableInf = solverInf.getAvgTable();
disp('INAP infinite-state (inapinf):');
disp(avgTableInf);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.15.3 ag_jackson_network

A Jackson network built agent by agent.

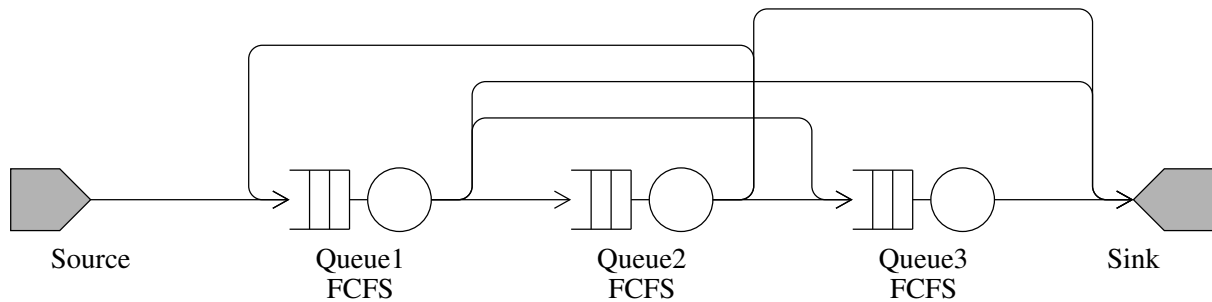


Figure 3.68: Topology of ag_jackson_network: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(3) Queue3 – Class1: Exp(2.5)
Solvers	MVA

Table 3.73: Features of ag_jackson_network.

The example is built and solved as follows.

```
%% Jackson Network with AG
%
% This example demonstrates SolverAG, the agent-based solver, on an open Jackson network with
% probabilistic routing. The solver models job transfers between
% queues as synchronization actions between interacting processes.
%
% Reference: Jackson, J.R. (1957). "Networks of waiting lines"
%
% Copyright (c) 2012-2025, Imperial College London
% All rights reserved.

clear; clc;

%% Parameters
lambda = 1.0;    % External arrival rate
mu = [2.0, 3.0, 2.5];    % Service rates
```

```
% Routing probabilities
p12 = 0.4; % Queue1 -> Queue2
p13 = 0.3; % Queue1 -> Queue3
p1s = 0.3; % Queue1 -> Sink
p21 = 0.2; % Queue2 -> Queue1
p23 = 0.3; % Queue2 -> Queue3
p2s = 0.5; % Queue2 -> Sink
p3s = 1.0; % Queue3 -> Sink

... (28 source lines omitted; full implementation: matlab/examples/advanced/)

solverINAP = AG(model, 'method', 'inap');
avgTableINAP = solverINAP.getAvgTable()

%% Solve with MVA for comparison
solverMVA = MVA(model);
avgTableMVA = solverMVA.getAvgTable()
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.15.4 ag_multiclass_closed

The multiclass closed version.

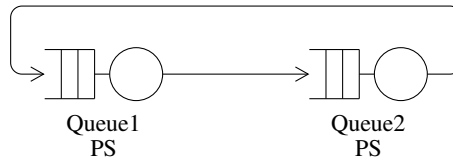


Figure 3.69: Topology of ag_multiclass_closed: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	2: Class1 (closed, $N = 5$), Class2 (closed, $N = 3$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(2); Class2: Exp(1.5) Queue2 – Class1: Exp(1); Class2: Exp(0.8)
Solvers	MVA

Table 3.74: Features of ag_multiclass_closed.

The example is built and solved as follows.

```
%% Multiclass Closed Network with AG
%
% This example demonstrates SolverAG, the agent-based solver, on a multiclass closed queueing
% network. The solver handles multiple job classes by creating
% separate processes for each (station, class) pair.
%
% Copyright (c) 2012-2025, Imperial College London
% All rights reserved.

clear; clc;

%% Parameters
N1 = 5; % Number of class 1 jobs
N2 = 3; % Number of class 2 jobs
mu1_q1 = 2.0; % Service rate for class 1 at queue 1
mu2_q1 = 1.5; % Service rate for class 2 at queue 1
mu1_q2 = 1.0; % Service rate for class 1 at queue 2
mu2_q2 = 0.8; % Service rate for class 2 at queue 2
```

```

%% Create model
model = Network('Multiclass-Closed');

queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);

class1 = ClosedClass(model, 'Class1', N1, queue1);

... (14 source lines omitted; full implementation: matlab/examples/advanced/)

solverINAP = AG(model, 'method', 'inap');
avgTableINAP = solverINAP.getAvgTable()

%% Solve with MVA for comparison
solverMVA = MVA(model);
avgTableMVA = solverMVA.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.15.5 ag_tandem_open

An open tandem written through the agent interface.

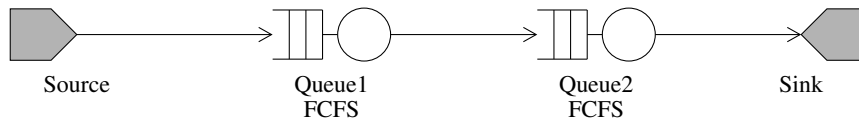


Figure 3.70: Topology of ag_tandem_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.5)
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(1.5)
Solvers	MVA

Table 3.75: Features of ag_tandem_open.

The example is built and solved as follows.

```

%% Open Tandem Queue with AG
%
% This example demonstrates SolverAG, the agent-based solver, on an open tandem queueing
% network
% (M/M/1 -> M/M/1) using the INAP algorithm.
%
% Reference: Marin and Rota-Bulo', "A Mean-Field Analysis of a Class of
%           Interactive Distributed Systems", MASCOTS 2009
%
% Copyright (c) 2012-2025, Imperial College London
% All rights reserved.

clear; clc;

%% Parameters
lambda = 0.5; % Arrival rate
mu1 = 1.0;    % Service rate at queue 1
mu2 = 1.5;    % Service rate at queue 2

%% Create model: Source -> Queue1 -> Queue2 -> Sink
model = Network('Tandem-MM1');

```



```

source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

... (8 source lines omitted; full implementation: matlab/examples/advanced/)

solverINAP = AG(model, 'method', 'inap');
avgTableINAP = solverINAP.getAvgTable()

%% Compare with MVA
solverMVA = MVA(model);
avgTableMVA = solverMVA.getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.15.6 ag_tandem_phasetype

An open tandem whose two queues share a mean service time and differ only in variability (Erlang-2 against a hyperexponential), solved by the RCAT methods, which carry the phase-type law exactly where an M/M/1 reading sees no difference.

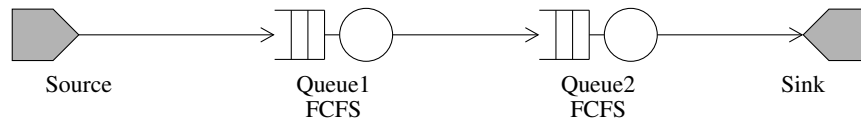


Figure 3.71: Topology of ag_tandem_phasetype: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.5)
Service	Queue1 – Class1: Erlang(2,2) Queue2 – Class1: HyperExp(0.99,0.01;1.14,0.07584)

Table 3.76: Features of ag_tandem_phasetype.

The example is built and solved as follows.

```

%% Open Tandem Queue with Phase-Type Service, solved by the agent-based solver
%
% This example demonstrates that the agent-based methods represent a phase-type
% service law exactly rather than collapsing it to its mean rate. Each
% component of the agent decomposition is a QBD whose level is the queue length
% and whose phase is the pair (arrival phase, service phase), so the first
% station -- an isolated M/PH/1, since it sees the Poisson source directly --
% comes out at the Pollaczek-Khinchine mean whatever the reversed-rate
% iteration does. Both stations here have the same mean service time and differ
% only in their variability, which is exactly what an M/M/1 reading cannot see.
%
% References: Casale and Harrison, "AutoCAT: Automated Product-Form Solution of
%             Stochastic Models", Stochastic Models 27, 2013
%             Neuts, "Matrix-Geometric Solutions in Stochastic Models", 1981
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

clear; clc;

%% Parameters
lambda = 0.5;    % Arrival rate

```

```

meanS = 1.0;    % Mean service time at both queues
scv1 = 0.5;    % Erlang-2 service at queue 1 (less variable than exponential)
scv2 = 4.0;    % HyperExp service at queue 2 (more variable)

... (24 source lines omitted; full implementation: matlab/examples/advanced/)

avgTableINAP = AG(model, 'method', 'inap').getAvgTable()

% 'inapinf' additionally drops the maxStates truncation, solving each open
% component on its infinite state space through Neuts' rate matrix R. That
% matters most at Queue2, whose service law has the heavier tail.
avgTableINAPINF = AG(model, 'method', 'inapinf').getAvgTable()

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.16 Large-scale models

When the state space cannot be enumerated, the exact solvers are out and the question becomes how much accuracy the approximations retain. The script here is a large closed PS network solved by several approximate methods.

3.16.1 largescale_tbi

A large closed PS network, and the accuracy the approximate solvers retain on it.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

%% Large-scale transient fluid analysis with trajectory-based iteration (TBI)
% This example builds a vehicle-sharing-style closed network with M
% queueing stations and one Erlang transit delay per ordered station pair,
% giving O(M^2) stations and, with 16 Erlang phases per delay, a fluid ODE
% with about 1500 state variables. At this size the monolithic stiff fluid
% solution (method 'closing') takes several minutes, dominated by the
% Jacobian factorizations, while trajectory-based iteration (TBI) solves
% each station cell separately against frozen inbound trajectories and
% completes in seconds. See:
%
% M. Sheldon, D. Tuncer, G. Casale, "TBI: Transient Hierarchical
% Modeling of Large-Scale Vehicle Sharing Systems", IEEE Transactions
% on Intelligent Transportation Systems.
%
% Each cell holds one queueing station together with its outbound transit
% delays, mirroring the spatial submodels of the paper.

clear model;
M = 10;    % queueing stations; the model has M + M*(M-1) stations in total
N = 250;   % closed population (vehicles)
kph = 16;  % Erlang phases per transit delay
Tend = 20; % transient horizon

model = Network('tbi_largescale');
rng(1, 'twister');
Q = cell(1,M);

... (54 source lines omitted; full implementation: matlab/examples/advanced/)

AvgTable(1:M,:) % queue-length summary at the queueing stations

% For comparison, the undecomposed solution of the same model:
solver = SolverFLD(model, 'method', 'closing', 'timespan', [0 Tend], 'stiff', true);

```

```
% takes several minutes on the same machine (about 80 seconds already at
% M=8, and beyond 10 minutes at M=12).
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.17 Custom solvers

A solver is a class with an analyzer and a feature set. The feature set is a claim about what the solver can represent, and the dispatcher uses it to decide whether a model may be given to it; the analyzer computes the metrics. These two scripts show the smallest pair that the dispatcher will accept.

3.17.1 solver_custom

A user-defined solver registered with the dispatcher.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
function [QN,UN,RN,TN,CN,XN] = solver_custom(sn, options)
% [Q,U,R,T,C,X] = SOLVER_CUSTOM(QN, OPTIONS)

% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

M = sn.nstations; % number of stations
K = sn.nclasses; % number of classes
N = sn.njobs'; % job populations
rates = sn.rates; % arrival and service rates
V = sn.visits; % visits

QN = zeros(M,K);
UN = zeros(M,K);
RN = zeros(M,K);
TN = zeros(M,K);
CN = zeros(1,K);
XN = zeros(1,K);

line_printf('The solution algorithm needs to be implemented in %s.m: returning with no result
.',mfilename);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.17.2 solver_custom_analyzer

Its analyzer, the function that actually computes the metrics.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
function [QN,UN,RN,TN,CN,XN, runtime] = solver_custom_analyzer(sn, options)
% [QN,UN,RN,TN,CN,XN,RUNTIME] = SOLVER_CUSTOM_ANALYZER(QN, OPTIONS)

% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.
```

```
Tstart = tic;

line_printf('Any activity prior or after launching the solution algorithm needs to be
            implemented in %s.m\n.',mfilename);
[QN,UN,RN,TN,CN,XN] = solver_custom(sn, options);

QN(isnan(QN))==0;
CN(isnan(CN))==0;
RN(isnan(RN))==0;
UN(isnan(UN))==0;
XN(isnan(XN))==0;
TN(isnan(TN))==0;

runtime = toc(Tstart);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Chapter 4

Model gallery

The gallery is a catalogue of small named models, one function each, returning a model rather than solving it. It exists twice over: as documentation, because a name in Kendall notation is the fastest way to find the model one means, and as the cross-codebase parity suite, because the same 71 functions exist in all four codebases and their metrics are compared numerically.

The models live in `matlab/examples/gallery/`, one file per model. Calling a gallery function builds and returns the model without solving it, so the caller chooses the solver. Running a Python gallery file as a script prints a short confirmation of the model it built.

```
function model = gallery_mml
model = Network('M/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

4.1 Reading the names

The single-station entries are named in Kendall notation, with the distributions abbreviated: `m` exponential, `d` deterministic, `erl` Erlang, `hyp` hyperexponential, `aph` acyclic phase-type, `cox` Coxian, `me` matrix-exponential, `map` Markovian arrival process, `mmap` marked MAP, `gam` Gamma, `par` Pareto, `u` uniform. So `gallery_erlml` is an Erl/M/1 queue and `gallery_mhypk` an M/HyperExp/k. A trailing `_ps` replaces FCFS by processor sharing, `k` in place of `1` makes the station multiserver, and the suffixes `_tandem`, `_linear`, `_reentrant`, `_feedback`, `_multiclass` and `_prio` describe the topology or the workload around the station.

The remaining entries are named after what they model rather than after a queueing notation: `gallery_cqn` and `gallery_repairmen` for the closed archetypes, `gallery_fj_open` and `gallery_fj_closed` for fork-join, `gallery_fcr` for a finite capacity region, `gallery_cache_lru` and `gallery_cache_routing` for caches, `gallery_renv_breakdown` for a random environment, `gallery_lqn_basic`, `gallery_lqn_workflows` and `gallery_lqn_random` for layered models, `gallery_multitier` and `gallery_multitier_storage` for multitier applications, `gallery_lukumar_reentrant` for the Lu-Kumar re-entrant line, and `gallery_qn_random` for a generated network.

4.2 The catalogue

The complete list, in the order in which the models are registered:

gallery_aphm1	gallery_merl1_linear
gallery_cache_lru	gallery_merl1_reentrant
gallery_cache_routing	gallery_merl1_tandem
gallery_coxm1	gallery_merlk
gallery_cqn	gallery_mhyp1
gallery_cqn_multiclass	gallery_mhyp1_linear
gallery_detm1	gallery_mhyp1_reentrant
gallery_dml	gallery_mhyp1_tandem
gallery_erldk	gallery_mhypk
gallery_erlerl1	gallery_mm1
gallery_erlerl1_reentrant	gallery_mm1_feedback
gallery_erlm1	gallery_mm1_linear
gallery_erlm1_ps	gallery_mm1_multiclass
gallery_erlm1_reentrant	gallery_mm1_prio
gallery_erlm1ps	gallery_mm1_ps
gallery_fcr	gallery_mm1_ps_feedback
gallery_fj_closed	gallery_mm1_ps_multiclass
gallery_fj_open	gallery_mm1_ps_reentrant
gallery_fj_quorum	gallery_mm1_reentrant
gallery_gamm1	gallery_mm1_tandem
gallery_hyperl1_feedback	gallery_mm1_tandem_multiclass
gallery_hyperl1_reentrant	gallery_mm1k
gallery_hyperlk	gallery_mmap1
gallery_hyphyp1_linear	gallery_mmap1_multiclass
gallery_hyphyp1_reentrant	gallery_mmapk
gallery_hyphyp1_tandem	gallery_mmk
gallery_hypm1	gallery_mpar1
gallery_hypm1_reentrant	gallery_multitier
gallery_lqn_basic	gallery_multitier_storage
gallery_lqn_random	gallery_parm1
gallery_lqn_workflows	gallery_qn_random
gallery_lukumar_reentrant	gallery_renv_breakdown
gallery_mapm1	gallery_repairmen
gallery_mapmk	gallery_replayerm1
gallery_mdk	gallery_um1
gallery_merl1	

4.2.1 gallery_aphm1

The APH/M/1 queue: acyclic phase-type interarrival times, exponential service, and a single server.

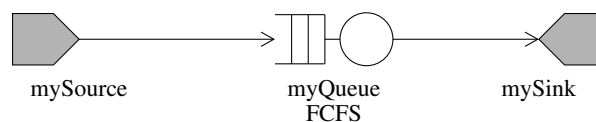


Figure 4.1: Topology of gallery_aphm1: an open network over 1 queueing station.

The example is built and solved as follows.

```

function model = gallery_aphm1
model = Network('APH/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
  
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: APH
Service	myQueue – myClass: Exp(2)

Table 4.2: Features of gallery_aphm1.

```

oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, APH.fitCentral(1,0.99,1.999));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.2 gallery_cache_lru

A cache node under least-recently-used replacement, read by a closed class.

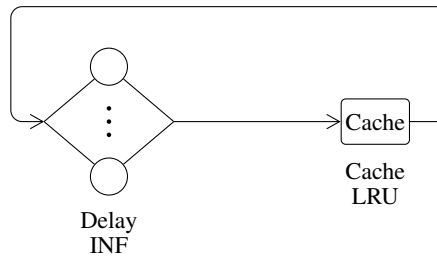


Figure 4.2: Topology of gallery_cache_lru: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: LRU replacement, 5 items, capacity 2; class switching on 2 links

Table 4.3: Features of gallery_cache_lru.

The example is built and solved as follows.

```

function model = gallery_cache_lru()
% GALLERY_CACHE_LRU Closed cache model with LRU replacement (n=5 items, m=2 slots)
model = Network('Cache-LRU');
%% Block 1: nodes
n = 5; % number of items
m = 2; % cache capacity
delay = Delay(model, 'Delay');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.LRU);
%% Block 2: classes
jobClass = ClosedClass(model, 'JobClass', 1, delay, 0);
hitClass = ClosedClass(model, 'HitClass', 0, delay, 0);
missClass = ClosedClass(model, 'MissClass', 0, delay, 0);
delay.setService(jobClass, Exp(1));
pAccess = DiscreteSampler((1/n)*ones(1,n)); % uniform item references

```

```

cacheNode.setRead(jobClass, pAccess);
cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);
%% Block 3: topology
P = model.initRoutingMatrix();
P{jobClass, jobClass}(delay, cacheNode) = 1.0;
P{hitClass, jobClass}(cacheNode, delay) = 1.0;
P{missClass, jobClass}(cacheNode, delay) = 1.0;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.3 gallery_cache_routing

A cache whose hit and miss outcomes route the job to different downstream stations.

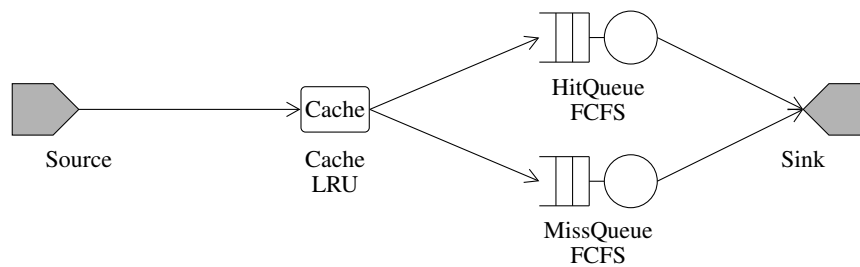


Figure 4.3: Topology of gallery_cache_routing: an open network over 2 queueing stations with a cache node.

Nodes	5: Source, Queue × 2, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Scheduling	HitQueue: FCFS; MissQueue: FCFS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Service	HitQueue – InitClass: Disabled; HitClass: Exp(2); MissClass: Disabled MissQueue – InitClass: Disabled; HitClass: Disabled; MissClass: Exp(1)
Features	cache: LRU replacement, 4 items, capacity 2; drop rule {'InitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at HitQueue; drop rule {'InitClass': 'waitingQueue', 'HitClass': 'waitingQueue'} at MissQueue; class switching on 4 links

Table 4.4: Features of gallery_cache_routing.

The example is built and solved as follows.

```

function model = gallery_cache_routing()
% GALLERY_CACHE_ROUTING Open cache with hit/miss routed to distinct queues
model = Network('Cache-Routing');
%% Block 1: nodes
n = 4; % number of items
m = 2; % cache capacity
source = Source(model, 'Source');
cacheNode = Cache(model, 'Cache', n, m, ReplacementStrategy.LRU);
hitQueue = Queue(model, 'HitQueue', SchedStrategy.FCFS);
missQueue = Queue(model, 'MissQueue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
jobClass = OpenClass(model, 'InitClass', 0);
hitClass = OpenClass(model, 'HitClass', 0);
missClass = OpenClass(model, 'MissClass', 0);
source.setArrival(jobClass, Exp(1));
hitQueue.setService(hitClass, Exp(2.0));

```



```

missQueue.setService(missClass, Exp(1.0));
pAccess = DiscreteSampler((1/n)*ones(1,n)); % uniform item references
cacheNode.setRead(jobClass, pAccess);
cacheNode.setHitClass(jobClass, hitClass);
cacheNode.setMissClass(jobClass, missClass);
%% Block 3: topology
P = model.initRoutingMatrix();
P{jobClass, jobClass}(source, cacheNode) = 1.0;
P{hitClass, hitClass}(cacheNode, hitQueue) = 1.0;
P{hitClass, hitClass}(hitQueue, sink) = 1.0;
P{missClass, missClass}(cacheNode, missQueue) = 1.0;
P{missClass, missClass}(missQueue, sink) = 1.0;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.4 gallery_coxm1

The Coxian/M/1 queue: Coxian interarrival times, exponential service, and a single server.

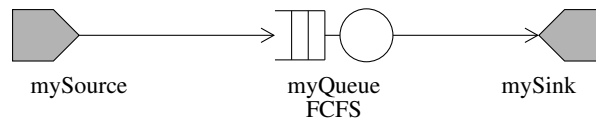


Figure 4.4: Topology of gallery_coxm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Coxian
Service	myQueue – myClass: Exp(2)

Table 4.5: Features of gallery_coxm1.

The example is built and solved as follows.

```

function model = gallery_coxm1
model = Network('Cox/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Coxian.fitCentral(1,0.99,1.999));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.5 gallery_cqn

The closed queueing network archetype: a fixed job population circulating between a Delay station and a queue.

The example is built and solved as follows.

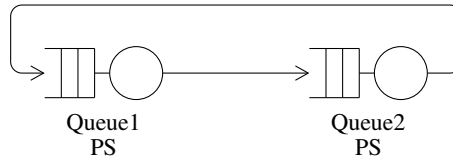


Figure 4.5: Topology of gallery_cqn: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 3$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(0.8271) Queue2 – Class1: Exp(0.4381)

Table 4.6: Features of gallery_cqn.

```
function model = gallery_cqn(M, useDelay, seed)
model = Network('Single-class CQN');
%% Block 1: nodes

if nargin==0
    M=2;
end
if nargin<2
    useDelay = false;
end
if nargin<3
    seed = 23000;
end
rng(seed);
for i=1:M
    station{i} = Queue(model, ['Queue',num2str(i)], SchedStrategy.PS);
end
if useDelay
    station{M+1} = Delay(model, 'Delay1');
end
%% Block 2: classes
jobclass = ClosedClass(model, 'Class1', round(rand*M+2), station{1}, 0);

for i=1:M
    station{i}.setService(jobclass, Exp.fitMean(rand()+i)); % (Queue 1,Class1)
end

... (4 source lines omitted; full implementation: matlab/examples/gallery/)

%% Block 3: topology
P = model.initRoutingMatrix(); % initialize routing matrix
P{jobclass,jobclass} = Network.serialRouting(station);
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.6 gallery_cqn_multiclass

The closed archetype with several classes, each with its own population.
The example is built and solved as follows.

```
function model = gallery_cqn_multiclass(m,r,wantdelay)
if nargin == 0
    m = 1;
    r = 2;
```

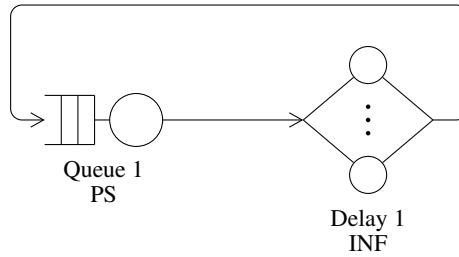


Figure 4.6: Topology of gallery_cqn_multiclass: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 5$), Class2 (closed, $N = 5$)
Scheduling	Queue 1: PS; Delay 1: INF
Service	Queue 1 – Class1: Exp(0.03226); Class2: Exp(0.07143) Delay 1 – Class1: Exp(0.04762); Class2: Exp(0.08333)

Table 4.7: Features of gallery_cqn_multiclass.

```

        wantdelay = true;
    end
    model = Network('Multi-class CQN');
    %% Block 1: nodes
    for i=1:m
        node{i} = Queue(model, ['Queue ', num2str(i)], SchedStrategy.PS);
    end
    if wantdelay
        node{end+1} = Delay(model, 'Delay 1');
    end
    %% Block 2: classes
    for s=1:r
        jobclass{s} = ClosedClass(model, ['Class', num2str(s)], 5, node{1}, 0);
    end

    for s=1:r
        for i=1:m
            node{i}.setService(jobclass{s}, Exp.fitMean(round(50*rand))); % (Queue 1, Class1)
        end
        if wantdelay
            node{end}.setService(jobclass{s}, Exp.fitMean(round(100*rand))); % (Delay 1, Class1)
        end

        ... (3 source lines omitted; full implementation: matlab/examples/gallery/)
    end

    P = model.initRoutingMatrix(); % initialize routing matrix
    for s=1:r
        P{jobclass{s}, jobclass{s}} = Network.serialRouting(node);
    end
    model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.7 gallery_detm1

The D/M/1 queue: deterministic interarrival times, exponential service, and a single server. The example is built and solved as follows.

```

function model = gallery_detm1()
% GALLERY_DETM1 Create a D/M/1 queue model (deterministic arrivals, exponential service)
%
% This model demonstrates the effect of deterministic (constant) inter-arrival

```

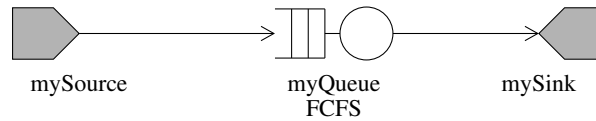


Figure 4.7: Topology of gallery_detm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Det(1)
Service	myQueue – myClass: Exp(2)

Table 4.8: Features of gallery_detm1.

```

% times compared to exponential arrivals. The deterministic arrival process
% has zero variance, which typically reduces queue length and response time
% variability compared to the M/M/1 model.
%
% Model Configuration:
% - Arrival process: Deterministic with rate 1.0 (inter-arrival time = 1.0)
% - Service process: Exponential with rate 2.0 (mean service time = 0.5)
% - Traffic intensity: rho = lambda/mu = 1.0/2.0 = 0.5 (stable)
% - Scheduling: First-Come-First-Served (FCFS)
%
% Returns:
%   model - Network object containing the configured D/M/1 queue
%
% Examples:
%   model = gallery_detm1();
%   solver = CTMC(model,'exact');
%   solver.solve();
%   avg_table = solver.getAvgTable()
%
% Notes:
% - D/M/1 queues typically have lower response time variance than M/M/1
% - The deterministic arrival process eliminates arrival variability
%
% ... (16 source lines omitted; full implementation: matlab/examples/gallery/)

queue.setService(oclass, Exp(2));

%% Block 3: topology
model.link(Network.serialRouting(source, queue, sink));

end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.8 gallery_dm1

The D/M/1 queue: deterministic interarrival times, exponential service, and a single server.

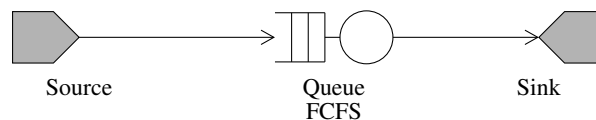


Figure 4.8: Topology of gallery_dm1: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Det(1)
Service	Queue – Class1: Exp(2)

Table 4.9: Features of gallery_dm1.

```
function model = gallery_dm1
model = Network('D/M/1');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Det(1));
queue.setService(oclass1, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.9 gallery_erldk

The Erl/D/ k queue: Erlang interarrival times, deterministic service, and k parallel servers.

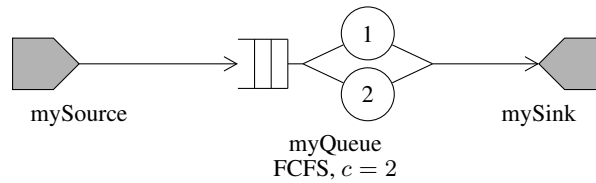


Figure 4.9: Topology of gallery_erldk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Det(1)

Table 4.10: Features of gallery_erldk.

The example is built and solved as follows.

```
function model = gallery_erldk(k)
if nargin<1
    k = 2;
end
model = Network('Erl/D/k');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Erlang.fitMeanAndOrder(1,5));
```

```

queue.setService(oclass, Det(2/k));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.10 gallery_erlerl1

The Erl/Erl/1 queue: Erlang interarrival times, Erlang service, and a single server.

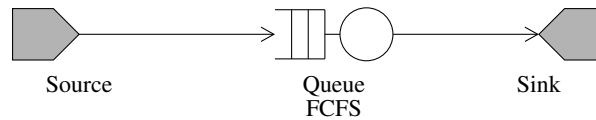


Figure 4.10: Topology of gallery_erlerl1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Erlang(5,5); Class2: Disabled
Service	Queue – Class1: Erlang(10,5); Class2: Exp(3)
Features	class switching on 2 links

Table 4.11: Features of gallery_erlerl1.

The example is built and solved as follows.

```

function model = gallery_erlerl1(n)
model = Network('Erl/Erl/1');
if nargin<1
    n=5;
end
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Erlang.fitMeanAndOrder(1,n));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Erlang.fitMeanAndOrder(0.5,n));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.11 gallery_erlerl1_reentrant

An Erl/Erl/1 queue with re-entrant visits.

The example is built and solved as follows.

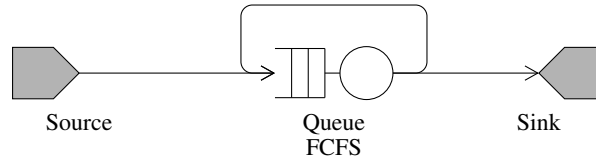


Figure 4.11: Topology of `gallery_erlerl1_reentrant`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Erlang(5,5); Class2: Disabled
Service	Queue – Class1: Erlang(50,5); Class2: Exp(10)
Features	class switching on 1 link

Table 4.12: Features of `gallery_erlerl1_reentrant`.

```
function model = gallery_erlerl1_reentrant
model = Network('Erl/Erl/1-Reentrant');
n=5;
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Erlang.fitMeanAndOrder(1,n));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Erlang.fitMeanAndOrder(0.1,n));
queue.setService(oclass2, Exp(10));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=0.5;
P{oclass2,oclass2}(queue,sink)=0.5;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.12 `gallery_erlm1`

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server.

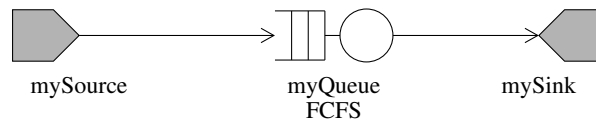


Figure 4.12: Topology of `gallery_erlm1`: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model = gallery_erlm1
model = Network('Er/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Exp(2)

Table 4.13: Features of gallery_erlm1.

```

sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Erlang.fitMeanAndOrder(1,5));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.13 gallery_erlm1_ps

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS.

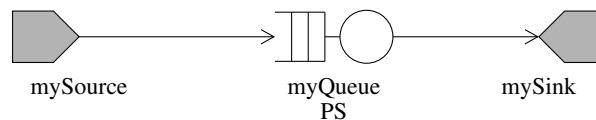


Figure 4.13: Topology of gallery_erlm1_ps: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Exp(2)

Table 4.14: Features of gallery_erlm1_ps.

The example is built and solved as follows.

```

function model = gallery_erlm1_ps
model = Network('Er/M/1-PS');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.PS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Erlang.fitMeanAndOrder(1,5));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.14 gallery_erlm1_reentrant

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

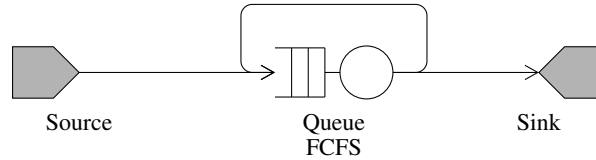


Figure 4.14: Topology of gallery_erlm1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Erlang(5,5); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.15: Features of gallery_erlm1_reentrant.

The example is built and solved as follows.

```
function model = gallery_erlm1_reentrant
model = Network('Er/M/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Erlang.fitMeanAndOrder(1,5));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Exp(2));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.15 gallery_erlm1ps

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS.

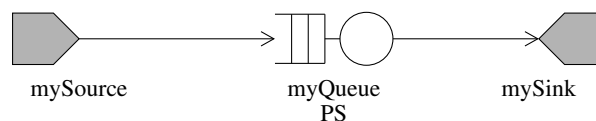


Figure 4.15: Topology of gallery_erlm1ps: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Exp(2)

Table 4.16: Features of gallery_erlm1ps.

The example is built and solved as follows.

```
function model = gallery_erlm1ps()
% GALLERY_ERLM1PS Create an Erlang/M/1-PS queue model (Erlang arrivals, exponential service,
% processor sharing)
%
% This model uses Erlang-distributed inter-arrival times with processor sharing
% scheduling. Processor sharing means all jobs in the system share the server
% capacity equally, which is different from FCFS scheduling.
%
% Model Configuration:
% - Arrival process: Erlang with mean 1.0 and 5 phases (lower variance)
% - Service process: Exponential with rate 2.0 (mean service time = 0.5)
% - Traffic intensity: rho = lambda/mu = 1.0/2.0 = 0.5 (stable)
% - Scheduling: Processor Sharing (PS)
%
% Returns:
% model - Network object containing the configured Erlang/M/1-PS queue
%
% Examples:
% model = gallery_erlm1ps();
% solver = MAM(model);
% solver.solve();
% avg_table = solver.getAvgTable()
%
% Notes:
% - Processor sharing provides fairness among jobs
% - All jobs receive service simultaneously with equal share
% - Response time independent of service order

... (15 source lines omitted; full implementation: matlab/examples/gallery/)

queue.setService(oclass, Exp(2));

%% Block 3: topology
model.link(Network.serialRouting(source, queue, sink));

end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.16 gallery_fcr

A finite capacity region, which caps the number of jobs admitted to a set of stations irrespective of their individual buffers.

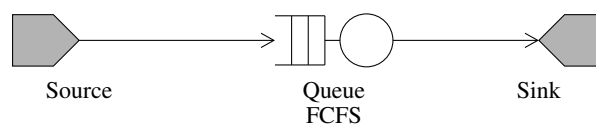


Figure 4.16: Topology of gallery_fcr: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.8)
Service	Queue – Class1: Exp(1)

Table 4.17: Features of `gallery_fcr`.

```
function model = gallery_fcr(K)
% GALLERY_FCR Finite capacity region with dropping around a single queue
% (equivalent to an M/M/1/K queue where K is the region capacity)
if nargin < 1
    K = 3;
end
model = Network('FCR-Dropping');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass = OpenClass(model, 'Class1', 0);
source.setArrival(oclass, Exp(0.8));
queue.setService(oclass, Exp(1.0));
%% Block 3: topology
P = model.initRoutingMatrix();
P.set(oclass, oclass, source, queue, 1.0);
P.set(oclass, oclass, queue, sink, 1.0);
model.link(P);
%% Block 4: finite capacity region with dropping
fcr = model.addRegion({queue});
fcr.setGlobalMaxJobs(K);
fcr.setDropRule(oclass, true);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.17 `gallery_fj_closed`

The closed fork-join counterpart, with a fixed population circulating through the block.

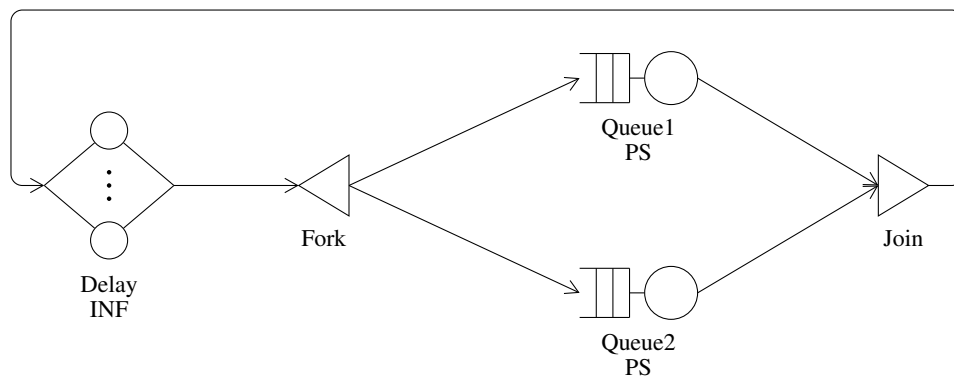


Figure 4.17: Topology of `gallery_fj_closed`: a closed network over 3 queueing stations with a fork-join block.

The example is built and solved as follows.

```
function model = gallery_fj_closed()
% GALLERY_FJ_CLOSED Closed fork-join network (single class, two parallel tasks)
```

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1)

Table 4.18: Features of gallery_fj_closed.

```

model = Network('Fork-Join-Closed');
%% Block 1: nodes
delay = Delay(model, 'Delay');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
%% Block 2: classes
oclass = ClosedClass(model, 'class1', 5, delay);
delay.setService(oclass, Exp(1.0));
queue1.setService(oclass, Exp(1.0));
queue2.setService(oclass, Exp(1.0));
%% Block 3: topology
P = model.initRoutingMatrix();
P{oclass,oclass}(delay,fork) = 1.0;
P{oclass,oclass}(fork,queue1) = 1.0;
P{oclass,oclass}(fork,queue2) = 1.0;
P{oclass,oclass}(queue1,join) = 1.0;
P{oclass,oclass}(queue2,join) = 1.0;
P{oclass,oclass}(join,delay) = 1.0;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.18 gallery_fj_open

An open fork-join block: a job forks into parallel tasks and its siblings are joined before departure.

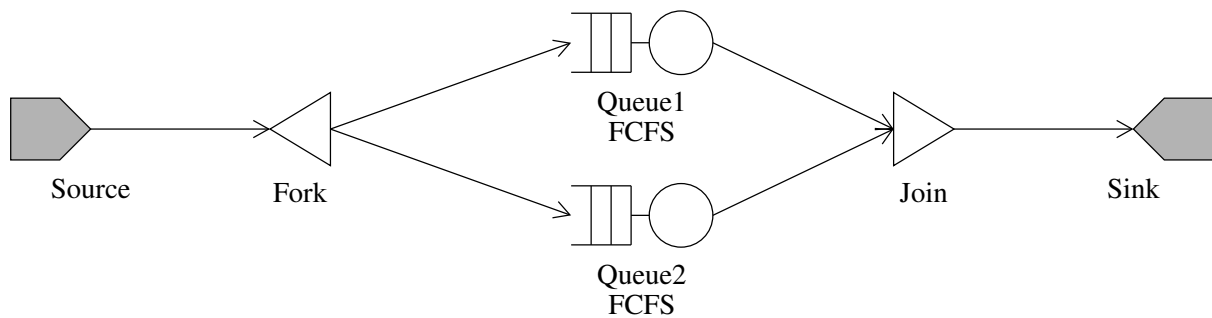


Figure 4.18: Topology of gallery_fj_open: an open network over 2 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	1: class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – class1: Exp(0.05)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(2)

Table 4.19: Features of gallery_fj_open.

The example is built and solved as follows.

```

function model = gallery_fj_open()
% GALLERY_FJ_OPEN Open fork-join network (single class, two parallel tasks)
model = Network('Fork-Join-Open');
%% Block 1: nodes
source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass = OpenClass(model, 'class1');
source.setArrival(oclass, Exp(0.05));
queue1.setService(oclass, Exp(1.0));
queue2.setService(oclass, Exp(2.0));
%% Block 3: topology
P = model.initRoutingMatrix();
P{oclass,oclass}(source,fork) = 1.0;
P{oclass,oclass}(fork,queue1) = 1.0;
P{oclass,oclass}(fork,queue2) = 1.0;
P{oclass,oclass}(queue1,join) = 1.0;
P{oclass,oclass}(queue2,join) = 1.0;
P{oclass,oclass}(join,sink) = 1.0;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.19 gallery_fj_quorum

A fork-join block whose join fires on a quorum of the siblings, the later ones being discarded when they arrive.

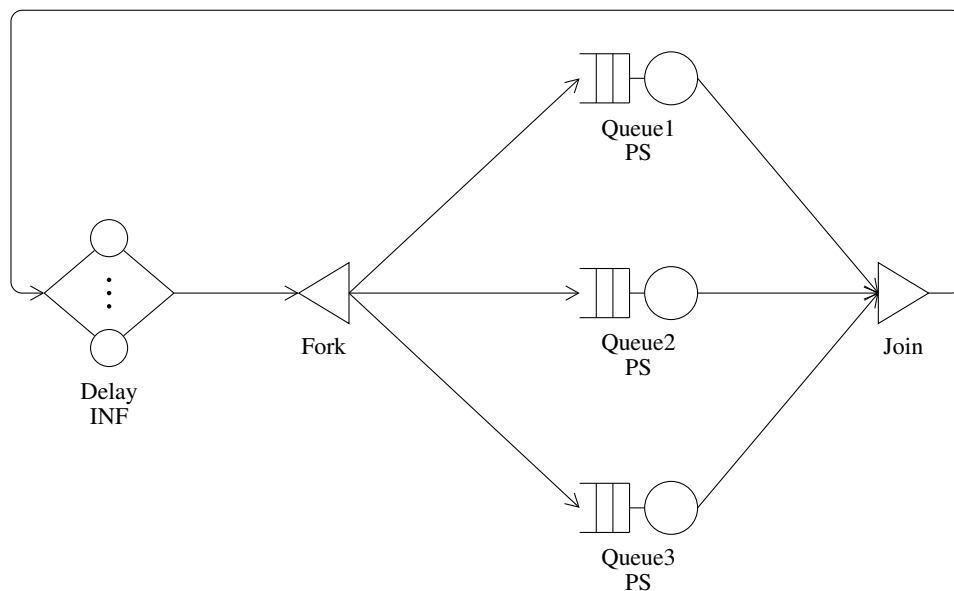


Figure 4.19: Topology of `gallery_fj_quorum`: a closed network over 4 queueing stations with a fork-join block.

The example is built and solved as follows.

```

function model = gallery_fj_quorum()
% GALLERY_FJ_QUORUM Closed fork-join network with a 2-of-3 quorum join
%

```

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	1: class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(2) Queue2 – class1: Exp(2) Queue3 – class1: Exp(2)

Table 4.20: Features of gallery_fj_quorum.

```
% The join fires on the SECOND of the three sibling tasks; the third is
% discarded when it arrives. Solved exactly by SolverLDES and SolverJMT;
% SolverMVA and SolverNC charge the second order statistic of the branch
% completion times at the join (fj_ordstat_exp).
model = Network('Fork-Join-Quorum');
%% Block 1: nodes
delay = Delay(model, 'Delay');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
queue3 = Queue(model, 'Queue3', SchedStrategy.PS);
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
%% Block 2: classes
oclass = ClosedClass(model, 'class1', 5, delay);
delay.setService(oclass, Exp(1.0));
queue1.setService(oclass, Exp(2.0));
queue2.setService(oclass, Exp(2.0));
queue3.setService(oclass, Exp(2.0));
join.setStrategy(oclass, JoinStrategy.PARTIAL);
join.setRequired(oclass, 2);
%% Block 3: topology
P = model.initRoutingMatrix();
P{oclass,oclass}(delay,fork) = 1.0;

... (3 source lines omitted; full implementation: matlab/examples/gallery/)

P{oclass,oclass}(queue1,join) = 1.0;
P{oclass,oclass}(queue2,join) = 1.0;
P{oclass,oclass}(queue3,join) = 1.0;
P{oclass,oclass}(join,delay) = 1.0;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.20 gallery_gamm1

The Gamma/M/1 queue: Gamma interarrival times, exponential service, and a single server.

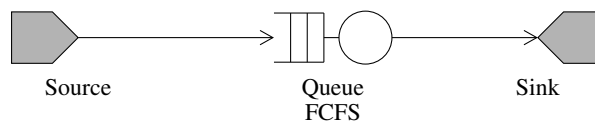


Figure 4.20: Topology of gallery_gamm1: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model = gallery_gamm1
model = Network('Gam/M/1');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Gamma(5,0.2)
Service	Queue – Class1: Exp(2)

Table 4.21: Features of gallery_gamml.

```

oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Gamma.fitMeanAndSCV(1,1/5));
queue.setService(oclass1, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.21 gallery_hyperl1_feedback

A HyperExp/Erl/1 queue with probabilistic feedback.

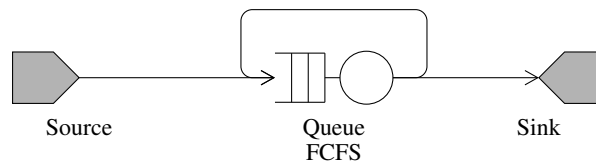


Figure 4.21: Topology of gallery_hyperl1_feedback: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: HyperExp(0.99,0.01;2.294,0.01759)
Service	Queue – Class1: Erlang(100,5)

Table 4.22: Features of gallery_hyperl1_feedback.

The example is built and solved as follows.

```

function model = gallery_hyperl1_feedback
model = Network('Hyper/Erl/1-Feedback');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, HyperExp.fitMeanAndSCV(1,64));
queue.setService(oclass1, Erlang.fitMeanAndOrder(0.05,5));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass1}(queue,queue)=0.9;
P{oclass1,oclass1}(queue,sink)=0.1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.22 gallery_hyperl1_reentrant

The HyperExp/Erl/1 queue: hyperexponential interarrival times, Erlang service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

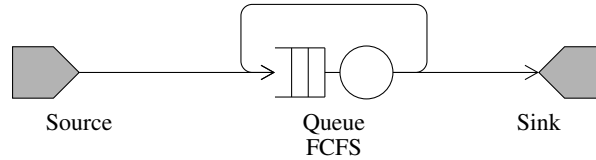


Figure 4.22: Topology of gallery_hyperl1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: HyperExp(0.99,0.01;2.294,0.01759); Class2: Disabled
Service	Queue – Class1: Erlang(10,5); Class2: Exp(3)
Features	class switching on 1 link

Table 4.23: Features of gallery_hyperl1_reentrant.

The example is built and solved as follows.

```
function model = gallery_hyperl1_reentrant
model = Network('Hyper/Erl/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, HyperExp.fitMeanAndSCV(1,64));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Erlang.fitMeanAndOrder(0.5,5));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.23 gallery_hyperlk

The HyperExp/Erl/ k queue: hyperexponential interarrival times, Erlang service, and k parallel servers.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: HyperExp(0.1127,0.8873;0.4057,3.194)
Service	myQueue – myClass: Erlang(4,4)

Table 4.24: Features of gallery_hyperlk.

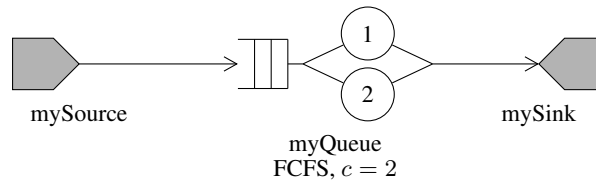


Figure 4.23: Topology of gallery_hyper1k: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model = gallery_hyper1k(k)
if nargin<1
    k = 2;
end
model = Network('Hyper/Erl/k');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, HyperExp.fitMeanAndSCVBalanced(1/1.8,4));
queue.setService(oclass, Erlang.fitMeanAndSCV(1,0.25));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.24 gallery_hyphyp1_linear

The HyperExp/HyperExp/1 queue: hyperexponential interarrival times, hyperexponential service, and a single server. In this variant, the model is a linear chain of such stations.

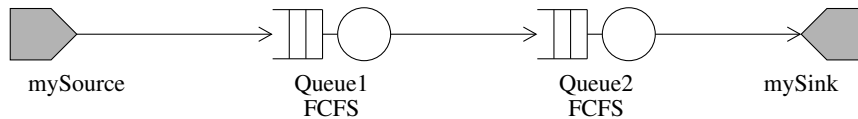


Figure 4.24: Topology of gallery_hyphyp1_linear: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: HyperExp(0.99,0.01;1.077,0.1244)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: Hyper-Exp(0.99,0.01;1.235,0.1015)

Table 4.25: Features of gallery_hyphyp1_linear.

The example is built and solved as follows.

```
function model = gallery_hyphyp1_linear(n,Umax)
if ~exist('Umax','var')
    Umax = 0.9;
end
if ~exist('n','var')
```

```

n = 2;
end
model = Network('Hyp/Hyp/1-Linear');
%% Block 1: nodes
line{1} = Source(model, 'mySource');
for i=1:n
    line{1+i} = Queue(model, ['Queue', num2str(i)], SchedStrategy.FCFS);
end
line{end+1} = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
line{1}.setArrival(oclass, HyperExp.fitMeanAndSCV(1,2));
means = linspace(0.1,Umax,n/2);
if mod(n,2)==0
    means=[means,means(end:-1:1)];
else
    means=[means,Umax,means(end:-1:1)];
end
for i=1:n
    line{1+i}.setService(oclass, HyperExp.fitMeanAndSCV(means(i),1+i));
end
%% Block 3: topology
model.link(Network.serialRouting(line));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.25 gallery_hyphyp1_reentrant

The HyperExp/HyperExp/1 queue: hyperexponential interarrival times, hyperexponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

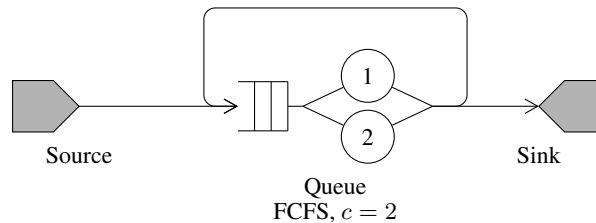


Figure 4.25: Topology of gallery_hyphyp1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS ($c = 2$)
Arrivals	Source – Class1: HyperExp(0.99,0.01;2.294,0.01759); Class2: Disabled
Service	Queue – Class1: HyperExp(0.99,0.01;2.281,0.1517); Class2: Exp(3)
Features	class switching on 1 link

Table 4.26: Features of gallery_hyphyp1_reentrant.

The example is built and solved as follows.

```

function model = gallery_hyphyp1_reentrant
model = Network('Hyper/Hyper/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
queue.setNumberOfServers(2);
sink = Sink(model, 'Sink');

```

```

%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, HyperExp.fitMeanAndSCV(1,64));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, HyperExp.fitMeanAndSCV(0.5,4));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.26 gallery_hyphyp1_tandem

Two HyperExp/HyperExp/1 queues in series.

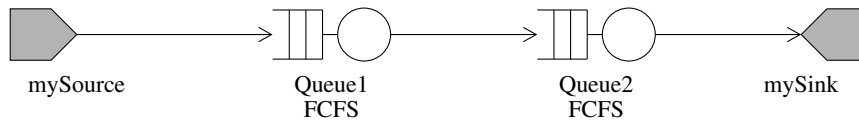


Figure 4.26: Topology of gallery_hyphyp1_tandem: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: HyperExp(0.99,0.01;1.077,0.1244)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: HyperExp(0.99,0.01;1.235,0.1015)

Table 4.27: Features of gallery_hyphyp1_tandem.

The example is built and solved as follows.

```

function model=gallery_hyphyp1_tandem
model = gallery_hyphyp1_linear(2);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.27 gallery_hypm1

The HyperExp/M/1 queue: hyperexponential interarrival times, exponential service, and a single server.

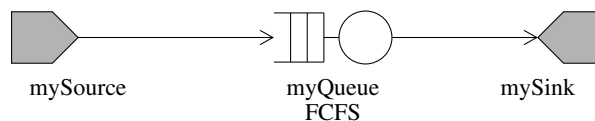


Figure 4.27: Topology of gallery_hypm1: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: HyperExp(0.99,0.01;2.294,0.01759)
Service	myQueue – myClass: Exp(2)

Table 4.28: Features of gallery_hypm1.

```
function model = gallery_hypm1
model = Network('H2/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, HyperExp.fitMeanAndSCV(1,64));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.28 gallery_hypm1_reentrant

The HyperExp/M/1 queue: hyperexponential interarrival times, exponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

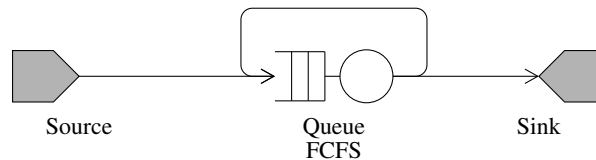


Figure 4.28: Topology of gallery_hypm1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: HyperExp(0.99,0.01;1.14,0.07584); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.29: Features of gallery_hypm1_reentrant.

The example is built and solved as follows.

```
function model = gallery_hypm1_reentrant
model = Network('Hyper/M/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, HyperExp.fitMeanAndSCV(1,4));
```

```

source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Exp(2));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.29 gallery_lqn_basic

The smallest layered queueing network: one reference task calling one server task.

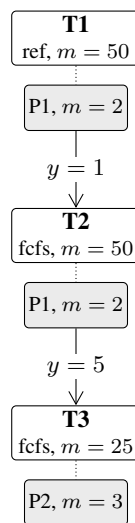


Figure 4.29: Call graph of `gallery_lqn_basic`: 3 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 2$), P2 (ps, $m = 3$)
Tasks	3: T1 (ref, $m = 50$), T2 (fcfs, $m = 50$), T3 (fcfs, $m = 25$)
Entries	3: E1, E2, E3
Activities	3, host demands AS1: 0.1, AS2: 0.05, AS3: 0.02
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 2.0

Table 4.30: Features of `gallery_lqn_basic`.

The example is built and solved as follows.

```

function model = gallery_lqn_basic()
% GALLERY_LQN_BASIC Basic 3-task layered queueing network (client/server chain)
model = LayeredNetwork('LQN-Basic');
%% Processors
P1 = Processor(model, 'P1', 2, SchedStrategy.PS);
P2 = Processor(model, 'P2', 3, SchedStrategy.PS);
%% Tasks
T1 = Task(model, 'T1', 50, SchedStrategy.REF).on(P1).setThinkTime(Exp(1/2));
T2 = Task(model, 'T2', 50, SchedStrategy.FCFS).on(P1).setThinkTime(Exp(1/3));
T3 = Task(model, 'T3', 25, SchedStrategy.FCFS).on(P2).setThinkTime(Exp(1/4));

```

```

%% Entries
E1 = Entry(model, 'E1').on(T1);
E2 = Entry(model, 'E2').on(T2);
E3 = Entry(model, 'E3').on(T3);
%% Activities
A1 = Activity(model, 'AS1', Exp(10)).on(T1).boundTo(E1).synchCall(E2, 1);
A2 = Activity(model, 'AS2', Exp(20)).on(T2).boundTo(E2).synchCall(E3, 5).repliesTo(E2);
A3 = Activity(model, 'AS3', Exp(50)).on(T3).boundTo(E3).repliesTo(E3);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.30 gallery_lqn_random

A randomly generated layered model, used to exercise the layered solvers beyond hand-written topologies.

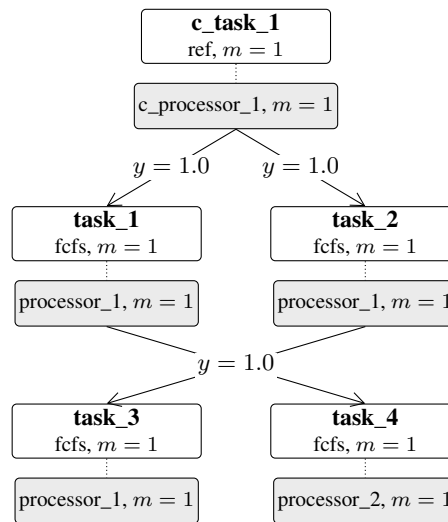


Figure 4.30: Call graph of `gallery_lqn_random`: 5 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: c_processor_1 (inf, m = 1), processor_1 (ps, m = 1), processor_2 (ps, m = 1)
Tasks	5: c_task_1 (ref, m = 1), task_1 (fcfs, m = 1), task_2 (fcfs, m = 1), task_3 (fcfs, m = 1), task_4 (fcfs, m = 1)
Entries	5: c_entry_1, entry_1, entry_2, entry_3, entry_4
Activities	5, host demands c_activity_1: 1.0, activity_1: 1.0, activity_2: 1.0, activity_3: 1.0, activity_4: 1.0
Calls	4 (4 synchronous, 0 asynchronous)

Table 4.31: Features of `gallery_lqn_random`.

The example is built and solved as follows.

```

function model = gallery_lqn_random(seed)
% GALLERY_LQN_RANDOM Randomly generated layered queueing network (reproducible).
% Uses LayeredNetworkGenerator with a fixed default seed so repeated calls
% yield the same model. 1 client, 2 levels, 4 tasks, 2 processors.
if nargin < 1
    seed = 23000;
end
rng(seed);
gen = LayeredNetworkGenerator();

```

```
model = gen.generate(1, 2, 4, 2);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.31 gallery_lqn_workflows

Workflow patterns (sequence, AND-fork and join, loop) as an activity graph in a layered model.

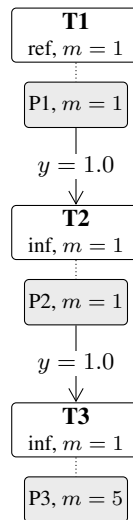


Figure 4.31: Call graph of gallery_lqn_workflows: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (inf, $m = 1$), P2 (inf, $m = 1$), P3 (ps, $m = 5$)
Tasks	3: T1 (ref, $m = 1$), T2 (inf, $m = 1$), T3 (inf, $m = 1$)
Entries	3: Entry, E2, E3
Activities	14, host demands A1: 1.0, A2: 2.0, A3: 3.0, B1: 0.1, B2: 0.2, B3: 0.3, B4: 0.4, B5: 0.5, ...
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 0.0

Table 4.32: Features of gallery_lqn_workflows.

The example is built and solved as follows.

```
function model = gallery_lqn_workflows()
% GALLERY_LQN_WORKFLOWS Layered network with loop, and-fork/join and or-fork/join
model = LayeredNetwork('LQN-Workflows');
%% Processors and tasks
P1 = Processor(model, 'P1', Inf, SchedStrategy.INF);
T1 = Task(model, 'T1', 1, SchedStrategy.REF).on(P1);
T1.setThinkTime(Immediate());
E1 = Entry(model, 'Entry').on(T1);

P2 = Processor(model, 'P2', Inf, SchedStrategy.INF);
T2 = Task(model, 'T2', Inf, SchedStrategy.INF).on(P2).setThinkTime(Immediate());
E2 = Entry(model, 'E2').on(T2);

P3 = Processor(model, 'P3', 5, SchedStrategy.PS);
T3 = Task(model, 'T3', Inf, SchedStrategy.INF).on(P3);
T3.setThinkTime(Exp.fitMean(10));
E3 = Entry(model, 'E3').on(T3);
```

```

%% Activities
A1 = Activity(model, 'A1', Exp.fitMean(1)).on(T1).boundTo(E1);
A2 = Activity(model, 'A2', Exp.fitMean(2)).on(T1);
A3 = Activity(model, 'A3', Exp.fitMean(3)).on(T1).synchCall(E2);

B1 = Activity(model, 'B1', Exp.fitMean(0.1)).on(T2).boundTo(E2);
B2 = Activity(model, 'B2', Exp.fitMean(0.2)).on(T2);
B3 = Activity(model, 'B3', Exp.fitMean(0.3)).on(T2);
B4 = Activity(model, 'B4', Exp.fitMean(0.4)).on(T2);

... (10 source lines omitted; full implementation: matlab/examples/gallery/)

T2.addPrecedence(ActivityPrecedence.Serial(B4, B5));
T2.addPrecedence(ActivityPrecedence.AndFork(B1, {B2, B3, B4}));
T2.addPrecedence(ActivityPrecedence.AndJoin({B2, B3, B5}, B6));
T3.addPrecedence(ActivityPrecedence.OrFork(C1, {C2, C3, C4}, [0.3, 0.3, 0.4]));
T3.addPrecedence(ActivityPrecedence.OrJoin({C2, C3, C4}, C5));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.32 gallery_lukumar_reentrant

The Lu-Kumar re-entrant line, the standard counterexample in which a work-conserving policy destabilizes a network whose stations are individually underloaded.

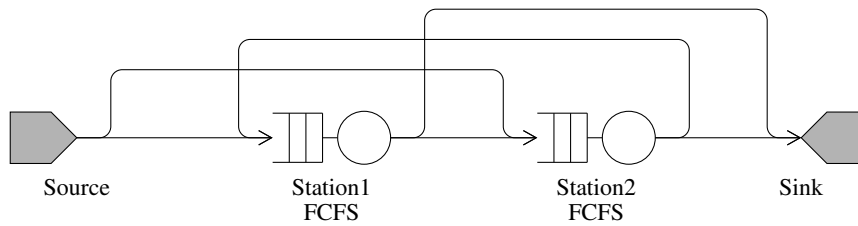


Figure 4.32: Topology of gallery_lukumar_reentrant: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	4: Class1 (open), Class2 (open), Class3 (open), Class4 (open)
Scheduling	Station1: FCFS; Station2: FCFS
Arrivals	Source – Class1: Exp(0.08); Class2: Disabled; Class3: Exp(0.08); Class4: Disabled
Service	Station1 – Class1: Exp(0.1); Class2: Disabled; Class3: Disabled; Class4: Exp(1) Station2 – Class1: Disabled; Class2: Exp(1); Class3: Exp(0.1); Class4: Disabled
Features	class switching on 1 link

Table 4.33: Features of gallery_lukumar_reentrant.

The example is built and solved as follows.

```

function model = gallery_lukumar_reentrant(schedStrategy)
% GALLERY_LUKUMAR_REENTRANT Lu-Kumar (Rybko-Stolyar) unstable network example
%
% This example implements the classic Lu-Kumar/Rybko-Stolyar network,
% which demonstrates that queueing networks can be unstable even when
% the utilization at each station is strictly less than 1.
%
% Network structure (two chains sharing two stations):
% - 2 stations (servers)
% - 4 job classes in 2 chains
% - Chain 1: Source -> Class1@Station1 -> Class2@Station2 -> Sink
% - Chain 2: Source -> Class3@Station2 -> Class4@Station1 -> Sink

```



```

%
% Under FCFS scheduling: The network is stable when all station
% utilizations are < 1.
%
% Under the "bad" priority policy (HOL with class 1 > class 4 at
% station 1, and class 3 > class 2 at station 2): The network becomes
% unstable due to the "virtual bottleneck" phenomenon, even with
% utilization < 1 at each station.
%
% Reference:
% Lu, S.H. and Kumar, P.R. (1991), "Distributed Scheduling Based on Due
% Dates and Buffer Priorities", IEEE Transactions on Automatic Control,
% Vol. 36, No. 12, pp. 1406-1416.
%

... (116 source lines omitted; full implementation: matlab/examples/gallery/)

P{class3, class4}(station2, station1) = 1; % Class3@Station2 -> Class4@Station1
P{class4, class4}(station1, sink) = 1; % Class4@Station1 -> Sink

model.link(P);

end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.33 gallery_mapm1

The MAP/M/1 queue: a Markovian arrival process, exponential service, and a single server.

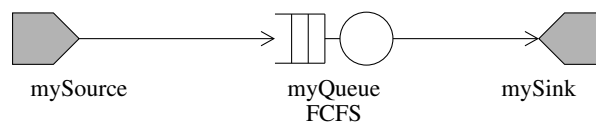


Figure 4.33: Topology of gallery_mapm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: MAP
Service	myQueue – myClass: Exp(2)

Table 4.34: Features of gallery_mapm1.

The example is built and solved as follows.

```

function model = gallery_mapm1(map)
if nargin < 1
    D0 = [-0.6984901916396979, 0.45234650636128054; 0.34690024319398277,
        -0.8194057961021199];
    D1 = [0.2125067546435463, 0.033636930634871165; 0.4441520099524867,
        0.028353542955650513];
    map = MAP({D0, D1});
end
model = Network('MAP/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, map);

```

```

queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.34 gallery_mapmk

The MAP/M/ k queue: a Markovian arrival process, exponential service, and k parallel servers.

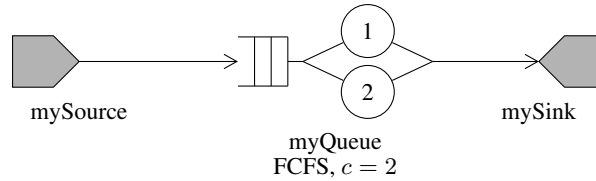


Figure 4.34: Topology of gallery_mapmk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: MAP
Service	myQueue – myClass: Exp(2)

Table 4.35: Features of gallery_mapmk.

The example is built and solved as follows.

```

function model = gallery_mapmk(map,k)
if nargin < 1
    map = MAP.rand;
end
if nargin<2
    k = 2;
end
model = Network('MAP/M/k');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, map);
queue.setService(oclass, Exp(2));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.35 gallery_mdk

The M/D/ k queue: Poisson arrivals, deterministic service, and k parallel servers.

The example is built and solved as follows.

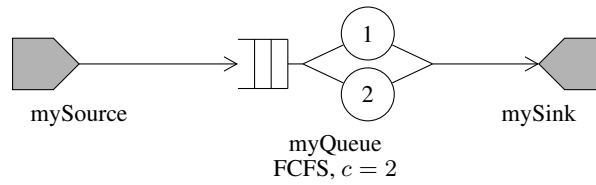


Figure 4.35: Topology of gallery_mdk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Det(1)

Table 4.36: Features of gallery_mdk.

```
function model = gallery_mdk(k)
if nargin<1
    k = 2;
end
model = Network('M/D/k');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp.fitMean(1));
queue.setService(oclass, Det(2/k));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.36 gallery_merl1

The M/Erl/1 queue: Poisson arrivals, Erlang service, and a single server.

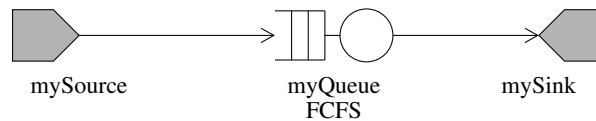


Figure 4.36: Topology of gallery_merl1: an open network over 1 queueing station.

The example is built and solved as follows.

```
function [model,source,queue,sink,oclass]=gallery_merl1
model = Network('M/E/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Erlang.fitMeanAndOrder(0.5,2));
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Erlang(4,2)

Table 4.37: Features of gallery_merl1.

```
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.37 gallery_merl1_linear

The M/Erl/1 queue: Poisson arrivals, Erlang service, and a single server. In this variant, the model is a linear chain of such stations.

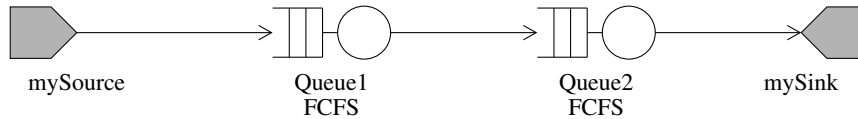


Figure 4.37: Topology of gallery_merl1_linear: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Erlang(1.111,1) Queue2 – myClass: Erlang(2.222,2)

Table 4.38: Features of gallery_merl1_linear.

The example is built and solved as follows.

```
function model = gallery_merl1_linear(n,Umax)
if ~exist('Umax','var')
    Umax = 0.9;
end
if ~exist('n','var')
    n = 2;
end
model = Network('M/Erl/1-Linear');
%% Block 1: nodes
line{1} = Source(model, 'mySource');
for i=1:n
    line{1+i} = Queue(model, ['Queue',num2str(i)], SchedStrategy.FCFS);
end
line{end+1} = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
line{1}.setArrival(oclass, Exp(1));
means = linspace(0.1,Umax,n/2);
if mod(n,2)==0
    means=[means,means(end:-1:1)];
else
    means=[means,Umax,means(end:-1:1)];
end
for i=1:n
```

```

    line{1+i}.setService(oclass, Erlang.fitMeanAndOrder(means(i),i));
end
%% Block 3: topology
model.link(Network.serialRouting(line));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.38 gallery_mer11_reentrant

The M/Erl/1 queue: Poisson arrivals, Erlang service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

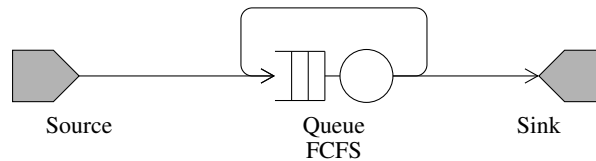


Figure 4.38: Topology of gallery_mer11_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Erlang(10,5); Class2: Exp(3)
Features	class switching on 1 link

Table 4.39: Features of gallery_mer11_reentrant.

The example is built and solved as follows.

```

function model = gallery_mer11_reentrant
model = Network('M/Erl/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Exp(1));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Erlang.fitMeanAndOrder(1/2,5));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.39 gallery_mer11_tandem

Two M/Erl/1 queues in series.

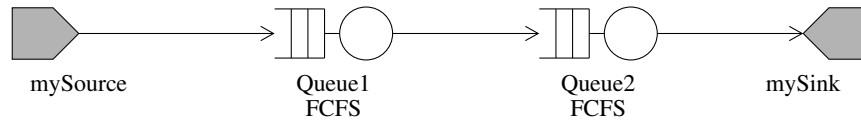


Figure 4.39: Topology of gallery_mer11_tandem: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Erlang(1.111,1) Queue2 – myClass: Erlang(2.222,2)

Table 4.40: Features of gallery_mer11_tandem.

The example is built and solved as follows.

```
function model = gallery_mer11_tandem()
model = gallery_mer11_linear(2);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.40 gallery_mer1k

The M/Erl/ k queue: Poisson arrivals, Erlang service, and k parallel servers.

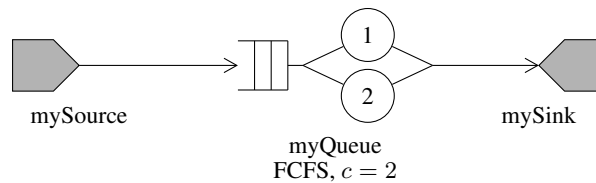


Figure 4.40: Topology of gallery_mer1k: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model=gallery_mer1k(k)
if ~exist('k','var')
    k = 2;
end
model = Network(['M/Erl/', num2str(k)]);
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Erlang.fitMeanAndOrder(0.5,2));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Erlang(4,2)

Table 4.41: Features of gallery_merlk.

4.2.41 gallery_mhyp1

The M/HyperExp/1 queue: Poisson arrivals, hyperexponential service, and a single server.

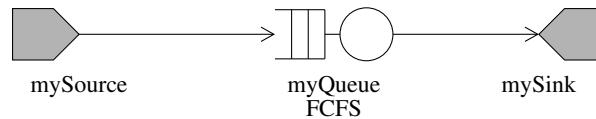


Figure 4.41: Topology of gallery_mhyp1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: HyperExp(0.99,0.01;2.281,0.1517)

Table 4.42: Features of gallery_mhyp1.

The example is built and solved as follows.

```

function model = gallery_mhyp1()
model = Network('M/H/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, HyperExp.fitMeanAndSCV(0.5,4));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
  
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.42 gallery_mhyp1_linear

The M/HyperExp/1 queue: Poisson arrivals, hyperexponential service, and a single server. In this variant, the model is a linear chain of such stations.

The example is built and solved as follows.

```

function model = gallery_mhyp1_linear(n,Umax)
if ~exist('Umax','var')
    Umax = 0.9;
end
if ~exist('n','var')
    n = 2;
end
model = Network('M/Hyp/1-Linear');
  
```

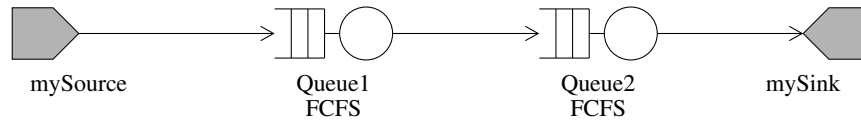


Figure 4.42: Topology of gallery_mhyp1_linear: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: HyperExp(0.99,0.01;1.196,0.1383)

Table 4.43: Features of gallery_mhyp1_linear.

```

%% Block 1: nodes
line{1} = Source(model, 'mySource');
for i=1:n
    line{1+i} = Queue(model, ['Queue',num2str(i)], SchedStrategy.FCFS);
end
line{end+1} = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
line{1}.setArrival(oclass, Exp(1));
means = linspace(0.1,Umax,n/2);
if mod(n,2)==0
    means=[means,means(end:-1:1)];
else
    means=[means,Umax,means(end:-1:1)];
end
for i=1:n
    line{1+i}.setService(oclass, HyperExp.fitMeanAndSCV(means(i),n));
end
%% Block 3: topology
model.link(Network.serialRouting(line));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.43 gallery_mhyp1_reentrant

The M/HyperExp/1 queue: Poisson arrivals, hyperexponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

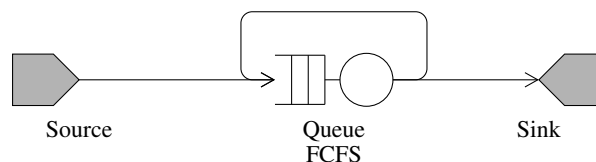


Figure 4.43: Topology of gallery_mhyp1_reentrant: an open network over 1 queueing station.

The example is built and solved as follows.

```

function model = gallery_mhyp1_reentrant
model = Network('M/Hyper/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);

```


Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Coxian; Class2: Exp(3)
Features	class switching on 1 link

Table 4.44: Features of gallery_mhyp1_reentrant.

```

sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Exp(1));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Coxian.fitMeanAndSCV(0.5,4));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.44 gallery_mhyp1_tandem

Two M/HyperExp/1 queues in series.

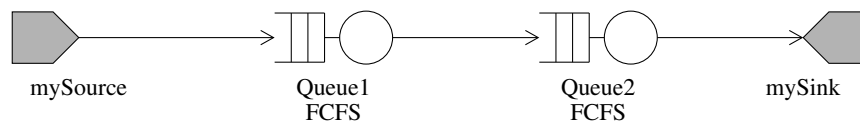


Figure 4.44: Topology of gallery_mhyp1_tandem: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: HyperExp(0.99,0.01;1.196,0.1383)

Table 4.45: Features of gallery_mhyp1_tandem.

The example is built and solved as follows.

```

function model=gallery_mhyp1_tandem
model = gallery_mhyp1_linear(2);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.45 gallery_mhypk

The M/HyperExp/ k queue: Poisson arrivals, hyperexponential service, and k parallel servers.

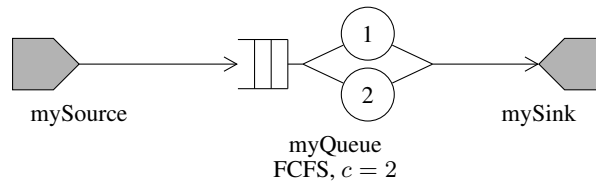


Figure 4.45: Topology of gallery_mhypk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Coxian

Table 4.46: Features of gallery_mhypk.

The example is built and solved as follows.

```
function model = gallery_mhypk(k)
if ~exist('k','var')
    k = 2;
end
model = Network(['M/Hyper/', num2str(k)]);
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Coxian.fitMeanAndSCV(0.5,4));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.46 gallery_mm1

The M/M/1 queue: Poisson arrivals, exponential service, and a single server.

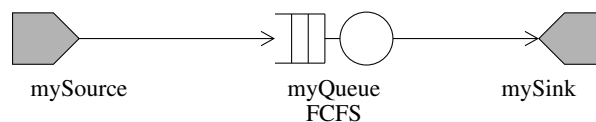


Figure 4.46: Topology of gallery_mm1: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model = gallery_mm1
model = Network('M/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Exp(2)

Table 4.47: Features of gallery_mm1.

```

queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.47 gallery_mm1_feedback

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, a completing job returns to the station with fixed probability.

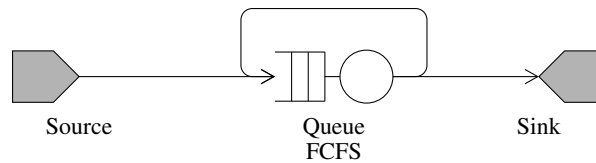


Figure 4.47: Topology of gallery_mm1_feedback: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue – Class1: Exp(2)

Table 4.48: Features of gallery_mm1_feedback.

The example is built and solved as follows.

```

function model = gallery_mm1_feedback(p)
if ~exist('p','var')
    p = 1/3;
end
model = Network('M/M/1-Feedback');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Exp.fitMean(1));
queue.setService(oclass1, Exp.fitMean(0.5));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;

```

```
P{oclass1,oclass1}(queue,queue)=p;
P{oclass1,oclass1}(queue,sink)=1-p;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.48 gallery_mm1_linear

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the model is a linear chain of such stations.

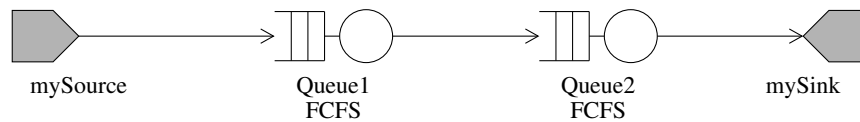


Figure 4.48: Topology of gallery_mm1_linear: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Exp(1.111) Queue2 – myClass: Exp(1.111)

Table 4.49: Features of gallery_mm1_linear.

The example is built and solved as follows.

```
function model = gallery_mm1_linear(n,Umax)
if ~exist('Umax','var')
    Umax = 0.9;
end
if ~exist('n','var')
    n = 2;
end
model = Network('M/M/1-Linear');
%% Block 1: nodes
line{1} = Source(model, 'mySource');
for i=1:n
    line{1+i} = Queue(model, ['Queue',num2str(i)], SchedStrategy.FCFS);
end
line{end+1} = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
line{1}.setArrival(oclass, Exp(1));
means = linspace(0.1,Umax,n/2);
if mod(n,2)==0
    means=[means,means(end:-1:1)];
else
    means=[means,Umax,means(end:-1:1)];
end
for i=1:n
    line{1+i}.setService(oclass, Exp.fitMean(means(i)));
end
%% Block 3: topology
model.link(Network.serialRouting(line));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.49 gallery_mm1_multiclass

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, several open classes share the station.

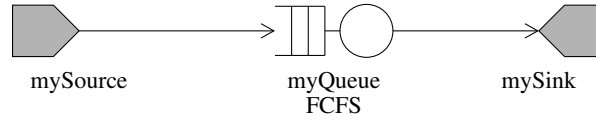


Figure 4.49: Topology of gallery_mm1_multiclass: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.5)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(4)

Table 4.50: Features of gallery_mm1_multiclass.

The example is built and solved as follows.

```

function model = gallery_mm1_multiclass
model = Network('M[2]/M[2]/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(1));
queue.setService(oclass1, Exp(4));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.5));
queue.setService(oclass2, Exp(4));
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue, sink);
P{2} = Network.serialRouting(source, queue, sink);
model.link(P);
end
  
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.50 gallery_mm1_prio

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the classes carry priorities, served head-of-line.

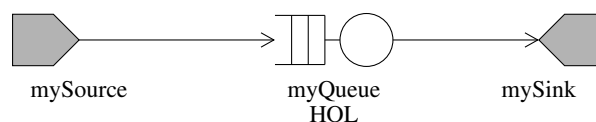


Figure 4.50: Topology of gallery_mm1_prio: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: HOL
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.5)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(4)

Table 4.51: Features of gallery_mm1_prio.

```
function model=gallery_mm1_prio()
model = Network('M[2]/M[2]/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.HOL);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1', 1);
source.setArrival(oclass1, Exp(1));
queue.setService(oclass1, Exp(4));

oclass2 = OpenClass(model, 'myClass2', 0);
source.setArrival(oclass2, Exp(0.5));
queue.setService(oclass2, Exp(4));
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source,queue,sink);
P{2} = Network.serialRouting(source,queue,sink);
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.51 gallery_mm1_ps

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS.

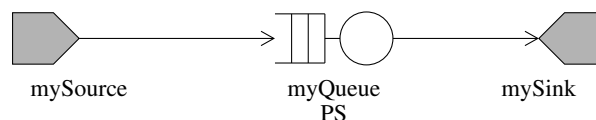


Figure 4.51: Topology of gallery_mm1_ps: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Exp(2)

Table 4.52: Features of gallery_mm1_ps.

The example is built and solved as follows.

```
function model = gallery_mm1_ps
model = Network('M/M/1-PS');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.PS);
sink = Sink(model, 'mySink');
```

```

%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Exp(2));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.52 gallery_mm1_ps_feedback

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS and a completing job returns to the station with fixed probability.

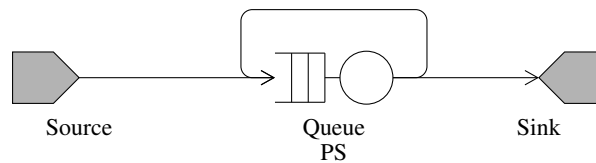


Figure 4.52: Topology of gallery_mm1_ps_feedback: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: PS
Arrivals	Source – Class1: Exp(1)
Service	Queue – Class1: Exp(2)

Table 4.53: Features of gallery_mm1_ps_feedback.

The example is built and solved as follows.

```

function model = gallery_mm1_ps_feedback(p)
if ~exist('p','var')
    p = 1/3;
end
model = Network('M/M/1-PS-Feedback');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.PS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Exp.fitMean(1));
queue.setService(oclass1, Exp.fitMean(0.5));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass1}(queue,queue)=p;
P{oclass1,oclass1}(queue,sink)=1-p;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.53 gallery_mm1_ps_multiclass

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS and several open classes share the station.

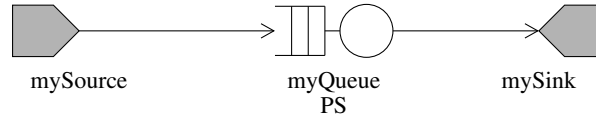


Figure 4.53: Topology of gallery_mm1_ps_multiclass: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.5)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(4)

Table 4.54: Features of gallery_mm1_ps_multiclass.

The example is built and solved as follows.

```
function model = gallery_mm1_ps_multiclass
model = Network('M[2]/M[2]/1-PS');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.PS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(1));
queue.setService(oclass1, Exp(4));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.5));
queue.setService(oclass2, Exp(4));
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue, sink);
P{2} = Network.serialRouting(source, queue, sink);
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.54 gallery_mm1_ps_reentrant

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS and jobs re-enter the station, so a visit is counted more than once.

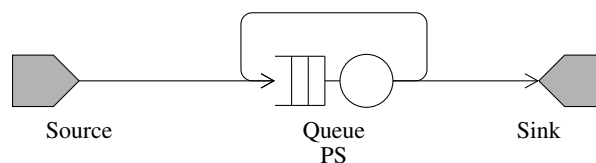


Figure 4.54: Topology of gallery_mm1_ps_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: PS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.55: Features of gallery_mm1_ps_reentrant.

The example is built and solved as follows.

```
function model = gallery_mm1_ps_reentrant
model = Network('M/M/1');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.PS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Exp(1));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Exp(2));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.55 gallery_mm1_reentrant

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

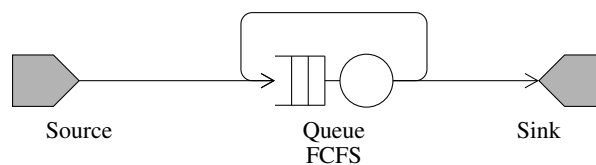


Figure 4.55: Topology of gallery_mm1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.56: Features of gallery_mm1_reentrant.

The example is built and solved as follows.

```
function model = gallery_mm1_reentrant
```

```
% A M/M/1 with two classes forming a re-entrant line
model = Network('M/M/1-Reentrant');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
oclass2 = OpenClass(model, 'Class2');
source.setArrival(oclass1, Exp(1));
source.setArrival(oclass2, Disabled());
queue.setService(oclass1, Exp(2));
queue.setService(oclass2, Exp(3));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass2}(queue,queue)=1;
P{oclass2,oclass2}(queue,sink)=1;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.56 gallery_mm1_tandem

Two M/M/1 queues in series.

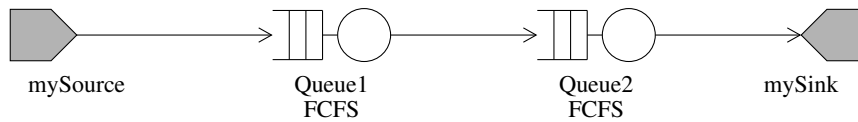


Figure 4.56: Topology of gallery_mm1_tandem: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Exp(1.111) Queue2 – myClass: Exp(1.111)

Table 4.57: Features of gallery_mm1_tandem.

The example is built and solved as follows.

```
function model = gallery_mm1_tandem
model = gallery_mm1_linear(2);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.57 gallery_mm1_tandem_multiclass

Two M/M/1 queues in series under a multiclass workload.

The example is built and solved as follows.

```
function model = gallery_mm1_tandem_multiclass
model = Network('M[2]/M[2]/1 -> -/M[2]/1');

%% Block 1: nodes
```

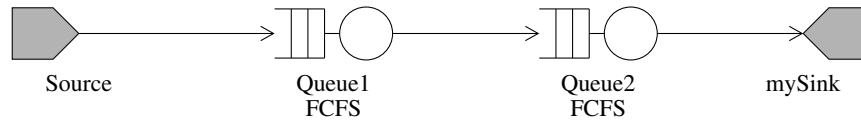


Figure 4.57: Topology of gallery_mm1_tandem_multiclass: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – myClass1: Exp(1); myClass2: Exp(0.5)
Service	Queue1 – myClass1: Exp(4); myClass2: Exp(2) Queue2 – myClass1: Exp(6); myClass2: Exp(6)

Table 4.58: Features of gallery_mm1_tandem_multiclass.

```
source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');

%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(1));
queue1.setService(oclass1, Exp(4));
queue2.setService(oclass1, Exp(6));

oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.5));
queue1.setService(oclass2, Exp(2));
queue2.setService(oclass2, Exp(6));

%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source, queue1, queue2, sink);
P{2} = Network.serialRouting(source, queue1, queue2, sink);
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.58 gallery_mm1k

The M/M/1/K queue: a single exponential server behind a buffer of K places, so an arrival finding the buffer full is lost.

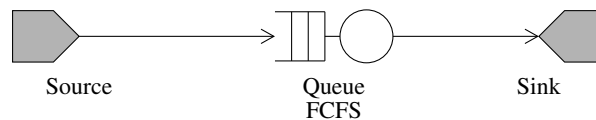


Figure 4.58: Topology of gallery_mm1k: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model = gallery_mm1k(K)
% GALLERY_MM1K M/M/1/K queue with finite capacity K (blocking / loss)
if nargin < 1
    K = 3;
end
model = Network('M/M/1/K');
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.8)
Service	Queue – Class1: Exp(1)
Features	finite buffer 3 at Queue

Table 4.59: Features of gallery_mm1k.

```

%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
queue.setNumberOfServers(1);
queue.setCapacity(K);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass = OpenClass(model, 'Class1', 0);
source.setArrival(oclass, Exp(0.8));
queue.setService(oclass, Exp(1.0));
%% Block 3: topology
P = model.initRoutingMatrix();
P.set(oclass, oclass, source, queue, 1.0);
P.set(oclass, oclass, queue, sink, 1.0);
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.59 gallery_mmap1

The M/MAP/1 queue: Poisson arrivals, MAP-modulated service, and a single server.

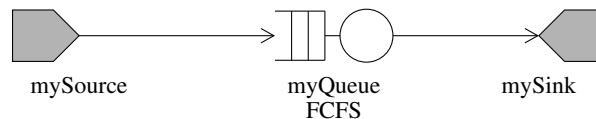


Figure 4.59: Topology of gallery_mmap1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: MAP

Table 4.60: Features of gallery_mmap1.

The example is built and solved as follows.

```

function model = gallery_mmap1(map)
if nargin < 1
    map = MAP.rand.setMean(0.5);
end
model = Network('M/MAP/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');

```

```

source.setArrival(oclass, Exp(1));
queue.setService(oclass, map);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.60 gallery_mmap1_multiclass

The M/MAP/1 queue: Poisson arrivals, MAP-modulated service, and a single server. In this variant, several open classes share the station.

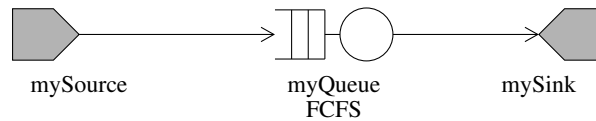


Figure 4.60: Topology of gallery_mmap1_multiclass: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass1: Exp(0.7); myClass2: Exp(0.3)
Service	myQueue – myClass1: MAP; myClass2: MAP

Table 4.61: Features of gallery_mmap1_multiclass.

The example is built and solved as follows.

```

function model = gallery_mmap1_multiclass(map1,map2)
if nargin < 1
    map1 = MAP.rand(2).setMean(0.5);
    map2 = MAP.rand(3).setMean(0.5);
end
model = Network('M/MAP/1');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass1 = OpenClass(model, 'myClass1');
source.setArrival(oclass1, Exp(0.35/map1.getMean));
queue.setService(oclass1, map1);
oclass2 = OpenClass(model, 'myClass2');
source.setArrival(oclass2, Exp(0.15/map2.getMean));
queue.setService(oclass2, map2);
%% Block 3: topology
P = model.initRoutingMatrix;
P{1} = Network.serialRouting(source,queue,sink);
P{2} = Network.serialRouting(source,queue,sink);
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.61 gallery_mmapk

The M/MAP/ k queue: Poisson arrivals, MAP-modulated service, and k parallel servers.

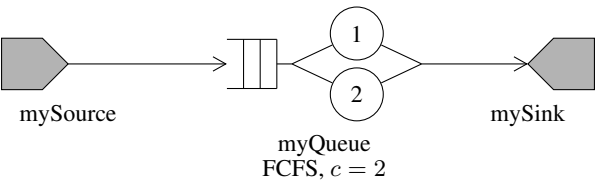


Figure 4.61: Topology of `gallery_mmapk`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: MAP

Table 4.62: Features of `gallery_mmapk`.

The example is built and solved as follows.

```
function model = gallery_mmapk(map)
if nargin < 1
    map = MAP.rand;
end
if nargin<2
    k = 2;
end
model = Network('M/MAP/k');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, map);
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.62 gallery_mmk

The M/M/ k queue: Poisson arrivals, exponential service, and k parallel servers.

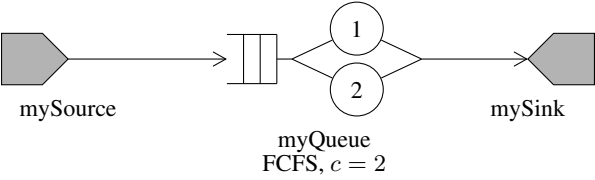


Figure 4.62: Topology of `gallery_mmk`: an open network over 1 queueing station.

The example is built and solved as follows.

```
function model=gallery_mmk(k)
if nargin<1
    k = 2;
end
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Exp(2)

Table 4.63: Features of gallery_mmk.

```

model = Network('M/M/k');
%% Block 1: nodes
source = Source(model, 'mySource');
queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
source.setArrival(oclass, Exp(1));
queue.setService(oclass, Exp(2));
queue.setNumberOfServers(k);
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.63 gallery_mpar1

The M/Pareto/1 queue: Poisson arrivals, Pareto service, and a single server.

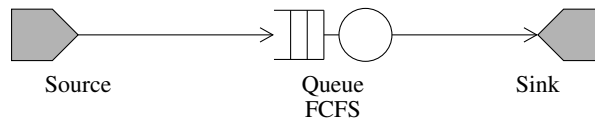


Figure 4.63: Topology of gallery_mpar1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue – Class1: Pareto(2.008,0.251)

Table 4.64: Features of gallery_mpar1.

The example is built and solved as follows.

```

function model = gallery_mpar1
model = Network('M/Par/1');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Exp(1));
queue.setService(oclass1, Pareto.fitMeanAndSCV(0.5,64));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass1}(queue,sink)=1;
model.link(P);

```

```
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.64 gallery_multitier

A multitier application: presentation, application and database tiers as a closed network.

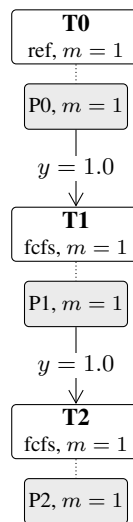


Figure 4.64: Call graph of `gallery_multitier`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P0 (ps, $m = 1$), P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	3: T0 (ref, $m = 1$), T1 (fcfs, $m = 1$), T2 (fcfs, $m = 1$)
Entries	9: E0, E10, E11, E12, E13, E20, E21, E22, E23
Activities	22, host demands A0: 1.0, A1: 1.0, A2: 1.0, A3: 1.0, B0: 1.0, B1: 1.0, B2: 1.0, B3: 1.0, ...
Calls	8 (8 synchronous, 0 asynchronous)

Table 4.65: Features of `gallery_multitier`.

The example is built and solved as follows.

```

function model = gallery_multitier()
% GALLERY_MULTITIER Layered Queueing Network (LQN) - Production 4-Tier J2EE
%
% This example demonstrates a comprehensive 4-layer J2EE system architecture:
%
%   Layer 1: CLIENT LAYER
%   - Reference task with 1 user (closed class)
%   - Represents browser clients with sequential user operations
%
%   Layer 2: APPLICATION LAYER
%   - Service task handling 4 concurrent web requests
%   - Multiple entries (renderHomePage, processProductPage, handleLogin, checkoutWorkflow)
%   - Complex activity precedence with or-fork/or-join branching
%
%   Layer 3: DATABASE LAYER
%   - Service task managing database operations
%   - Multiple entries for different query types (read, write, query, log)
%   - Sequential activity chains
%
% The model demonstrates:

```



```

% - Multiple entries per task
% - Complex activity precedence patterns (serial, or-fork, or-join)
% - Synchronous call routing between layers
% - Load-dependent processing characteristics
% - Production-grade LQN features
%
... (93 source lines omitted; full implementation: matlab/examples/gallery/)

% A gallery entry is a model factory and nothing else: it returns the model and
% leaves the choice of solver to the caller, so this file ends here. To solve it,
%
% model = gallery_multitier();
% AvgTable = LN(model, @(layer) MVA(layer)).getAvgTable
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.65 gallery_multitier_storage

The multitier application with an explicit storage tier behind the database.

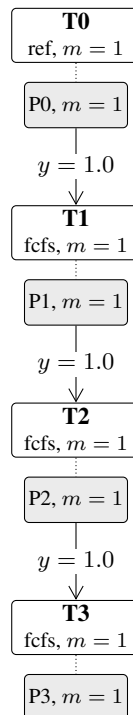


Figure 4.65: Call graph of gallery_multitier_storage: 4 tasks over 4 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	4: P0 (ps, $m = 1$), P1 (ps, $m = 1$), P2 (ps, $m = 1$), P3 (ps, $m = 1$)
Tasks	4: T0 (ref, $m = 1$), T1 (fcfs, $m = 1$), T2 (fcfs, $m = 1$), T3 (fcfs, $m = 1$)
Entries	10: E0, E10, E11, E12, E13, E20, E21, E22, E23, E3
Activities	24, host demands A0: 1.0, A1: 1.0, A2: 1.0, A3: 1.0, B0: 1.0, B1: 1.0, B2: 1.0, B3: 1.0, ...
Calls	10 (10 synchronous, 0 asynchronous)

Table 4.66: Features of gallery_multitier_storage.

The example is built and solved as follows.

```
function model = gallery_multitier_storage()
% GALLERY_MULTITIER_STORAGE Layered Queueing Network (LQN) - Production 4-Tier J2EE with
%   Cache
%
% This example extends the 4-tier J2EE model (example_layered_production_4tier.m)
% with a dedicated CACHE LAYER (Layer 4), demonstrating:
%
%   Layer 1: CLIENT LAYER
%   - Reference task with 1 user (closed class)
%   - Represents browser clients with sequential user operations
%
%   Layer 2: APPLICATION LAYER
%   - Service task handling web requests
%   - Multiple entries for different operations
%   - Complex activity precedence with branching
%
%   Layer 3: DATABASE LAYER
%   - Primary data storage
%   - Cache lookup operations (via calls to cache layer)
%   - Write and log operations
%
%   Layer 4: CACHE LAYER (NEW)
%   - Dedicated cache with LRU replacement strategy
%   - Cache capacity: 2 items out of 10 total items
%   - Uniform access distribution
%   - Cache hit/miss modeling with different service times
%
% ... (123 source lines omitted; full implementation: matlab/examples/gallery/)
% A gallery entry is a model factory and nothing else: it returns the model and
% leaves the choice of solver to the caller, so this file ends here. To solve it,
%
%   model = gallery_multitier_storage();
%   AvgTable = LN(model, @(layer) MVA(layer)).getAvgTable
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.66 gallery_parm1

The Pareto/M/1 queue: Pareto interarrival times, exponential service, and a single server.

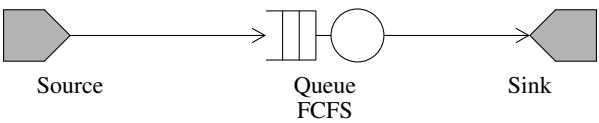


Figure 4.66: Topology of gallery_parm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Pareto(2.008,0.5019)
Service	Queue – Class1: Exp(2)

Table 4.67: Features of gallery_parm1.

The example is built and solved as follows.

```
function model = gallery_parm1
```

```

model = Network('Par/M/1');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Pareto.fitMeanAndSCV(1,64));
queue.setService(oclass1, Exp(2));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass1}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.67 gallery_qn_random

A randomly generated queueing network, used as a generator-driven parity fixture.

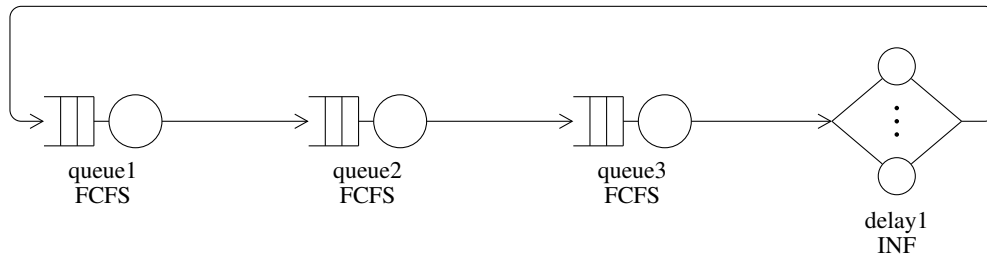


Figure 4.67: Topology of gallery_qn_random: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	2: CClass1 (closed, $N = 14$), CClass2 (closed, $N = 11$)
Scheduling	queue1: FCFS; queue2: FCFS; queue3: FCFS; delay1: INF
Service	queue1 – CClass1: Exp(1); CClass2: Exp(1) queue2 – CClass1: Exp(1); CClass2: Exp(1) queue3 – CClass1: Exp(1); CClass2: Exp(1) delay1 – CClass1: Exp(1); CClass2: Exp(1)

Table 4.68: Features of gallery_qn_random.

The example is built and solved as follows.

```

function model = gallery_qn_random(seed)
% GALLERY_QN_RANDOM Randomly generated mixed queueing network (reproducible).
% Uses NetworkGenerator with a deterministic (cyclic) topology and a fixed
% default seed so repeated calls yield the same model.
% Closed network: 3 queues, 1 delay, 2 closed classes (always stable).
if nargin < 1
    seed = 23000;
end
rng(seed);
gen = NetworkGenerator('schedStrat', 'fcfs', 'routingStrat', 'Probabilities', ...
    'distribution', 'exp', 'cclassJobLoad', 'medium', ...
    'hasVaryingServiceRates', false, 'hasMultiServerQueues', false, ...
    'hasRandomCSNodes', false, 'hasMultiChainCS', false, ...
    'topologyFcn', @NetworkGenerator.cyclicGraph);
model = gen.generate(3, 1, 0, 2);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.68 gallery_renv_breakdown

A station in a random environment that alternates between an operational and a failed stage.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
function env = gallery_renv_breakdown()
% GALLERY_RENV_BREAKDOWN Random environment: single server with breakdown/repair
% Returns an Environment whose base model is an M/M/1 queue that alternates
% between an UP stage (fast service) and a DOWN stage (degraded service).
model = Network('ServerWithFailures');
%% Base model nodes
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
%% Base model class
jobclass = OpenClass(model, 'Jobs');
source.setArrival(jobclass, Exp(0.8));
queue.setService(jobclass, Exp(2.0));
queue.setNumberOfServers(1);
%% Base model routing
P = model.initRoutingMatrix();
P.set(jobclass, jobclass, source, queue, 1.0);
P.set(jobclass, jobclass, queue, sink, 1.0);
model.link(P);
%% Environment: breakdown Exp(0.1), repair Exp(1.0), degraded service Exp(0.5)
env = Environment('ServerEnv');
env.addNodeFailureRepair(model, queue, Exp(0.1), Exp(1.0), Exp(0.5));
env.init();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.69 gallery_repairmen

The machine repairman model: machines think, fail, and queue for one repair station.

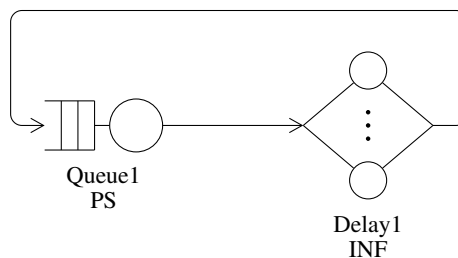


Figure 4.68: Topology of gallery_repairmen: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
function model = gallery_repairmen(nservers, seed)
model = Network('Finite repairmen CQN');
%% Block 1: nodes
M=1;
```

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 8$)
Scheduling	Queue1: PS; Delay1: INF
Service	Queue1 – Class1: Exp(0.7685) Delay1 – Class1: Exp(0.5)

Table 4.69: Features of gallery_repairmen.

```

if nargin<=1
    nservers=1;
end

if nargin<3
    seed = 2300;
end
rng(seed);
node{1} = Queue(model, ['Queue', num2str(1)], SchedStrategy.PS);
node{1}.setNumberOfServers(nservers);
node{M+1} = Delay(model, 'Delay1');
%% Block 2: classes
jobclass = ClosedClass(model, 'Class1', round(rand*10*M+3), node{1}, 0);

node{1}.setService(jobclass, Exp.fitMean(rand()+1)); % (Queue 1,Class1)
node{M+1}.setService(jobclass, Exp.fitMean(2.000000)); % (Delay 1,Class1)

%% Block 3: topology
P = model.initRoutingMatrix(); % initialize routing matrix
P{jobclass,jobclass} = Network.serialRouting(node);
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.70 gallery_replayerm1

A single queue whose interarrival times are replayed from a measurement trace.

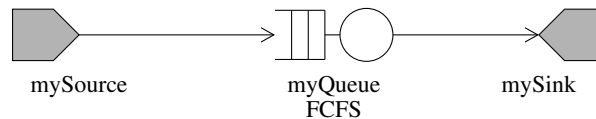


Figure 4.69: Topology of gallery_replayerm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Replayer(trace)
Service	myQueue – myClass: Exp(29.68)

Table 4.70: Features of gallery_replayerm1.

The example is built and solved as follows.

```

function model = gallery_replayerm1(filename)
if ~exist('filename','var')
    filename = which('example_trace.txt');
end
model = Network('Trace/M/1');
%% Block 1: nodes
source = Source(model, 'mySource');

```

```

queue = Queue(model, 'myQueue', SchedStrategy.FCFS);
sink = Sink(model, 'mySink');
%% Block 2: classes
oclass = OpenClass(model, 'myClass');
replayer = Replayer(filename);
source.setArrival(oclass, replayer);
queue.setService(oclass, Exp(3/replayer.getMean));
%% Block 3: topology
model.link(Network.serialRouting(source,queue,sink));
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

4.2.71 gallery_um1

The U/M/1 queue: uniform interarrival times, exponential service, and a single server.

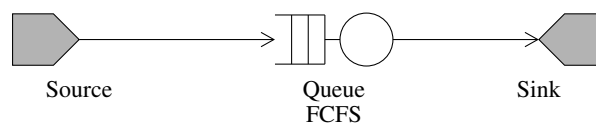


Figure 4.70: Topology of gallery_um1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Uniform(1,2)
Service	Queue – Class1: Exp(2)

Table 4.71: Features of gallery_um1.

The example is built and solved as follows.

```

function model = gallery_um1
model = Network('U/M/1');
%% Block 1: nodes
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
%% Block 2: classes
oclass1 = OpenClass(model, 'Class1');
source.setArrival(oclass1, Uniform(1,2));
queue.setService(oclass1, Exp(2));
%% Block 3: topology
P = model.initRoutingMatrix;
P{oclass1,oclass1}(source,queue)=1;
P{oclass1,oclass1}(queue,sink)=1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Each of these is also a parity fixture: the four codebases build the same model and the harness compares the metrics solver by solver, so a divergence in any one of them is caught here before it reaches a larger model.

Chapter 5

Discrete-time models

In a slotted model time advances in slots of fixed length, and every sampled interarrival and service time must fall on the slot lattice. A sample that does not is an error rather than something to round: refusing is what keeps a discrete-time result exact. Within a slot the events are ordered by a fixed phase – completions, internal movement, arrivals, bookkeeping – so that a departure and an arrival in the same slot cannot be confused.

Geometric interarrival and service times give the Geo/Geo/1 queue, whose exact closed form is known, and it is against that form that the solvers are checked. The models live under `matlab/examples/discrete/`, and slotted mode is requested through the solver options rather than by the model.

```
a = 0.2;    % per-slot arrival probability
s = 0.5;    % per-slot service completion probability

model = Network('GeoGeo1');
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');
jobClass = OpenClass(model, 'Class1');
source.setArrival(jobClass, Geometric(a));
queue.setService(jobClass, Geometric(s));
model.link(Network.serialRouting(source, queue, sink));

options = SolverNC.defaultOptions;
options.config.slotted = true;    % run on the slot lattice
AvgTable = SolverNC(model, options).getAvgTable

fprintf('closed form: E[N] = %g, E[T] = %g slots\n', ...
        a*(1-a)/(s-a), (1-a)/(s-a));
```

5.1 The slotted models

5.1.1 dt_bernoulli_loaddep

A load-dependent Bernoulli server, $p(n) = s \min(n, c)$, the discrete-time multiserver approximation.

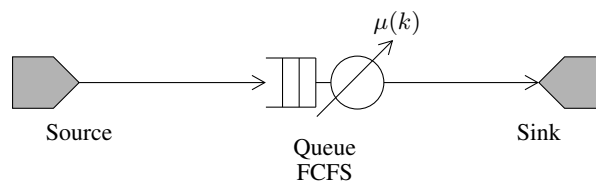


Figure 5.1: Topology of `dt_bernoulli_loaddep`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Geometric(0.6)
Service	Queue – Class1: Geometric(0.3)
Features	finite buffer 20 at Queue; load-dependent rates at Queue
Solvers	NC

Table 5.1: Features of dt_bernoulli_loaddep.

The example is built and solved as follows.

```
% Load-dependent Bernoulli server, and the discrete-time arrival theorem.
%
% Example 2.10 of Daduna (2001) notes that a discrete-time M/M/c queue has no
% exactly equivalent state dependent single server, but that  $p(n) = p \min(n, c)$ 
% reproduces its conditional service intensity. That is a load dependence in
% LINE terms, so the model is a single Bernoulli server whose service
% probability rises with the queue length up to c servers' worth of capacity.
%
% The example also prints the arrival distribution of theorem 2.11, the law an
% arriving job sees with itself not counted. In continuous time with Poisson
% arrivals that law would coincide with the time-stationary one (PASTA);
% discrete time has no such analogue, and the two rows below differ.

a = 0.6;      % per-slot arrival probability
s = 0.3;      % per-slot service completion probability of one server
c = 3;        % servers' worth of capacity
L = 20;       % buffer capacity, which bounds the state space

model = Network('LoadDepBernoulli');

source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'Class1');
source.setArrival(jobClass, Geometric(a));

... (10 source lines omitted; full implementation: matlab/examples/discrete/)

AvgTable = solver.getAvgTable; AvgTable

r = dqsys_bernoulli1(a, s * min(1:L, c), L);
fprintf('time-stationary law pi(0..6) : %s\n', mat2str(round(r.pmf(1:7), 6)));
fprintf('arrival law pi_1(0..6): %s\n', mat2str(round(r.arrivalPmf(1:7), 6)));
fprintf('no PASTA in discrete time: the two rows above are different laws\n');
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

5.1.2 dt_cycle

A closed cycle of slotted stations.

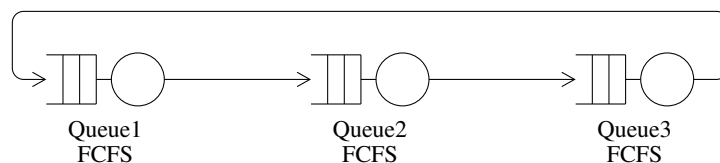


Figure 5.2: Topology of dt_cycle: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 3
Classes	1: Jobs (closed, $N = 5$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Jobs: Geometric(0.5) Queue2 – Jobs: Geometric(0.25) Queue3 – Jobs: Geometric(0.7)
Solvers	NC

Table 5.2: Features of `dt_cycle`.

```
% Closed cycle of Bernoulli servers on a discrete time scale.
%
% N jobs circulate through J FCFS single-server stations; station j completes
% a service with probability p_j in each slot. The stationary queue length
% vector has the product form of Daduna (2001), corollary 3.4,
%
%   pi(n_1,...,n_J) = prod_j (q_j/p_j)^{n_j} (1/q_j)^{\{1\{n_j>0\}\}} / G(N,J),
%
% whose extra factor on the busy nodes is what separates it from the
% continuous-time Gordon-Newell form: a homogeneous cycle is uniform on the
% state space in continuous time and is not here. SolverNC evaluates G with
% the three-term recursion of proposition 3.18 and the arrival constants with
% proposition 3.19, and reads the marginals off corollary 3.20.

p = [0.5 0.25 0.7]; % per-slot service completion probabilities
N = 5;              % jobs cycling

model = Network('BernoulliCycle');

station = cell(1, numel(p));
for j = 1:numel(p)
    station{j} = Queue(model, sprintf('Queue%d', j), SchedStrategy.FCFS);
end

jobClass = ClosedClass(model, 'Jobs', N, station{1});
for j = 1:numel(p)

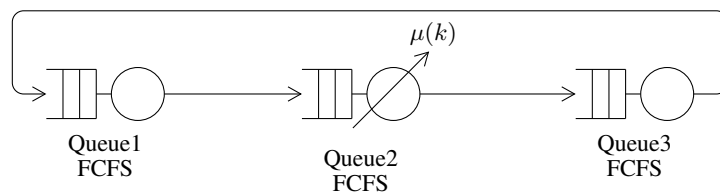
    ... (12 source lines omitted; full implementation: matlab/examples/discrete/)

% G_1(N,J)/G(N,J), the discrete-time counterpart of G(N-1)/G(N), and it is the
% same at every node of the cycle.
[lG, G, G1] = dpfqnc(p, N);
fprintf('log G(N,J)    = %g\n', lG);
fprintf('throughput    = %g jobs per slot\n', G1(N+1)/G);
fprintf('utilizations = %s\n', mat2str(round((G1(N+1)/G) ./ p, 6)));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

5.1.3 `dt_cycle_loaddep`

The cycle with a load-dependent slotted server.

Figure 5.3: Topology of `dt_cycle_loaddep`: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 3
Classes	1: Jobs (closed, $N = 5$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Jobs: Geometric(0.5) Queue2 – Jobs: Geometric(0.25) Queue3 – Jobs: Geometric(0.7)
Features	load-dependent rates at Queue2
Solvers	NC

Table 5.3: Features of dt_cycle_loaddep.

```
% Closed cycle of state dependent Bernoulli servers.
%
% Theorem 3.2 of Daduna (2001) keeps the product form when the service
% probability of station j depends on its own queue length, with node weight
%
%   w_j(n) = prod_{h=1}^{n-1} q_j(h) / prod_{h=1}^n p_j(h).
%
% Note where the state dependence sits: the missing q_j in the numerator is
% tied to the actual queue length, not to the node being non-empty, so the
% tidy (1/q_j)^{1{n>0}} of the state independent case cannot be factored out.
%
% Station 2 below runs at p_2(n) = p_2 min(n,2), the discrete-time analogue of
% adding a second server. That is admissible only because the dependence is
% expressed as a state dependent single server: a genuine multiserver node
% inside a cycle of geometrical queues destroys the product form for every
% finite server count (Pestien and Ramakrishnan, cited before example 2.10),
% and SolverNC rejects one rather than approximating it.

p = [0.5 0.25 0.7];
N = 5;

model = Network('BernoulliCycleLD');

station = cell(1, numel(p));
for j = 1:numel(p)
    station{j} = Queue(model, sprintf('Queue%d', j), SchedStrategy.FCFS);

    ... (16 source lines omitted; full implementation: matlab/examples/discrete/)

% Marginal queue length law of station 2 from the convolution constants.
P = repmat(p(:), 1, N);
P(2, :) = p(2) * min(1:N, 2);
[lG, G, W, Gc] = dpfqncld(P, N);
marg = W(2, :) .* Gc(2, N+1:-1:1) / G(N+1);
fprintf('P(X_2 = 0..%d) = %s\n', N, mat2str(round(marg, 6)));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

5.1.4 dt_cycle_multiclass

The multiclass cycle.

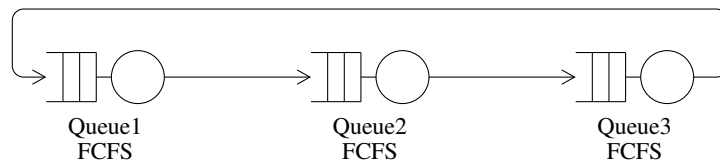


Figure 5.4: Topology of dt_cycle_multiclass: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 3
Classes	2: Chain1 (closed, $N = 3$), Chain2 (closed, $N = 2$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Chain1: Geometric(0.5); Chain2: Geometric(0.5) Queue2 – Chain1: Geometric(0.25); Chain2: Geometric(0.25) Queue3 – Chain1: Geometric(0.7); Chain2: Geometric(0.7)
Solvers	NC

Table 5.4: Features of dt_cycle_multiclass.

```
% Multichain closed cycle of Bernoulli servers.
%
% Section 3.2 of Daduna (2001) splits the circulating jobs into chains that
% never mix. Service in the cycle is type independent and FCFS forbids
% overtaking, so the cyclic order of the jobs is frozen for all time and the
% joint queue length law is the unichain one at the aggregate population. Each
% chain then holds a share of every station equal to its share of the
% population, which is what SolverNC reports below: the per-station totals
% reproduce dt_cycle at  $N = N_1 + N_2$ , and the per-class rows split them in the
% ratio  $N_1 : N_2$ .

p = [0.5 0.25 0.7];
N1 = 3;    % jobs in chain 1
N2 = 2;    % jobs in chain 2

model = Network('BernoulliCycleMC');

station = cell(1, numel(p));
for j = 1:numel(p)
    station{j} = Queue(model, sprintf('Queue%d', j), SchedStrategy.FCFS);
end

class1 = ClosedClass(model, 'Chain1', N1, station{1});
class2 = ClosedClass(model, 'Chain2', N2, station{1});
for j = 1:numel(p)
    station{j}.setService(class1, Geometric(p(j)));

    ... (16 source lines omitted; full implementation: matlab/examples/discrete/)

AvgTable = solver.getAvgTable; AvgTable

[lG, G, G1] = dpfqnc(p, N1 + N2);
X = G1(N1 + N2 + 1) / G;
fprintf('aggregate throughput = %g jobs per slot\n', X);
fprintf('per-chain throughput = %g and %g\n', X*N1/(N1+N2), X*N2/(N1+N2));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

5.1.5 dt_geogeo1

The Geo/Geo/1 queue: Bernoulli arrivals and Bernoulli service on the slot lattice, against the closed form of Daduna.

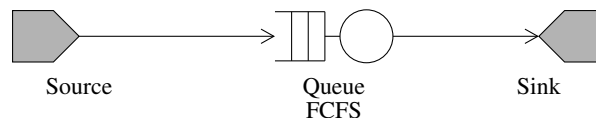


Figure 5.5: Topology of dt_geogeo1: an open network over 1 queueing station.

The example is built and solved as follows.

```
% Geo/Geo/1: the discrete-time single-server queue with an unbounded buffer.
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Geometric(0.2)
Service	Queue – Class1: Geometric(0.5)
Solvers	NC

Table 5.5: Features of dt_geogeo1.

```
%
% Arrivals occur with probability a in each slot, a service completes with
% probability s. SolverNC recognizes the slotted model and returns the exact
% closed form of Daduna (2001), theorem 2.3, which for constant a and s is
% the geometric law of corollary 2.7. Mean queue length  $a(1-a)/(s-a)$  and mean
% sojourn time  $(1-a)/(s-a)$  slots.

a = 0.2;    % per-slot arrival probability
s = 0.5;    % per-slot service completion probability

model = Network('GeoGeo1');

source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'Class1');
source.setArrival(jobClass, Geometric(a));
queue.setService(jobClass, Geometric(s));

model.link(Network.serialRouting(source, queue, sink));

%%
options = SolverNC.defaultOptions;
options.config.slotted = true;    % run on the slot lattice
solver = SolverNC(model, options);
AvgTable = solver.getAvgTable; AvgTable

fprintf('closed form: E[N] = %g, E[T] = %g slots\n', ...
    a*(1-a)/(s-a), (1-a)/(s-a));

% The same numbers straight from the single-queue formula.
r = dqsys_bernoulli1(a, s)
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

5.1.6 dt_geogeo1_loss

The Geo/Geo/1/L loss system, and the loss probability.

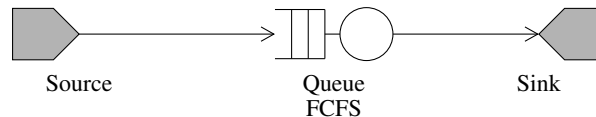


Figure 5.6: Topology of dt_geogeo1_loss: an open network over 1 queueing station.

The example is built and solved as follows.

```
% Geo/Geo/1/L: the discrete-time loss system of Daduna (2001), corollary 2.8.
%
% The buffer holds at most L jobs. An arrival that lands in a slot which
% already holds L jobs is lost, i.e.  $b(n) = 0$  for  $n \geq L$ , which is exactly the
% assumption corollary 2.8 places on the arrival probabilities. The queue
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Geometric(0.2)
Service	Queue – Class1: Geometric(0.5)
Features	finite buffer 4 at Queue
Solvers	NC

Table 5.6: Features of dt_geogeo1_loss.

```
% length law stays the birth-death form of theorem 2.3, now normalized over
% the finite state space, and the loss probability is the stationary
% probability of a full system.

a = 0.2;      % per-slot arrival probability
s = 0.5;      % per-slot service completion probability
L = 4;        % buffer capacity in jobs

model = Network('GeoGeo1L');

source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
sink = Sink(model, 'Sink');

jobClass = OpenClass(model, 'Class1');
source.setArrival(jobClass, Geometric(a));
queue.setService(jobClass, Geometric(s));
queue.setCapacity(L);

model.link(Network.serialRouting(source, queue, sink));

... (4 source lines omitted; full implementation: matlab/examples/discrete/)

AvgTable = solver.getAvgTable; AvgTable

r = dqsys_bernoulli1(a, s, L);
fprintf('queue length law   : %s\n', mat2str(round(r.pmf, 6)));
fprintf('loss probability    : %g\n', r.lossProb);
fprintf('carried throughput   : %g of the %g offered per slot\n', r.throughput, a);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Superposing two slotted streams produces batches, not a slotted stream of the same kind, so a tandem of slotted queues is the first model in which a discrete-time implementation can go wrong while the single queue still looks right. dt_cycle exists for that reason.

Chapter 6

Solver-specific examples

Most examples in this manual are written around a model and let the solver vary. The scripts of this chapter do the opposite: they are written around one solver, and print what it builds rather than only what it concludes. They live under `matlab/examples/solvers/`.

The CTMC solver is the natural subject, because its intermediate objects are the model: the state space, the infinitesimal generator and the stationary distribution are all printable, and the averages are read off them. Printing them is how one checks that a model means what it was intended to mean.

6.0.1 `ctmc_spn_mm1`

The same M/M/1 system written as a queueing model and as a Petri net, and the two generators compared.



Figure 6.1: Topology of `ctmc_spn_mm1`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the net is open, fed by a Source and drained into a Sink.

Nodes	6: Source, Place $\times$ 2, Transition $\times$ 2, Sink
Classes	1: jobs (open)
Arrivals	source – jobs: Exp(0.8)
Features	2 transitions with 1 firing mode each
Solvers	CTMC

Table 6.1: Features of `ctmc_spn_mm1`.

The example is built and solved as follows.

```
%% M/M/1 Queue Modeled as Stochastic Petri Net (SPN) with CTMC Solver
%
% This example demonstrates:
% - Creating an M/M/1 queue as an SPN (not a traditional queueing network)
% - Using CTMC solver for exact analysis of SPN models
% - Comparing SPN results with theoretical M/M/1 performance
% - Verifying that SPN feature support in CTMC works correctly
%
% The SPN model represents an M/M/1 queue as:
```

```

% - Source: generates arrivals (lambda=0.8)
% - Place P: customers waiting in queue
% - Transition T_serve: service start (mu=1.0)
% - Place S: customer in service
% - Transition T_complete: service completion (return to queue)
% - Sink: customers depart after service completion

clear all; close all;

fprintf('=====\\n');
fprintf('M/M/1 Queue as Stochastic Petri Net (SPN)\\n');
fprintf('=====\\n');

lineStart;

% =====
% Create network

... (132 source lines omitted; full implementation: matlab/examples/solvers/)

fprintf('=====\\n');
fprintf('OK CTMC successfully analyzes M/M/1 modeled as SPN\\n');
fprintf('OK SPN feature support in CTMC is working correctly\\n');
fprintf('Differences from theory are due to CTMC state space truncation\\n');
fprintf(' (open networks with infinite arrivals truncated at cutoff=10)\\n');
fprintf('=====\\n');

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

6.0.2 ctmc_spn_vs_nc_closed_qn

A closed network solved exactly by CTMC over its Petri net form and by the normalizing constant method.

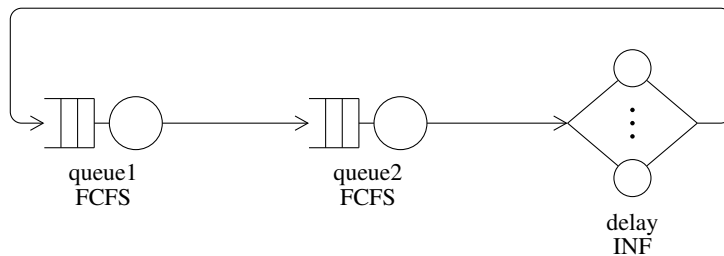


Figure 6.2: Topology of ctmc_spn_vs_nc_closed_qn: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	1: jobs (closed, $N = 3$)
Scheduling	queue1: FCFS; queue2: FCFS; delay: INF
Service	queue1 – jobs: Exp(1) queue2 – jobs: Exp(0.8) delay – jobs: Exp(0.5)

Table 6.2: Features of ctmc_spn_vs_nc_closed_qn.

The example is built and solved as follows.

```

%% Closed Queueing Network: CTMC (SPN) vs NC Solver Comparison
%
% This example demonstrates:
% - Single-class closed queueing network (product-form)
% - Solves the same model two different ways:
%   1. Traditional QN approach: solved with NC (Normalizing Constant) solver
%   2. SPN approach: same network modeled as Petri Net, solved with CTMC

```

```
% - Verifies that both approaches give identical results
% - Demonstrates CTMC's capability on Petri Net models
%
% Network Configuration:
% - Population: N=3 jobs circulating continuously
% - Station 1: Queue (FCFS, mu1=1.0)
% - Station 2: Queue (FCFS, mu2=0.8)
% - Station 3: Delay (think time, mean=2.0)
% - Routing: 1 -> 2 -> 3 -> 1 (cycle)
%
% This is product-form under Jackson's theorem:
% - FCFS disciplines
% - Exponential service times
% - Single job class
% - Product-form solution: pi(n1,n2,n3) = pi1(n1)pi2(n2)pi3(n3)

clear all; close all;

fprintf('=====\\n');

... (209 source lines omitted; full implementation: matlab/examples/solvers/)

fprintf(' - Total population conserved (=3 jobs)\\n');
fprintf(' - System throughput matches\\n');
fprintf(' - Individual station metrics match\\n');
fprintf('\\nOK CTMC successfully models closed QN as SPN\\n');
fprintf('OK Different modeling approaches yield same answers\\n');
fprintf('=====\\n');
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

6.0.3 ctmc_tandem_mm1

The generator of a two-station tandem, its state space and its stationary distribution, printed in full.

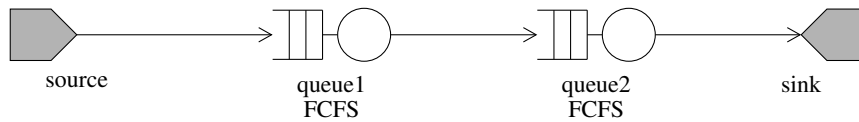


Figure 6.3: Topology of ctmc_tandem_mm1: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: jobs (open)
Scheduling	queue1: FCFS; queue2: FCFS
Arrivals	source – jobs: Exp(0.6)
Service	queue1 – jobs: Exp(1) queue2 – jobs: Exp(1.2)
Solvers	CTMC

Table 6.3: Features of ctmc_tandem_mm1.

The example is built and solved as follows.

```
%% Tandem M/M/1 Queues (Series of Two M/M/1 Stations)
%
% This example demonstrates:
% - Product-form open queueing network (OQN)
% - Series (tandem) configuration: Source -> Queue1 -> Queue2 -> Sink
% - CTMC solver for exact analysis of multi-station networks
% - Verification against theoretical results
% - Comparison with traditional queueing theory
%
% For a tandem M/M/1-M/M/1 system:
```



```

% - Station 1: lambda1 = 0.6, mu1 = 1.0
% - Station 2: lambda2 = 0.6 (same as lambda1, since all jobs from Q1 go to Q2), mu2 = 1.2
% - rho1 = 0.6, rho2 = 0.5
% - System is product-form: pi(n1,n2) = pi1(n1)pi2(n2)

clear all; close all;

fprintf('=====\n');
fprintf('Tandem M/M/1 Queues (Series of Two Stations)\n');
fprintf('=====\n');

lineStart;

% =====
% Define nodes
% =====

... (198 source lines omitted; full implementation: matlab/examples/solvers/)

fprintf('OK Product-form property verified:\n');
fprintf(' - Each station behaves independently as M/M/1\n');
fprintf(' - System decomposition holds\n');
fprintf('OK Results match theoretical predictions\n');
fprintf('OK CTMC feature support is working correctly!\n');
fprintf('=====\n');

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Two of the three scripts build the same system twice, once as a queueing model and once as a stochastic Petri net, and compare the generators. That is the strongest available check on either notation: the two descriptions are independent, and if they agree state by state then neither has been misunderstood.

Chapter 7

Model interchange

A model can be written to, and read back from, a JSON document. The format is the interchange between the codebases and the wire protocol of the LDES engine, which is driven entirely by it: the client serializes `model.json`, the engine solves it and returns `result.json`.

The examples under `matlab/examples/json/` are round-trip fixtures: each builds a model that uses one group of wire keys, writes it, reads it back and checks that the reconstructed model has the same structure. What a round trip must preserve is the declared state of the model; what it must not carry is the derived state, which is recomputed on load.

```
model = json_hetero_servers(); % a model using one group of wire keys
linemodel_save(model, 'model.json');
reloaded = linemodel_load('model.json');
```

7.1 The fixtures

7.1.1 json_balking_retrial

Balking thresholds and retrial orbits over the wire.

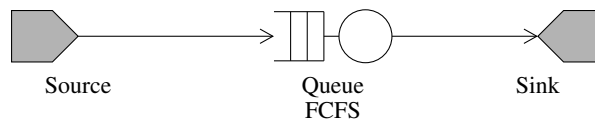


Figure 7.1: Topology of `json_balking_retrial`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	finite buffer 15 at Queue; drop rule {'Class1': 'retrialWithLimit', 'Class2': 'retrialWithLimit'} at Queue

Table 7.1: Features of `json_balking_retrial`.

The example is built and solved as follows.

```
function model = json_balking_retrial()
% Open 2-class network with balking, retrial, and patience.
model = Network('Balking_Retrial');
```

```
source = Source(model, 'Source');
queue  = Queue(model, 'Queue', SchedStrategy.FCFS);
sink   = Sink(model, 'Sink');

queue.setCapacity(15);

class1 = OpenClass(model, 'Class1');
class2 = OpenClass(model, 'Class2');

source.setArrival(class1, Exp(1));
source.setArrival(class2, Exp(0.5));
queue.setService(class1, Exp(2));
queue.setService(class2, Exp(3));

% Class1: balking based on queue length
queue.setBalking(class1, BalkingStrategy.QUEUE_LENGTH, ...
    {[5, 10, 0.3], {11, Inf, 1.0}});

% Class1: patience (reneging)
queue.setPatience(class1, Exp(0.1));

% Class2: retrial with max attempts
queue.setRetrial(class2, Exp(0.5), 3);

P = model.initRoutingMatrix();
P{class1} = Network.serialRouting(source, queue, sink);
P{class2} = Network.serialRouting(source, queue, sink);
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

7.1.2 json_classcap_droprule

Per-class capacities and drop rules.

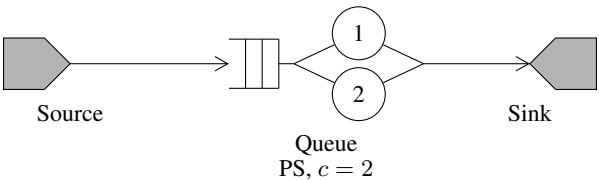


Figure 7.2: Topology of json_classcap_droprule: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: PS ($c = 2$)
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Queue – Class1: Exp(3); Class2: Exp(2)
Features	finite buffer 20 at Queue; load-dependent rates at Queue; drop rule {'Class1': 'drop', 'Class2': 'blockingAfterService'} at Queue

Table 7.2: Features of json_classcap_droprule.

The example is built and solved as follows.

```
function model = json_classcap_droprule()
% Open 2-class network with per-class capacity, drop rules, and load-dependence.
model = Network('ClassCap_DropRule');

source = Source(model, 'Source');
```

```
queue = Queue(model, 'Queue', SchedStrategy.PS);
sink   = Sink(model, 'Sink');

queue.setNumberOfServers(2);
queue.setCapacity(20);

class1 = OpenClass(model, 'Class1');
class2 = OpenClass(model, 'Class2');

source.setArrival(class1, Exp(1));
source.setArrival(class2, Exp(0.5));
queue.setService(class1, Exp(3));
queue.setService(class2, Exp(2));

% Per-class capacity
queue.classCap(class1.index) = 8;
queue.classCap(class2.index) = 15;

% Per-class drop rules. WAITQ is not usable here: no solver honours "wait
% upstream" for an open class at a plain finite capacity (CTMC drops the
% arrival, JMT blocks at the source and ignores the cap), so refreshCapacity

    ... (6 source lines omitted; full implementation: matlab/examples/json/)

P = model.initRoutingMatrix();
P{class1} = Network.serialRouting(source, queue, sink);
P{class2} = Network.serialRouting(source, queue, sink);
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

7.1.3 json_fcr_details

The finite capacity region block, with its members, limits and drop rules.

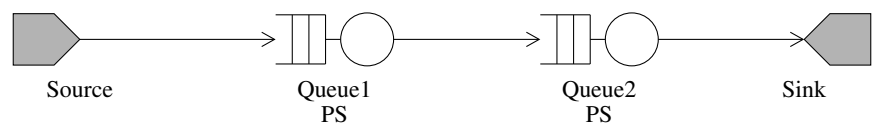


Figure 7.3: Topology of json_fcr_details: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: PS; Queue2: PS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Queue1 – Class1: Exp(3); Class2: Exp(2) Queue2 – Class1: Exp(4); Class2: Exp(3)

Table 7.3: Features of json_fcr_details.

The example is built and solved as follows.

```
function model = json_fcr_details()
% Open 2-class network with a Finite Capacity Region.
model = Network('FCR_Details');

source = Source(model, 'Source');
queue1 = Queue(model, 'Queue1', SchedStrategy.PS);
queue2 = Queue(model, 'Queue2', SchedStrategy.PS);
sink   = Sink(model, 'Sink');
```

```

class1 = OpenClass(model, 'Class1');
class2 = OpenClass(model, 'Class2');

source.setArrival(class1, Exp(1));
source.setArrival(class2, Exp(0.5));
queue1.setService(class1, Exp(3));
queue1.setService(class2, Exp(2));
queue2.setService(class1, Exp(4));
queue2.setService(class2, Exp(3));

P = model.initRoutingMatrix();
P{class1} = Network.serialRouting(source, queue1, queue2, sink);
P{class2} = Network.serialRouting(source, queue1, queue2, sink);
model.link(P);

% Finite Capacity Region
fcr = Region({queue1, queue2}, {class1, class2});
fcr.setGlobalMaxJobs(15);
fcr.setClassMaxJobs(class1, 8);
fcr.setClassMaxJobs(class2, 10);
fcr.setClassWeight(class1, 1.0);
fcr.setClassWeight(class2, 2.0);
fcr.setClassSize(class1, 1);
fcr.setClassSize(class2, 3);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

7.1.4 json_hetero_servers

Heterogeneous server types at one station, their per-type service rates and the policy that orders them.

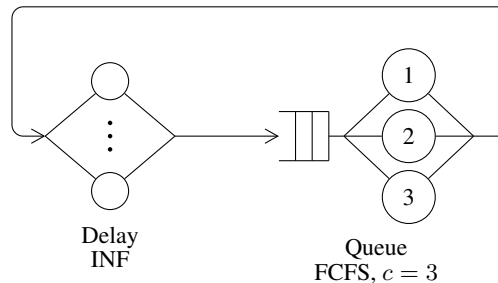


Figure 7.4: Topology of json_hetero_servers: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(2) Queue – Class1: Exp(3); Class2: Exp(2)

Table 7.4: Features of json_hetero_servers.

The example is built and solved as follows.

```

function model = json_hetero_servers()
% Closed 2-class network with heterogeneous servers.
model = Network('HeteroServers');

delay = Delay(model, 'Delay');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
queue.setNumberOfServers(3);

```

```

class1 = ClosedClass(model, 'Class1', 3, delay);
class2 = ClosedClass(model, 'Class2', 2, delay);

delay.setService(class1, Exp(1));
delay.setService(class2, Exp(2));

% Default service (required before hetero setup)
queue.setService(class1, Exp(3));
queue.setService(class2, Exp(2));

% Heterogeneous server types
fast = ServerType('Fast', 2, {class1, class2});
slow = ServerType('Slow', 1, {class1});

queue.addServerType(fast);
queue.addServerType(slow);

queue.setHeteroSchedPolicy(HeteroSchedPolicy.ORDER);

... (4 source lines omitted; full implementation: matlab/examples/json/)

P = model.initRoutingMatrix();
P{class1} = Network.serialRouting(delay, queue);
P{class2} = Network.serialRouting(delay, queue);
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

7.1.5 json_join_deadline

Join deadlines and the quorum they imply.

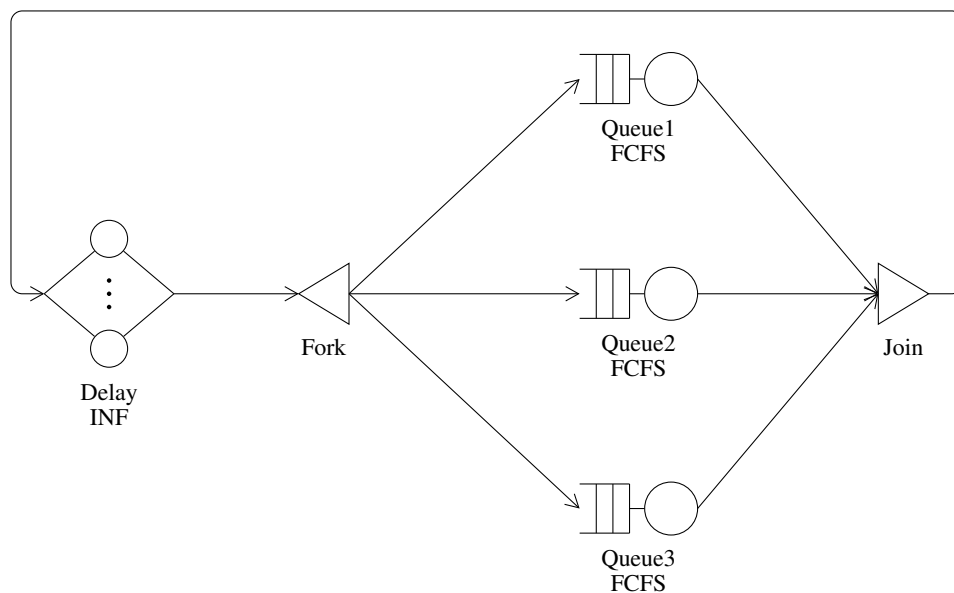


Figure 7.5: Topology of json_join_deadline: a closed network over 4 queueing stations with a fork-join block.

The example is built and solved as follows.

```

function model = json_join_deadline()
% Closed 1-class fork-join with quorum join and deadline.
model = Network('JoinDeadline');

```

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(3) Queue2 – Class1: Exp(4) Queue3 – Class1: Exp(2)

Table 7.5: Features of `json_join_deadline`.

```

delay = Delay(model, 'Delay');
fork = Fork(model, 'Fork');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
queue3 = Queue(model, 'Queue3', SchedStrategy.FCFS);
join = Join(model, 'Join', fork);

class1 = ClosedClass(model, 'Class1', 5, delay);
class1.deadline = 5.0;

delay.setService(class1, Exp(1));
queue1.setService(class1, Exp(3));
queue2.setService(class1, Exp(4));
queue3.setService(class1, Exp(2));

join.setStrategy(class1, JoinStrategy.PARTIAL);
join.setRequired(class1, 2);

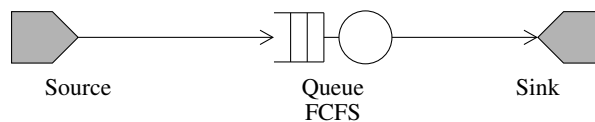
P = model.initRoutingMatrix();
P{class1} = zeros(6);
P{class1}(delay, fork) = 1;
P{class1}(fork, queue1) = 1;
P{class1}(fork, queue2) = 1;
P{class1}(fork, queue3) = 1;
P{class1}(queue1, join) = 1;
P{class1}(queue2, join) = 1;
P{class1}(queue3, join) = 1;
P{class1}(join, delay) = 1;
model.link(P);
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

7.1.6 `json_signal_classes`

G-network signal classes.

Figure 7.6: Topology of `json_signal_classes`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Signal1 (signal)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(2); Signal1: Exp(0.5)
Service	Queue – Class1: Exp(5); Signal1: Immediate

Table 7.6: Features of `json_signal_classes`.

The example is built and solved as follows.

```
function model = json_signal_classes()
% Open network with G-network negative signal.
model = Network('SignalClasses');

source = Source(model, 'Source');
queue  = Queue(model, 'Queue', SchedStrategy.FCFS);
sink   = Sink(model, 'Sink');

class1 = OpenClass(model, 'Class1');
signal1 = OpenSignal(model, 'Signal1', SignalType.NEGATIVE);
signal1 = signal1.forJobClass(class1);
signal1.setRemovalPolicy(RemovalPolicy.RANDOM);

source.setArrival(class1, Exp(2));
source.setArrival(signal1, Exp(0.5));
queue.setService(class1, Exp(5));
queue.setService(signal1, Immediate());

P = model.initRoutingMatrix();
P{class1}(source, queue) = 1;
P{class1}(queue, sink) = 1;
P{signal1}(source, queue) = 1;
P{signal1}(queue, sink) = 1;
model.link(P);
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

The fixtures are grouped by wire key rather than by model family on purpose: a key that no fixture exercises is a key that no round trip tests, and grouping them this way makes the gap visible.

Chapter 8

Inference and estimation

Inference runs the modeling process backwards: given measurements of a system – arrival rates, utilizations, response times, or a raw trace – it estimates the model parameters that would produce them. The unknown parameters are marked in the model by a distribution with a NaN parameter, and the estimator fills them in.

The estimators divide into three families: likelihood-based (MLE and its quasi-likelihood variant), recursive (the extended Kalman filter and its smoother, which track a parameter that drifts), and Bayesian (sampling and variational, which return a posterior rather than a point estimate). The changepoint scripts test all three against a workload that changes abruptly, which is where a recursive estimator earns its cost. The reference material is in the Inference chapter of the LINE user manual.

The scripts live under `matlab/examples/inference/`, and each one generates a synthetic dataset from known parameters, so the estimate can be compared against the truth.

```
% The unknown demands are declared as NaN and estimated from the data
node{2}.setService(jobclass{1}, Exp(NaN)); % true demand 0.1
node{2}.setService(jobclass{2}, Exp(NaN)); % true demand 0.3

% Synthetic dataset consistent with the model
n = 100;
util_samples = 0.25*ones(n,1) + randn(n,1)*0.02;
respt1_samples = 0.1./(1 - util_samples);
respt2_samples = 0.3./(1 - util_samples);
```

8.1 The estimators

8.1.1 `est_ekf_changepoint`

The filter under a changepoint in the workload.

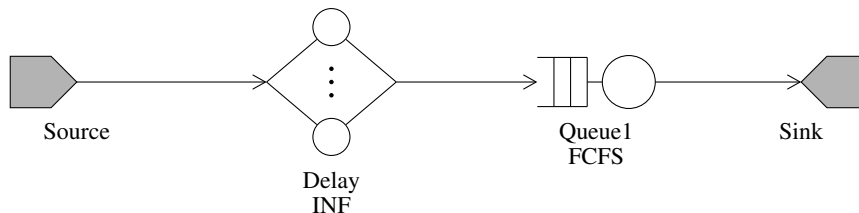


Figure 8.1: Topology of `est_ekf_changepoint`: an open network over 2 queueing stations.

The example is built and solved as follows.

```
%% Example: EKF sliding-window change-point detection (open network)
```

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(3.616)

Table 8.1: Features of `est_ekf_changepoint`.

```
%
% Service demand changes from D=0.3 to D=0.6 at t=100.
% A sliding window of size W feeds the EKF, which is warm-started
% from the previous window's estimate for stable tracking.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = OpenClass(model, 'Class1', 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN));
node{3}.setArrival(jobclass{1}, Exp(1.0));

P = model.initRoutingMatrix;
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);

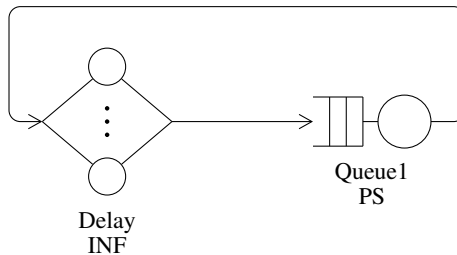
... (78 source lines omitted; full implementation: matlab/examples/inference/)

xline(changeT, 'k:', 'Change point', 'LineWidth', 1);
xlabel('Time');
ylabel('Estimated Demand');
title('EKF Sliding-Window Change-Point Detection (warm-start)');
legend('EKF estimate', 'Location', 'southeast');
grid on;
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.2 `est_ekf_closed`

The closed-network filter.

Figure 8.2: Topology of `est_ekf_closed`: a closed network over 2 queueing stations.

The example is built and solved as follows.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(2.5)
Solvers	MVA

Table 8.2: Features of `est_ekf_closed`.

```
% Example: EKF estimation with autoMethod selection
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
model.link(P);

%% Generate synthetic dataset
n = 100;
ts = 1:n;
arvr_samples = 1.5*ones(n,1) - rand(n,1)*0.1;
util_samples = 0.4*arvr_samples;
respt_samples = 0.4./(1 - util_samples);

%% Create SampledMetric objects

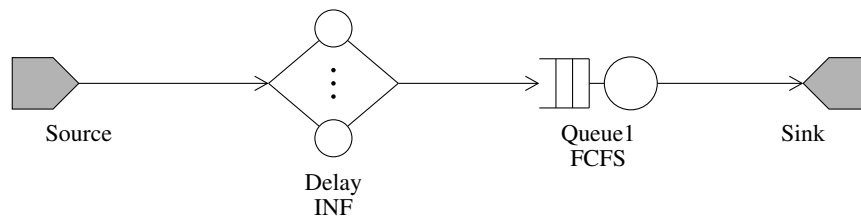
... (33 source lines omitted; full implementation: matlab/examples/inference/)

%% Solve model
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.3 `est_ekf_open`

An extended Kalman filter tracking the demands of an open network online.

Figure 8.3: Topology of `est_ekf_open`: an open network over 2 queueing stations.

The example is built and solved as follows.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(4.123)
Solvers	MVA

Table 8.3: Features of `est_ekf_open`.

```

%% Example: EKF estimation on an open network
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = OpenClass(model, 'Class1', 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated (true demand = 0.4)
node{3}.setArrival(jobclass{1}, Exp(1.0)); % arrival rate = 1.0

P = model.initRoutingMatrix;
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);

%% Generate synthetic dataset consistent with model
% True demand D=0.4, arrival rate lambda=1.0
% True utilization U = lambda*D = 0.4
% True response time R = D/(1-U) = 0.4/0.6 = 0.667

... (23 source lines omitted; full implementation: matlab/examples/inference/)

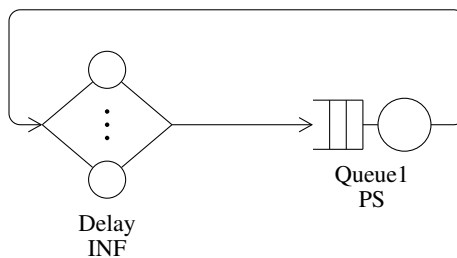
%% Solve model
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.4 `est_erps_closed`

The closed-network smoother.

Figure 8.4: Topology of `est_erps_closed`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.4: Features of `est_erps_closed`.

The example is built and solved as follows.

```
%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 3, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
P{2} = [0,1; 1,0];
model.link(P);

%% Generate random dataset for utilization and average arrival rate
n = 1000;
ts = 1:n;
ts2 = 1:1:n;
arvr1_samples = ones(n,1)-rand(n,1)*0.15;
arvr2_samples = 2*ones(n,1)-rand(n,1)*0.15;
util_samples = 0.1 * arvr1_samples + 0.3 * arvr2_samples;

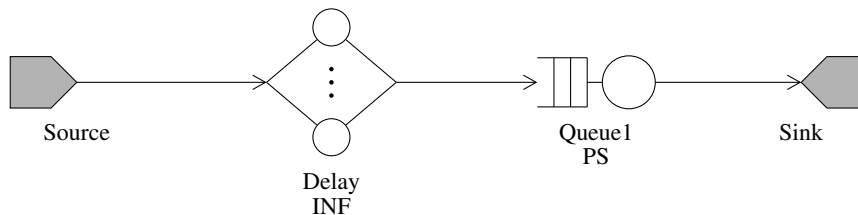
... (30 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.5 `est_erps_open`

The Rauch-Tung-Striebel smoother on an open network.

Figure 8.5: Topology of `est_erps_open`: an open network over 2 queueing stations.

The example is built and solved as follows.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: PS
Arrivals	Source – Class1: Exp(0.1); Class2: Exp(0.05)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10); Class2: HyperExp(0.5,0.5;2,8) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.5: Features of `est_erps_open`.

```

%% Example: ERPS estimation on an open network
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);

node{1}.setService(jobclass{1}, HyperExp(0.5, 3.0, 10.0));
node{1}.setService(jobclass{2}, HyperExp(0.5, 2.0, 8.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated
node{3}.setArrival(jobclass{1}, Exp(0.1));
node{3}.setArrival(jobclass{2}, Exp(0.05));

P = model.initRoutingMatrix;
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
P{2,2} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);

... (32 source lines omitted; full implementation: matlab/examples/inference/)

%% Solve model
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.6 `est_mcmc_closed`

Markov chain Monte Carlo sampling of the posterior over the demands.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(0.5703); Class2: Exp(0.5703)
Solvers	MVA

Table 8.6: Features of `est_mcmc_closed`.

The example is built and solved as follows.

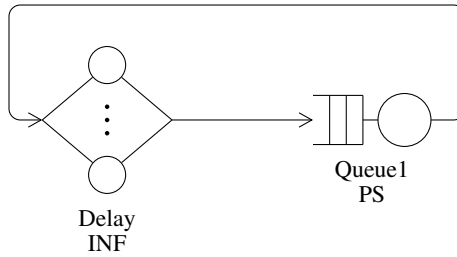


Figure 8.6: Topology of `est_mcmc_closed`: a closed network over 2 queueing stations.

```
%% Example: MCMC estimation on a closed network
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 3, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
P{2} = [0,1; 1,0];
model.link(P);

%% Generate synthetic dataset
n = 500;
ts = 1:n;

... (14 source lines omitted; full implementation: matlab/examples/inference/)

%% Solve model
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

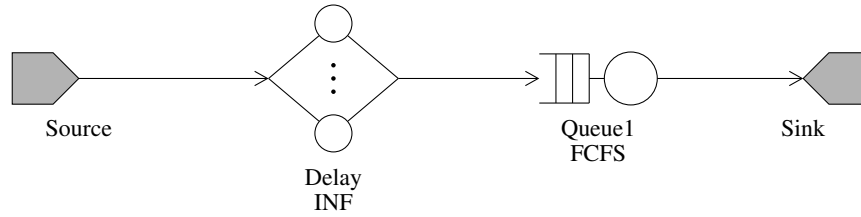
8.1.7 `est_mle_open`

Maximum likelihood estimation of the service demands of an open network from arrival rate, utilization and response time samples.

The example is built and solved as follows.

```
%% Example: MLE estimation on an open network
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');
```

Figure 8.7: Topology of `est_mle_open`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(7.962); Class2: Exp(4.77)
Solvers	MVA

Table 8.7: Features of `est_mle_open`.

```

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated (true demand = 0.1)
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated (true demand = 0.3)
node{3}.setArrival(jobclass{1}, Exp(1.0)); % arrival rate = 1.0
node{3}.setArrival(jobclass{2}, Exp(0.5)); % arrival rate = 0.5

P = model.initRoutingMatrix;
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
P{2,2} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);

... (34 source lines omitted; full implementation: matlab/examples/inference/)

%% Solve model
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.8 `est_qmle_closed`

Quasi-likelihood estimation on a closed network.

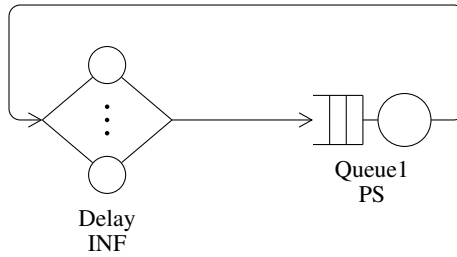
The example is built and solved as follows.

```

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 2, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 3, node{1}, 0);

```


Figure 8.8: Topology of `est_qmle_closed`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(9.553); Class2: Exp(4.837)
Solvers	MVA

Table 8.8: Features of `est_qmle_closed`.

```

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
P{2} = [0,1; 1,0];
model.link(P);

%% Generate random dataset for queue-lengths
n = 1000;
ts = 1:n;
% synthetic queue-length data consistent with PS demands of 0.2 and 0.4
qlen1_samples = 0.3 * ones(n,1) + rand(n,1) * 0.1;
qlen2_samples = 0.8 * ones(n,1) + rand(n,1) * 0.1;

... (17 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.9 `est_trace_closed`

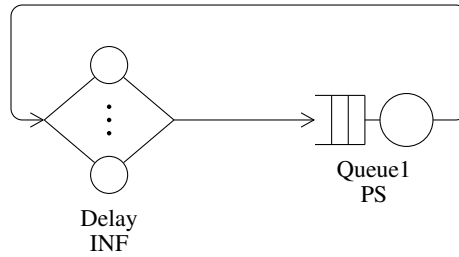
Estimation from a raw arrival and completion trace rather than from aggregate samples. The example is built and solved as follows.

```

%% Example: Compare trace-based estimators (MLPS, FMLPS, Gibbs) on a PS station
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
N = 5; % population
model = Network('model');

```

Figure 8.9: Topology of `est_trace_closed`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(5.43)
Solvers	MVA, CTMC

Table 8.9: Features of `est_trace_closed`.

```

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
model.link(P);

%% Generate synthetic trace data
n = 200;
arrival_times = sort(rand(n,1) * 100);
response_times = 0.5 + rand(n,1) * 0.3;
tput_samples = ones(n,1) * (N / (1.0 + 0.5)); % approximate throughput

%% Create trace-format SampledMetric objects
arvData = SampledMetric(MetricType.ArR, arrival_times, arrival_times, node{2}, jobclass{1});

... (63 source lines omitted; full implementation: matlab/examples/inference/)

%% Solve model with final estimates
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.10 `est_ubo_changepoint`

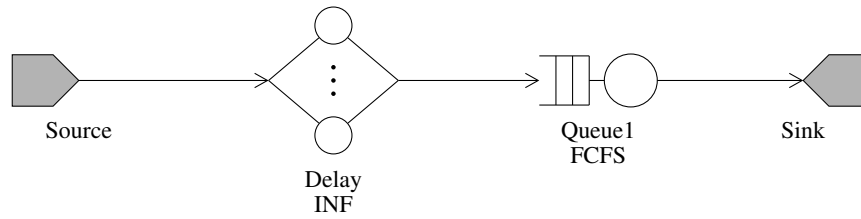
The variational estimator under a changepoint.

The example is built and solved as follows.

```

%% Example: UBO sliding-window change-point detection (open network)
%
% Service demand changes from D=0.3 to D=0.6 at t=100.
% A sliding window of size W is used to estimate demands online.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

```

Figure 8.10: Topology of `est_ubo_changepoint`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3.338); Class2: Exp(5.012)

Table 8.10: Features of `est_ubo_changepoint`.

```
%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN));
node{2}.setService(jobclass{2}, Exp(NaN));
node{3}.setArrival(jobclass{1}, Exp(1.0));
node{3}.setArrival(jobclass{2}, Exp(0.5));

... (96 source lines omitted; full implementation: matlab/examples/inference/)

subplot(2,1,2);
plot(tAxis, est2, 'b-', 'LineWidth', 1.5); hold on;
yline(D2, 'r--', 'LineWidth', 1);
xline(changeT, 'k:', 'Change point', 'LineWidth', 1);
xlabel('Time'); ylabel('Demand'); title('Class 2 (constant D=0.2)');
grid on;
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.11 `est_ubo_closed`

The closed-network version.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.11: Features of `est_ubo_closed`.

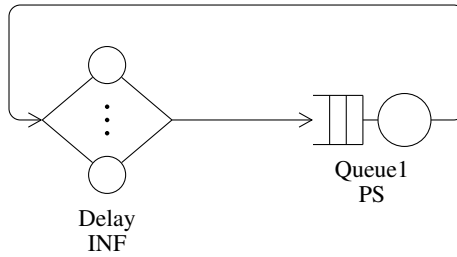


Figure 8.11: Topology of `est_ubo_closed`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
P{2} = [0,1; 1,0];
model.link(P);

%% Generate random dataset for utilization and average arrival rate
n = 30; % 30 measured points
ts = 1:n;
ts2 = 1:1:n;
arvr2_samples = ones(n,1)-rand(n,1)*0.15;
arvr1_samples = 2*ones(n,1)-rand(n,1)*0.15;
util_samples = 0.1 * arvr1_samples + 0.3 * arvr2_samples;

... (26 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

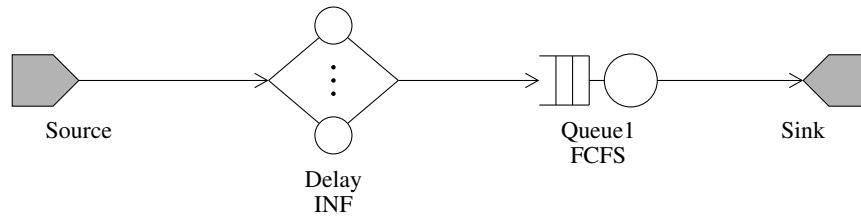
8.1.12 `est_ubo_open`

Variational Bayesian estimation on an open network.

The example is built and solved as follows.

```
clearvars -except exampleName;
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');
```

Figure 8.12: Topology of `est_ubo_open`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(0.1); Class2: Exp(0.05)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10); Class2: HyperExp(0.5,0.5;2,8) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.12: Features of `est_ubo_open`.

```

jobclass{1} = OpenClass(model, 'Class1', 0);
jobclass{2} = OpenClass(model, 'Class2', 0);

node{1}.setService(jobclass{1}, HyperExp(0.5, 3.0, 10.0));
node{1}.setService(jobclass{2}, HyperExp(0.5, 2.0, 8.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated
node{3}.setArrival(jobclass{1}, Exp(0.1));
node{3}.setArrival(jobclass{2}, Exp(0.05));

P = model.initRoutingMatrix;
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
P{2,2} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);

%% Generate random dataset for utilization, arrival rate, and response time
n = 1000;
ts = 1:n;

... (29 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.13 `est_ubo_variants_closed`

The variants of the variational estimator, side by side.

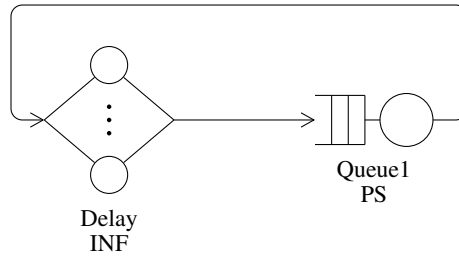
The example is built and solved as follows.

```

%% Example: Compare UBO and MLE estimators on a closed network
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

%% define model
model = Network('model');

```

Figure 8.13: Topology of `est_ubo_variants_closed`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(11.21); Class2: Exp(4.547)
Solvers	MVA

Table 8.13: Features of `est_ubo_variants_closed`.

```

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 3, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
P{2} = [0,1; 1,0];
model.link(P);

%% Generate synthetic dataset
n = 1000;
ts = 1:n;

... (45 source lines omitted; full implementation: matlab/examples/inference/)

%% Solve model with final estimates
solver{1} = MVA(model);
fprintf(1, '\nSOLVER: %s\n', solver{1}.getName());
AvgTable{1} = solver{1}.getAvgTable();
AvgTable{1}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.14 `est_ubr_changepoint`

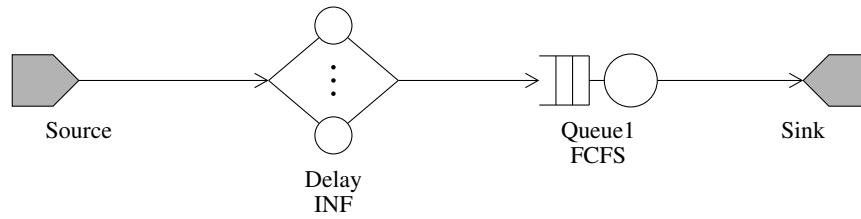
The regression estimator under a changepoint.

The example is built and solved as follows.

```

%% Example: UBR sliding-window change-point detection (open network)
%
% Service demand changes from D=0.3 to D=0.6 at t=100.
% A sliding window of size W is used to estimate demands online.
%
% Copyright (c) 2012-2026, Imperial College London
% All rights reserved.

```

Figure 8.14: Topology of `est_ubr_changepoint`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(1.665)

Table 8.14: Features of `est_ubr_changepoint`.

```

%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');

jobclass{1} = OpenClass(model, 'Class1', 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{2}.setService(jobclass{1}, Exp(NaN));
node{3}.setArrival(jobclass{1}, Exp(1.0));

P = model.initRoutingMatrix;
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];
model.link(P);

... (75 source lines omitted; full implementation: matlab/examples/inference/)

xline(changeT, 'k:', 'Change point', 'LineWidth', 1);
xlabel('Time');
ylabel('Estimated Demand');
title('UBR Sliding-Window Change-Point Detection');
legend('UBR estimate', 'Location', 'southeast');
grid on;

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.15 `est_ubr_closed`

The closed-network version.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(2.502); Class2: Exp(13.93)
Solvers	MVA

Table 8.15: Features of `est_ubr_closed`.

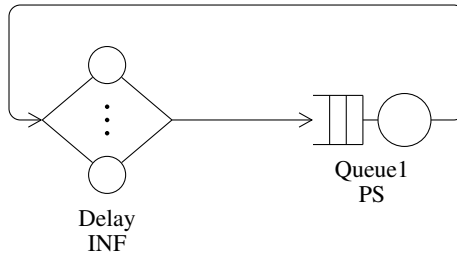


Figure 8.15: Topology of `est_ubr_closed`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
%% define model
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);

node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{2}.setService(jobclass{2}, Exp(NaN)); % NaN = to be estimated

P = model.initRoutingMatrix;
P{1} = [0,1; 1,0];
P{2} = [0,1; 1,0];
model.link(P);

%% Generate random dataset for utilization and average arrival rate
n = 1000;
ts = 1:n;
ts2 = 1:1:n;
arvr1_samples = 2*ones(n,1)-rand(n,1)*0.15;
arvr2_samples = 3*ones(length(ts2),1)-rand(length(ts2),1)*0.25;
util_samples = ones(n,1)-rand(n,1)*0.05;

... (23 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

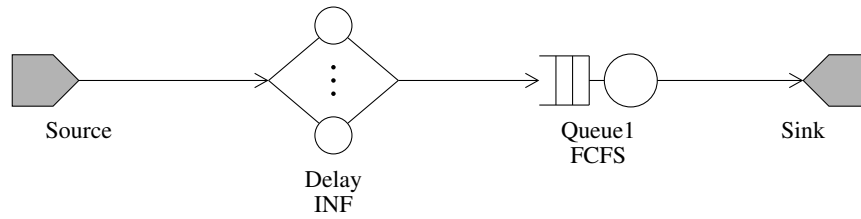
8.1.16 `est_ubr_open`

The regression-based estimator on an open network.

The example is built and solved as follows.

```
clearvars -except exampleName;
model = Network('model');

node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
node{3} = Source(model, 'Source');
node{4} = Sink(model, 'Sink');
```


Figure 8.16: Topology of `est_ubr_open`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(0.1)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10) Queue1 – Class1: Exp(1.976)
Solvers	MVA

Table 8.16: Features of `est_ubr_open`.

```

jobclass{1} = OpenClass(model, 'Class1', 0);

node{1}.setService(jobclass{1}, HyperExp(0.5,3.0,10.0));
node{2}.setService(jobclass{1}, Exp(NaN)); % NaN = to be estimated
node{3}.setArrival(jobclass{1}, Exp(0.1));

M = model.getNumberOfStations();
K = model.getNumberOfClasses();

P = cell(K,K);
P{1,1} = [0,1,0,0; 0,0,0,1; 1,0,0,0; 0,0,0,0];

model.link(P);

%% Generate random dataset for utilization and average arrival rate
n = 1000;
ts = 1:n; % timestamps at which the system is sampled
arvrates_samples = 2*ones(n,1)-rand(n,1)*0.15; % sampled arrival rate values

... (29 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1,length(solver));
for s=1:length(solver)
    fprintf(1,'SOLVER: %s\n',solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

8.1.17 `est_vi_closed`

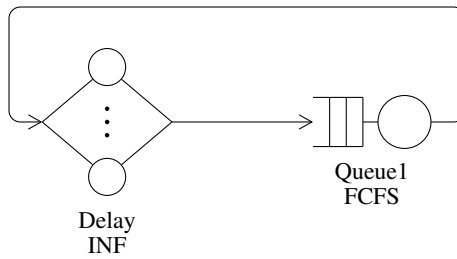
Variational inference over the transition counts of a closed loop, from noisy queue-length readings, returning a Gamma posterior per rate rather than a point estimate.

The example is built and solved as follows.

```

% Variational inference for Markovian queueing networks, on a closed loop.
%
% Method 'vi' (I. Perez, G. Casale, Adv. Appl. Prob. 53(3), 2021) infers
% service rates from NOISY QUEUE-LENGTH READINGS taken over time: each reading
% is exact with probability 1-epsilon and uniform over the remaining feasible
% values otherwise. Unlike the other estimators it returns a conjugate Gamma
% POSTERIOR per rate, not only a point estimate.

```

Figure 8.17: Topology of `est_vi_closed`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Delay: INF; Queue1: FCFS
Service	Delay – Class1: Exp(0.5) Queue1 – Class1: Exp(1.994)
Solvers	MVA

Table 8.17: Features of `est_vi_closed`.

```
%
% It reads the queue lengths of EVERY station, not only the estimated one: the
% transition counts the method is written in are pinned by the whole picture.

%% define model, with the queue rate to be estimated
N = 10;
model = Network('model');
node{1} = Delay(model, 'Delay');
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
jobclass{1} = ClosedClass(model, 'Class1', N, node{1}, 0);
node{1}.setService(jobclass{1}, Exp(0.5));
node{2}.setService(jobclass{1}, Exp(2.0)); % starting point of the estimate
model.link(Network.serialRouting(node{1}, node{2}));

%% queue-length readings, one per unit time, 10% of them faulty
ts = (1:10)';
qlen = [1 2 3 2 4 3 5 4 3 4]';

%% estimate the service rate of the queue

... (22 source lines omitted; full implementation: matlab/examples/inference/)

AvgTable = cell(1, length(solver));
for s=1:length(solver)
    fprintf(1, 'SOLVER: %s\n', solver{s}.getName());
    AvgTable{s} = solver{s}.getAvgTable();
    AvgTable{s}
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Chapter 9

Optimization

Besides evaluating a given model, LINE can solve design problems on queueing networks. While the solvers answer evaluation questions (given a model, what are its performance metrics?), the optimization layer answers design questions: how many servers, what service rates, which routing probabilities, or what job populations optimize a cost or performance objective subject to service-level constraints. The reference material on decision variables, objectives, constraints and solvers is in the Optimization chapter of the LINE user manual; this chapter collects the worked examples.

The relevant scripts are included under the `examples/opt/` and each runs in seconds. The general structure of an optimization script consists of four blocks:

1. Definition of the LINE network model
2. Declaration of the decision variables
3. Specification of the objective and constraints
4. Solution

Cost coefficients are supplied as dictionary objects created with `configureDictionary`, keyed by station name, and solver options are collected in an `opt.LineOptSolverOptions` object whose setters return the object itself.

9.1 Sizing an M/M/c queue

The M/M/c queue is a classic model of a service center with c parallel servers fed by a common infinite-capacity buffer. In this example, we wish to find the minimum number of servers such that the utilization does not exceed 50%. Jobs arrive at rate $\lambda = 3$ job/s and each server works at rate $\mu = 1$ job/s, hence the utilization with c servers is $\rho = \lambda/(c\mu) = 3/c$. It is known from theory that the optimum is therefore $c^* = 6$; we wish to recover this value. The following script (`opt_server_sizing.m`) solves the problem with the default differential evolution solver, at a cost of 10 units per server:

```
%% Block 1: LINE model (Source -> Queue -> Sink)
model = Network('MMc');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0)); % lambda = 3
queue.setService(jobs, Exp(1.0)); % mu = 1 per server
model.link(Network.serialRouting(source, queue, sink));

%% Block 2: decision variables
problem = opt.OptimizationProblem(model);
```

```

problem.addVariable(opt.ServerAllocation(queue, [1 10]));

%% Block 3: objective and constraints
serverCost = configureDictionary('string','double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));

%% Block 4: solution
options = opt.LineOptSolverOptions(); options.setSeed(42);
result = problem.solve(options);

fprintf('Optimal servers : %d\n', result.getVariableValue('Server_servers'));
fprintf('Cost           : %.1f\n', result.objectiveValue);

```

Obtaining the following output:

```

Optimal servers : 6
Cost           : 60.0
Feasible       : True
LINE evaluations: 10

```

Note that only 10 distinct configurations required a LINE solve: the evaluator caches results at the level of the decoded configuration, so the hundreds of candidate vectors explored by differential evolution collapse onto the 10 possible server counts.

9.2 Exact sizing by bisection

The sizing problem of Tutorial 1 has a special structure: feasibility is monotone in the decision variable, since adding servers can only decrease utilization. In such cases `BisectionSolver` finds the exact optimum in $O(\log n)$ LINE evaluations. Model and problem definition are identical to Tutorial 1 (`opt_bisection_sizing.m`); only the solution block changes:

```

result = opt.BisectionSolver(problem).solve();

```

Obtaining the following output:

```

Optimal servers : 6
Cost           : 60.0
LINE evaluations: 4 (vs hundreds for DE)

```

9.3 Continuous service rate tuning

We now choose the cheapest service rate, a continuous decision, meeting a response time SLA on an M/M/1 queue. With $\lambda = 3$, the response time is $R = 1/(\mu - 3)$, so $R \leq 0.5$ requires $\mu \geq 5$; the rate is billed at 20 cost units per unit, hence the optimum is $\mu^* = 5$ at cost 100. In the script (`opt_service_rate.m`), the model is an M/M/1 queue built as in Tutorial 1 with initial rate $\mu = 4$, and the problem is declared as:

```

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServiceRate(queue, jobs, [3.5 8.0]));
rateCost = configureDictionary('string','double');
rateCost('Server') = 20.0;
problem.setObjective(opt.MinimizeCost([], rateCost, [], {}));
problem.addConstraint(opt.ResponseTimeConstraint(queue, jobs, 0.5));

options = opt.LineOptSolverOptions();
options.setSeed(42); options.setMaxIterations(60);
result = problem.solve(options);

```

Obtaining the following output:

```
Optimal rate      : 5.001 (theory: 5.0)
Cost              : 100.0
Feasible         : True
```

9.4 Load balancing across heterogeneous servers

An arrival stream of rate $\lambda = 2$ must be split between a fast queue ($\mu_1 = 4$) and a slow queue ($\mu_2 = 2.5$) so that the mean end-to-end response time is minimized. Modeling each queue as M/M/1, the mean number of jobs in the system is $N(p) = \frac{\rho_1}{1-\rho_1} + \frac{\rho_2}{1-\rho_2}$ with $\rho_1 = p\lambda/\mu_1$ and $\rho_2 = (1-p)\lambda/\mu_2$, and by Little's law the system response time is $N(p)/\lambda$. Setting $dN/dp = 0$ yields $p^* \approx 0.743$ with response time 0.4264: the optimal policy does not saturate the fast server. The script (`opt_load_balancing.m`) declares the routing split as the decision variable:

```
problem = opt.OptimizationProblem(model);
problem.addVariable(opt.RoutingProbabilities(jobs, source, {fast, slow}));
problem.setObjective(opt.MinimizeSystemResponseTime(jobs));

options = opt.LineOptSolverOptions();
options.setSeed(42); options.setMaxIterations(60);
result = problem.solve(options);
probs = result.getVariableValue('Jobs_routing_from_S');
```

Obtaining the following output:

```
Fraction to Fast: 0.736 (theory: 0.743)
Fraction to Slow: 0.264
System RT       : 0.4248 (theory: 0.4264)
```

The system response time is computed by Little's law from the chain queue lengths and throughput, which remains correct on parallel topologies.

9.5 Population sizing in a closed network

An interactive system is modeled as a closed network with a Delay station (mean think time 1) and a processor sharing server (rate 2). We seek the largest number of concurrent users the system can sustain while the server response time stays within an SLA of 2 time units. Feasibility is monotone non-increasing in the population, so bisection applies with the `max_feasible` direction (`opt_population_sizing.m`):

```
model = Network('Interactive');
think = Delay(model, 'Think');
server = Queue(model, 'AppServer', SchedStrategy.PS);
users = ClosedClass(model, 'Users', 1, think);
think.setService(users, Exp(1.0));
server.setService(users, Exp(2.0));
model.link(Network.serialRouting(think, server));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.JobPopulation(users, [1 50]));
problem.setObjective(opt.MinimizeCost([], [], [], {})); % feasibility sizing
problem.addConstraint(opt.ResponseTimeConstraint(server, users, 2.0));

result = opt.BisectionSolver(problem, 'max_feasible').solve();
```

Obtaining the following output:

```
Max users      : 5
Feasible       : True
```

```
LINE evaluations: 6
```

9.6 Robust sizing under workload scenarios

Capacity decisions should often hold under load uncertainty. Here the server pool of Tutorial 1 is sized so that the utilization SLA holds both under the average load ($\lambda = 3$) and under a peak-load scenario ($\lambda = 4.5$). Scenario models share the node and class names of the base model and are registered with `addScenario`; the solver enforces every constraint on all scenarios (`opt_robust_sizing.m`):

```
base = buildMMc(3.0);      % average load
peak = buildMMc(4.5);     % peak load scenario
nodes = base.getNodes(); queue = nodes{2};

problem = opt.OptimizationProblem(base);
problem.addVariable(opt.ServerAllocation(queue, [1 12]));
serverCost = configureDictionary('string','double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));
problem.addScenario(peak, 1.0);

result = opt.BisectionSolver(problem).solve();
```

Obtaining the following output:

```
Robust servers : 9 (6 for average load only)
Cost           : 90.0
Feasible       : True
```

The average load alone needs 6 servers; the peak scenario pushes the answer to $\lceil 4.5/0.5 \rceil = 9$.

9.7 Cost-performance tradeoff frontier

How much does tightening the utilization SLA cost? `ParetoSweep` re-solves the sizing problem for a sweep of utilization bounds (epsilon-constraint method) and reports the non-dominated cost frontier, here with each point solved exactly by bisection (`opt_pareto_frontier.m`):

```
factory = @(eps) opt.UtilizationConstraint(queue, eps);
sweep = opt.ParetoSweep(problem, factory, ...
    [0.3 0.4 0.5 0.6 0.75 0.9], 'bisection');
sweep.solve();
frontier = sweep.getFrontier();
for i = 1:numel(frontier)
    p = frontier{i};
    fprintf(' util <= %-4g -> %6.1f (%d)\n', p.epsilon, p.objectiveValue, ...
        p.result.getVariableValue('Server_servers'));
end
```

Obtaining the following output:

```
Utilization bound -> optimal cost (servers)
util <= 0.3 -> 100.0 (10)
util <= 0.4 -> 80.0 (8)
util <= 0.5 -> 60.0 (6)
util <= 0.6 -> 50.0 (5)
util <= 0.75 -> 40.0 (4)
```

The bound 0.9 is absent from the frontier: it requires the same 4 servers as 0.75, so its point is dominated. Frontier points are returned as a cell array of `opt.ParetoPoint` objects carrying the bound (`epsilon`), the optimal objective (`objectiveValue`) and the underlying

`opt.OptimizationResult (result)`, from which a plot can be drawn with the standard MATLAB `stairs` function.

9.8 Decomposing a multi-variable problem

Jointly choosing the number of servers and the service rate couples two variable types. The decomposition workflow splits them into blocks (servers first, then rates), solves the blocks in sequence with the other block's values held fixed, and cycles until the full penalized objective stabilizes, i.e., block coordinate descent (`opt_decomposition.m`):

```
problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));
problem.addVariable(opt.ServiceRate(queue, jobs, [1.0 4.0]));
serverCost = configureDictionary('string', 'double');
serverCost('Server') = 10.0;
rateCost = configureDictionary('string', 'double');
rateCost('Server') = 20.0;
problem.setObjective(opt.MinimizeCost(serverCost, rateCost, [], {}));
problem.addConstraint(opt.ResponseTimeConstraint(queue, jobs, 0.5));

workflow = problem.decompose();
workflow.autoDecompose();
options = opt.LineOptSolverOptions(); options.setSeed(42);
workflow.setSolverOptions(options);
result = workflow.solveSequential(4, 1e-3);
```

Obtaining the following output:

```
Converged      : True after 1 cycle(s)
Servers        : 10
Rate           : 2.045
Objective      : 140.90
```

As with any coordinate descent scheme on a non-convex problem, the workflow converges to a block-wise optimum that may differ from the joint optimum; a joint solve on this instance finds cost 76. Decomposition remains preferable when the joint dimensionality is intractable.

9.9 The models behind the tutorials

The tutorials above are design problems, and each one optimizes a queueing network that the narrative describes but does not draw. This section gives that network for every script of `examples/opt/`: its topology, its features, and the output the script prints once the optimizer has run.

9.9.1 `opt_bisection_sizing`

The same sizing problem solved exactly by bisection, which is admissible because the constraint is monotone in the number of servers.

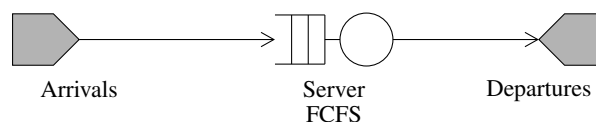


Figure 9.1: Topology of `opt_bisection_sizing`: an open network over 1 queueing station.

The example is built and solved as follows.

```
% opt_bisection_sizing Exact server sizing via BisectionSolver (O(log n)) LINE
```

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.1: Features of `opt_bisection_sizing`.

```
% solves). Same M/M/c as opt_server_sizing -> 6 servers, cost 60, 4 probes.
% Mirrors python/examples/opt/opt_bisection_sizing.py.

model = Network('MMc');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0));
queue.setService(jobs, Exp(1.0));
model.link(Network.serialRouting(source, queue, sink));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));
serverCost = configureDictionary('string', 'double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));

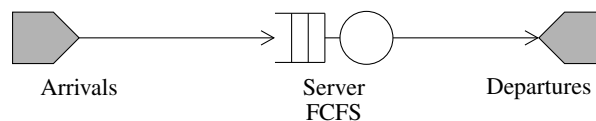
result = opt.BisectionSolver(problem).solve();

fprintf('Optimal servers : %d\n', result.getVariableValue('Server_servers'));
fprintf('Cost           : %.1f\n', result.objectiveValue);
fprintf('LINE evaluations: %d (vs hundreds for DE)\n', result.modelEvaluations);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.2 `opt_decomposition`

Decomposing a multi-variable problem into blocks solved in turn, the coordinate-descent workflow and its blockwise optimum.

Figure 9.2: Topology of `opt_decomposition`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.2: Features of `opt_decomposition`.

The example is built and solved as follows.


```
% opt_decomposition Two-variable problem (server count + service rate) solved
% by a DecompositionWorkflow: auto-decompose by variable type, then Gauss-Seidel
% cycling to a fixed point. Mirrors opt_decomposition.py.

model = Network('MMc');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0));
queue.setService(jobs, Exp(1.0));
model.link(Network.serialRouting(source, queue, sink));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));
problem.addVariable(opt.ServiceRate(queue, jobs, [1.0 4.0]));
serverCost = configureDictionary('string','double'); serverCost('Server') = 10.0;
rateCost = configureDictionary('string','double'); rateCost('Server') = 20.0;
problem.setObjective(opt.MinimizeCost(serverCost, rateCost, [], {}));
problem.addConstraint(opt.ResponseTimeConstraint(queue, jobs, 0.5));

workflow = problem.decompose();
workflow.autoDecompose();
options = opt.LineOptSolverOptions(); options.setSeed(42);
workflow.setSolverOptions(options);
result = workflow.solveSequential(4, 1e-3);

fprintf('Converged      : %d after %d cycle(s)\n', result.converged, result.cyclesCompleted)
;
fprintf('Servers        : %d\n', result.getFinalVariableValue('Server_servers'));
fprintf('Rate           : %.3f\n', result.getFinalVariableValue('Server_Jobs_rate'));
fprintf('Objective       : %.2f\n', result.finalObjective);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.3 opt_load_balancing

Load balancing across heterogeneous servers: the decision variables are the routing probabilities, which must form a simplex.

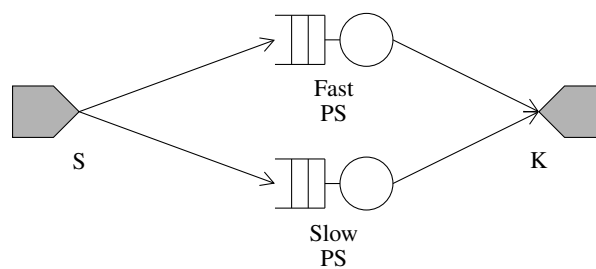


Figure 9.3: Topology of `opt_load_balancing`: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Jobs (open)
Scheduling	Fast: PS; Slow: PS
Arrivals	S – Jobs: Exp(2)
Service	Fast – Jobs: Exp(4) Slow – Jobs: Exp(2.5)
Features	class switching on 4 links
Solvers	MVA

Table 9.3: Features of `opt_load_balancing`.

The example is built and solved as follows.

```
% opt_load_balancing Routing split from a source to a fast and a slow PS
% server minimizing end-to-end response time (theory fast fraction ~0.743).
% Mirrors opt_load_balancing.py.

model = Network('LoadBalance');
source = Source(model, 'S');
fast = Queue(model, 'Fast', SchedStrategy.PS);
slow = Queue(model, 'Slow', SchedStrategy.PS);
sink = Sink(model, 'K');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(2.0));
fast.setService(jobs, Exp(4.0));
slow.setService(jobs, Exp(2.5));
model.addLink(source, fast);
model.addLink(source, slow);
model.addLink(fast, sink);
model.addLink(slow, sink);

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.RoutingProbabilities(jobs, source, {fast, slow}));
problem.setObjective(opt.MinimizeSystemResponseTime(jobs));

options = opt.LineOptSolverOptions(); options.setSeed(42); options.setMaxIterations(60);
result = problem.solve(options);

probs = result.getVariableValue('Jobs_routing_from_S');
fprintf('Fraction to Fast: %.3f (theory: 0.743)\n', probs(1));
fprintf('Fraction to Slow: %.3f\n', probs(2));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.4 opt_lqn_host_demand

Host demand tuning on a layered model, with the gradient optimizer and layer freezing.

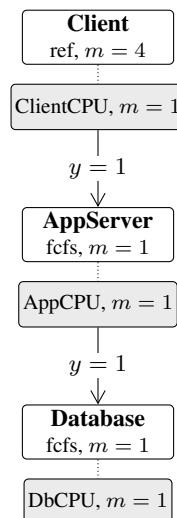


Figure 9.4: Call graph of `opt_lqn_host_demand`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

The example is built and solved as follows.

```
% opt_lqn_host_demand Tune the host demands of a two-layer LQN under an
```

Processors	3: ClientCPU (inf, $m = 1$), AppCPU (ps, $m = 1$), DbCPU (ps, $m = 1$)
Tasks	3: Client (ref, $m = 4$), AppServer (fcfs, $m = 1$), Database (fcfs, $m = 1$)
Entries	3: Browse, Render, Query
Activities	3, host demands BrowseAct: 0.5, RenderAct: 0.25, QueryAct: 0.2
Calls	2 (2 synchronous, 0 asynchronous)
Think times	Client: 1.0

Table 9.4: Features of `opt_lqn_host_demand`.

```
% end-to-end response-time budget, using the partial-sensitivity gradient.
%
% lqnGradient='partial_plus_fd' drives the analytic path: SolverLN's per-layer
% d(metric)/d(service rate) table is reshaped into a sensitivity dictionary and
% chained with d(rate)/d(demand) = -1/D^2, with a periodic whole-model finite
% difference (every fdRefresh gradient calls) correcting the bias.

model = LayeredNetwork('BookstoreLQN');

P1 = Processor(model, 'ClientCPU', 1, SchedStrategy.INF);
T1 = Task(model, 'Client', 4, SchedStrategy.REF).on(P1);
T1.setThinkTime(Exp(1.0));
E1 = Entry(model, 'Browse').on(T1);

P2 = Processor(model, 'AppCPU', 1, SchedStrategy.PS);
T2 = Task(model, 'AppServer', 1, SchedStrategy.FCFS).on(P2);
E2 = Entry(model, 'Render').on(T2);

P3 = Processor(model, 'DbCPU', 1, SchedStrategy.PS);
T3 = Task(model, 'Database', 1, SchedStrategy.FCFS).on(P3);
E3 = Entry(model, 'Query').on(T3);

A1 = Activity(model, 'BrowseAct', Exp(2.0)).on(T1).boundTo(E1).synchCall(E2, 1);
A2 = Activity(model, 'RenderAct', Exp(4.0)).on(T2).boundTo(E2).synchCall(E3, 1);
A2.repliesTo(E2);

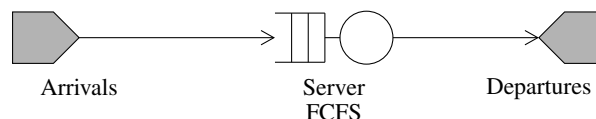
... (22 source lines omitted; full implementation: matlab/examples/opt/)

fprintf('Render demand    : %.4f\n', result.getVariableValue('RenderAct_hostdemand'));
fprintf('Query demand     : %.4f\n', result.getVariableValue('QueryAct_hostdemand'));
fprintf('System RespT       : %.4f\n', result.objectiveValue);
fprintf('Feasible           : %d\n', result.feasible);
fprintf('Total violation     : %.4f\n', result.getTotalViolation());
fprintf('LINE evaluations: %d\n', result.modelEvaluations);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.5 `opt_pareto_frontier`

The cost-performance tradeoff frontier, swept rather than solved at one weighting, through ParetoSweep.

Figure 9.5: Topology of `opt_pareto_frontier`: an open network over 1 queueing station.

The example is built and solved as follows.

```
% opt_pareto_frontier Cost vs utilization-bound frontier via ParetoSweep
% (epsilon-constraint method, bisection per point). Mirrors
```

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.5: Features of `opt_pareto_frontier`.

```
% opt_pareto_frontier.py.

model = Network('MMc');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0));
queue.setService(jobs, Exp(1.0));
model.link(Network.serialRouting(source, queue, sink));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));
serverCost = configureDictionary('string', 'double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));

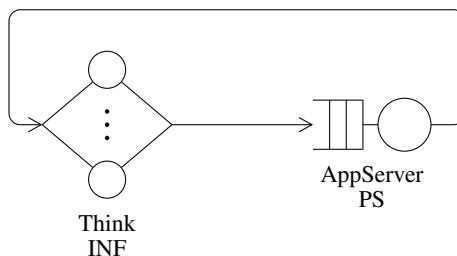
factory = @(eps) opt.UtilizationConstraint(queue, eps);
sweep = opt.ParetoSweep(problem, factory, [0.3 0.4 0.5 0.6 0.75 0.9], 'bisection');
sweep.solve();

fprintf('Utilization bound -> optimal cost (servers)\n');
frontier = sweep.getFrontier();
for i = 1:numel(frontier)
    p = frontier{i};
    fprintf(' util <= %-4g -> %6.1f (%d)\n', p.epsilon, p.objectiveValue, ...
        p.result.getVariableValue('Server_servers'));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.6 `opt_population_sizing`

Population sizing in a closed network: how many jobs to admit so that throughput is maximized without breaching a residence-time bound.

Figure 9.6: Topology of `opt_population_sizing`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
% opt_population_sizing Largest closed-class population sustaining an
% interactive response-time SLA, via BisectionSolver('max_feasible').
```

Nodes	2: Queue, Delay
Classes	1: Users (closed, $N = 1$)
Scheduling	Think: INF; AppServer: PS
Service	Think – Users: Exp(1) AppServer – Users: Exp(2)
Features	class switching on 2 links
Solvers	MVA

Table 9.6: Features of `opt_population_sizing`.

```
% Interactive system: Delay (think) + PS server. Mirrors opt_population_sizing.py.

model = Network('Interactive');
think = Delay(model, 'Think');
server = Queue(model, 'AppServer', SchedStrategy.PS);
users = ClosedClass(model, 'Users', 1, think);
think.setService(users, Exp(1.0));
server.setService(users, Exp(2.0));
model.link(Network.serialRouting(think, server));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.JobPopulation(users, [1 50]));
problem.setObjective(opt.MinimizeCost([], [], {}, {})); % pure feasibility sizing
problem.addConstraint(opt.ResponseTimeConstraint(server, users, 2.0));

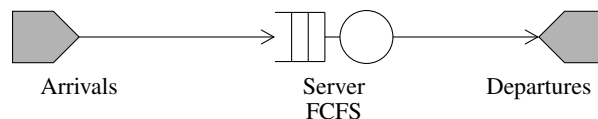
result = opt.BisectionSolver(problem, 'max_feasible').solve();

fprintf('Max users      : %d\n', result.getVariableValue('Users_population'));
fprintf('Feasible       : %d\n', result.feasible);
fprintf('LINE evaluations: %d\n', result.modelEvaluations);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.7 `opt_robust_sizing`

Robust sizing, in which the constraint must hold across a set of workload scenarios rather than at one nominal load.

Figure 9.7: Topology of `opt_robust_sizing`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.7: Features of `opt_robust_sizing`.

The example is built and solved as follows.

```
% opt_robust_sizing Worst-case server sizing across an average-load and a
% peak-load workload scenario (constraints enforced on all scenarios). Average
% load needs 6 servers; the peak scenario pushes it to 9. Mirrors
```

```
% opt_robust_sizing.py.

base = buildMMc(3.0);      % average load
peak = buildMMc(4.5);     % peak load scenario
nodes = base.getNodes(); queue = nodes{2};

problem = opt.OptimizationProblem(base);
problem.addVariable(opt.ServerAllocation(queue, [1 12]));
serverCost = configureDictionary('string','double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));
problem.addScenario(peak, 1.0);

result = opt.BisectionSolver(problem).solve();

fprintf('Robust servers : %d (6 for average load only)\n', result.getVariableValue('
    Server_servers'));
fprintf('Cost          : %.1f\n', result.objectiveValue);
fprintf('Feasible       : %d\n', result.feasible);

function model = buildMMc(arrivalRate)
    model = Network('MMc');
    source = Source(model, 'Arrivals');
    queue = Queue(model, 'Server', SchedStrategy.FCFS);
    sink = Sink(model, 'Departures');
    jobs = OpenClass(model, 'Jobs');
    source.setArrival(jobs, Exp(arrivalRate));
    queue.setService(jobs, Exp(1.0));
    model.link(Network.serialRouting(source, queue, sink));
end
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.8 opt_sensitivity_report

The sensitivity of the optimum to each parameter, reported alongside the solution.

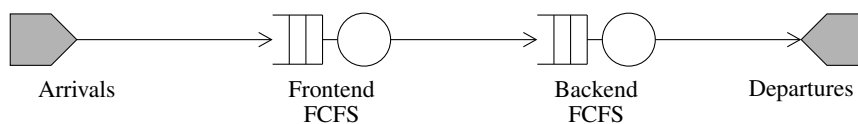


Figure 9.8: Topology of opt_sensitivity_report: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Jobs (open)
Scheduling	Frontend: FCFS; Backend: FCFS
Arrivals	Arrivals – Jobs: Exp(1)
Service	Frontend – Jobs: Exp(2) Backend – Jobs: Exp(1.5)

Table 9.8: Features of opt_sensitivity_report.

The example is built and solved as follows.

```
% opt_sensitivity_report Read back the analytic d(metric)/d(service rate)
% table that opt.LineEvaluator attaches to every flat-model evaluation.
%
% opt.sens.computeModelSensitivities fills an opt.SensitivityData, a three-level
% nested dictionary (metric kind -> metric key -> parameter key -> derivative).
% The optimizer consumes it internally; this example walks it directly via
% forKind, which is also how one inspects which station a rate change moves.
```

```

model = Network('Tandem');
source = Source(model, 'Arrivals');
q1 = Queue(model, 'Frontend', SchedStrategy.FCFS);
q2 = Queue(model, 'Backend', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(1.0));
q1.setService(jobs, Exp(2.0));
q2.setService(jobs, Exp(1.5));
model.link(Network.serialRouting(source, q1, q2, sink));

evaluator = opt.LineEvaluator(model, {}, {});
result = evaluator.evaluateValues(configureDictionary('string', 'cell'));

fprintf('Solver used : %s\n', result.solverUsed);
fprintf('Feasible      : %d\n', result.feasible);
fprintf('Frontend Q    : %.4f\n', result.getQueueLength('Frontend'));
fprintf('Backend Q     : %.4f\n', result.getQueueLength('Backend'));

... (17 source lines omitted; full implementation: matlab/examples/opt/)

pk = sort(keys(byParam));
for j = 1:numel(pk)
    fprintf(' %24s %32s % .6f\n', mk(i), pk(j), byParam(pk(j)));
end
end
end

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.9 opt_server_sizing

Sizing an M/M/c queue: choose the number of servers that minimizes cost subject to a response-time bound, by differential evolution over an integer decision variable.

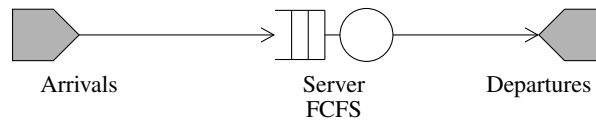


Figure 9.9: Topology of opt_server_sizing: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.9: Features of opt_server_sizing.

The example is built and solved as follows.

```

% opt_server_sizing Minimum servers of an M/M/c queue under a 50% utilization
% SLA, via differential evolution. lambda=3, mu=1 -> optimum c*=6 (cost 60).
% Mirrors python/examples/opt/opt_server_sizing.py.

model = Network('MMc');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');

```

```

jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0));
queue.setService(jobs, Exp(1.0));
model.link(Network.serialRouting(source, queue, sink));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));
serverCost = configureDictionary('string', 'double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));

options = opt.LineOptSolverOptions(); options.setSeed(42);
result = problem.solve(options);

fprintf('Optimal servers : %d\n', result.getVariableValue('Server_servers'));
fprintf('Cost           : %.1f\n', result.objectiveValue);
fprintf('Feasible        : %d\n', result.feasible);
fprintf('LINE evaluations: %d\n', result.modelEvaluations);

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.9.10 opt_service_rate

Continuous tuning of a service rate, the archetype of a continuous decision variable under a service-level constraint.

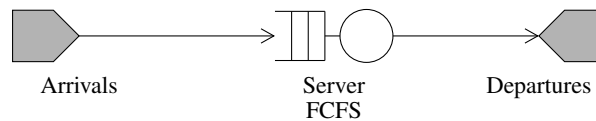


Figure 9.10: Topology of opt_service_rate: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(4)
Features	class switching on 3 links
Solvers	MVA

Table 9.10: Features of opt_service_rate.

The example is built and solved as follows.

```

% opt_service_rate Cheapest continuous service rate meeting a response-time
% SLA on an M/M/1 (theory rate 5.0). Mirrors opt_service_rate.py.

model = Network('MM1');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0));
queue.setService(jobs, Exp(4.0));
model.link(Network.serialRouting(source, queue, sink));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServiceRate(queue, jobs, [3.5 8.0]));
rateCost = configureDictionary('string', 'double');
rateCost('Server') = 20.0;

```



```
problem.setObjective(opt.MinimizeCost([], rateCost, [], {}));  
problem.addConstraint(opt.ResponseTimeConstraint(queue, jobs, 0.5));  
  
options = opt.LineOptSolverOptions(); options.setSeed(42); options.setMaxIterations(60);  
result = problem.solve(options);  
  
fprintf('Optimal rate      : %.3f (theory: 5.0)\n', result.getVariableValue('Server_Jobs_rate'))  
);  
fprintf('Cost              : %.1f\n', result.objectiveValue);  
fprintf('Feasible           : %d\n', result.feasible);
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Index

- ag_closed_network, 227
- ag_gnetwork, 228
- ag_jackson_network, 229
- ag_multiclass_closed, 230
- ag_tandem_open, 231
- ag_tandem_phasetype, 232
- agent models, 227
- bisection, 331
- block coordinate descent, 334
- busy period, 198
- busyp_subnetwork, 198
- Cache node, 96
- cache task, 212
- cache_compare_replc, 97
- cache_itemsize_costcap, 98
- cache_mmap_rr_env, 99
- cache_replc_climb, 101
- cache_replc_fifo, 102
- cache_replc_hlru, 104
- cache_replc_lru, 105
- cache_replc_qlru, 107
- cache_replc_routing, 108
- cache_replc_rr, 110
- cache_rmf_transient, 111
- cachenet_tandem, 112
- caching, 96, 236
- capacity planning, 333
- cdf_respt_closed, 193
- cdf_respt_closed_threeclasses, 194
- cdf_respt_distrib, 195
- cdf_respt_open_twoclasses, 196
- cdf_respt_populations, 197
- cl_basic, 122
- cl_closed, 123
- cl_compare, 123
- cl_multiclass, 124
- cl_scheduling, 125
- cl_stations, 125
- cl_sweep, 126
- class priorities, 44
- class switching, 39
- ClassSwitch node, 39
- closed queueing network, 16
- cluster network, 122
- constraint, 330
- cqn_bas_blocking, 16
- cqn_bcmp_theorem, 17
- cqn_lcfs_multiclass, 18
- cqn_mmp2_service, 19
- cqn_multichain_cs, 20
- cqn_multiserver, 21
- cqn_multiserver_nc, 22
- cqn_online, 23
- cqn_repairmen, 24
- cqn_repairmen_multi, 25
- cqn_scheduling_dps, 26
- cqn_threeclass_hyperl, 27
- cqn_twoclass_erl, 28
- cqn_twoclass_hyperl, 29
- cqn_twoqueues_multi, 30
- cross-codebase parity, 1
- cs_implicit, 39
- cs_multi_diamond, 40
- cs_single_diamond, 41
- cs_transient_class, 42
- CTMC solver, 301
- ctmc_spn_mm1, 301
- ctmc_spn_vs_nc_closed_qn, 302
- ctmc_tandem_mm1, 303
- custom solver, 234
- cyclic polling, 189
- decision variable, 330
- design problem, 330, 334
- differential evolution, 330
- discrete time, 294
- dispatch_closed, 127
- dispatch_mixed, 128
- dispatch_open, 128
- distributions
 - Geometric, 294
- dt_bernoulli_loaddep, 294

[dt_cycle](#), [295](#)
[dt_cycle_loaddep](#), [296](#)
[dt_cycle_multiclass](#), [297](#)
[dt_geogeo1](#), [298](#)
[dt_geogeo1_loss](#), [299](#)

epsilon-constraint method, [333](#)
[est_ekf_changepoint](#), [312](#)
[est_ekf_closed](#), [313](#)
[est_ekf_open](#), [314](#)
[est_erps_closed](#), [315](#)
[est_erps_open](#), [316](#)
[est_mcmc_closed](#), [317](#)
[est_mle_open](#), [318](#)
[est_qmle_closed](#), [319](#)
[est_trace_closed](#), [320](#)
[est_ubo_changepoint](#), [321](#)
[est_ubo_closed](#), [322](#)
[est_ubo_open](#), [323](#)
[est_ubo_variants_closed](#), [324](#)
[est_ubr_changepoint](#), [325](#)
[est_ubr_closed](#), [326](#)
[est_ubr_open](#), [327](#)
[est_vi_closed](#), [328](#)
[example_mapqn2renv](#), [202](#)
Extended Kalman Filter, [312](#)

[fcr_constraints](#), [163](#)
[fcr_lincon](#), [164](#)
[fcr_lossn](#), [165](#)
[fcr_mmlkdrop](#), [166](#)
[fcr_mmlwaitq](#), [167](#)
[fcr_oqndrop](#), [168](#)
[fcr_oqnwaitq](#), [169](#)
feature table, [1](#)
[fes_aggregation](#), [151](#)
[fes_single_class](#), [152](#)
file formats
 JSON, [305](#)
finite capacity region, [163](#), [236](#)
[fj_asymm](#), [49](#)
[fj_basic_closed](#), [50](#)
[fj_basic_nesting](#), [51](#)
[fj_basic_open](#), [52](#)
[fj_complex_serial](#), [53](#)
[fj_cs_multi_visits](#), [54](#)
[fj_cs_postfork](#), [55](#)
[fj_cs_prefork](#), [57](#)
[fj_deep_nesting](#), [58](#)
[fj_delays](#), [59](#)

[fj_mixed_openclosed](#), [60](#)
[fj_nojoin](#), [62](#)
[fj_route_overlap](#), [63](#)
[fj_serialfjs_closed](#), [64](#)
[fj_serialfjs_open](#), [65](#)
[fj_threebranches](#), [66](#)
[fj_tiny_closed](#), [67](#)
[fj_twoclasses_forked](#), [68](#)
[fj_variable_fanout](#), [70](#)
flow-equivalent server, [151](#)
Fork node, [49](#)
fork-join, [49](#), [236](#)

[gallery_aphm1](#), [237](#)
[gallery_cache_lru](#), [238](#)
[gallery_cache_routing](#), [239](#)
[gallery_coxm1](#), [240](#)
[gallery_cqn](#), [240](#)
[gallery_cqn_multiclass](#), [241](#)
[gallery_detm1](#), [242](#)
[gallery_dm1](#), [243](#)
[gallery_erldk](#), [244](#)
[gallery_erlerl1](#), [245](#)
[gallery_erlerl1_reentrant](#), [245](#)
[gallery_erlm1](#), [246](#)
[gallery_erlm1_ps](#), [247](#)
[gallery_erlm1_reentrant](#), [248](#)
[gallery_erlm1ps](#), [248](#)
[gallery_fcr](#), [249](#)
[gallery_fj_closed](#), [250](#)
[gallery_fj_open](#), [251](#)
[gallery_fj_quorum](#), [252](#)
[gallery_gamm1](#), [253](#)
[gallery_hyperl1_feedback](#), [254](#)
[gallery_hyperl1_reentrant](#), [255](#)
[gallery_hyperlk](#), [255](#)
[gallery_hyphyp1_linear](#), [256](#)
[gallery_hyphyp1_reentrant](#), [257](#)
[gallery_hyphyp1_tandem](#), [258](#)
[gallery_hypm1](#), [258](#)
[gallery_hypm1_reentrant](#), [259](#)
[gallery_lqn_basic](#), [260](#)
[gallery_lqn_random](#), [261](#)
[gallery_lqn_workflows](#), [262](#)
[gallery_lukumar_reentrant](#), [263](#)
[gallery_mapm1](#), [264](#)
[gallery_mapmk](#), [265](#)
[gallery_mdk](#), [265](#)
[gallery_merl1](#), [266](#)
[gallery_merl1_linear](#), [267](#)

[gallery_mer11_reentrant](#), 268
[gallery_mer11_tandem](#), 268
[gallery_mer1k](#), 269
[gallery_mhyp1](#), 270
[gallery_mhyp1_linear](#), 270
[gallery_mhyp1_reentrant](#), 271
[gallery_mhyp1_tandem](#), 272
[gallery_mhypk](#), 273
[gallery_mm1](#), 273
[gallery_mm1_feedback](#), 274
[gallery_mm1_linear](#), 275
[gallery_mm1_multiclass](#), 276
[gallery_mm1_prio](#), 276
[gallery_mm1_ps](#), 277
[gallery_mm1_ps_feedback](#), 278
[gallery_mm1_ps_multiclass](#), 279
[gallery_mm1_ps_reentrant](#), 279
[gallery_mm1_reentrant](#), 280
[gallery_mm1_tandem](#), 281
[gallery_mm1_tandem_multiclass](#), 281
[gallery_mm1k](#), 282
[gallery_mmap1](#), 283
[gallery_mmap1_multiclass](#), 284
[gallery_mmapk](#), 284
[gallery_mmk](#), 285
[gallery_mpar1](#), 286
[gallery_multitier](#), 287
[gallery_multitier_storage](#), 288
[gallery_parm1](#), 289
[gallery_qn_random](#), 290
[gallery_renv_breakdown](#), 291
[gallery_repairmen](#), 291
[gallery_replayerm1](#), 292
[gallery_um1](#), 293
[Geo/Geo/1 queue](#), 294
[Geometric distribution](#), 294

[HOL scheduling](#), 44

[infinitesimal generator](#), 301

[init_state_fcfs_exp](#), 178

[init_state_fcfs_nonexp](#), 179

[init_state_ps](#), 180

[Join node](#), 49

[JSON format](#), 305

[json_balking_retrial](#), 305

[json_classcap_droprule](#), 306

[json_fcr_details](#), 307

[json_hetero_servers](#), 308

[json_join_deadline](#), 309

[json_signal_classes](#), 310

[Kendall notation](#), 236

[large-scale models](#), 233

[largescale_tbi](#), 233

[layered queueing network](#), 71, 236

[lcq_async_cache](#), 212

[lcq_singlehost](#), 213

[lcq_threehosts](#), 214

[ld_class_dependence](#), 153

[ld_fes_multiclass](#), 154

[ld_fes_singleclass](#), 155

[ld_global_dependence](#), 156

[ld_joint_dependence](#), 157

[ld_multiserver_fcfs](#), 158

[ld_multiserver_ps](#), 159

[ld_multiserver_ps_twoclasses](#), 160

[ld_whittle_bandwidth](#), 161

[ldes_warmstart](#), 181

[load balancing](#), 332, 334

[load dependence](#), 151

[lqn_basic](#), 71

[lqn_bpmn](#), 72

[lqn_bpmn_trace](#), 74

[lqn_flatph](#), 75

[lqn_fork_open_arrival](#), 77

[lqn_init](#), 78

[lqn_jsq](#), 79

[lqn_moment3](#), 80

[lqn_multi_solvers](#), 81

[lqn_ofbiz](#), 82

[lqn_open_arrival](#), 84

[lqn_paramident](#), 85

[lqn_rrobin](#), 86

[lqn_serial](#), 87

[lqn_server_pools](#), 88

[lqn_setup](#), 89

[lqn_sockshop](#), 90

[lqn_srvnph](#), 91

[lqn_transient](#), 93

[lqn_twotasks](#), 94

[lqn_workflows](#), 95

[Lu-Kumar re-entrant line](#), 236

[mam_transient_mapmap1](#), 2

[maximum likelihood](#), 312

[MCMC](#), 312

[mixed network](#), 31

[model features](#)

[caching](#), 96, 236

- class priorities, [44](#)
- class switching, [39](#)
- cyclic polling, [189](#)
- finite capacity region, [163](#), [236](#)
- fork-join, [49](#), [236](#)
- load balancing, [332](#), [334](#)
- load dependence, [151](#)
- random environment, [202](#), [236](#)
- reward models, [223](#)
- slotted time, [294](#)
- state-dependent routing, [182](#)
- switchover times, [192](#)
- warm start, [178](#)
- model gallery, [236](#)
- model interchange, [305](#)
- mqn_basic, [32](#)
- mqn_multichain_cs, [33](#)
- mqn_multiserver_fcfs, [34](#)
- mqn_multiserver_ps, [35](#)
- mqn_singleserver_fcfs, [36](#)
- mqn_singleserver_ps, [38](#)
- node types
 - Cache, [96](#)
 - ClassSwitch, [39](#)
 - Fork, [49](#)
 - Join, [49](#)
 - Place, [129](#)
 - Transition, [129](#)
- objective function, [330](#)
- open queueing network, [2](#)
- opt_bisection_sizing, [334](#)
- opt_decomposition, [335](#)
- opt_load_balancing, [336](#)
- opt_lqn_host_demand, [337](#)
- opt_pareto_frontier, [338](#)
- opt_population_sizing, [339](#)
- opt_robust_sizing, [340](#)
- opt_sensitivity_report, [341](#)
- opt_server_sizing, [342](#)
- opt_service_rate, [343](#)
- oqn_basic, [3](#)
- oqn_bmmapt, [4](#)
- oqn_cs_routing, [5](#)
- oqn_fourqueues, [6](#)
- oqn_mapt, [8](#)
- oqn_mapt_service, [9](#)
- oqn_mmapt, [10](#)
- oqn_multichain_cs, [11](#)
- oqn_nhpp, [12](#)
- oqn_oneline, [13](#)
- oqn_trace_driven, [14](#)
- oqn_vsinks, [15](#)
- parameter inference, [312](#)
- Pareto frontier, [333](#), [334](#)
- pas_closed_tandem_fig6, [216](#)
- pas_compatibility_5class, [217](#)
- pas_cyclic_ctmc_vs_bruteforce, [218](#)
- pas_cyclic_normconst_bruteforce, [218](#)
- pas_mmk, [219](#)
- pas_nc_sampling, [220](#)
- pas_saturation, [221](#)
- pas_selfloop, [222](#)
- pass and swap, [215](#)
- Place node, [129](#)
- polling_exhaustive_det, [189](#)
- polling_exhaustive_exp, [190](#)
- polling_gated, [190](#)
- polling_klimited, [191](#)
- prio_hol_closed, [44](#)
- prio_hol_open, [45](#)
- prio_identical, [46](#)
- prio_psprio, [48](#)
- QMLE, [312](#)
- random environment, [202](#), [236](#)
- renv_basic, [203](#)
- renv_container_terminal, [204](#)
- renv_fourstages_repairmen, [205](#)
- renv_genqn, [206](#)
- renv_lqn_twostages, [206](#)
- renv_map_fallback, [207](#)
- renv_node_breakdown, [208](#)
- renv_rotterdam_blending, [209](#)
- renv_threestages_repairmen, [210](#)
- renv_twostages_repairmen, [211](#)
- response time distribution, [193](#)
- retrieval_chain, [115](#)
- retrieval_default, [116](#)
- retrieval_ps, [117](#)
- retrieval_routing, [119](#)
- retrieval_simple, [120](#)
- reward models, [223](#)
- rewardModel_aggregation, [223](#)
- rewardModel_mmlk, [224](#)
- rewardModel_multiclass, [225](#)
- rewardModel_templates, [226](#)

- round-trip fixture, [305](#)
- routing matrix, [1](#)
- runnable entry, [1](#)
- scheduling strategies
 - HOL, [44](#)
- script structure, [1](#)
- sdroute_closed, [182](#)
- sdroute_jsq, [183](#)
- sdroute_krzesinski, [184](#)
- sdroute_multibranch, [185](#)
- sdroute_open, [186](#)
- sdroute_twoclasses_closed, [187](#)
- serial routing, [1](#)
- server sizing, [330](#), [334](#)
- slot, [294](#)
- slotted time, [294](#)
- snc_delay_quantile, [200](#)
- snc_tandem_multiclass, [201](#)
- solution methods
 - bisection, [331](#)
 - block coordinate descent, [334](#)
 - differential evolution, [330](#)
 - epsilon-constraint method, [333](#)
 - Extended Kalman Filter, [312](#)
 - flow-equivalent server, [151](#)
 - maximum likelihood, [312](#)
 - MCMC, [312](#)
 - QMLE, [312](#)
 - stochastic network calculus, [199](#)
 - variational inference, [312](#)
- solver_custom, [234](#)
- solver_custom_analyzer, [234](#)
- solvers
 - CTMC, [301](#)
- spn_basic_closed, [130](#)
- spn_basic_open, [131](#)
- spn_closed_fourplaces, [132](#)
- spn_closed_twoplaces, [133](#)
- spn_colored_gspn, [134](#)
- spn_fluid_dae, [135](#)
- spn_fourmodes, [136](#)
- spn_inhibiting, [137](#)
- spn_lpbounds, [138](#)
- spn_open_sevenplaces, [139](#)
- spn_pareto_service, [140](#)
- spn_productform_nc, [141](#)
- spn_queueing_place, [142](#)
- spn_twomodes, [143](#)
- state probability, [170](#)
- state-dependent routing, [182](#)
- statepr_aggr, [170](#)
- statepr_aggr_large, [171](#)
- statepr_allprobs_fcfs, [172](#)
- statepr_allprobs_ps, [173](#)
- statepr_sys_aggr, [174](#)
- statepr_sys_aggr_large, [175](#)
- statepr_sys_marg, [177](#)
- stochastic network calculus, [199](#)
- stochastic Petri net, [129](#), [304](#)
- switchover times, [192](#)
- switchover_basic, [192](#)
- test_spn_nrm_open, [144](#)
- topology figure, [1](#)
- Transition node, [129](#)
- variational inference, [312](#)
- warm start, [178](#)
- wf_aph, [145](#)
- wf_branch, [146](#)
- wf_complex, [146](#)
- wf_erlang, [147](#)
- wf_loop, [148](#)
- wf_parallel, [149](#)
- wf_serial, [149](#)
- wire key, [311](#)
- worked example, [1](#)
- workflow model, [145](#)
- workload scenario, [333](#)